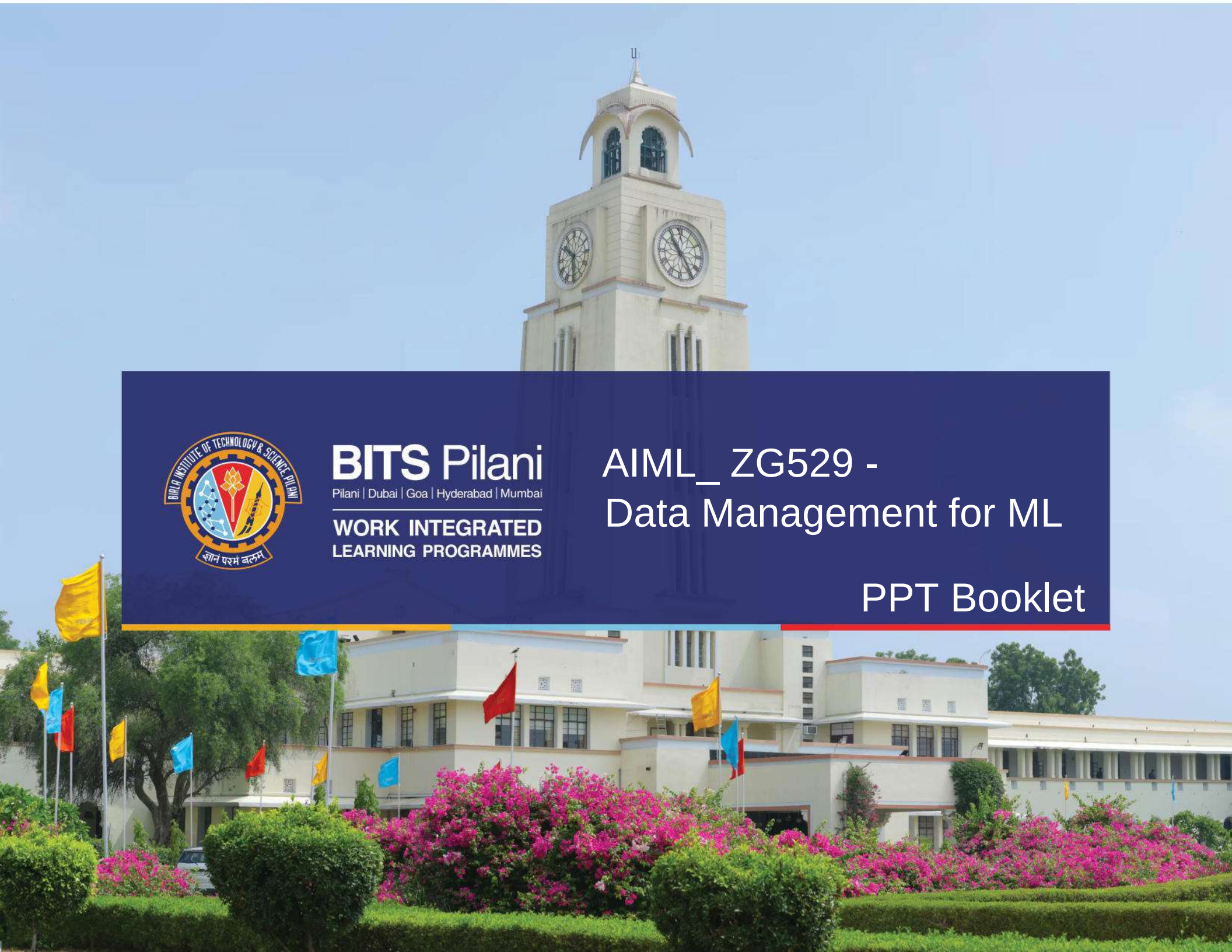


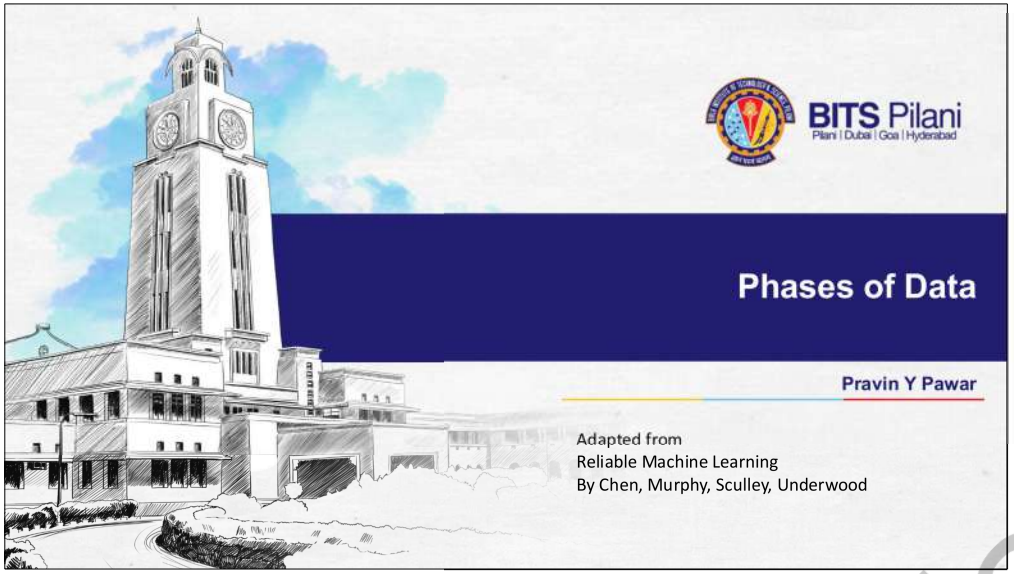
BITS Pilani

Pilani | Dubai | Goa | Hyderabad | Mumbai

WORK INTEGRATED LEARNING PROGRAMMES

AIML_ ZG529 - Data Management for ML

PPT Booklet



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Phases of Data

Pravin Y Pawar

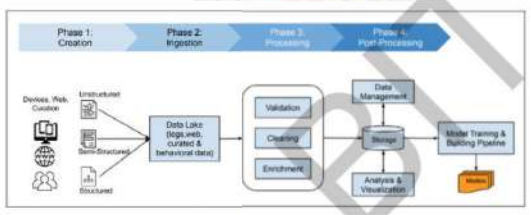
Adapted from
Reliable Machine Learning
By Chen, Murphy, Sculley, Underwood

Data Management

- The data management stage is about transforming the data into a format and storage organization suitable for using it for later stages of the process
- During this process
  - may apply a range of data transformations that are model specific to prepare the data for training
  - next operations on the data will be to
    - train ML models,
    - anonymize certain sensitive items of data
    - delete data when we no longer need it or are asked to do so
- To prepare for these operations on the data,
  - take input from the business leaders to answer questions about primary use cases for the data
  - explore possible areas for future exploration

Phases of Data

- Data management in modern ML environments consists of multiple phases before feeding the data into model-training pipelines
  - Creation
  - Ingestion
  - Processing (which includes validation, cleaning, and enrichment)
  - Post-processing (which includes data management, storage, and analysis)



ML data management phases

Phases of Data(2) - Creation

Sources

- Odd or obvious to state, but ML training data comes from somewhere
  - Datasets are all created at some point via some process
  - The process of generating or capturing the data in some data storage system but not our data storage system
  - Examples - logs from serving systems, large collections of images captured from an event, diagnostic data from medical procedures
- Some datasets work well when they are static (or at least don't change quickly), and others are useful only when frequently updated
- For example,
  - A photo-recognition dataset, may be usable for many months, as long as it is suitably representative of the types of photos would like to recognize with model.
  - On the other hand, if the outside photos represent only winter environments from a temperate climate, the distribution of the photo set will be significantly diversified

BITS Pilani WILP

Phases of Data(3) - Creation

Kinds of data

- Structured data
  - quantitative with a predefined data model/schema, highly organized, and stored in tabular formats like spreadsheets or relational databases
  - can be easily processed by relatively simple code using obvious heuristics.
- Unstructured data
  - qualitative without a standard data model/schema
  - cannot be processed and analyzed using conventional data methods and tools
- Semi-structured data
  - doesn't have a specific data model/schema but includes tags and semantic markers
  - a type of structured data that lies between structured and unstructured data
- The character of the internal structure of the data will have significant implications for the way we process, store, and use it.

Phases of Data(4) - Creation

Dataset augmentation

- Small amount of training data is available - not enough to train a high-quality model on
- then need to augment that data
  - Tools are available that can do so
- For example, Snorkel
  - is one good place to start, as it allows us to easily expand a small dataset into a large one
  - has good evidence that this approach can yield extremely good results at low cost, although it does need to be used with caution

Phases of Data(5) - Ingestion

Filter / Selection / Sampling

- The data needs to be received into the system and written to storage for further processing
  - not all data created may be ingested or collected, because we don't want or need to
  - a filtering and selection step necessarily occurs
- Sampling
  - may also simply sample at this stage if have so much data which can not be processed all of it
    - as ML training and other data processing is often extremely computationally expensive
  - an effective way to save money on intermediate processing and training costs
  - important to measure the quality cost of sampling and compare that to the savings
  - is likely to occasionally loose some detail for some events – unavoidable!
- In general, ML training systems perform better with more data. Any reduction in data may well impact quality at the same time as reducing costs.

Phases of Data(6) - Ingestion

Mechanism

- Depending on the volume of data and the complexity of our service, the ingestion phase
  - may be as simple as "dump some files in that directory over there"
  - or as sophisticated as a remote procedure call (RPC) endpoint that receives specifically formatted files and confirms a reference to the data bundle that was received so that its progress through the system can be tracked
- Reliability concerns during the ingestion phase typically focus on correctness and throughput
  - Correctness is the general property that the data is properly read and written in the correct place without being unnecessarily skipped or misplaced
  - Monitoring the existence and condition of data before and during ingestion is the most difficult part of the data pipeline.

BITS Pilani WILP

Phases of Data(7) - Processing

Validation

- Set of common operations to make the data ready for training
  - Validation
  - cleaning and ensuring data consistency
  - enriching and extending
- Validation
  - Efficient and powerful ML models can never do what we want them to do with bad data
  - In production, a common reason for errors in the data is bugs in the code that is collecting the data in the first place
  - Data ingested from external sources might have a lot of errors
  - extremely important to validate the incoming data especially when there is a well-defined schema and/or an ability to compare against the last known valid feed
  - performed against a common definition of the field
  - need to both store and be able to reference those standard definitions

Phases of Data(8) - Processing

Cleaning and ensuring data consistency

- Even with a decent validation framework in place, most data is still messy
  - may have missing fields, duplicates, misclassifications, or even encoding errors
  - more data we have, the more likely that cleaning and data consistency will be its own stage of processing
- ML pipeline will assuredly have code whose job is to clean the data
  - can either put this code in a single place, where it can be reviewed and improved,
  - or put aspects of it throughout the training pipeline
- Another set of data consistency tasks during this portion is the normalization of data
  - refers to a set of techniques used to transform the input data into a similar scale
- Common technique is that of putting the data into buckets
  - map a range of data into a much smaller set of groups that represent the same range

Phases of Data(9) - Processing

Enriching and extending

- Enriching
  - Combine data with data from other sources
- Labeling
  - most common and fundamental way to extend the data
  - identifies a given event or record by bringing in confirmation from an outside source of data (sometimes a human)
  - Labeled data is the key driver of all supervised ML and is often one of the most challenging and expensive parts of the whole ML process
- Joins
  - Might use many external data sources to extend training data

Phases of Data(10) - Post-processing

Storage

- Need to store the data somewhere
  - How and where we store the data is mostly driven by how we tend to use it
  - which is really a set of questions about training and serving systems
- Two predominant concerns here:
  - efficiency of storage
  - metadata
- The efficiency of a storage system is driven by access patterns
  - that are driven by the model structure, team structure, and training process
- Metadata helps humans interact with the storage
  - When multiple people work on building models on the same data (or when the same person works on this over time), metadata about the stored features provides huge value
  - Roadmap to understanding how the last model was put together and how we might want to put together the model
  - More commonly undervalued parts of an ML system.

BITS Pilani WILP

Phases of Data(11) - Post-processing

Management

- Data management system is primarily motivated by two factors
  - The business purpose to which we intend to put the data
    - What problem are we trying to solve?
    - What is the value of that problem to our organization or our customers?
  - The model structure and strategy
    - What models do we plan to build?
    - How are they put together?
    - How often are they refreshed?
    - How many of them are there?
    - How similar to one another are they?
- Every choice we make about our data management system is constrained by, and constrains, these two factors!
- If data management is about how and why we write data, ML training pipelines are about how and why we read it.

Phases of Data(12) - Post-processing

Analysis and Visualization

- Process of transforming large amounts of data into an easy-to-navigate representation using statistical and/or graphical techniques and tools
  - Essential task of ML architectures and knowledge discovery techniques to make data less confusing and more accessible
- Not enough just to show a pie chart or bar graph
  - Need to provide the human reader an explanation of
    - what each record in the dataset means,
    - how it is linked with the records in other datasets
    - and whether it is clean and safe to use for training models
- Practically impossible for data scientists to look at large datasets without well-defined and high-performant data visualization tools and processes

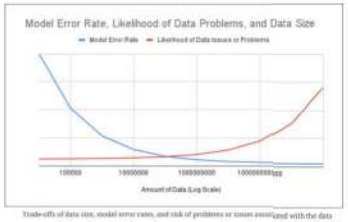
Data and ML Pipelines

Relation

- ML systems are data processing pipelines
  - with a purpose to extract usable and repeatable insights from data
  - fundamentally, they consume data, and output a processed representation of that data
- Key differences between ML pipelines and conventional log processing or analysis pipelines
  - have some very different and specific constraints and fail in different ways
  - success is hard to measure,
  - many failures are difficult to detect
  - depend thoroughly and completely on the structure, performance, accuracy, and reliability of their underlying data systems
- Overwhelmingly interested in two things:
  - the data used to construct the models
  - the processing pipeline that takes the data and transforms it into models

Data as Asset

- Unarguably data is an important asset in ML systems
  - certainly impossible to have an ML system without data
  - often true that a simple (or even simplistic) ML system with more (and higher-quality) training data can outperform a more sophisticated system with less, or less representative, data



- Organizations continue to scramble to collect as much data as possible, hoping to find ways to turn that data into value
  - many organizations have made this into a profoundly successful business model
  - Netflix, whose ability to recommend high-quality shows and movies to customers was an early differentiator

BITS Pilani WILP

Data as Liability

- Anything could be an asset, under the right (wrong) circumstances, it can also be a liability
  - For data, the acquisition, collection, and curation of data can expose areas of unexpected nuance and complexity in the data
  - can lead to potential harm for us and for our users
- The intent is to enumerate enough of the complexity to dissuade any readers from
  - simply thinking "more data == better" or
  - thinking "this stuff is easy

Data as Liability(2)

Questions with data collection

- Data must be collected in compliance with applicable laws, which might be based on
  - where organization is located
  - where the data originates
  - on organizational policies
- Significant restrictions on
  - what counts as data about people
  - how to get permission to store the data
  - how to store and retrieve the permission that was granted
  - whether we need to provide access to the data to the people who provided it and under what circumstances
- Restrictions may come from
  - laws
  - industry practices
  - insurance regulations
  - corporate governance policies
  - or any range of other sources
- For example, some jurisdictions include prohibition against
  - collecting personally identifiable information (PII) about an individual without their explicit written consent
  - along with the requirement to delete that data upon request by the data subject

Not technical question, but policy question!

Data as Liability(3)

Questions with data storage

- Data must be secured from external and internal access
  - Employees should not be able to view or change private user data without restriction and without detailed logging of that
- Pseudonymization
  - approach to reducing data access and reducing the auditing surface is to anonymize the data
  - private identifiers are replaced with others in a reversible fashion, and reversing the pseudonymization requires access to an additional data or system
  - protects the data from casual inspection by engineers working on the pipeline but permits discovery if we find that we need to reverse the anonymization
  - preserves the properties of data that are relevant to ML model
- A better approach is permanently removing any direct connection between private data about a person and the data that we use to train on
  - substantially reduce the risk of the data and increase the flexibility to handle it
  - harder than it seems!
  - Doing this in a way that's not trivially reversible but still valuable can be difficult

Data as Liability(4)

Questions with data deletion

- Might need to delete data
  - at the request of individual users, local laws, regulations, compliances
  - in cases where we no longer have permission to store the data
- Deleting data and having it actually be deleted is surprisingly hard!
  - been true since at least the early MS-DOS days
  - today's computing environment makes deletion even harder due to multiple copies of the data and metadata management
- Important to be certain that people want their data deleted without putting up arbitrary barriers
  - impose a short delay before really deleting the data
  - also underlying data structures, indices, and backups may be reconstructed

BITS Pilani WILP

The Data Sensitivity of ML Pipelines

- ML pipelines are unusually sensitive to their input data compared to most other data processing pipelines
  - All data processing pipelines are subject to the correctness and volume of their input data
  - ML pipelines are furthermore sensitive to subtle changes in distribution of the data
    - can easily go from mostly right to significantly wrong simply by omitting a small fraction of the data
- For example, consider a real-world system that somehow loses all of the data from a particular country, region, or language
  - drop all of the data from December 31 of a given year, lose the ability to detect New Year's Eve shopping trends
  - treads may be substantially different from the surrounding days in December and January
  - results into systematically biased results in significant confusion in models' understanding and predictions
- As a result of this sensitivity, the ability to aggregate, process, and monitor data is critical to successfully managing ML data pipelines.

Data Reliability

- Because our data processing system needs to work, the data must have several characteristics as it traverses the system
- Needs to know basics of making sure that
  - the data is not lost (durability)
  - is the same for all copies (consistency)
  - is tracked carefully as it changes over time (version control)
  - is read fast enough (performance)
  - is ready to be read (availability)

Data Management

What and Why?

- Data management is the practice of ingesting, processing, securing and storing an organization's data,
  - where it is then utilized for strategic decision-making to improve business outcomes
- Over the last decade, developments within hybrid cloud, artificial intelligence, the Internet of Things (IoT), and edge computing have led to the exponential growth of big data
  - creating even more complexity for enterprises to manage
- A data management discipline within an organization has become an increasing priority as this growth has created significant challenges,
  - such as data silos, security risks, and general bottlenecks to decision-making
- Teams address these challenges head on with a number of data management solutions, which are aimed to clean, unify, and secure data
  - Allows leaders to glean insights through dashboards and other data visualization tools, enabling informed business decisions
  - Empowers data science teams to investigate more complex questions, allowing them to leverage more advanced analytical capabilities

Data Management Framework

Components

Source : trellance

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Data Management Framework

Components : described

- Data Governance
  - provides the overarching support to data management through stewardship, policies, processes, standards, and adherence to leading practices
- Data Architecture
  - provides the infrastructure for the storage, integration, and use of data throughout the organization
- Metadata
  - allows to use data more efficiently by providing critical information about data attributes
- Data Quality
  - provides the structure necessary to have data that fulfills the needs of the business
- Data Lifecycle
  - follows the data throughout the company, providing integrity from the initial introduction into the company through the final deletion from the company
- Analytics
  - applies statistical and visualization techniques that lead to valuable insights that can help the company make better business decisions.
- Data Privacy
  - supports the needs of the business to share data internally and externally.

Data Management Components

- The scope of a data management discipline is quite broad!
- A strong data management strategy typically implements the following components to streamline their strategy and operations throughout an organization:
  - Data processing
  - Data storage
  - Data governance
  - Data security

Data Management Components(2)

Data processing

- In this stage,
  - Raw data is ingested from a range of data sources, such as web APIs, mobile apps, Internet of Things (IoT) devices, forms, surveys, and more
  - Then, usually processed or loaded, via data integration techniques, such as extract, transform, load (ETL) or extract, load, transform (ELT)
    - ETL has historically been the standard method to integrate and organize data across different datasets
    - ELT has been growing in popularity with the emergence of cloud data platforms and the increasing demand for real-time data
- Independently of the data integration technique used,
  - the data is usually filtered, merged, or aggregated during the data processing stage to meet the requirements for its intended purpose,
    - which can range from a business intelligence dashboard to a predictive machine learning algorithm

Data Management Components(3)

Data storage

- Data can be stored before or after data processing, the type of data and purpose of it will usually dictate the storage repository that is leveraged
- Data warehousing
  - requires a defined schema to meet specific data analytics requirements for data outputs,
    - such as dashboards, data visualizations, and other business intelligence tasks
    - which are usually directed and documented by business users in partnership with data engineers,
    - who will ultimately execute against the defined data model
  - underlying structure of a data warehouse is typically organized as a relational system (i.e. in a structured data format),
    - sourcing data from transactional databases
- Other storage systems, such as data lakes,
  - incorporate data from both relational and non-relational systems, becoming a sandbox for innovative data projects
  - benefit data scientists in particular, as they allow them to incorporate both structured and unstructured data into their data science projects

BITS Pilani WILP

Data Management Components(4)

Data governance

- A set of standards and business processes which ensure that data assets are leveraged effectively within an organization
  - includes processes around data quality, data access, usability, and data security
- Data governance councils
  - tend align on taxonomies to ensure that metadata is added consistently across various data sources.
  - help to define roles and responsibilities to ensure that data access is provided appropriately; this is particularly important to maintain data privacy.

Data Management Components(5)

Data security

- Data security sets guardrails in place to protect digital information from unauthorized access, corruption, or theft
  - more scrutiny is placed upon the security practices of modern businesses
  - to ensure that customer data is protected from cybercriminals or disaster recovery incidents
- While data loss can be devastating to any business, data breaches, in particular, can reap costly consequences from both a financial and brand standpoint
- Data security teams can better secure their data by leveraging encryption and data masking within their data security strategy.

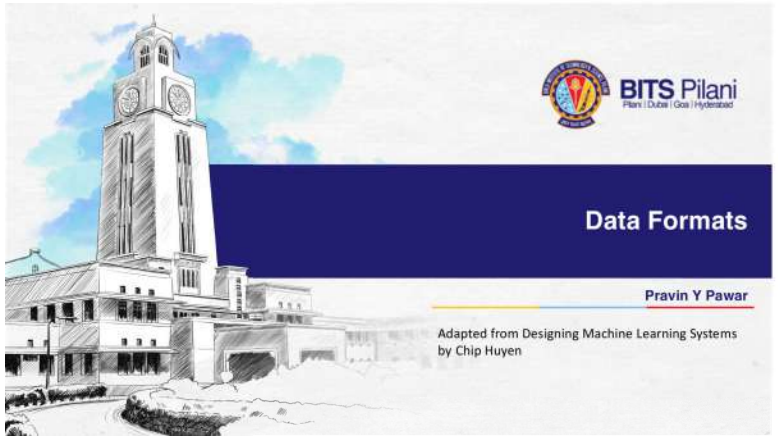
Data Platform

- Data management tools require a robust data and computing infrastructure
  - that scales to meet business demand without impacting performance
- Most companies now build a data lake to store raw data of any type from any system, often in Hadoop
  - but increasingly in a public cloud (e.g., Microsoft Azure)
- Companies also build
  - data warehouses to support performance reporting (e.g., reports, dashboards, and light analysis) on structured data
  - temporary sandboxes for testing, analysis, and prototyping purposes
- Data environments require
  - fast query processing
  - large storage capacity
  - elastic computing
  - In-memory caches
  - columnar storage
  - and large in-memory data grids and computing clusters

Thank You!

In our next session:

BITS Pilani WILP



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Data Formats

Pravin Y Pawar

Adapted from Designing Machine Learning Systems by Chip Huyen

Data Sources

- An ML system can work with data from many different sources
  - have different characteristics with different access patterns
  - can be used for different purposes
  - and require different processing methods
- Understanding the sources of data can help access and manipulate data more efficiently

Data Sources(2)

User Generated data

- One common source is user input data, data explicitly input by users, which is often the input on which ML models can make predictions
  - can be texts, images, videos, uploaded files, etc.
- If there are wrong way for humans to input data, humans are going to do it, and as a result, user input data can be easily mal-formatted
- If user input is supposed to be
  - texts, they might be too long or too short
  - numerical values, users might accidentally enter texts
  - fie uploading, they might upload files in the wrong formats
- Challenges
  - User input data requires more heavy-duty checking and processing
  - Users also have little patience. In most cases, when we input data, we expect to get results back immediately
  - Therefore, user input data tends to require fast processing.

Data Sources(3)

System-generated data

- Data generated by different components of systems,
  - which include various types of logs and system outputs such as model predictions
- Logs
  - Can record the state of the system and significant events in the system, such as memory usage, number of instances, services called, packages used, etc. Can record the results of different jobs, including large batch jobs for data processing and model training
  - Provides visibility into how the system is doing, and the main purpose of this visibility is for debugging and possibly improving the application
  - Much less likely to be mal-formatted as its system generated
  - Don't need to be processed as soon as they arrive
    - acceptable to process logs periodically, such as hourly or even daily
    - might still want to process logs fast to be able to detect and be notified whenever something interesting happens
- Because debugging ML systems is hard,
  - Common practice to log everything you can
  - Means volume of logs can grow very, very quickly
- Leads to two problems
  - The first is that it can be hard to know where to look because signals are lost in the noise
    - Many services that process and analyze logs, such as Logstash, DataDog, Logz, etc.
  - The second problem is how to store a rapidly growing amount of logs
    - In most cases, store logs for as long as they are useful, and can discard them when they are no longer relevant
    - Can also be stored in low-access storage that costs much less than higher-frequency-access storage

BITS Pilani WILP

Data Sources(4)

System-generated users data

- Users' behaveiors
  - such as clicking, choosing a suggestion, scrolling, zooming, ignoring a popup, or spending an unusual amount of time on certain pages
  - system-generated data, it's still considered part of user data
  - might be subject to privacy regulations
  - Can be used for ML systems to make predictions and to train their future versions
- Internal databases
  - generated by various services and enterprise applications in a company
  - manage their assets such as inventory, customer relationship, users, and more
  - can be used by ML models directly or by various components of an ML system
  - For example,

    when users enter a search query on Amazon, one or more ML models will process that query to detect the intention of that query — what products users are actually looking for? — then Amazon will need to check their internal databases for the availability of these products before ranking them and showing them to users.

Data Sources(5)

Third-party data - wonderfully weird world

- Types of data
  - First-party data is the data that company already collects about users or customers
  - Second-party data is the data collected by another company on their own customers that they make available to you
  - Third-party data companies collect data on the public who aren't their customers
- The rise of the Internet and smartphones has made it much easier for all types of data to be collected
  - Riddled with privacy concerns
  - Data from apps, websites, check-in services, etc. are collected and (hopefully) anonymized to generate activity history for each person
  - Third-party data is usually sold as structured data after being cleaned and processed by vendors.

Data Formats

Need

- Once have data, need to store data (or "persist" it, in technical terms)!
- Since data comes from multiple sources with different access patterns
  - storing your data isn't always straightforward and can be costly
  - important to think about how the data will be used in the future so that the format will make sense
- Questions needs to be considered
  - How do I store multimodal data?
  - When each sample might contain both images and texts?
  - Where to store your data so that it's cheap and still fast to access?
  - How to store complex models so that they can be loaded and run correctly on different hardware?
- Data serialization
  - The process of converting a data structure or object state into a format that can be stored or transmitted and reconstructed later
  - Many data serialization formats
- When considering a format to work with, need to consider different characteristics such as
  - human readability
  - access patterns
  - and whether it's based on text or binary, which influences the size of its files

Data Formats(2)

Common data formats

Format	Binary/Text	Human-readable	Example use cases
JSON	Text	Yes	Everywhere
CSV	Text	Yes	Everywhere
Parquet	Binary	No	Hadoop, Amazon Redshift
Avro	Binary primary	No	Hadoop
Protobuf	Binary primary	No	Google, TensorFlow (TFRecord)
Pickle	Binary	No	Python, PyTorch serialization

Common data formats and where they are used.

BITS Pilani WILP

Data Formats(3)

JSON

- JSON, JavaScript Object Notation, is everywhere
  - Though it was derived from JavaScript, it's language-independent — most modern programming languages can generate and parse JSON
  - Human-readable
  - Key-value pair paradigm is simple but powerful, capable of handling data of different levels of structuredness
- For example, persons data can be stored in a structured format like the following

```
{
  "firstName": "Bootie",
  "lastName": "McBootface",
  "isViking": true,
  "age": 12,
  "address": {
    "streetAddress": "12 Ocean Drive",
    "city": "Port Royal",
    "postalCode": "10021-3100"
  }
}
```

- The same data can also be stored in an unstructured blob of text like the following

```
{
  "text": "Bootie McBootface, aged 12, is viking, at 12 Ocean Drive, Port Royal,
10021-3100"
}
```

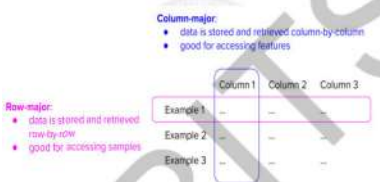
Data Formats(4)

Row-major vs. Column-major Format

- Two common formats that represents distinct paradigms are CSV and Parquet
  - CSV is row-major, which means consecutive elements in a row are stored next to each other in memory
  - Parquet is column-major, which means consecutive elements in a column are stored next to each other
- Modern computers process sequential data more efficiently than non-sequential data
  - If a table is row-major, accessing its rows will be faster than accessing its columns in expectation
  - For row-major formats, accessing data by rows is expected to be faster than accessing data by columns
- Column-major formats allow flexible column-based reads, especially if data is large with thousands, if not millions, of features
  - Row-major formats allow faster data writes
- Overall,
  - row-major formats are better when you have to do a lot of writes
  - column-major ones are better when you have to do a lot of column-based reads.

Data Formats(5)

Row-major vs. Column-major Format



Data Formats(6)

Text vs. Binary Format

- CSV and JSON are text files whereas Parquet files are binary files
  - Text files are files that are in plain text, which usually mean they are human-readable
  - Binary files are files that contain 0's and 1's, and meant to be read or used by programs that know how to interpret the raw bytes
  - A program has to know exactly how the data inside the binary file is laid out to make use of the file
- Binary files are more compact.
  - AWS recommends using the Parquet format because

"the Parquet format is up to 2x faster to unload and consumes up to 6x less storage in Amazon S3, compared to text formats."

BITS Pilani WILP

Structured vs Unstructured data

Structured data

- Follows a predefined data mode – aka schema
  - For example, Person model has two data values, age of type number and name as string
  - Predefined structure makes data analysis easier
- Disadvantage – commitment to predefined schema!
  - If schema changes, retrospectively update all data – causes mysterious bugs in the process
  - As requirements changes over time,
    - Committing to a predefined schema can be restricting
    - If data coming from multiple sources, don't have any control over schema
  - Unstructured data becomes appealing!

Structured vs Unstructured data(2)

Unstructured data

- Does not adhere to predefined schema!
- Usually a text but can also be numbers, dates, images, audio
- Might still contain intrinsic patterns that helps to extract meaningful structures out of data
- Allows for more flexible storage options
  - If storage follows schema, can only store data following that schema
  - If storage does not follows schema, can store any type of data!
  - Can convert all data, regardless of types and formats into byte strings and store them together
- Data Warehouse vs Data Lake
  - Data warehouse is repository to store structured data
  - Data Lake is repository to store unstructured data
  - Data lakes are usually used to store raw data before processing
  - Data warehouses are used to store data that has been processed into formats ready to be used

Structured vs Unstructured data(3)

Key Differences

Structured data	Unstructured data
Schema already defined	Does not have to follow a schema
Easy to search and analyze	Fast arrival
Can only handle data with a specific schema	Can handle data from any source
Schema changes will cause a lot of troubles	No need to worry about schema changes, as its shifted to the downstream applications that uses this data
Stored in data warehouses	Stored in data lakes

Data Models

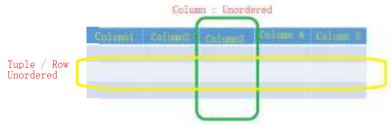
- Most important part of developing software, because they have such a profound effect
  - on how we think about solving the problem
  - on how the software is written
- Describes how data is represented in terms of attributes
  - Car can be represented by its make, model, year, color etc., as well by its owner, license plate etc.
  - Most applications are built by layering one data model on top of another
- Many different kinds of data models
  - Every data model embodies assumptions about how it is going to be used
  - Building software is hard enough, even when working with just one data model, and without worrying about its inner workings
- Selecting a data representation affects
  - the ways systems are built
  - The problems systems can solve

BITS Pilani WILP

Relational Model

Most persistent ideas in computer science

- Invented by Edgar Codd in 1970, but still going strong today, even getting more popular!
- Simple and Powerful
  - Data is organized into relations; Each relation is a set of tuples
  - Table is visual representation of relation, each row of table makes up a tuple
  - Relations are unordered – can shuffle order of rows or columns
  - Usually stored in CSV or Parquet format



Relational Model(2)

Normalization

- Desirable for relations to be normalized
  - Can follow forms such as first normal form (1NF), second normal form (2NF) etc.
  - Can reduce data redundancy and improve data integrity
- For example, consider Books relation
  - Lot of duplicate data – publisher , country
  - If something change, needs to be reflected everywhere
  - If normalized, updates needs to be managed in single place

Title	Author	Format	Publisher	County	Price
Book1	Pawar	Paperback	Press1	UK	20
Book1	Pawar	E-book	Press1	UK	10
Book2	Pravin	Paperback	NotedDraft	US	30
Book3	PYP	Paperback	Press1	UK	30
Book2	Pravin	Paperback	NotedDraft	US	15

Title	Author	Format	PubID	Price
Book1	Pawar	Paperback	1	20
Book1	Pawar	E-book	1	10
Book2	Pravin	Paperback	2	30
Book3	PYP	Paperback	1	30
Book2	Pravin	Paperback	2	15

PubID	Pulisher	Country
1	Press1	UK
2	NotedDraft	US

Major downside : need to join data back from multiple tables – expensive for large tables!

Relational Model(3)

Querying

- Databases built around relational data model are relational databases
- Language used to specify data to be retrieved from database is query language
  - Most popular is SQL
- SQL
  - Declarative query language – specify the output needed rather than steps needed to get it
  - Specify the pattern of data –
    - Tables from which data is needed
    - Conditions the results must meet
    - Transformations, such as joins, sort, group, aggregate
  - Database figures out how to provide those results
    - How to break query in parts
    - What methods to user execute each part of query
    - Order in which the parts needs to be executed

NoSQL

Need and Origin

- Relational model generalizes a lot but still restrictive
  - Needs a strict schema – schema management is painful!
  - Per Couchbase 2014 survey, frustration with schema management is #1 reason for adoption of non relational databases
- Latest movement against relational data model is NoSQL
  - Started as hashtag for meetup to discuss non-relational databases
  - Retroactively reinterpreted as Not Only SQL – many NoSQL data systems supports relational models
- Two major types – Document and Graph model
  - Document model targets use cases where data comes in self-contained documents
  - Graph model targeting use cases where relationships between data items are common and important

BITS Pilani WILP

NoSQL(2)

Document Model

- Built around concept of "document"
  - Single continuous string, encoded as JSON, XML or binary format like BSON
  - Each document has a unique key that represents that document, which can be used to retrieve it
  - Collection of document analogous to table in relational database, document analogous to row
    - Can convert relation into a collection of documents
    - Collection of document much more flexible than table
    - All rows in table must follow same schema while documents in same collection can have different schemas
- For example, a book can be represented as document as follows

```
{
    "Title": "Book1",
    "Author" : "Pravin",
    "Publisher": " Press1",
    "Country": "US",
    "Sold as": [
        { "format": "Paperback", "Price": "20"},
        { "format": "E-books", "Price": "10"},
    ]
}
```

NoSQL(3)

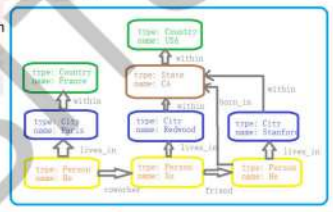
Document Model – Schema less – does not enforce schema

- Misleading!
  - Application that reads documents assumes some kind of structure of documents
  - Document databases just shifts responsibility of assuming structures from application that writes data to the application that reads that data!
- Has better locality than relational model
  - In relational model, data is spread across different tables and needs to be joined when required
  - In document model, all information is stored in single, self-contained place making it easier to read
  - Difficult to execute joins across documents compared to across tables
- Because of different strengths of document and relational data models,
  - Common to use both models for different tasks in same database systems
  - PostgreSQL and MySQL support them both!

NoSQL(4)

Graph Model

- Built around concept of "Graph"
  - Graph consists of nodes and edges, where edges represents relationship between the nodes
  - Database that uses graph structures to store its data is graph database
  - In document database, content of each document is priority
  - In graph database, relationships between data items are priority!
  - Faster to retrieve data faster as relationships are models in that way
- For example, consider the following representation
  - that may come from social media
  - Find person born in USA
  - Which one is simpler – graph or relational model?



Query Languages for Data

- Common ways of querying data
  - Declarative queries
  - Imperative code
- When the relational model was introduced, it included a new way of querying data:
  - SQL is a declarative query language
- whereas IBM IMS and CODASYL queried the database using imperative code

BITS Pilani WILP

Imperative code vs Declarative queries

- Many commonly-used programming languages are imperative.
- For example, following code retrieves the sharks from the list of animals

```
function getSharks() {
    var sharks = [];
    for (var i = 0; i < animals.length; i++) {
        if (animals[i].family === "Sharks") {
            sharks.push(animals[i]);
        }
    }
    return sharks;
}
```

- In the relational algebra, you would instead write: $sharks = \sigma_{family = \text{"Sharks"}}(animals)$

  where σ is the selection operator, returning only those animals that match the condition family = "Sharks"

- When SQL was defined, it followed the structure of the relational algebra fairly closely:

```
SELECT * FROM animals WHERE family = 'Sharks';
```

Imperative code vs Declarative queries(2)

- In Imperative language
  - instructs the computer to perform certain operations in a certain order
    - stepping through the code, line by line,
    - evaluating conditions
    - updating variables
    - and deciding whether to go around the loop one more time
- In a declarative query language, like SQL or relational algebra,
  - specify the pattern of the data is needed
  - what conditions the results must meet,
  - how you want it to be transformed e.g. sorted, grouped and aggregated
  - but not how to achieve that goal
    - Up to the database system's query optimizer to decide
      - which indexes and which join methods to use
      - in which order to execute various parts of the query

Imperative code vs Declarative queries(4)

- Today, CPUs are getting faster by adding more cores
  - not by running at significantly higher clock speeds than before
- Imperative code is very hard to parallelize across multiple cores and multiple machines
  - because it specifies instructions that must be performed in a particular order
- Declarative languages often lend themselves to parallel execution
  - specify only the pattern of the results, but not the algorithm that is used to determine the results
  - database is free to use a parallel implementation of the query language, if appropriate

MapReduce querying

- Programming model for processing large amounts of data in bulk across many machines,
  - popularized by Google
- A limited form of MapReduce is supported by some NoSQL data stores MongoDB and CouchDB
  - as a mechanism for performing read-only queries across many documents
- MapReduce is neither a declarative query language, nor a fully imperative query API, but somewhere in between!
  - the logic of the query is expressed with snippets of code, which are called repeatedly by the processing framework
  - based on the map (also known as collect) and reduce (also known as fold or inject) functions that exist in many functional programming languages

BITS Pilani

MapReduce querying(2)

- For example, imagine you are a marine biologist, and you add an observation record to your database every time you see animals in the ocean
  - Now you want to generate a report saying how many sharks have been sighted per month
- In PostgreSQL you might express that query like this:

```
SELECT date_trunc('month', observation_timestamp) AS observation_month,
       sum(num_animals) AS total_animals
FROM observations
WHERE family = 'Sharks'
GROUP BY observation_month;
```

- This query
  - first filters the observations to only show species in the Sharks family
  - then groups the observations by the calendar month in which they occurred
  - and finally adds up the number of animals seen in all observations in that month

MapReduce querying(3)

- The same can be expressed with MongoDB's MapReduce feature as follows

```
db.observations.mapReduce(
    function map() { ❷
        var year  = this.observationTimestamp.getFullYear();
        var month = this.observationTimestamp.getMonth() + 1;
        emit(year + "-" + month, this.numAnimals); ❸
    },
    function reduce(key, values) { ❹
        return Array.sum(values); ❺
    },
    {
        query: { family: "Sharks" }, ❶
        out: "monthlySharkReport" ❻
    }
);
```

❷ The JavaScript function `map` is called once for every document that matches `query`, with `this` set to the document object.

❸ The `map` function emits a key (a string consisting of year and month, such as `"2013-12"` or `"2014-1"`) and a value (the number of animals in that observation).

❹ The key-value pairs emitted by `map` are grouped by key. For all key-value pairs with the same key (i.e. the same month and year), the `reduce` function is called once.

❺ The `reduce` function adds up the number of animals from all observations in a particular month.

❻ The final output is written to the collection `monthlySharkReport`.

MapReduce querying(4)

- For example, say the observations collection contains these two documents:

```
{
    observationTimestamp: Date.parse("Mon, 25 Dec 1995 12:34:56 GMT"),
    family:    "Sharks",
    species:   "Carcharodon carcharias",
    numAnimals: 3
}
{
    observationTimestamp: Date.parse("Tue, 12 Dec 1995 16:17:18 GMT"),
    family:    "Sharks",
    species:   "Carcharias taurus",
    numAnimals: 4
}
```

- The map function would called once for each document, resulting in
  - emit("1995-12", 3)
  - emit("1995-12", 4)
- Subsequently, the reduce function would be called with
  - reduce("1995-12", [3, 4]), returning 7.

MapReduce querying(5)

- The map and reduce function are somewhat restricted in what they are allowed to do
- Must be pure functions
  - only use the data that is passed to them as input
  - cannot perform additional database queries
  - must not have any side-effects
- Restrictions allow the database to run the functions anywhere, in any order, and re-run them on failure
- However, they are nevertheless powerful
  - parse strings
  - call library functions
  - perform calculations and more

BITS Pilani WILP

MapReduce querying(6)

- MapReduce is a fairly low-level programming model for distributed execution on a cluster of machines
  - A usability problem with MapReduce is need to write two carefully coordinated functions
  - often harder than writing a single query
- Higher-level query languages like SQL can be implemented as a pipeline of MapReduce operations
  - A declarative query language offers more opportunities for a query optimizer to improve the performance of a query
  - MongoDB added support for a declarative query language called aggregation pipeline
- The aggregation pipeline language's expressiveness
  - is similar to a subset of SQL,
  - but it uses a JSON-based syntax rather than SQL's English-sentence-styl
- The moral of the story is that a NoSQL system may find itself accidentally reinventing SQL, albeit in disguise

```
db.observations.aggregate([
    { $match: { family: "Sharks" } },
    { $group: {
        _id: {
            year:  { $year:  "$observationTimestamp" },
            month: { $month: "$observationTimestamp" }
        },
        totalAnimals: { $sum: "$numAnimals" }
    } }
]);
```

Formats for Encoding Data

(at least) two different data representations

- In memory,
  - data is kept in objects, structs, lists, arrays, hash tables, trees etc.
  - data structures are optimized for efficient access and manipulation by the CPU
- In disks, Over networks
  - to write data to a file, or send it over the network, need to encode it
  - as some kind of self-contained sequence of bytes (for example, a JSON document)
  - sequence-of-bytes representation looks quite different from the data structures that are normally used in memory
- Need some kind of translation between the two representations
  - The translation from the in-memory representation to a byte sequence is called encoding
    - aka serialization or marshalling
  - the reverse is called decoding
    - aka parsing, deserialization, unmarshalling

Language-specific formats

Encoding Libraries

- Programming languages come with built-in support for encoding in-memory objects into byte sequences
  - Java has java.io.Serializable
  - Ruby has Marshal
  - Python has pickle
- Encoding libraries are very convenient!
  - allow in-memory objects to be saved and restored with minimal additional code

Language-specific formats(2)

Deep Problems

- Integration
  - If tied to a particular programming language, then reading the data in another language is very difficult
  - Store or transmit data in such an encoding, preclude integrating systems with those of other organizations (which may use different languages)
- Decoding
  - To restore data in the same object types, the decoding process needs to instantiate arbitrary classes
  - Frequently a source of security problems
  - if an attacker can get your application to decode an arbitrary byte sequence, they can instantiate arbitrary classes, which in turn often allows them to do terrible things such as remotely executing arbitrary code
- Versioning
  - Is often an afterthought in these libraries
  - Intended for quick and easy encoding of data, they often neglect the inconvenient problems of forward and backward compatibility
- Efficiency
  - CPU time taken to encode or decode, and the size of the encoded structure is also often an afterthought
  - Java's built-in serialization is notorious for its bad performance and bloated encoding
- A bad idea to use language's built-in encoding for anything other than very transient purposes!

BITS Pilani WILP

Standardized encodings

Textual format

- Standardized encodings can be written and read by many programming languages
  - XML
  - JSON
  - CSV
- Textual formats, and thus somewhat human-readable!
- XML is often criticized for being too verbose and unnecessarily complicated
- JSON's popularity is mainly due to its built-in support by web browsers and simplicity relative to XML
- CSV is albeit less powerful
- Widely known, widely supported, and almost as widely disliked!

Standardized encodings(2)

Subtle problems

- Encoding of numbers
  - Lot of ambiguity around the encoding of numbers
  - In XML and CSV, cannot distinguish between a number and a string that happens to consist of digits
  - JSON distinguishes strings and numbers, but it doesn't distinguish integers and floating-point, and doesn't specify a precision
  - When dealing with large numbers, integers greater than 253 cannot be exactly represented, so such numbers become inaccurate when parsed
  - Workaround is to have two numbers: once as a JSON number and once as a decimal string
- Binary strings
  - JSON and XML have good support for Unicode character strings, i.e. human readable text
  - but they don't support binary strings (sequences of bytes without a character encoding)
  - get around this limitation by encoding the binary data as text using Base64

Standardized encodings(2)

Subtle problems(2)

- Optional schema support
  - Schema support for both XML and JSON is optional
    - Schema languages are quite powerful, and thus quite complicated to learn and implement
    - Use of XML schemas is fairly widespread, but many JSON-based tools don't bother using schemas
    - Applications that don't use XML/JSON schemas need to potentially include additional code to encode/decode data correctly
  - CSV does not have any schema, so it is up to the application to define the meaning of each row and column
    - If an application change adds a new row or column, need to handle that change manually
- Common (data interchange) formats
  - JSON, XML and CSV are good enough for many purposes
  - Likely that they will remain popular, especially as data interchange formats (i.e. sending data from one organization to another)
  - As long as people agree on what the format is, it often doesn't matter how pretty or efficient the format is
  - The difficulty of getting different organizations to agree on anything outweighs most other concerns!

Modes of Data Flow

- Many ways how data can flow from one process to another
  - Who encodes the data, and who decodes it?
- Most common ways how data flows between processes:
  - Via Databases, where the process writing to the database encodes the data, and the process reading from the database decodes it.
  - Via RPC and REST APIs, where the client encodes a request, the server decodes the request and encodes a response, and the client finally decodes the response.
  - Via Asynchronous message-passing (using message brokers or actors), where nodes communicate by sending each other messages that are encoded by the sender and decoded by the recipient

BITS Pilani WILP

Data flow through databases

Data Compatibility

- In a database, the process that writes to the database encodes the data, and the process that reads from the database decodes it
- May just be a single process accessing the database, in which case the reader is simply a later version of the same process
  - Backward compatibility is clearly necessary here, otherwise won't be able to decode what was previously wrote
- Common for several different processes to be accessing a database at the same time
  - For example, several different applications or services, or several instances of the same service
  - Either way, in an environment where the application is changing,
    - Likely that some processes accessing the database will be running newer code and some will be running older code
    - A new version is currently being deployed in a rolling upgrade, so some instances have been updated
    - Others haven't yet!
  - Means that a value in the database may be written by a newer version of the code, and subsequently read by an older version of the code that is still running
  - Forward compatibility is also often required for databases

Data flow through databases(3)

Different values written at different times

- A database generally allows any value to be updated at any time
  - Means within a single database may have some values that were written five milliseconds ago, and some values that were written five years ago
- Data outlives code
  - When deploy a new version of your application, may entirely replace the old version with the new version within a few minutes
  - Same is not true of database contents: the five-year-old data will still be there
- Schema Evolution
  - Rewriting (migrating) data to a new schema is certainly possible, but it's an expensive thing to do on a large dataset, so mostly avoided
  - Simple schema changes, such as adding a new column with a null default value, without rewriting existing data
  - Schema evolution thus allows the entire database to appear as if it was encoded with a single schema, even though the underlying storage may contain records encoded with various historical versions of the schema

Data flow through databases(4)

Archival storage

- Snapshot of database can be taken from time to time, say for backup purposes or for loading into a data warehouse
  - Data dump will typically be encoded using the latest schema, even if the original encoding in the source database contained a mixture of schema versions from different eras
  - Copying the data anyway, might as well encode the copy of the data consistently
- As the data dump is written in one go, and is thereafter immutable, formats like Avro object container files are a good fit
  - Good opportunity to encode the data in an analytics-friendly column-oriented format such as Parquet

Data flow through services

Client and Server

- Processes that need to communicate over a network, there are a few different ways of arranging that communication
  - The most common arrangement is to have two roles: clients and servers
  - The servers expose an API over the network, and the clients can connect to the servers to make requests to that API
  - The API exposed by the server is known as a service
- Web Clients
  - Clients (web browsers) make requests to web servers, making GET requests to download HTML, CSS, JavaScript, images etc. or making POST requests to submit data to the server
  - The API consists of a standardized set of protocols and data formats (HTTP, URLs, SSL/TLS, HTML, etc.)
  - Because web browsers, web servers and website authors mostly agree on these standards, can use any web browser to access any website
- Other Clients
  - A native app running on a mobile device or a desktop computer can also make network requests to a server
  - A client-side JavaScript application running inside a web browser can use XMLHttpRequest to become a HTTP client (aka Ajax)
  - The server's response is typically not HTML for displaying to a human, but rather data in an encoding that is convenient for further processing by the clientside
  - Application code (such as JSON) API is implemented on top of HTTP is application-specific,
  - Client and server need to agree on the details of that API

BITS Pilani WILP

Data flow through services(2)

Service-oriented architecture (SOA) / Microservices Architecture

- Service-oriented architecture (SOA)
  - Server can itself be a client to another service
  - Often used to decompose a large application into smaller services by area of functionality,
    - such that one service makes a request to another when it requires some functionality or data from that other service
  - more recently refined and rebranded as microservices architecture!
- A key design goal is to make the application easier to change and maintain by making services independently deployable and evolvable
  - For example, each service should be owned by one team, and that team should be able to release new versions of the service frequently without having to coordinate with other teams
- Should expect old and new versions of servers and clients to be running at the same time
  - Data encoding used by servers and clients must be compatible across versions of the service API

Data flow through services(3)

Web services

- HTTP is used as the underlying protocol for talking to the service
  - Two popular approaches: REST and SOAP
- REST is not a protocol, but rather a design philosophy that builds upon the principles of HTTP
  - emphasizes simple data formats, using URLs for identifying resources, and using HTTP features for cache control, authentication, and content type negotiation
  - gaining popularity compared to SOAP, at least in the context of cross-organizational service integration and is often associated with microservices
  - An API designed according to the principles of REST is called RESTful
- SOAP is an XML-based protocol for making network API requests
  - Most commonly used over HTTP, it aims to be independent from HTTP and avoids using most HTTP features
  - Instead comes with a sprawling and complex multitude of related standards (the web service framework, known as WS-*)
  - The API of a SOAP web service is described using an XML-based language called WSDL
  - As WSDL is not designed to be human-readable, and as SOAP messages are often too complex to construct manually, users of SOAP rely heavily on tool support, code generation and IDEs
  - Although SOAP is still used in many large enterprises, it has fallen out of favor in most smaller companies.

Data flow through services(4)

Remote procedure calls (RPC)

- Web services are merely the latest incarnation of a long lineage of technologies for making API requests over a network, many of which received a lot of hype but have serious problems
  - Enterprise JavaBeans (EJB) and Java Remote Method Invocation (RMI) are limited to Java
  - DCOM is limited to Microsoft platforms
  - CORBA is excessively complex, and does not provide backward or forward compatibility
- All of these are based on the idea of a Remote Procedure Call (RPC), which has been around since the 1970s
  - RPC tries to make a request to a remote network service look the same as calling a function or method in programming language, within the same process (aka location transparency)
  - Although this seems convenient at first, the approach is fundamentally flawed
  - No point trying to make a remote service look too much like a local object in programming language, because it's a fundamentally different thing

Data flow through services(5)

Current directions for RPC

- RPC isn't going away!
  - Various RPC frameworks have been built on top of all the encodings:
  - Thrift and Avro come with RPC support included
  - gRPC is a RPC implementation using Protocol Buffers
- New generation of RPC frameworks is more explicit about the fact that a remote request is different from a local function call
  - Also provide service discovery — that is, allowing a client to find out at which IP address and port number it can find a particular service
  - Custom RPC protocols with a binary encoding format can achieve better performance than something generic like JSON over REST
- RESTful API has other significant advantages
  - good for experimentation and debugging (can simply make requests to it using a web browser or the command-line tool curl)
  - supported by all main stream programming languages and platforms
  - a vast ecosystem of tools(servers, caches, load balancers, proxies, firewalls, monitoring, debugging tools, testing tools, etc.)
- For these reasons, REST seems to be the predominant style for public APIs.
  - Focus of RPC frameworks is on requests between services owned by the same organization, typically within the same datacenter

BITS Pilani WILP

Message passing data flow

Asynchronous message-passing systems

- Common ways of data exchange
  - REST and RPC (where one process sends a request over the network to another process, and expects a response as quickly as possible),
  - Databases (where one process writes encoded data, and another process reads it again sometime in the future)
- Asynchronous message-passing systems, which are somewhere between RPC and databases
  - similar to RPC in that a client's request (usually called a message) is delivered to another process with low latency
  - similar to databases in that the message is not sent via a direct network connection, but goes via an intermediary called a message broker which stores the message temporarily
  - also called a message queue or message-oriented middleware

Message passing data flow(2)

Message broker's Advantages compared to direct RPC

- can act as a buffer if the recipient is unavailable or overloaded, and thus improve system reliability
- can automatically redeliver messages to a process that crashed, and thus prevent messages from being lost
- avoids the sender needing to know the IP address and port number of the recipient
- allows one message to be sent to several recipients
- logically decouples the sender from the recipient (the sender just publishes messages and doesn't care who consumes them)
- Message-passing communication is usually one-way
  - sender normally doesn't expect to receive a reply to its messages
  - possible for a process to send a response, but this would usually be done on a separate channel
- This is what makes it asynchronous: the sender doesn't wait for the message to be delivered, but simply sends it and then forgets about it.

Message passing data flow(3)

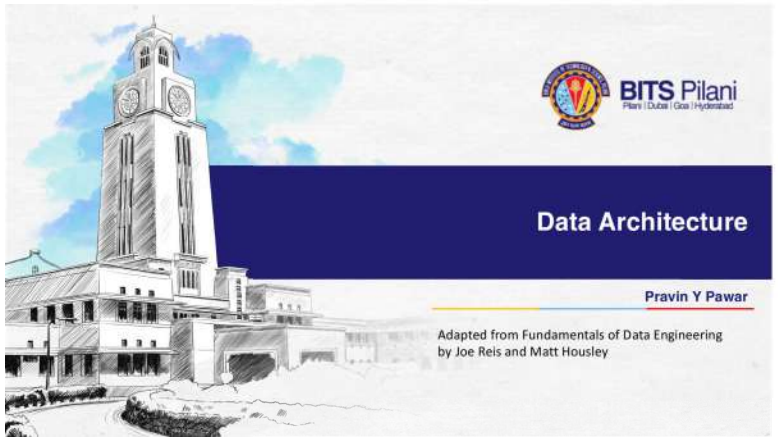
Message brokers

- Landscape
  - In past, dominated by commercial enterprise software by companies such as TIBCO, IBM WebSphere, and webMethods.
  - More recently, open source implementations such as RabbitMQ, ActiveMQ, HornetQ, NATS and Apache Kafka have become popular
- In general, message brokers are used as follows:
  - one process sends a message to a named queue or topic
  - broker ensures that the message is delivered to one or more consumers or subscribers of that queue or topic
  - can be many producers and many consumers on the same topic
- A topic provides only one-way data flow!
- Message brokers typically don't enforce any particular data model
  - message is just a sequence of bytes, with some metadata, so can use any encoding format
  - If the encoding is backward and forward compatible, have the greatest flexibility to change publishers and consumers independently, and deploy them in any order

Thank You!

In our next session:

BITS Pilani WILP



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Data Architecture

Pravin Y Pawar

Adapted from Fundamentals of Data Engineering by Joe Reis and Matt Housley

Architecture

- "Never shoot for the best architecture but rather the least worst architecture"
  - Mark Richards and Neal Ford, Fundamentals of Software Architecture
- "Architecture represents the significant design decisions that shape a system, where significant is measure by cost of change". – Grady Booch

Data Architecture

Need

- Reflects current and future state of data systems
  - That supports organization's long-term data needs and strategy
- Data requirements changes rapidly
  - New tools and practices seem to arrive on a near-daily basis
  - Gives rise to the need of understanding good data architecture
- Needs to understand needs of business and gather requirements for new use cases
- Needs to translates requirements to design new ways to capture and serve data
  - Knowing the trade-offs with design patterns, technologies and tools in source systems, ingestion, storage, transformation and serving data
  - Successful data engineering is build upon rock-solid data architecture!

Data Architecture(2)

Defined

- Per TOGAF
  - A description of the structure and interaction of the enterprise's major types and sources of data, logical data assets, physical data assets and data management resources.
- Per DAMA DMBOK (Data Management Book of Knowledge)
  - Identifying the data needs of the enterprise and designing and maintaining the master blueprint to meet those needs. Using master blueprints to guide data integration, control data assets and align data investments with business strategy.
- Definition
  - Data architecture is design of systems to *support evolving data needs of an enterprise*, achieved *by flexible and reversible decisions* reached through *a careful evaluation of trade-offs*.

BITS Pilani WILP

Data Architecture(3)

Other Aspects

- Operational and Technical
- Operational Architecture
  - Encompasses the functional requirements of what needs to happen related to people, processes and technology
    - What business processes does the data serve?
    - How does organization manages data quality?
  - Describes what needs to be done
- Technical Architecture
  - Outlines how data is ingested, stored, transformed and served along the data engineering life cycle
    - How will you move 10 TB of data every hour from source database to data lake?
  - Describes how it will happen

Data Warehouse

- Is a central data hub used for reporting and analysis
  - Data is typically highly formatted and structured for analytics use cases
  - Among oldest and most well-established data architectures
- Per Bill Inmon, father of data warehouse, it is
  - A subject oriented, integrated, non-volatile and time-variant collection of data in support of management's decisions
- In past, widely used at enterprises with significant budgets – expensive and labour-intensive
- Since then, scalable, pay-as-you-go model has made it accessible even to tiny companies

Data Warehouse(2)

Organizational data warehouse architecture

- Organizes data associated with business team structures and processes
  - Separates online analytical processing (OLAP) from production databases (OLTP)
  - Centralizes and organizes data
- ETL (Extract, Transfer, Load)
  - DW pulls data from application systems using ETL
  - Extract phase pull data from source systems
  - Transformation phase cleaned and standardizes data, applies business logic
  - Load phase pushes data into DW target database system
  - Data is then loaded into multiple data marts that serve analytical needs for specific lines of business

Data Warehouse(3)

ELT DW Architecture

- ELT – Extract, Load and Transfer
- Data gets moved more or less directly from production systems into a staging area in the data warehouse
  - Staging stage indicates that data is in raw form
  - Transformations are directly handled by data warehouse system
  - Intention is to take advantage of massive computational power of cloud data processing tools
  - Data is processed in batches and transformed output is written into tables and views for analytics
- Transformation-on-Read ELT
  - Popularized during big data growth in Hadoop ecosystem

BITS Pilani WILP

Data Warehouse(4)

Cloud data warehouse

- Represents evolution of on-premises data warehouse architecture and have led to significant changes to organizational architecture
  - Amazon Redshift kicked off cloud data warehouse revolution
  - Google BigQuery, Snowflake popularized the approach
    - Can spin up a Redshift cluster on demand, scaling it up over time as data and analytics grows
    - Can spin up cluster to serve specific workloads and quickly delete clusters when not needed
- Separate compute from storage
  - Data is stored in object storage allowing virtually limitless storage
  - Gives option to spin up computing power on demand providing ad hoc big data capabilities without long-term cost of thousands of nodes
- As cloud data warehouse matures, line between data warehouse and date lake will continue to blur!

Data Warehouse(5)

Data Marts

- More refined subset of a warehouse designed to serve analytics and reporting, focused on a single sub organization, department or line of business
  - Every department has its own data mart, specific to its needs
  - Contrast to full data warehouse that serves the broader organization or business
- Exists for two reasons
  - Makes data more easily accessible to analysts and report developers
  - Provide additional stage of transformation beyond that provided by initial ETL/ELT pipelines
- Significantly improves performance if reports or analytics requires complex joins and aggregations of data, especially when raw data is large
  - Transform processes can populate data mart with joined and aggregated data to improve performance of live queries

Data Lake

- Most popular architectures that appeared during big data era
  - Instead of imposing tight structural limitations on data, simply dump all data – structured and unstructured – into central location
- Data lake 1.0
  - Started with HDFS , moved to cloud based object storage – extremely limitless and cheap storage costs
  - Allows immerse amount of data of any size and type to be stored
  - When needs to be queried or transformed, have access to nearly unlimited computing power by spinning cluster on demand with favourite tolls – MapReduce, Spark, Presto, Hive
- Had serious shortcomings – dumping ground – data swamps, dark data and WORN
  - Data grew to unmanageable sizes, with little in way of schema management, data cataloguing and discovery tools
  - Write-only – updates and deletes become headaches
  - Processing also challenge – joins were nightmares!
- Many organizations found significant value in data lakes – Netflix and Facebook
  - Had resources to build successful data practices and create their Hadoop based tools

Data lakehouse

- Various players sought to enhance limitations of first generations data lakes
  - Databricks introduced data lakehouse – convergence between data warehouse and data lakes
  - Incorporates the controls, data management and data structures found in data warehouse while still housing data in object storage and supporting variety of query and transformation engines
  - Supports Atomicity, consistency, durability and isolation (ACID)
- Technical architecture of cloud data warehouses have evolved to be very similar to data lake architecture
  - Separates compute from storage
  - Support petabyte-scale queries
  - Store a variety of unstructured and semi-structured objects
  - Integrate with advanced processing technologies such as Spark and Beam
- Convergence will continue but still will exist as different architectures
  - Vendors offering data platforms that combine data lake and data warehouse capabilities
  - AWS, Azure, Google Cloud and Snowflake, Databricks are class leaders
  - Offering tightly integrated tools for working with data, running the gamut from relational to completely unstructured

BITS Pilani WILP

Modern Data Stack

- Currently trendy analytics architecture
  - Core aim is to reduce complexity and increase modularization
- Past data stacks relied on expensive, monolithic toolsets
- MDS uses cloud-based, plug-and-play, easy-to-use, off-the-shelf components to create a modular and cost-effective data architecture
  - Data Pipelines
  - Cloud Storage
  - Transformation
  - Data Management / Governance
  - Monitoring, Visualization and exploration
- Still in flux, tools are changing and evolving rapidly
  - Key concept of plug-and-play modularity with easy-to-understand pricing and implementation is way of future
  - In analytics engineering, MDS is and will continue to be default choice of data architecture!

Lambda Architecture

- Old days (early to mid-2010s) popularity of working with streaming data exploded
  - Kafka as highly scalable message queue and f/w Apache Storm and Samza for analytics
  - Allowed to perform new types of analytics and modeling on large amounts of data
  - Needed to figure out how to reconcile batch and streaming data in single architecture
  - Answer: Lambda architecture!
- Three systems : batch, streaming and serving are operating independently
  - Source is immutable and append-only, sending data to two destinations : stream and batch
  - In-stream processing serves data with lowest possible latency in speed layer, usually NoSQL DB
  - Batch layer process and transforms data in data warehouse, creating precomputed and aggregated views of data
  - Serving layer provides a combined view by aggregating query results from two layers
- Managing multiple systems with different codebases is difficult, creating error-prone systems with code and data that are extremely difficult to reconcile!

Kappa Architecture

- As a response to shortcomings of Lambda architecture, Jay Kreps proposed an alternative
  - Just use stream processing platform as backbone for all data handling
    - Ingestion, storage and serving – facilitating a true event based architecture
  - Real-time and batch processing can be applied seamlessly to same data
    - By reading live event stream directly and replaying large chucks of data for batch processing
- Not yet widely adopted!
  - Streaming itself is still a bit of mystery to many companies – easy to talk, harder to execute
  - Turns out to be complicated and expensive in practice
  - Batch storage and processing remain much more efficient and cost-effective for enormous historical data

Dataflow model

Unified batch and streaming

- Lambda & Kappa both provided inspiration and groundwork for unification of batch and stream
  - Trying to tie together complicated tools that were not natural fits in the first place
  - Central problem is managing batch and stream processing with multiple code paths
  - Kappa relies on unified queuing and storage layer, still depends on different tools
- Google attempted to solve it by Dataflow model and Apache Beam as its implementation
  - Core idea is to view all data as events, aggregation is performed over various types of windows
  - Ongoing real-time event streams are unbounded data
  - Data batches are simply bounded event streams and boundaries provides natural window
    - Windows can tumbling , sliding
  - Real-time and batch processing happens in same system using nearly identical code
- Philosophy "batch as a special case of streaming" is now more pervasive
  - Frameworks such as Flink and Spark have adopted a similar approach!

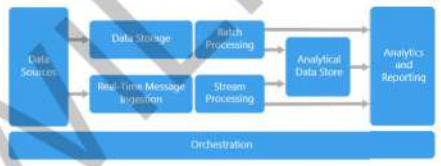
BITS Pilani WILP

Data Mesh

- Recent response to sprawling monolithic data platforms, such as centralized data lakes and warehouse and great divide of data – operational and analytical data
  - Attempts to invert the challenges of centralized data architecture – by applying concepts of domain driven design on data architecture
- Big part of data mesh is decentralization,
- Per Zhamak Dehghani,
  - In order to decentralize the monolithic data platforms, we need to reverse how we think about data, its locality and ownership. Instead of flowing the data from domains into a centrally owned data lake or platform, domain need to host and serve their domain datasets in easily consumable way.
- Four key components
  - Domain-oriented decentralized data ownership and architecture
  - Data as a product
  - Self-serve data infrastructure as a platform
  - Federated computational governance

Big data architecture style

- is designed to handle the ingestion, processing, and analysis of data that is too large or complex for traditional database systems.



Source : https://docs.microsoft.com/en-us/azure/architecture/guide/architecture-styles/big-data

Big Data Applications

Workloads

- Big data solutions typically involve one or more of the following types of workload:
  - ✓ Batch processing of big data sources at rest
  - ✓ Real-time processing of big data in motion
  - ✓ Interactive exploration of big data
  - ✓ Predictive analytics and machine learning

Big Data Systems Components

Components

- Most big data architectures include some or all of the following components:
  - ✓ Data sources
    All big data solutions start with one or more data sources like databases, files, IoT devices etc
  - ✓ Data Storage
    Data for batch processing operations is typically stored in a distributed file store that can hold high volumes of large files in various formats.
  - ✓ Batch processing
    Because the data sets are so large, often a big data solution must process data files using long-running batch jobs to filter, aggregate, and otherwise prepare the data for analysis. Usually these jobs involve reading source files, processing them, and writing the output to new files.
  - ✓ Real-time message ingestion
    - ✓ If the solution includes real-time sources, the architecture must include a way to capture and store real-time messages for stream processing.
  - ✓ Stream processing
    After capturing real-time messages, the solution must process them by filtering, aggregating, and otherwise preparing the data for analysis. The processed stream data is then written to an output sink.
  - ✓ Analytical data store
    Many big data solutions prepare data for analysis and then serve the processed data in a structured format that can be queried using analytical tools. The analytical data store used to serve these queries can be a Kimball-style relational data warehouse, as seen in most traditional business intelligence (BI) solutions.
  - ✓ Analysis and reporting
    The goal of most big data solutions is to provide insights into the data through analysis and reporting.
  - ✓ Orchestration
    Most big data solutions consist of repeated data processing operations, encapsulated in workflows, that transform source data, move data between multiple sources and sinks, load the processed data into an analytical data store, or push the results straight to a report or dashboard. To automate these workflows, you can use an orchestration technology such Azure Data Factory or Apache Oozie and Sqoop.

BITS Pilani WILP

Big data architecture Usage

When to use this architecture

- Consider this architecture style when you need to:
  - ✓ Store and process data in volumes too large for a traditional database
  - ✓ Transform unstructured data for analysis and reporting
  - ✓ Capture, process, and analyze unbounded streams of data in real time, or with low latency

Big data architecture Benefits

Advantages

- Technology choices
  - ✓ Variety of technology options in open source and from vendors are available
- Performance through parallelism
  - ✓ Big data solutions take advantage of parallelism, enabling high-performance solutions that scale to large volumes of data.
- Elastic scale
  - ✓ All of the components in the big data architecture support scale-out provisioning, so that you can adjust your solution to small or large workloads, and pay only for the resources that you use.
- Interoperability with existing solutions
  - ✓ The components of the big data architecture are also used for IoT processing and enterprise BI solutions, enabling you to create an integrated solution across data workloads.

Big data architecture Challenges

Things to ponder upon

- Complexity
  - ✓ Big data solutions can be extremely complex, with numerous components to handle data ingestion from multiple data sources. It can be challenging to build, test, and troubleshoot big data processes.
- Skillset
  - ✓ Many big data technologies are highly specialized, and use frameworks and languages that are not typical of more general application architectures. On the other hand, big data technologies are evolving new APIs that build on more established languages.
- Technology maturity
  - Many of the technologies used in big data are evolving. While core Hadoop technologies such as Hive and Pig have stabilized, emerging technologies such as Spark introduce extensive changes and enhancements with each new release.

A typical TDS setup

Three major problems

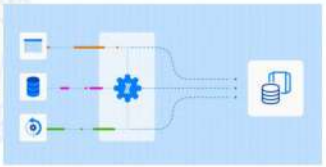
- Long turn-around time to untangle and set up infrastructure
  - Companies making use of on-premise infrastructure are responsible for all the costs associated with it
    - such as the army of engineers required to keep everything maintained and running smoothly.
  - As setup is so deeply interconnected, what may seem like a minor change might break other parts of the system
  - Finding the exact logical coupling between the systems requires a lot of work-hours to analyze before any improvements can be made to the existing landscape
- Slow response to new information
  - As the company grows, so does its data and computational power needs
  - Very costly in terms of resources and time when it comes to scaling out (expanding) on-premises infrastructure
    - leads to a limit in how much computational power there is to analyze data
    - data pipelines can take hours to complete, a problem that is compounded as the organization grows
  - A TDS requires slow ETL (Extract, Transform, Load) operations before newly ingested data can conform to the rest of the data model
    - A new data update can take weeks and many hours of refactoring before insights appear
    - By the time the data is ready, the organization is unable to act in time resulting in missed opportunities.
- Expensive journey to insights
  - A lot of the report generation is manually done, especially when the data is coming from different sources
    - The report is manually generated, manually cleaned, and manually transferred to Excel
  - Leads to errors being made, time being taken away from other business-critical tasks, and the inability to scale.
    - Analysts are unable to efficiently perform their roles due to the complex landscape

Source: neptune

BITS Pilani WILP

Modern Data Stack

- A radically new approach to data integration saves engineering time, allowing engineers and analysts to pursue higher-value activities!
- The modern data stack (MDS) is a suite of tools used for data integration. These tools include, in order of how the data flows:
  - a fully managed ELT data pipeline
  - a cloud-based columnar warehouse or data lake as a destination
  - a data transformation tool
  - a business intelligence or data visualization platform.
- The goal of an MDS is to analyze business's data to proactively uncover new areas of opportunity and improve efficiency



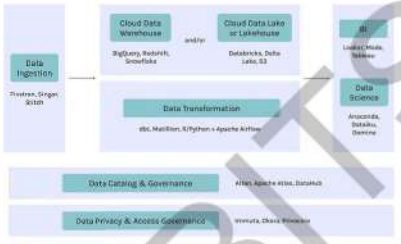
Source: fivetran

The 3 characteristics of a modern data platform

- A modern platform should be…
  - Easy to set up
    - no lengthy sales process, demo calls, and implementation cycles. Just login, pay via credit card, and go!
  - Pay as you go
    - no up-front payments and million dollar licensing fees. The "modern" stack is all about putting power in the hands of the consumer, i.e. paying only for what you use
  - Plug and play
    - the modern data stack will continue to evolve and innovate, and the best of breed tools do not enforce "lock in" like tools in the legacy era, and are instead built on open standards & APIs allowing easy integration with the rest of the stack

Source: the-building-blocks-of-a-modern-data-platfor

The key building blocks of a modern data platform



Source: atlan

The key building blocks of a MDS(2)

Modern data ingestion

- Data ingestion is likely where efforts to build out a modern data platform get started
  - i.e. how to bring data from a variety of different data sources and ingest it into core data storage layer?
- The majority of data pipelines capture data in batch, using SQL queries or extract routines, but this is changing
  - Many companies now prefer a streaming-first architecture that captures and processes data in real time in a continuous stream
  - Streaming tools use change data capture, replication, and/or publish/subscribe messaging systems to capture events as they happen, move them into the data environment, and analyze them.
- Some key modern data ingestion tools:
  - SAAS tools: Fivetran, Hevo Data, Stitch
  - Open-source tools: Singer, StreamSets
  - Custom data ingestion pipelines built on orchestration engines like Airflow

BITS Pilani WILP

The key building blocks of a MDS(3)

Modern data storage and processing

- The data storage and processing layer is fundamental to the modern data platform
- Architecture is evolving, typically see 3 kinds of tools or frameworks:
- Data warehouses
  - The cornerstone of this architecture is a modern data warehouse
  - generally the system of choice for analysts since they optimize compute and processing speed
  - Key data warehouse tools include BigQuery, Redshift, and Snowflake.
- Data lakes
  - Refers to data being stored on an object storage like Amazon S3, coupled with tools to process the data such as Spark
  - cheap storage systems are often used to store vast amounts of raw or even unstructured data
  - Key tools for data lakes:
    - Data storage: Amazon S3, Azure Data Lake Storage Gen2, Google Cloud Storage
    - Data processing engines: Databricks or Spark; Athena, Presto, or Starburst; Dremio
- Data lakehouses
  - One of the trends is seeing this year is the long-awaited convergence of data warehouses and lakes
  - help unify the siloed systems that most companies have created over the last decade
  - a system design that combines the data management features such as ACID transactions and change data capture from a data warehouse with the low-cost storage of a data lake.

The key building blocks of a MDS(4)

Modern data transformation

- Two core implementations seeing today for the data transformation layer
- For companies with data warehouse–first architectures
  - tools like dbt that leverage native SQL for transformation have emerged as the top choice for data transformation
- The other common implementation is using Airflow as an orchestration engine coupled with custom transformation in a programming language like Python
- Key tools for data transformation:
  - With data warehouses: dbt, Matillion
  - With an orchestration engine: Apache Airflow + Python, R, or SQL

The key building blocks of a MDS(5)

Modern business intelligence and analytics

- BI dashboards have been around for ages
  - but the latest breed of BI and analytics tools are built to fit within a larger modern data platform
  - generally more self-service platforms that allow users to explore data, rather than just consuming graphs and charts
- Modern BI tools include Looker, Mode, Redash, Sigma, Sisense, Superset, and Tableau.

The key building blocks of a MDS(6)

Data cataloguing and governance

- While the modern data platform is great in some areas (super fast, easy to scale up, little overhead),
  - it struggles with bringing discovery, trust and context to data.
- Key tools for modern data cataloguing and governance:
  - SAAS tools: Atlan
  - Open-source tools: Apache Atlas, LinkedIn's DataHub, Lyft's Amundsen
  - In-house tools: Airbnb's Dataportal, Facebook's Nemo, Uber's Databook

BITS Pilani WILP

The key building blocks of a MDS(7)

Modern data privacy and access governance

- A major challenge is to be able to manage privacy controls and access governance across entire stack
- While it's still early, there have been some recent players and developments in the space and players emerging with tools
  - that can act as an entitlement engine to apply privacy and security policies across the data stack
- Key tools for data privacy and access governance:
  - SAAS services: Immuta, Okera, Privacera
  - Open-source engines: Apache Ranger

Other modern data tools you should know about

Other modern data tools you should know about

- Each company uses data differently, so many have added additional layers and tools for specific use cases
- Real-time data processing tools
  - Companies that need real-time data processing generally add two additional types of tools to their data platform:
  - Real-time streaming pipelines: Confluent, Kafka
  - Real-time analytics: Druid, Imply
- Data science tools
  - Companies that have moved past BI and analytics onto strong predictive and data science analytics often add a specific data science tool (or tools) to their data stack:
  - Preferred by data scientists: Jupyter Notebooks
  - Other options: Dataiku, DataRobot, Domino, SageMaker
- Event collectors
  - Companies with a significant digital presence often add event collectors to log and store external events
  - Tools like Segment and Snowplow are key additions to their data ingestion stack.

Other modern data tools you should know about(2)

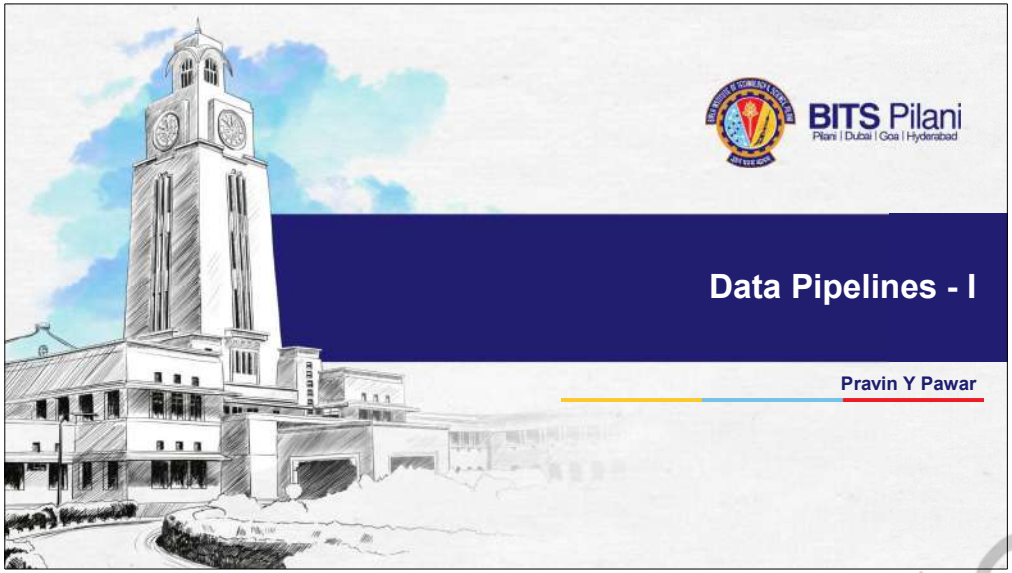
- Data quality tools
  - This space is still relatively nascent, but it's seeing a lot of activity nowadays.
  - seeing a few aspects of data quality incorporated throughout the data stack — data quality checks during profiling, business-driven quality rules, and unit testing frameworks within the pipeline
- Unit testing
  - Frameworks like the open-source Great Expectations are emerging and evolving, allowing unit tests to be written as a part of data pipelines itself.

Thank You!

In our next session:

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES



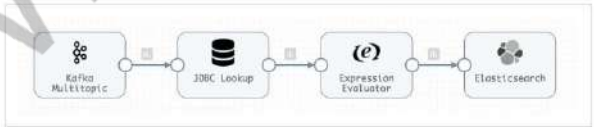
Data Pipelines - I

Pravin Y Pawar

Data Pipeline

What?

- Series of steps that allow data from one system to move to become useful in another system
  - particularly analytics, data science, or AI and machine learning systems
- Works by pulling data from the source, applying rules for transformation and processing, then pushing data to its destination.



Data Pipeline(2)

Purpose

- Lot of data out there!
- Data pipelines transform raw data into data ready for analytics, applications, machine learning and AI systems
  - keep data flowing to solve problems, inform decisions
- Used to:
  - Deliver sales data to sales and marketing for customer 360
  - Link a global network of scientists and doctors to speed drug discovery
  - Recommend financial services to help a small business owner thrive
  - Track COVID-19 cases and inform community health decisions
  - Combine diverse sensor data with AI for predictive maintenance
- With so much work to do, data pipelines can get pretty complicated pretty fast.

Data Pipeline(3)

Benefits

- Data pipelines done right have incredible advantages for companies and organizations, and the work they do.
- Self-service Data
  - Can be created ad hoc by data scientists and business analysts unstick the IT bottleneck
  - Means when people have brilliant ideas, they can test them, fail fast, and innovate faster
- Accelerate Cloud Migration and Adoption
  - Helps to expand cloud presence and migrate data to cloud platforms
  - Cloud computing helps to feed many new use cases at processing speeds, cost-effectiveness, and bursting capacity unheard of in traditional on-premises data centers
  - Teams can take advantage of rapid innovation happening on those cloud platforms such as
    - natural language processing
    - sentiment analysis
    - image processing
    - and more.
- Real-time Analytics and Applications
  - Real-time or near real-time functionality in consumer and business applications puts the pressure on data pipelines
    - to deliver the right data, to the right place, right now
  - Streaming data pipelines deliver continuous data to real-time analytics and applications

BITS Pilani WILP

Data Pipeline(4)

Challenges

- Always Under Construction Data
  - Building and debugging data pipelines takes time
  - Have to align with the schema, set sources and destinations, check work, find errors, and back and forth until one can finally go live
  - by which time the business requirements may have changed again
  - That's why many data engineers have such a backlog of work!
- Out of Order Data Pipelines
  - Even a small change to a row or a table can mean hours of rework, updating each stage in the pipeline, debugging, and then deploying the new data pipeline
  - Data pipelines often have to go offline to make updates or fixes
  - Unplanned changes can cause hidden breakages that take months of engineering time to uncover and fix
  - These unexpected, unplanned, and unrelenting changes are referred to as “data drift”.
- Build It and They Will Come
  - Data pipelines are built for specific frameworks, processors, and platforms.
  - Changing any one of those infrastructure technologies to take advantage of cost savings or other optimizations can mean weeks or months of rebuilding and testing pipelines before deployment.

Data Pipeline(5)

Working

- When a data pipeline is deployed and running
  - pulls data from the source
  - applies rules for transformation and processing
  - then pushes data to its destination
- Data sources
  - handle data in very different ways and might include applications, messaging systems, data streams, relational and NoSQL databases, cloud object storage, data warehouses, and data lakes
  - data structure varies significantly, depending on the source
- Transformations
  - Changes to data structure, format, or values as well as calculations and modifications to the data itself
  - A pipeline might have any number of transformations embedded to prepare data for use or route it correctly
- Destinations
  - systems where the data is ready to use, put directly into use, or stored for potential use
  - include applications, messaging systems, data streams, relational and NoSQL databases, data warehouses, data lakes, and cloud object storage

Data Pipeline(6)

Working

- 5 Common Data Pipeline Sources
  - JDBC
  - Oracle CDC
  - HTTP Client
  - HDFS
  - Apache Kafka
- Common Transformations
  - Masking PII (personally identifiable information) for protection and compliance
  - Converting data type for fields
  - Calculating based on a formula or expression
  - Renaming fields, columns, and features
  - Joining or merging datasets
  - Converting data formats (JSON to Parquet, for example)
  - Generating Avro Schema and other schema types on the fly
  - Handling Slowly Changing Dimensions
- 5 Common Data Pipeline Destinations
  - Apache Kafka
  - JDBC
  - Snowflake
  - Amazon S3
  - Databricks

Data Pipeline(7)

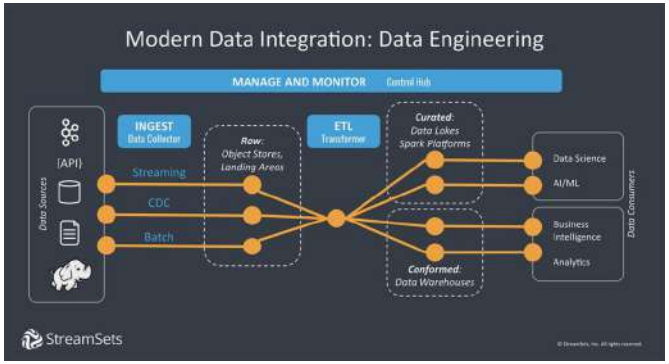
Architectures

- Batch Data Pipeline
  - move large sets of data at a particular time or in response to a behavior or when a threshold is met
  - often used for bulk ingestion or ETL processing
  - might be used to deliver data weekly or daily from a CRM system to a data warehouse for use in a dashboard for reporting and business intelligence
- Streaming Data Pipeline
  - flow data continuously from source to destination as it is created
  - used to populate data lakes or as part of data warehouse integration, or to publish to a messaging system or data stream
  - used in event processing for real-time applications
  - streaming data pipelines might be used to provide real-time data to a fraud detection system and to monitor quality of service.
- Change Data Capture Pipeline (CDC)
  - used to refresh data and keep multiple systems in sync
  - Instead of copying the entire database, only changes to data since the last sync are shared
  - can be particularly useful during a cloud migration project when 2 systems are operating with the same data sets

BITS Pilani (WILP)

Data Pipeline(8)

Architectures



Data Pipeline Tools

- Building a single pipeline for a single purpose at a given time is no problem
  - Can grab a simple tool for setting up a data pipeline or hand code the steps
- But how to scale that process to thousands of data pipelines in support of increasing demand for data across organization, over months or years?
  - When considering data pipeline tools, it is important to think ahead to where your data platform is headed.
- Questions to be pondered upon
  - Are you grabbing data from one place to put it somewhere else? Or do you need to transform it to fit the downstream analytics requirements?
  - Is your data environment stable and fully under your control? Or is it dynamic and pulling data from systems or apps outside of your control?
  - Will the pipeline move data once, for a short-term analysis project? Or will the pipelines you build need to be operationalized to handle data flows over time?

Data Pipeline Tools(2)

Data Ingestion and Data Loading Tools

- These tools make data pipelines easy to set up, easy to build, and easy to deploy solve the "under construction" issue
  - but only if you can count on your data scientists to do the data prep work later
- Only support the most basic transformations and work best for simply copying data
  - Instead of being intent-driven, these data pipelines are rigid and embedded with the specifics of data structure and semantics
  - Instead of adapting to change when data drift happens, they have to be rebuilt and deployed again

Data Pipeline Tools(3)

Data Integration, Data Transformation Platforms

- More complex data integration or ETL software may have a solution for every possible scenario
  - with hundreds of connectors, integrations, and transformations
- But these platforms were designed for an era when data drift hardly happened
  - Things stayed the same for years
  - At the first hint of change, these data pipelines break and require massive rework
  - The demands for digital transformation are moving fast and planning for every possible outcome may not be possible

BITS Pilani WILP

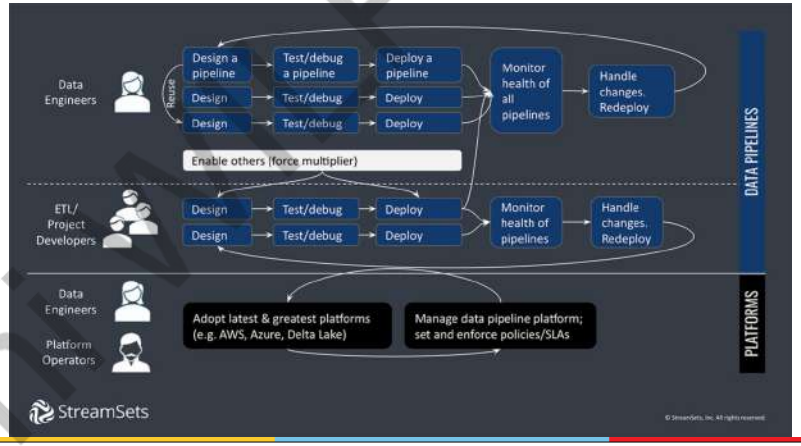
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Data Pipeline Tools(4)

Data Engineering Platforms

- Third way!
- A data engineering platform builds smart data pipelines according to DataOps principles
  - Smart data pipelines abstract away the "how" so you can focus on the what, who, and where of the data
  - This is the fundamental difference between data integration and data engineering
- Instead of being perpetually under construction, out of order, or limited to a single platform
  - smart data pipelines allow you to move fast with confidence that your data will continue to flow with little to no intervention
- DataOps tools allow you to:
  - Design and deploy data pipelines in hours, not weeks or months
  - Build in as much resiliency as possible to handle changes
  - Adopt to new platforms by pointing to them, a task that takes minutes not months

Data Pipeline People



Data Analytics Pipelines

Immature Data and Analytics Processes

Drawing Swords!

- Though data is a critical business asset
  - most organizations still don't have mature processes for converting data into insights that drive business value
- The development of data and analytics pipelines,
  - is still a handcrafted and largely non-repeatable process with minimal reuse,
  - managed by individuals working in isolation with different tools and approaches
- The result is
  - a plodding development environment that can't keep pace with the demands of a data-driven business
  - an error-prone operational environment that is slow to respond to change requests
- Drawing Swords
  - The lack of efficiency on the development side leads to a lack of effectiveness on the business side
  - Business and IT draw swords and go to war instead of partnering

BITS Pilani WILP

Data Pipelines

Origin

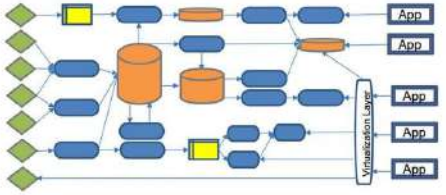
- Emerged from the big data community
- Data lakes created a feeding frenzy among developers, data scientists, and data analysts
  - gained unfettered access for the first time
  - to a nearly limitless supply of raw data sourced
  - from a multiplicity of internal and external systems
- Created a wide diversity of workflows and programs—or data pipelines—to process that data
  - (e.g., clean, filter, aggregate, move, load)
  - for a variety of purposes and use cases

Data Pipelines(2)

Simple vs Complex

- Represents the encoded flow of data from source to consumption
- Some data pipelines are simple:
  - "Export this data into a CSV file and place into this folder."
- But others are complex:
  - "Move tables from 10 sources into a target database, merge common fields, array into a dimensional schema, aggregate by year, flag null values, convert into an extract for a BI tool, and generate personalized dashboards based on the data."

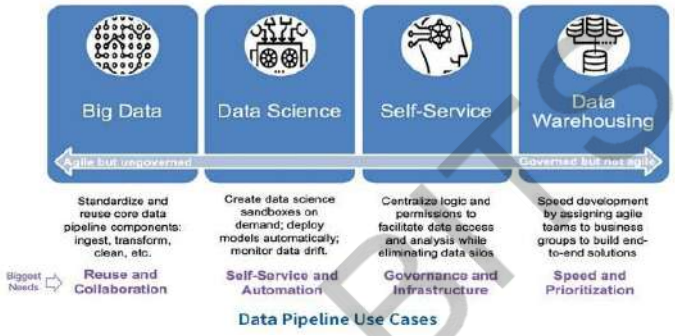
SIMPLE - "Export this data into a CSV file and place into this file folder."



COMPLEX - "Move tables from 10 sources into a target database, merge common fields, array into a dimensional schema, aggregate by year, flag null values, convert into an extract for a BI tool, and generate personalized dashboards based on the data."

Data Pipelines(3)

Use Cases



Data Pipeline Use Cases

Data Pipelines(4)

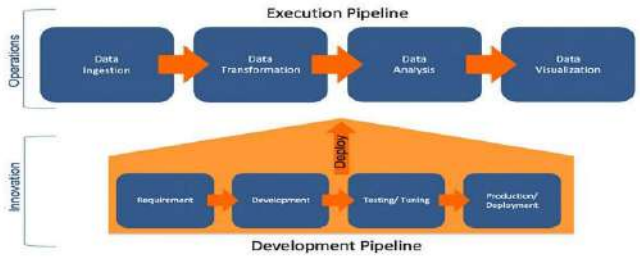
Two types

- A development pipeline
  - creates code to process data for a new pipeline
- An execution pipeline
  - runs the pipeline in production
- In most IT shops
  - these are managed by different teams, creating tremendous inefficiencies
  - developers can build or change code without being responsible for its downstream impacts
  - inevitably increases error rates, delays delivery and frustrates business users

BITS Pilani WILP

Data Pipelines(4)

Development and Execution Pipelines



Development and Operations Pipelines

Data Pipelines(5)

Generic Pipeline (Major)

- Although most data analytics pipelines are fairly complex, with dozens or hundreds of steps, most consist of the following stages:

- Data Ingestion
  - Data from various sources is extracted, validated, and loaded into downstream systems.
- Data Transformation
  - Data is cleansed, enriched, integrated, and modeled to support the target application or data structure.
- Data Analysis
  - Data is analyzed refined, and packaged in data models to provide certain insights or fulfill a task.
- Data Visualization
  - Data is visualized in a suitable manner, e.g., a static report or an interactive dashboard.

Data Pipelines(6)

People and Tools

- Data analytics pipelines
  - often span multiple functions supported by different specialists
  - who use a variety of tools and technologies

- A data engineer
  - uses ETL or data preparation tools
  - to extract data from a source and load it to a central data warehouse or data set
- A BI developer
  - uses a visualization tool
  - to build a chart, report or dashboard which is consumed by business users
- A data scientist
  - might then use Python or R
  - to build a predictive model that provides data for a business application managed by a software developer

References

- The Ultimate Guide to DataOps Product Evaluation and Selection Criteria
- DataOps: Industrializing Data and Analytics Strategies for Streamlining the Delivery of Insights
- IBM DataOps How and Why Whitepaper

BITS Pilani WILP

DataOps

DataOps

New Approach

- Data Analytics pipelines becoming more complex and development teams grow in size
  - organizations need better collaboration and development processes
  - to govern the flow of data and code from one step of the data lifecycle to the next
  - – from data ingestion and transformation to analysis and reporting
- DataOps
  - is an emerging set of practices, processes, and technologies
  - for building and enhancing data and analytics pipelines
  - to meet business needs quickly
- The goal is to increase agility and cycle times,
- While
  - reducing data defects,
  - giving developers and business users greater confidence in data analytics output

DataOps(2)

Foundations

- Builds on concepts popular in the software engineering field,
  - such as agile, lean, and continuous integration/continuous delivery
- but addresses the unique needs of data and analytics environments,
  - including the use of multiple data sources and varied use cases that range from data warehousing to data science
- Relies heavily on
  - test automation
  - code repositories
  - collaboration tools,
  - orchestration frameworks
  - and workflow automation
- to accelerate delivery times while minimizing defects

DataOps(3)

Data Stakeholders

BITS Pilani WILP

DataOps(4)

DataOps for Data Warehousing and Data Management

Table 1. DataOps for Data Warehousing and Data Management

Current Challenges:	DataOps' Answers:
• Complex data landscapes and processes with heterogeneous data sources and tools that are hard to manage. • A lack of agility makes it hard to respond to rapidly changing business requirements.	• Analytics pipelines that streamline processes, improve collaboration, and establish a culture of continuous improvement. • Use of automation to increase agility and speed and free resources for more valuable activities.

DataOps(5)

DataOps for Dashboards and Reports

Table 2. DataOps for Dashboards and Reports

Current Challenges:	DataOps' Answers:
• Manage a growing number of reports and dashboards over the entire lifecycle. • Understand what the business needs and quickly respond to changing require-ments.	• Establish an end-to-end responsibility to bridge the gap between data and business. • Ensure relevancy with short cycles, fast feedback, and data-driven improvements. • Improve speed, quality, and scalability with defined analytics pipelines.

DataOps(6)

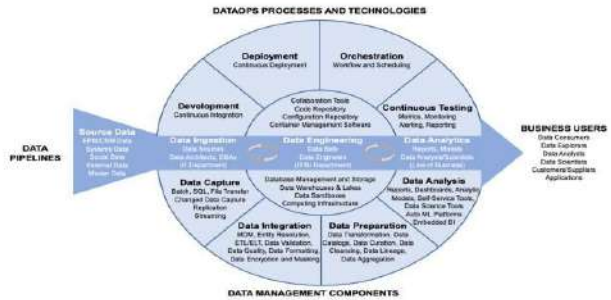
DataOps for Data Science

Table 3. DataOps for Data Science

Current Challenges:	DataOps' Answers:
• Transform statistical models from an experimental stage to production so they deliver ROI. • Alignment among data scientists, product owners, and data engineers.	• Establish analytics pipelines that keep the data science process flexible. • Use tools to orchestrate stakeholders and technologies throughout the entire data science process. • Combine different skillsets with cross functional thinking.

DataOps(7)

Technology



DataOps Technical Framework

BITS Pilani WILP

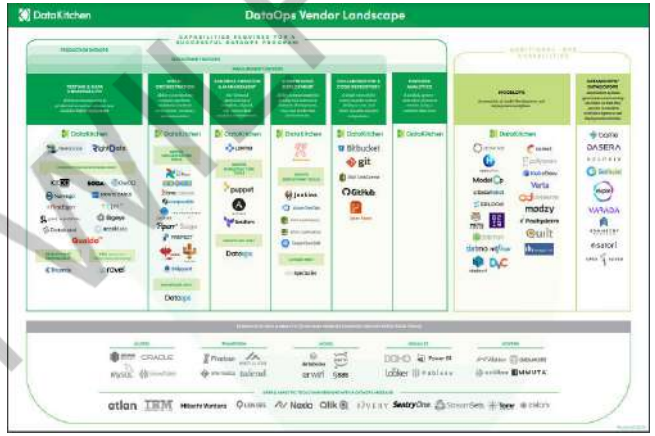
DataOps(8)

Tools

- There are five categories of DataOps products on the market today:
  - All-in-one tools
  - Orchestration tools
  - Component tools
  - Case-specific tools
  - Open source tools

DataOps(9)

Vendor Landscape



DataOps(10)

Benefits of DataOps

- Improve Collaboration and Communication
  - comes with a change in culture that promotes collaboration, trust, and responsibility
  - goals are to blur the lines between departments and functions,
  - encourage the exchange of knowledge, reduce conflicts, and eventually increase productivity
- Accelerate Time to Production
  - A major driver for DataOps is speed
  - Idea of streamlined and largely automated analytics pipelines helps deliver new features and insights quickly and reduces manual effort
  - Short feedback and testing cycles help speed up reactions to changing business requirements and increase flexibility
- Increase Quality and Reliability
  - Well-defined analytics pipelines enhance both speed and robustness
  - For instance, multiple stages of automated and manual tests prevent the deployment of flawed updates
  - Besides, DataOps also includes monitoring of rolled-out changes to identify bottlenecks and potential issues
  - Lastly, the convergence of different roles helps align changes throughout various stages,
  - such as when a data engineer is informed about the later cleansing issues encountered by a data scientist

DataOps(11)

Common Pitfalls of DataOps

- The following common pitfalls should be considered on the way to a DataOps culture
- Acceptance:
  - Don't make it technical!
  - DataOps has the same burden as other related ideas in software engineering, which are often characterized as technical Knick knacks
  - It is important to show that DataOps is not a technology trend, but a business trend
  - Communicate the holistic idea of DataOps, get everybody on board early, and show that business value is the goal, rather than having a fancy tech stack
- Engineering:
  - Keep it simple!
  - DataOps combines many concepts which can be extended almost endlessly, running the risk of over-complication
  - Always take care to balance effort and benefit
  - This applies to both technical solutions, such as unnecessary tests and overly complex deployment processes, and to organizational structures, which can easily become burdened with too many roles, feedback loops, and overly rigid processes.

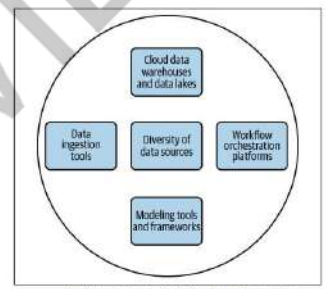
BITS Pilani WILP

References

- The Ultimate Guide to DataOps Product Evaluation and Selection Criteria
- DataOps: Industrializing Data and Analytics Strategies for Streamlining the Delivery of Insights
- IBM DataOps How and Why Whitepaper

Key components of modern data infrastructure

- No single right way to design analytics ecosystem or choose products and vendors
- Before deciding on products and design for building data pipelines,
  - it's worth understanding what makes up a modern data infrastructure

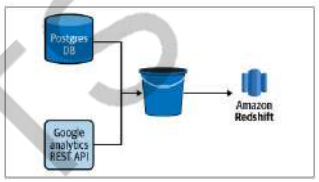


The key components of a modern data infrastructure.

Diversity of Data Sources

Source Systems

- The majority of organizations have dozens, if not hundreds, of data sources
  - that feed their analytics endeavors
- Typical for an analytics team to ingest data from source systems
  - that are built and owned by the organization as well as from third-party tools and vendors
- For example, an ecommerce company might
  - store data from their shopping cart in a PostgreSQL database behind their web app
  - also use a third-party web analytics tool such as Google Analytics to track usage on their website
  - combination of the two data sources is required to get a full understanding of customer behavior leading up to a purchase
- A data pipeline that ends with an analysis of such behavior starts with the ingestion of data from both sources.



Ingestion Interface and Data Structure

Data Ingestion - extracting data from one source and loading it into another

- When building a new data ingestion
  - Essential to know how to get data in and in what form
- First, what is the interface to the data?
  - A database behind an application, such as a Postgres or MySQL database
  - A layer of abstraction on top of a system such as a REST API
  - A stream processing platform such as Apache Kafka
  - A shared network file system or cloud storage bucket containing logs, comma-separated value (CSV) files, and other flat files
  - A data warehouse or data lake
  - Data in HDFS or HBase database
- In addition to the interface, the structure of the data will vary.
  - JSON from a REST API
  - Well-structured data from a MySQL database
  - JSON within columns of a MySQL database table
  - Semi-structured log data
  - CSV, fixed-width format (FWF), and other flat file formats
  - JSON in flat files
  - Stream output from Kafka
- Each interface and data structure presents its own challenges and opportunities

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Cloud Data Warehouses and Data Lakes

- Three things transformed the landscape of analytics and data warehousing over the last 10 years
  - All related to the emergence of the major public cloud providers (Amazon, Google, and Microsoft)
  - The ease of building and deploying data pipelines, data lakes, warehouses, and analytics processing in the cloud
  - Continued drop-in storage costs in the cloud
  - The emergence of highly scalable, columnar databases, such as Amazon Redshift, Snowflake, and Google Big Query
- These changes breathed new life into data warehouses and introduced the concept of a data lake!
- A data warehouse is a database where data from different systems is stored and modeled to support analysis and other activities related to answering questions with it
  - Data in a data warehouse is structured and optimized for reporting and analysis queries
- A data lake is where data is stored, but without the structure or query optimization of a data warehouse
  - likely contain a high volume of data as well as a variety of data types
- There is a place for both data warehouses and data lakes in the same data ecosystem, and data pipelines often move data between both.

Data Ingestion Tools

- The need to ingest data from one system to another is common to nearly all data pipelines
  - data teams must contend with a diversity of data sources from which to ingest data from
- A number of commercial and open source tools are available in a modern data infrastructure.
  - Singer
  - Stitch
  - Fivetran
- Despite the prevalence of these tools,
  - some teams decide to build custom code to ingest data
  - Some even develop their own frameworks
- Reasons vary by organization but are often related to
  - cost, a culture of building over buying, and concerns about the legal and security risks of trusting an external vendor.
  - need to assess the value of a commercial solution is to make it easier for data engineers to build data ingestions into their pipelines or to enable non-data engineers (such as data analysts) to build ingestions themselves
- Data ingestion is traditionally both the extract and load steps of an ETL or ELT process
  - Some tools focus on just these steps, while others provide the user with some transform capabilities as well

Data Transformation and Modeling Tools

- Pipelines are also made up of tasks that transform and model data for new purposes
  - such as machine learning, analysis, and reporting
  - terms often used interchangeably
- Data transformation
  - Transforming data is a broad term that is signified by the T in an ETL or ELT process
  - A transformation can be something
    - as simple as converting a timestamp stored in a table from one time zone to another
    - a more complex operation that creates a new metric from multiple source columns that are aggregated and filtered through some business logic
- Data modeling
  - Data modeling is a more specific type of data transformation
  - A data model structures and defines data in a format that is understood and optimized for data analysis
  - A data model is usually represented as one or more tables in a data warehouse

Data Transformation and Modeling Tools(2)

- Data Transformation Tools
  - Some data ingestion tools provide some level of data transformation capabilities, but these are often quite simple
    - for the sake of protecting personally identifiable information (PII) it may be desirable to turn an email address into a hashed value
  - For more complex data transformations and data modeling, find it desirable to seek out tools and frameworks specifically designed for the task, such as dbt
  - Additionally data transformation is often context-specific and can be written in a language familiar to data engineers and data analysts, such as SQL or Python.
- Data modeling Tools
  - typically defined and written in SQL or via point-and-click user interfaces
  - SQL is a highly accessible language that is common to both data engineers and analysts
    - empowers the analyst to work directly with the data and optimize the design of models for their needs
    - used in nearly every organization, thus providing a familiar entry point for new hires to a team
- In most cases, choosing a transformation framework that supports building data models in SQL rather than via a point-and click user interface is desirable!

BITS Pilani WILP

Workflow Orchestration Platforms

- As the complexity and number of data pipelines in an organization grows
  - it's important to introduce a workflow orchestration platform to data infrastructure
  - manages the scheduling and flow of tasks in a pipeline
- Imagine a pipeline with a dozen tasks ranging from
  - data ingestions written in Python
  - data transformations written in SQL
  - that must run in a particular sequence throughout the day
  - not a simple challenge to schedule and manage dependencies between each task
- There are numerous workflow orchestration platforms available to alleviate the pain
  - Apache Airflow, Luigi, and AWS Glue, are designed for more general use cases and are thus used for a wide variety of data pipelines
  - Kubeflow Pipelines, are designed for more specific use cases and platforms (machine learning workflows built on Docker containers in the case of Kubeflow Pipelines

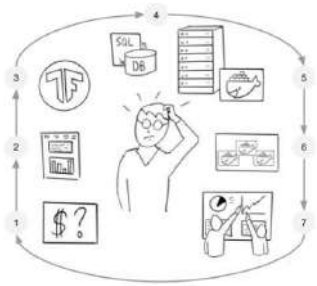
Customizing Data Infrastructure

- It's rare to find two organizations with exactly the same data infrastructure
  - Most pick and choose tools and vendors that meet their specific needs and build the rest on their own
  - many more come to market each year
- Depending on the culture and resources in organization,
  - may be encouraged to build most of data infrastructure on their own
  - or to rely on SaaS vendors instead
- Regardless of which way to lean on the build versus-buy scale
  - need to build the high-quality data infrastructure necessary to build high-quality data pipelines
- Most important is understanding constraints (dollars, engineering resources, security, and legal risk tolerance) and the resulting trade-offs!

Paradigm Shift

- The biggest paradigm shifts in computing happen not when impossible things become possible, but when possible things become easy.
- Call it machine learning, artificial intelligence, or data science
  - is on the cusp of becoming radically more approachable than ever before
    - More people will be able to build more, smarter applications
  - This is super exciting!

Lifecycle of a Data Science Project



Lifecycle of a data science project

BITS Pilani WILP

Lifecycle of a Data Science Project

Common Elements

- Details of the life cycle will naturally vary between companies and projects:
  - How you develop a predictive model for customer lifetime value differs greatly from building self-driving cars.
- However, all data science and machine learning projects have a few key elements in common:
  - In the technical point of view, all projects involve data and computation at their foundation.
  - All projects will eventually need to address the question of integrating results into production systems which typically involves a great deal of software engineering.
  - Finally, in the human point of view, all projects involve experimentation and iteration, which many consider to be the central activity of data science.
- Certainly possible for individuals companies or teams to come up with their own bespoke processes and practices to conduct data science projects,
  - a common infrastructure can help to increase the number of projects that can be
    - executed simultaneously (volume),
    - speed up the time-to-market (velocity),
    - ensure that the results are robust (validity), and
    - make it possible to support a larger variety of projects.

Data Science Infrastructure

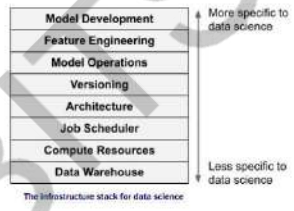
Evolution

- The infrastructure for data science is going through an evolution
  - Primordial machine learning and optimization libraries have existed for decades without much other infrastructure
  - As of 2020, experiencing a Cambrian explosion of data science libraries, frameworks, and infrastructures
    - often driven by commercial interests, similar to what happened during the Dotcom boom.
  - Widely shared patterns will emerge from this soup which will form the basis of a common, open-source infrastructure stack for data science

Data Science Infrastructure (2)

The stack

- Because of culture of open source and relatively free technical information sharing between companies,
  - been able to observe and collect common patterns in data science projects and infrastructure components
    - implementation details vary, the major infrastructural layers are relatively uniform across a large number of projects
    - each layer can be implemented in various ways, driven by the specific needs of their environment and use cases
  - but the big picture is remarkably consistent!
  - The most fundamental, generic components
    - are at the bottom of the stack
  - The layers become more specific to data science
    - towards the top of the stack

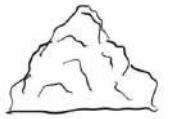


The infrastructure stack for data science

Data Science Infrastructure (3)

Components : Data Warehouse & Compute Resources

- Data Warehouse
  - Place where data is stored
  - Doesn't do anything by itself but how data is stored is critical
  - A key challenge is to lay out the data in a way that makes it easily discoverable and efficiently accessible by data science applications while maintaining a high degree of durability
    - naturally the data warehouse must not lose data.


Data Warehouse

- Compute Resources
  - Need to have a way to do something with data, in other words, take some data, apply an algorithm and get results
  - A challenge is that algorithms come in many shapes and sizes and need to run many of them, often in parallel, very frequently
  - Need a system that can provide computing resources on demand
  - The compute layer doesn't dictate what is computed
    - blindly accepts an algorithm from the outside world and executes it, consuming and producing data

Compute Resources

BITS Pilani WILP

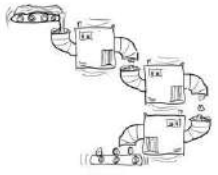
Data Science Infrastructure (4)

Components: Job Scheduler & Architecture

- Job Scheduler
  - Training a machine learning model or other data science application is rarely a one-time operation
    - More often it is a repeating process
    - Need to feed fresh data in the algorithm at a regular cadence
  - Job of the scheduling layer to keep all the data flowing and all the machines humming uninterruptedly
  - A key challenge is to keep the scheduling system itself up all the time and make sure it can manage almost any number of concurrent machines and data pipelines

Job Scheduler

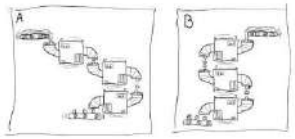
- Architecture
  - Data warehouse, compute resources, and job scheduler, are generic in nature: can be used to execute any workloads
  - At the architecture layer defines what application is and how it is going to work
    - includes defining an algorithm or several algorithms and how the scheduling layer is supposed to connect the algorithms and the data pipelines together
  - A key challenge is that of software engineering in general
    - How to map a real-world problem into an effective and robust software architecture.


Architecture

Data Science Infrastructure (5)

Components: Versioning

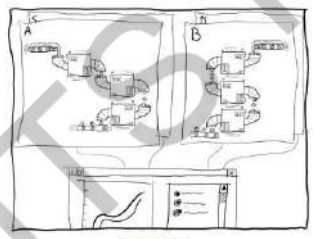
- Versioning
  - "experimentation and iteration" as one of the common themes in data science projects
  - A concrete ramification of this is that architectures we define are always subject to change
    - makes sense to organize the work accordingly
  - To complicate matters further, in the case of data science the question of which version of the application is better is often non-trivial
    - To judge the versions properly, you may need to run multiple versions side by side.
  - A key challenge is that can evaluate results reliably only if the applications are well isolated, that is, they must not interfere with each other
    - Must isolate the algorithms themselves, the input and output data, and the scheduled pipelines


Versioning

Data Science Infrastructure (6)

Components: Model Operations

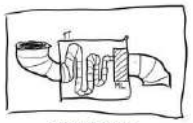
- Model Operations
  - Imagine four versions of an application running, each with their own version of code and data
    - Each version may be built by a separate data scientist, all of whom are all iterating on their versions independently.
  - The result, unless organized properly, can be a logistical nightmare!
  - Role of the model operations layer to help
    - to deploy, monitor, and assure validity of all models at all times,
    - without hindering the speed of experimentation
    - To make this possible, the infrastructure needs to track metadata about all executions and models, from prototype to production


Model Operations

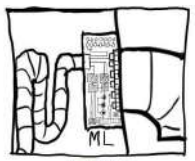
Data Science Infrastructure (7)

Components: Feature Engineering & Model Development

- Feature Engineering
  - Typically, the raw input data is not fed directly into the model
    - but it goes through a sequence of data transformations
  - Designing this sequence is a major part of the data scientist's work
    - when they are developing a new model or trying to improve performance of an existing model
  - A key challenge is to make this process as efficient as possible


Feature Engineering

- Model Development
  - At the very top of the stack there's the layer of model development
    - finding and describing a mathematical model that transforms features into desired outputs
  - A key challenge, and the key expertise of a data scientist, is to understand what modeling approach is most appropriate for the task at hand and how to implement it typically using off-the-shelf libraries.
  - Occupies only a tiny part of the end-to-end machinery that makes an effective data science application
    - shouldn't underestimate the effort that it takes to embed intelligence in the surrounding environment, which is what this stack is all about!

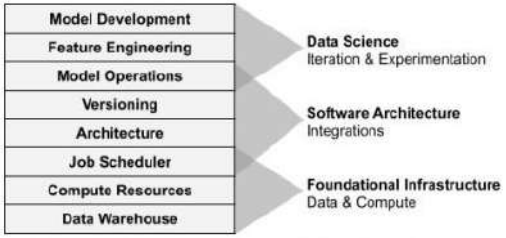


Model Development

BITS Pilani WILP

Data Science Infrastructure (8)

Stack again



Concerns of a data science project mapped to the infrastructure layers

Data Science Infrastructure (9)

WHY THE STACK

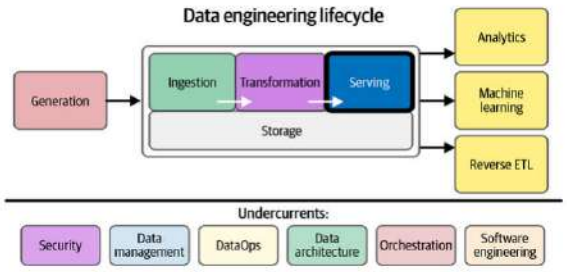
- Each layer of the stack is a complex system of its own
  - meant to cover the needs of a typical data science project, end-to-end,
- Three common themes that are common to most data science projects
  - Every data science project regardless of the problem domain needs to deal with data and compute
    - so these layers form the foundational infrastructure
    - layers are agnostic of what exactly gets executed
  - The middle layers
    - define the software architecture of an individual data science application
    - What gets executed and how: the algorithms, data pipelines, deployment strategies, and how results should be distribute
    - Much about the work is about integrating existing software components together.
  - The top of the stack
    - realm of data science:
    - Defining a mathematical model and how to transform raw input to something that the model can process
    - can evolve quickly as the data scientist experiments with different approaches

Data Science Infrastructure (11)

Data Scientists Freedom and Responsibility

Infrastructure complements the interests of a data scientist

Data Serving



Serving delivers data for use cases

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Data Serving

Three Ways

- Serve data for analytics and BI
  - most traditional area of data serving
  - prepare data for use in statistical analysis, reporting, and dashboards
  - important as ever for stakeholders to have visibility into the business, organizational, and financial processes
- Serve data for ML applications
  - ML is not possible without high quality data, appropriately prepared
  - work with data scientists and ML engineers to acquire, transform, and deliver the data necessary for model training
- Serve data through reverse ETL
  - process of sending data back to data sources
  - For example, might acquire data from an adtech platform, run a statistical process on this data to determine cost-per-click bids, and then feed this data back into the ad tech platform

Analytics

- Discovering, exploring, identifying, and making visible key insights and patterns within data
  - Carried out using statistical methods, reporting, BI tools, and more
- Business Analytics
  - uses historical and current data to make strategic and actionable decisions
  - types of decisions tend to factor in longer-term trends and often involve a mix of statistical and trend analysis, alongside domain expertise and human judgment
  - typically falls into a few big areas—dashboards, reports, and ad hoc analysis
- Operational Analytics
  - uses data to take immediate action:
  - Operational analytics versus business analytics = immediate action versus actionable insights
  - big difference between operational and business analytics is time
  - as real-time updates can be impactful in addressing a problem when it occurs
- The line between business and operational analytics has begun to blur!
- Embedded Analytics
  - Whereas business and operational analytics are internally focused, a recent trend is external-facing or embedded analytics
  - With so much data powering applications, companies increasingly provide analytics to end users
  - Typically referred to as data applications, often with analytics dashboards embedded within the application itself
  - The landscape of embedded analytics is snowballing, and expect that such data applications will become increasingly pervasive within the next few years.

Machine Learning

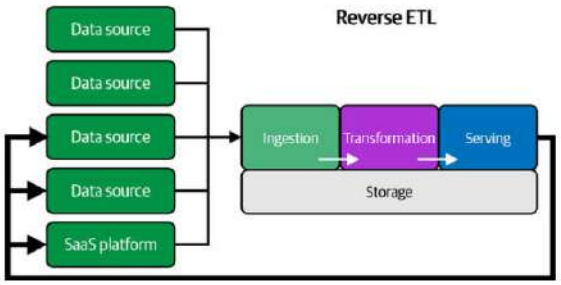
- ML is increasingly common!
- ML engineering is rising (itself almost a parallel universe to data engineering)
  - Boundary between ML, data science, data engineering, and ML engineering is increasingly fuzzy, and this boundary varies dramatically between organizations
- In some organizations, ML engineers take over data processing for ML applications right after data collection or may even form an entirely separate and parallel data organization that handles the entire lifecycle for all ML applications.
  - Data engineers handle all data processing in other settings and then hand off data to ML engineers for model training
  - Data engineers may even handle some extremely ML-specific tasks, such as featuarization of data

Reverse ETL

- Today, reverse ETL is a buzzword that describes serving data by loading it from an OLAP database back into a source system
  - increasingly recognized as a formal data engineering responsibility
- For example,
- A data engineer might pull customers and order data from a CRM and store it in a data warehouse
- This data is used to train a lead scoring model, whose results are returned to the data warehouse
- Company's sales team wants access to these scored leads to try to generate more sales
- Have a few options to get the results of this lead scoring model into the hands of the sales team
  - Can put the results in a dashboard for them to view
  - Or might email the results to them as an Excel file
- The challenge with these approaches is that they are not connected to the CRM, where a salesperson does their work
  - Why not just put the scored leads back into the CRM?
- Using reverse ETL and loading the scored leads back into the CRM is the easiest and best approach for this data product
- Reverse ETL takes processed data from the output side of the data engineering lifecycle and feeds it back into source systems

BITS Pilani WILP

Reverse ETL(2)



Ways to Serve Data for Analytics and ML

File Exchange

- Is ubiquitous in data serving. We process data and generate files to pass to data consumers
  - A data scientist might load a text file (unstructured data) of customer messages to analyze the sentiments of customer complaints
  - A business unit might receive invoice data from a partner company as a collection of CSVs (structured data)
  - An analyst must perform some statistical analysis on these files
- The way files are served depends on several factors, such as these:
  - Use case — business analytics, operational analytics, embedded analytics
  - The data consumer's data-handling processes
  - The size and number of individual files in storage
  - Who is accessing this file
  - Data type — structured, semi-structured, or unstructured

Ways to Serve Data for Analytics and ML(2)

Databases

- Critical layer in serving data for analytics and ML - serving data from OLAP databases (e.g.,data warehouses and data lakes)
  - Serving data involves querying a database and then consuming those results for a use case
  - An analyst or data scientist might query a database by using a SQL editor and export those results to a CSV file for consumption by a downstream application, or analyze the results in a notebook
- Carries a variety of benefits
  - A database imposes order and structure on the data through schema
  - Databases can offer fine-grained permission controls at the table, column, and row level, allowing database administrators to craft complex access policies for various roles
  - Databases can offer high serving performance for large, computationally intensive queries and high query concurrency
- A data scientist might connect to a database, extract data, and perform feature engineering and selection
  - Converted dataset is then fed into an ML model
  - offline model is trained and produces predictive results
  - Data engineers are quite often tasked with managing the database-serving layer.

Ways to Serve Data for Analytics and ML(3)

Streaming Systems

- Increasingly important in the realm of serving
  - type of serving may involve emitted metrics, which are different from traditional queries
- Operational analytics databases playing a growing role in this area
  - allow queries to run across a large range of historical data, encompassing up-to-the-second current data
  - combine aspects of OLAP databases with stream-processing systems
  - increasingly work with streaming systems to serve data for analytics and ML, so get familiar with this paradigm

BITS Pilani WILP

Ways to Serve Data for Analytics and ML(4)

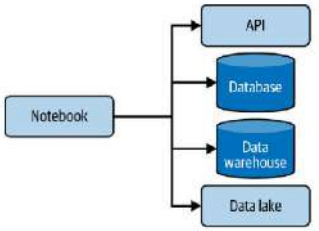
Query Federation

- Query federation pulls data from multiple sources, such as data lakes, RDBMSs, and data warehouses
  - becoming more popular as distributed query virtualization engines gain recognition as ways to serve queries without going through the trouble of centralizing data in an OLAP system
  - Trino and Presto and managed services such as Starburst
- End user might be querying several systems—OLTP, OLAP, APIs, filesystems, etc.
  - Instead of serving data from a single system, now serving data from multiple systems, each with its usage patterns, quirks, and nuances
  - If federated queries touch live production source systems, need to ensure that the federated query won't consume excessive resources in the source
- Federation allows ad hoc queries for performing exploratory analysis, blending data from various systems without the complexity of setting up data pipelines or ETL
  - Federated queries also provide read-only access to source systems, which is great when you don't want to serve files, database access, or data dump
  - The end user reads only the version of the data they're supposed to access and nothing more
  - Query federation is a great option to explore for situations where access and compliance are critical

Ways to Serve Data for Analytics and ML(5)

Serving Data in Notebooks

- Data scientists often use notebooks in their day-to-day work
  - for exploring data, engineering features, or training a model
- Jupyter Notebook, along with its next-generation iteration, JupyterLab
  - open source and can be hosted locally on a laptop, on a server, or through various cloud-managed services
  - Jupyter stands for Julia, Python, and R
- Data scientists will programmatically connect to a data source, such as an API, a database, a data warehouse, or a data lake
- In a notebook, all connections are created using
  - the appropriate built-in or imported libraries to load a file from a filepath
  - connect to an API endpoint
  - or make an ODBC connection to a database



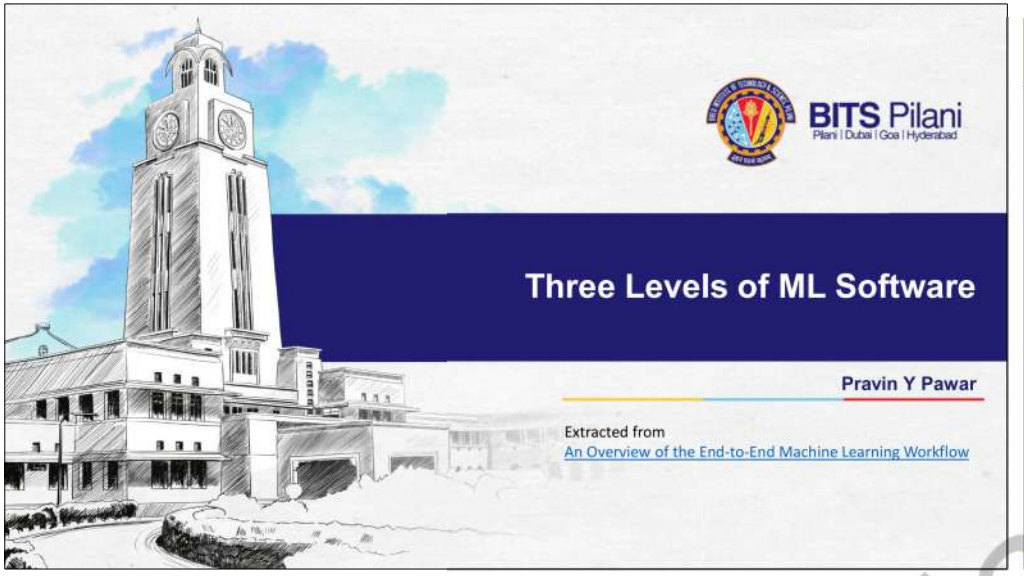
Reference

Smart Data Pipelines: Tools,Techniques, and Key Concepts

Thank You!

In our next session:

BITS Pilani WILP



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Three Levels of ML Software

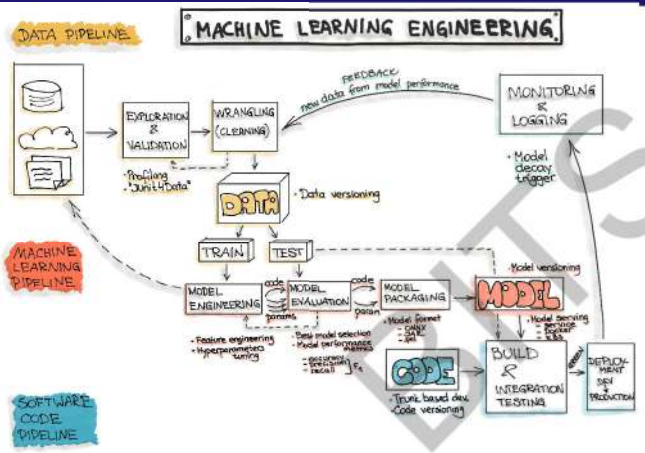
Pravin Y Pawar

Extracted from
An Overview of the End-to-End Machine Learning Workflow

Three levels of ML Software

- ML/AI is rapidly adopted by new applications and industries!
- Goal of a machine learning project is to build a statistical model by using collected data and applying machine learning algorithms
  - Yet building successful ML-based software projects is still difficult!
- Every ML-based software needs to manage three main assets:
  - Data
  - Model
  - and Code
- Essential technical methodologies - involved in the development of the Machine Learning-based software
  - Data Engineering: data acquisition & data preparation,
  - ML Model Engineering: ML model training & serving, and
  - Code Engineering : integrating ML model into the final product

Machine Learning Engineering



Data Engineering

- The initial step in any data science workflow is to acquire and prepare the data to be analyzed
  - Typically, data is being integrated from various resources and has different formats
- Data Preparation
  - according to Gartner "an iterative and agile process for exploring, combining, cleaning and transforming raw data into curated datasets for data integration, data science, data discovery and analytics/business intelligence (BI) use cases".
  - is reported to be the most expensive with respect to resources and time
- Data preparation is a critical activity in the data science workflow
  - important to avoid the propagation of data errors to the next phase, data analysis
  - would result in the derivation of wrong insights from the data

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Data Engineering(2)

Sequence of operations on the data leading to training and testing datasets for ML algorithms

- Data Ingestion
  - Collecting data by using various frameworks and formats, such as Spark, HDFS, CSV, etc
  - Might also include synthetic data generation or data enrichment.
- Data Exploration
  - Includes data profiling to obtain information about the content and structure of the data
  - The output of this step is a set of metadata, such as max, min, avg of values
- Data Validation
  - Operations are user-defined error detection functions, which scan the dataset in order to spot some errors
- Data Wrangling (Cleaning)
  - The process of re-formatting particular attributes and correcting errors in data, such as missing values imputation
- Data Labeling
  - The operation of the Data Engineering pipeline, where each data point is assigned to a specific category
- Data Splitting
  - Splitting the data into training, validation, and test datasets to be used during the core machine learning stages to produce the ML model

Model Engineering

The core of the ML workflow

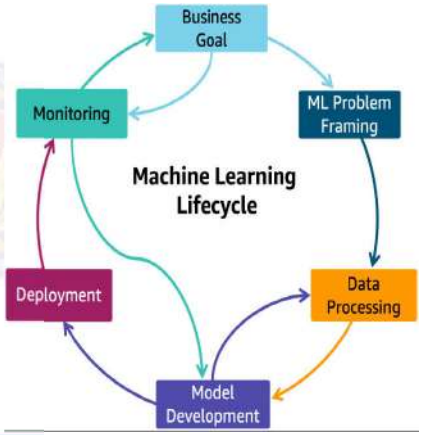
- Phase of writing and executing machine learning algorithms to obtain an ML model
- The Model Engineering pipeline includes a number of operations that lead to a final model:
- Model Training
  - The process of applying the machine learning algorithm on training data to train an ML model
  - includes feature engineering and the hyperparameter tuning for the model training activity
- Model Evaluation
  - Validating the trained model to ensure it meets original codified objectives before serving the ML model in production to the end-user
- Model Testing
  - Performing the final "Model Acceptance Test" by using the hold backtest dataset
- Model Packaging
  - The process of exporting the final ML model into a specific format (e.g. PMML, PFA, or ONNX)
  - which describes the model, in order to be consumed by the business application

Model Deployment

- Need to deploy model as part of a business application such as a mobile or desktop application
  - The ML models require various data points (feature vector) to produce predictions
  - The final stage of the ML workflow is the integration of the previously engineered ML model into existing software
- Includes the following operations:
- Model Serving
  - The process of addressing the ML model artifact in a production environment
- Model Performance Monitoring
  - The process of observing the ML model performance based on live and previously unseen data, such as prediction or recommendation
  - interested in ML-specific signals, such as prediction deviation from previous model performance
  - signals might be used as triggers for model re-training
- Model Performance Logging
  - Every inference request results in the log-record

Machine Learning Lifecycle

- Cyclic iterative process with instructions, and best practices to use across defined phases while developing an ML workload
  - adds clarity and structure for making a machine learning project successful
- The end-to-end machine learning lifecycle process includes the following phases:
  - Business goal identification
  - ML problem framing
  - Data processing (data collection, data preprocessing, feature engineering)
  - Model development (training, tuning, evaluation)
  - Model deployment (inference, prediction)
  - Model monitoring
- The phases of the ML lifecycle are not necessarily sequential in nature and can have feedback loops
  - to interrupt the cycle across the lifecycle phases



BITS Pilani WILP

Machine Learning Lifecycle(2)

Phases

- Business goal
  - should have a clear idea of the problem, and the business value to be gained by solving that problem
  - must be able to measure business value against specific business objectives and success criteria
- ML problem framing
  - the business problem is framed as a machine learning problem
  - what is observed and what should be predicted (known as a label or target variable)
  - Determining what to predict and how performance and error metrics must be optimized is a key step in this phase
- Data processing
  - Training an accurate ML model requires data processing to convert data into a usable format
  - includes collecting data, preparing data
  - and feature engineering that is the process of creating, transforming, extracting, and selecting variables from data
- Model development
  - consists of model building, training, tuning, and evaluation
  - includes creating a CI/CD pipeline that automates the build, train and release to staging and production environments
- Deployment
  - After a model is trained, tuned, evaluated and validated, one can deploy the model into production
  - can then make predictions and inferences against the model
- Monitoring
  - ensures model is maintaining a desired level of performance through early detection and mitigation

ML lifecycle phase — Business goal

- The most important phase particularly challenging when considering ML solutions because ML is a constantly evolving technology!
- After success criteria is defined, evaluate organization's ability to move toward that target
  - The target should be achievable and provide a clear path to production
  - Involve
    - all relevant stakeholders from the beginning to align them to this target
    - any new business processes that will result from this initiative
- Steps in this phase:
  - Understand business requirements
  - Form a business question
  - Review a project's ML feasibility and data requirements
  - Evaluate the cost of data acquisition, training, inference, and wrong predictions
  - Review proven or published work in similar domains, if available
  - Determine key performance metrics, including acceptable errors
  - Define the machine learning task based on the business question
  - Identify critical, must have features
  - Design small, focused POCs to validate all of the preceding
  - Evaluate if bringing in external data sources will improve model performance
  - Establish pathways to production
  - Consider new business processes that may come out of this implementation
  - Align relevant stakeholders with this initiative

ML lifecycle phase — ML problem framing

- Business problem is framed as a machine learning problem
  - what is observed and what should be predicted (known as a label or target variable)
  - Determining what to predict and how performance must be optimized is a key step in ML
- Steps in this phase:
  - Define criteria for a successful outcome of the project
  - Establish an observable and quantifiable performance metric for the project, such as accuracy
  - Help ensure business stakeholders understand and agree with the defined performance metrics
  - Formulate the ML question in terms of inputs, desired outputs, and the performance metric to be optimized
  - Evaluate whether ML is the right approach
  - Some business problems don't need ML, simple business rules can do a much better job
  - For other business problems, there might not be sufficient data to apply ML as a solution
  - Create a strategy to achieve the data sourcing and data annotation objective
  - Start with a simple model that is easy to interpret, and which makes debugging more manageable

ML lifecycle architecture diagram

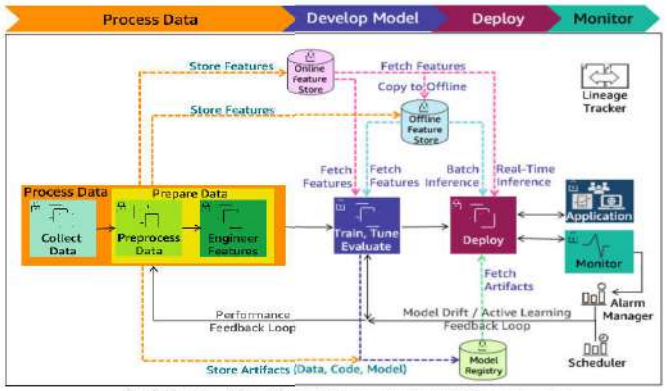
ML lifecycle with data processing sub-phases included

ML lifecycle with data processing sub-phases included

BITS Pilani WILP

ML lifecycle architecture diagram(2)

ML lifecycle with detailed phases and expanded components



ML lifecycle with detailed phases and expanded components

ML lifecycle architecture diagram(3)

Components

- Online/Offline feature store
  - reduces duplication and rerun of feature engineering code across teams and projects
  - Online store with low-latency retrieval capabilities is ideal for real-time inference
  - Offline store should maintain a history of feature values and is suited for training and batch scoring
- Model registry
  - A repository for storing ML model artifacts including trained model and related metadata (data, code, model)
  - enables lineage for ML models as it can act as a version control system
- Performance feedback loop
  - Automates model performance evaluation tasks initiated from the model development to data processing phase
- Model drift feedback loop
  - Automates model update re-training tasks initiated from the production deployment to data processing phase

ML lifecycle architecture diagram(4)

Components

- Alarm manager
  - receives the alerts from the model monitoring system
  - runs actions by publishing notifications to services that can deliver alerts to target applications to handle them
  - The model update re-training pipeline is one such target application
- Scheduler
  - A scheduler can initiate a re-training at business defined intervals
- Lineage tracker
  - enables reproducible machine learning experiences
  - enables re-creating the ML environment at a specific point-in-time, reflecting the versions of all resources and environments at that time
  - collects references to traceable data, model and infrastructure resource changes

ML lifecycle phase - Data processing

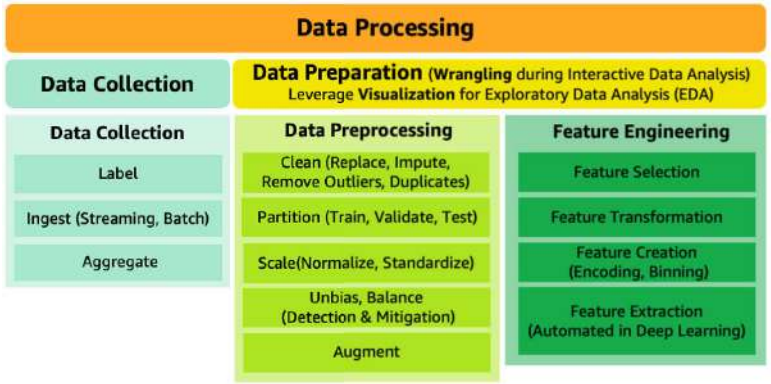
Need

- In ML workloads, the data (inputs and corresponding desired output) serves important functions including:
  - Defining the goal of the system: the output representation and the relationship of each output to each input, by means of input/output pairs
  - Training the algorithm that associates inputs to outputs
  - Measuring the performance of the trained model, and evaluating whether the performance target was met
  - Building baselines to monitor the performance of the models deployed to production
- Data processing includes data collection and data preparation
  - Data preparation includes data preprocessing and feature engineering
  - Data wrangling is data preparation during the interactive data analysis and model development
  - Data Visualization can help with exploratory data analysis (EDA)
  - EDA can help with understanding data, sanity checks, and validating the quality of the data
- The same sequence of data processing steps that you apply to the training data is also applied to the inference requests.

BITS Pilani WILP

ML lifecycle phase - Data processing(2)

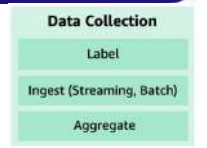
Data processing components

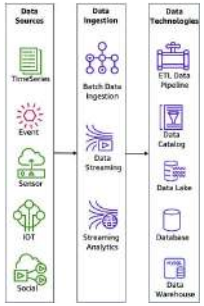


ML lifecycle phase - Data processing(3)

Data collection

- One of the first steps in the ML lifecycle is to identify what data is needed
  - Then evaluate the various means available for collecting that data to train model
- Activities in the data collection phase:
  - Label — Data for which already know the target answer is called labeled data
    - If labels are missing, then some effort is required to label it (manual or automated)
  - Ingest & Aggregate
    - Data collection includes ingesting and aggregating data from multiple data sources
- Some components of the ingest and aggregate:
  - Data sources
    - include time-series, events, sensors, IoT devices, and social networks, depending on the nature of the use case
  - Data ingestion
    - processes and technologies capture and store data on storage media
    - can occur in real-time using streaming technologies or historical mode using batch technologies
  - Data technologies
    - Data storage technologies vary from transactional (SQL) databases, to data lakes and data warehouses
    - ETL pipeline technology automates and orchestrates the data movement and transformations across cloud services and resources
    - A data lake technology enables storing and analyzing structured and unstructured data





ML lifecycle phase - Data processing(4)

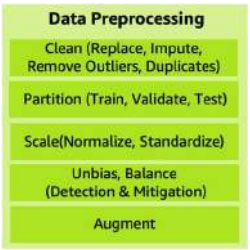
Data preparation

- ML models are only as good as the data that is used to train them
  - Ensure that suitable training data is available and is optimized for learning and generalization
- Data preparation includes
  - data preprocessing
  - feature engineering
- A key aspect to understanding data is to identify patterns which are often not evident with data in tables
  - Exploratory data analysis (EDA) with visualization tools can help quickly gain a deeper understanding of data
  - Prepare data using wrangler tools for interactive data analysis and model building
  - The no-code/low-code, automation, and visual capabilities improve the productivity and reduces the cost for interactive analysis

ML lifecycle phase - Data processing(5)

Data preprocessing

- Data preprocessing puts data into the right shape and quality for training
  - Many strategies: data cleaning, balancing, replacing, imputing, partitioning, scaling, augmenting and unbiasing
- Clean (replace, impute, remove outliers and duplicates)
  - Remove outliers and duplicates, replace inaccurate or irrelevant data
  - Correct missing data using imputation techniques that will minimize bias as part of data cleaning
- Partition
  - To prevent ML models from overfitting and evaluate trained model accurately, randomly split data into train, validate, and test sets
  - Care is needed to avoid data leakage
  - Data leakage happens when information from hold-out test dataset leaks into the training data
  - One way to avoid data leakage is to remove duplicates before splitting of the data



BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

ML lifecycle phase - Data processing(6)

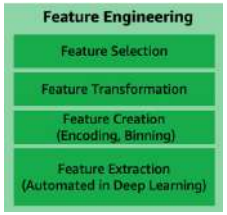
Data preprocessing

- Scale (normalize, standardize)
  - Having features on a similar scale close to normally distributed will ensure that each feature is equally important
  - make it easier for most ML algorithms including K-Means, KNN, PCA, gradient descent
  - Standardization better handles the outliers
- Unbias, balance (detection & mitigation)
  - Detecting and mitigating bias will help avoiding inaccurate model results
  - Biases are imbalances in the accuracy of predictions across different groups, such as age or income bracket
  - Biases can come from the data or algorithm used to train your model
- Augment
  - increases the amount of data artificially by synthesizing new data from existing data
  - can help regularize and reduce overfitting

ML lifecycle phase - Data processing(7)

Feature engineering

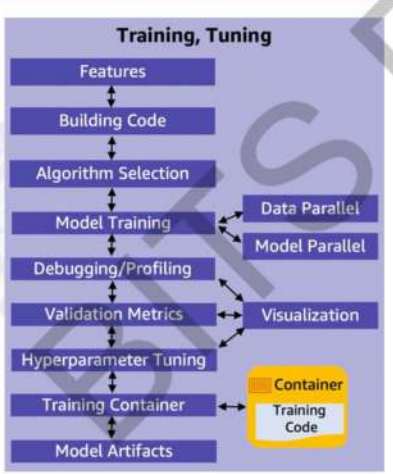
- Every unique attribute of the data is considered a feature
- Feature engineering is a process to select and transform variables when creating a predictive model using machine learning or statistical modeling
  - includes feature creation, feature transformation, feature extraction, and feature selection
  - With deep learning, the feature engineering is automated as part of the algorithm learning
- Feature creation
  - is creating new features from existing data to help with better predictions
  - Examples of feature creation techniques include: one-hot-encoding, binning, splitting, and calculated features
- Feature transformation and imputation
  - manage replacing missing features or features that are not valid
  - Some techniques include: forming Cartesian products of features, non-linear transformations (such as binning numeric variables into categories), and creating domain-specific features
- Feature extraction
  - involves reducing the amount of data to be processed using dimensionality reduction techniques
  - reduces the amount of memory and computing power required, while still accurately maintaining original data characteristics
  - Techniques include: PCA, ICA, and LDA
- Feature selection
  - process of selecting subset of extracted features which is relevant and contributes to minimizing the error rate of a trained model
  - Feature importance score and correlation matrix can be factors in selecting the most relevant features for model training



ML lifecycle phase – Model development

Model building, training, tuning

- In this phase, select a machine learning algorithm that is appropriate for problem and then train the ML model
  - provide the algorithm with the training data
  - set an objective metric for the ML model to optimize on
  - and set the hyperparameters to optimize the training process



ML lifecycle phase – Model development(2)

Model training, tuning activities

- Model training, tuning, and evaluation require prepared data and engineered features
- Features selection
  - Features are selected as part of the data processing after a bias strategy is implemented
- Building code
  - Model development includes building the algorithm and its supporting code
  - should support version control, and continuous build, test, and integration through a pipeline
- Algorithm selection
  - Selecting the right algorithm involves running many experiments with parameter tunings across available options
  - Factors to consider when evaluating each option can include accuracy, explainability, training/prediction time, memory requirements
- Model training (data training parallel, model training parallel)
  - The process of training an ML model involves providing an ML algorithm with training data to learn from
  - Distributed training enables splitting large models and training datasets across computing instances to reduce runtime to fraction of it takes to do manually
  - Model parallelism is the process of splitting a model up between multiple devices or nodes.
  - Data parallelism is the process of splitting the training set in mini-batches evenly distributed across nodes
    - each node only trains the model on a fraction of the total dataset

BITS Pilani WILP

ML lifecycle phase – Model development(3)

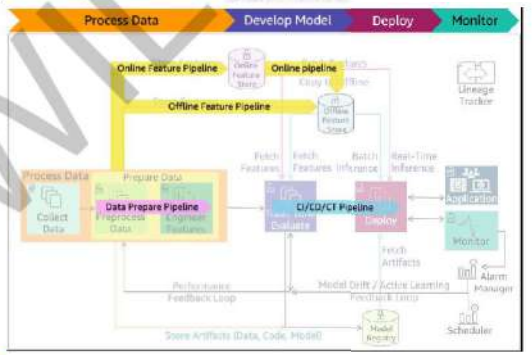
Model training, tuning activities

- Debugging/profiling
  - A machine learning training job can have problems including:
  - system bottlenecks, overfitting, saturated activation functions, and vanishing gradients. These problems can compromise model performance
  - A debugger provides visibility into the ML training process through monitoring, recording, and analyzing data
  - captures the state of a training job at periodic intervals
- Validation metrics
  - Typically, a training algorithm computes several metrics, such as loss and prediction accuracy
  - determine if the model is learning and generalizing well for making predictions on unseen data
  - Metrics reported by the algorithm depend on the business problem and the ML technique used
- Hyperparameter tuning
  - Settings that can be tuned to control the behavior of the ML algorithm are referred to as hyperparameters
  - The number and type of hyperparameters in ML algorithms are specific to each model
  - Examples of commonly used hyperparameters include: learning rate, number of epochs, hidden layers, hidden units, and activation functions
  - Hyperparameter tuning, or optimization, is the process of choosing the optimal hyperparameters for a learning algorithm
- Training code container
  - Create container images with training code and its entire dependency stack
  - will enable training machine learning algorithms and deploy models quickly and reliably at any scale
- Model artifacts
  - Model artifacts are the output that results from training a model
  - typically consist of trained parameters, a model definition that describes how to compute inferences, and other metadata
- Visualization
  - Enables exploring and understanding data during metrics validation, debugging, profiling, and hyperparameter tuning.

ML lifecycle phase – Model development(4)

ML lifecycle with pre-production pipelines

- The data prepare pipeline automates data preparation tasks
- The feature pipeline automates the storing, fetching, and copying of the features into and from online/offline store
- The CI/CD/CT pipeline automates the build, train, and release to staging and production environments



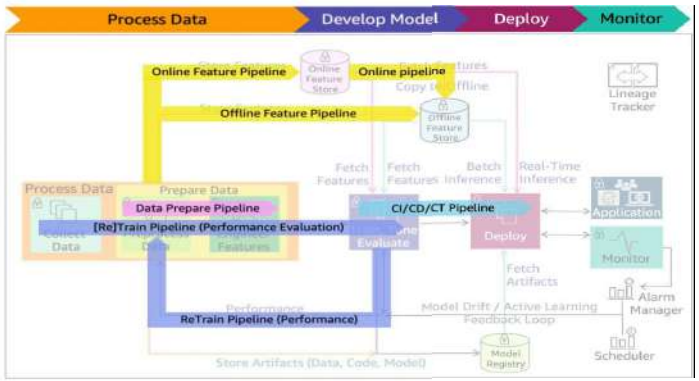
ML lifecycle phase – Model development(5)

Model evaluation

- After the model has been trained, evaluate it for its performance and accuracy
  - generate multiple models using different methods and evaluate the effectiveness of each model
  - for multiclass models, determine error rates for each class separately
- Can evaluate model using historical data (offline evaluation) or live data (online evaluation)
- Offline evaluation
  - the trained model is evaluated with a portion of the dataset that has been set aside as a holdout set
  - never used for model training or validation—it's only used to evaluate errors in the final model
  - The holdout data annotations must have high accuracy for the evaluation to make sense
  - Allocate additional resources to verify the accuracy of the holdout data
- Based on the evaluation results, might fine-tune the data, the algorithm, or both
  - may apply the concepts of data cleansing, preparation, and feature engineering again

ML lifecycle phase – Model development(6)

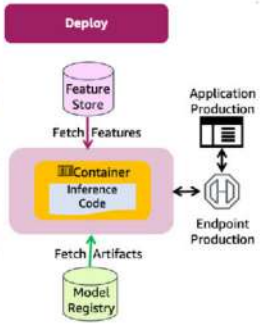
ML lifecycle with performance evaluation pipeline added



BITS Pilani WILP

ML lifecycle phase - Deployment

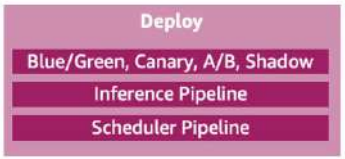
- After model have been trained, tuned, and evaluated, then can deploy the model into production
  - can make predictions and inferences against the deployed model
  - Use a manual governance process - ensures the model has fully been tested and evaluated before it's released to production
- Deploy production phase of the ML lifecycle
  - Features are retrieved from the feature store
  - Artifacts are taken from the model registry
  - Inference code used container from the container repository



ML lifecycle phase – Deployment(2)

Deployment main components

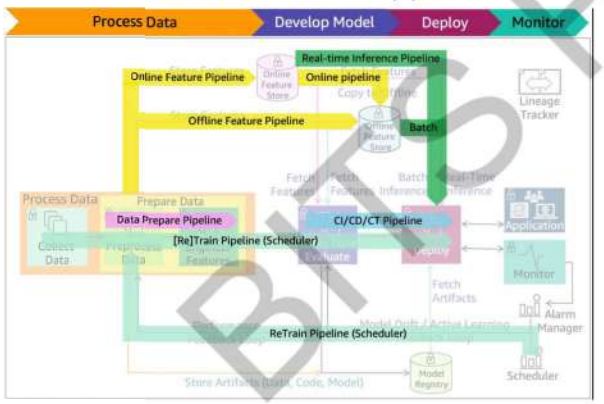
- Blue/green, canary, A/B, shadow deployment/testing
  - Deployment and testing strategies that reduce downtime and risks when releasing a new or updated version
- The blue/green deployment technique
  - provides two identical production environments
  - can use this technique when need to deploy a new version of the model to production
  - Once testing is done on the green environment, live application traffic is directed to the green environment
  - Blue environment is maintained as backup
- A canary deployment
  - first deploy the new release to a small group of users
  - Other users continue to use the previous version until satisfied with the new release
  - Then, can gradually roll the new release out to all users
- A/B testing strategy
  - enables deploying changes to a model
  - Direct a defined portion of traffic to the new model. Direct the remaining traffic to the old model
  - similar to canary testing, but has larger user groups and a longer time scale, typically days or even weeks
- Shadow deployment strategy
  - New version is available alongside the old version
  - The input data is run through both versions
  - The older version is used for servicing the production and the new one is used for testing and analysis



ML lifecycle phase – Deployment(3)

ML lifecycle with scheduler re-train, and batch/real-time Inference pipelines

- Inference pipeline
  - automates capturing of the prepared data, performing predictions and post-processing for real-time or batch inferences
- Scheduler pipeline
  - Deployed model is representative of the latest data patterns
  - Re-training at intervals can minimize the risk of data and concept drifts
  - A scheduler can initiate a re-training at business defined intervals
  - Data prepare, CI/CD/CT, and feature pipelines will also be active during this process



ML lifecycle phase – Monitoring

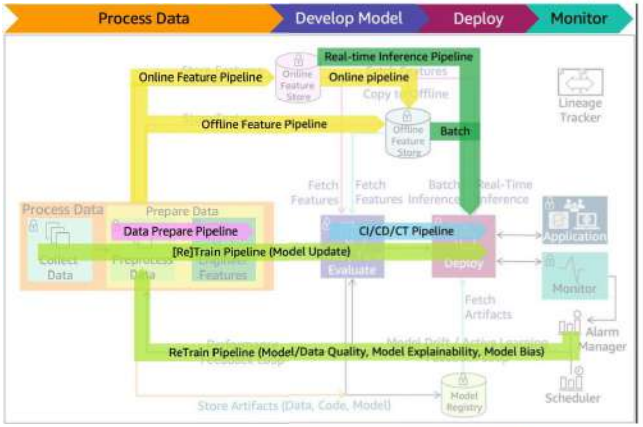
Post deployment monitor main components

- The model monitoring system must
  - capture data,
  - compare that data to the training set, define rules to detect issues
  - and send alerts
- Process repeats on a defined schedule, when initiated by an event, or when initiated by human intervention
  - The issues detected in the monitoring phase include: data quality, model quality, bias drift, and feature attribution drift
- Key components of monitoring including:
  - Model explainability
    - uses explainability to evaluate the soundness of the model and if the predictions can be trusted
  - Detect drift
    - detects data and concept drifts
    - Data drift is the significant changes to data distribution compared to data used for training
    - Concept drift is when the properties of target variables change
    - Any kind of drift will result in model performance degradation
    - will initiate an alert and send it to alarm manager system
  - Model update pipeline
    - If alarm manager identifies any violations, it launches the model update pipeline for a re-train
    - Data prepare, CI/CD/CT, and feature pipelines will also be active during this process

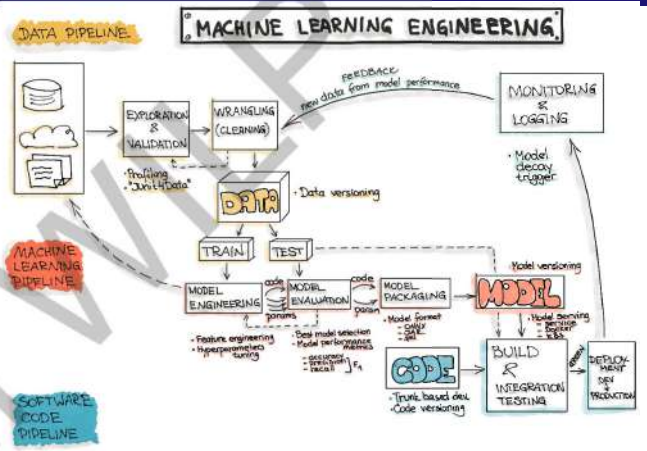
BITS Pilani WILP

ML lifecycle phase – Monitoring(2)

ML lifecycle with model update re-train and batch/real-time Inference pipelines



Machine Learning Workflow



Data: Data Engineering Pipelines

- Fundamental part of any machine learning workflow is Data
  - Collecting good data sets has a huge impact on the quality and performance of the ML model
- "Garbage In, Garbage Out",
  - In the ML context means that the ML model is only as good as data using which its built
  - Data indirectly influence the overall performance of the production system
  - The amount and quality of the data set are usually problem-specific and can be empirically discovered
- Data engineering is reported as heavily time-consuming
  - might spend the majority of time on a machine learning project constructing data sets, cleaning, and transforming data
  - includes a sequence of operations on the available data
  - final goal of these operations is to create training and testing datasets for the ML algorithms
- Stages of the data engineering pipeline
  - Data Ingestion
  - Exploration and Validation
  - Data Wrangling (Cleaning)
  - Data Splitting

Data Ingestion

- Collecting data by using various systems, frameworks and formats
  - such as internal/external databases, data marts, OLAP cubes, data warehouses, OLTP systems, Spark, HDFS etc
  - might also include synthetic data generation or data enrichment
- Contains actions that should be maximally automated:
  - Data Sources Identification: Find the data and document its origin (data provenance)
  - Space Estimation: Check how much storage space it will take
  - Space Location: Create a workspace with enough storage space
  - Obtaining Data: Get the data and convert them to a format that can be easily manipulated without changing the data itself
  - Back up Data: Always work on a copy of the data and keep the original dataset untouched
  - Privacy Compliance: Ensure sensitive information is deleted or protected (e.g., anonymized) to ensure GDPR compliance
  - Metadata Catalog: Start documenting the metadata of the dataset by recording the basic information about the size, format, aliases, last modified time, and access control lists
  - Test Data: Sample a test set, put it aside, and never look at it to avoid the "data snooping" bias

BITS Pilani WILP

Exploration and Validation

- Exploration includes data profiling to obtain information about the content and structure of the data
  - The output of this step is a set of metadata, such as max, min, avg of values
- Data validation operations are user-defined error detection functions, which scan the dataset to spot some errors
  - process of assessing the quality of the data by running dataset validation routines (error detection methods)
- Includes actions:
  - Use RAD tools: Using Jupyter notebooks is a good way to keep records of data exploration and experimentation
  - Attribute Profiling: Obtain and document the metadata about each attribute, such as
    - Name
    - Number of Records
    - Data Type (categorical, numerical, int/float, text, structured, etc.)
    - Numerical Measures (min, max, avg, median, etc. for numerical data)
    - Amount of missing values (or "missing value ratio" = Number of absent values/ Number of records)
    - Type of distribution (Gaussian, uniform, logarithmic, etc.)
  - Label Attribute Identification: For supervised learning tasks, identify the target attribute(s).
  - Data Visualization: Build a visual representation for value distribution
  - Attributes Correlation: Compute and analyze the correlations between attributes
  - Additional Data: Identify data that would be useful for building the model (go back to "Data Ingestion")

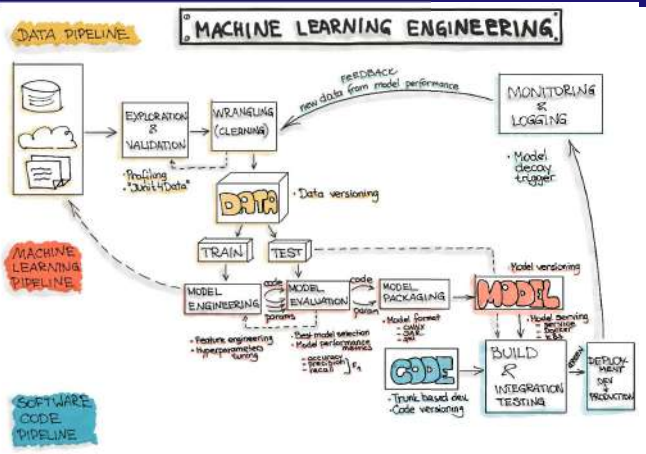
Data Wrangling (Cleaning)

- Is a Data preparation step where we programmatically wrangle data,
  - e.g., by re-formatting or re-structuring particular attributes that might change the form of the data's schema
  - recommended to write scripts or functions for all data transformations in the data pipeline to reuse all these functionalities on future data
- Includes:
  - Transformations: Identify the promising transformations you may want to apply
  - Outliers: Fix or remove outliers (optional)
  - Missing Values: Fill in missing values (e.g., with zero, mean, median) or drop their rows or columns
  - Not relevant Data: Drop the attributes that provide no useful information for the task (relevant for feature engineering)
  - Restructure Data: Might include the following operations (from the book "Principles of Data Wrangling")
    - Reordering record fields by moving columns
    - Creating new record fields through extracting values
    - Combining multiple record fields into a single record field
    - Filtering datasets by removing sets of records
    - Shifting the granularity of the dataset and the fields associated with records through aggregations and pivots

Data Splitting

- Splitting the data into
  - Training
  - Validation
  - Test
- datasets to be used during the core machine learning stages to produce the ML model

Machine Learning Workflow



BITS Pilani WILP

Model: Machine Learning Pipelines

- The core of the ML workflow is the phase of writing and executing machine learning algorithms to obtain an ML model
- The model engineering pipeline is
  - usually utilized by a data science team
  - includes a number of operations that lead to a final model
- Operations include
  - Model Training
  - Model Evaluation
  - Model Testing
  - Model Packaging
- Recommended to automate these steps as much as possible!

Model Training

Feature engineering

- The process of applying the machine learning algorithm on training data to train an ML model
  - includes feature engineering the hyperparameter tuning for the model training activity
- Feature engineering might include:
  - Discretize continuous features
  - Decompose features (e.g., categorical, date/time, etc.)
  - Add transformations of features (e.g., log(x), sqrt(x), x2, etc.)
  - Aggregate features into promising new features
  - Feature scaling: Standardize or normalize features
  - New features should be added quickly to get fast from a feature idea to the feature running in production

Model Training(2)

Model Engineering

- Might be an iterative process and include the following workflow:
  - Every ML model specification (code that creates an ML model) should go through a code review and be versioned.
  - Train many ML models from different categories
    - (e.g., linear regression, logistic regression, k-means, naive Bayes, SVM, Random Forest, etc.) using standard parameters
  - Measure and compare their performance
    - For each model, use N-fold cross-validation and compute the mean and standard deviation of the performance measure on the N folds.
  - Error Analysis
    - analyze the types of errors the ML models make.
  - Consider further feature selection and engineering
    - Identify the top three to five most promising models, preferring models that make different types of errors.
    - Hyperparameters tuning by using cross-validation.
    - Consider Ensemble methods such as majority vote, bagging, boosting,
    - Combining ML models should produce better performance than running them individually

Model Evaluation, Testing and Packaging

- Model Evaluation
  - Validate the trained model to ensure it meets original business objectives before serving the ML model in production to the end-user
- Model Testing
  - Once the final ML model is trained, its performance needs to be measured
  - by using the hold-back test dataset to estimate the generalization error by performing the final “Model Acceptance Test”
- Model Packaging
  - The process of exporting the final ML model into a specific format (e.g. PMML, PFA, or ONNX)
  - which describes the model to be consumed by the business application

BITS Pilani WILP

ML Model serialization formats

- Various formats to distribute ML models
  - In order to achieve a distributable format, the ML model should be present and should be executable as an independent asset
  - means that the ML models should work outside of the model-training environment
- Two flavors
  - Language-agnostic exchange formats
  - Vendor-specific exchange formats
- Language-agnostic exchange formats
  - Amalgamation is the simplest way to export an ML model as portable - model and all necessary code to run are bundled as one package
  - Usually, a single source code file that can be compiled on nearly any platform as a standalone program
  - Create a standalone version of an ML model by using SKompiler to transform trained Scikit-learn models into other forms,
    - such as SQL queries, Excel formulas,
    - Portable Format for Analytics (PFA) files
    - or SymPy expressions - translated to code in a variety of languages, such as C, Javascript, Rust, Julia, etc.

ML Model serialization formats(2)

Commom Language-agnostic exchange formats

- PMML
  - format for model serving based on XML with the file extension .pmml - standardized by the Data Mining Group (DMG)
  - Basically, .ppml describes a model and pipeline in XML
  - The PMML supports not all of the ML algorithms, and its usage in open source-driven tools is limited due to licensing issues
- PFA (Portable Format for Analytics) - designed as a replacement for PMML
  - is a string of JSON-formatted text that describes an executable called a scoring engine
  - each engine has a well-defined input, a well-defined output, and functions for combining inputs to construct the output in an expression-centric syntax tree
  - To run ML models as PFA files, need a PFA-enabled environment
- ONNX (Open Neural Network eXchange)
  - ML framework independent file format
  - created to allow any ML tool to share a single model format - supported by many big tech companies such as Microsoft, Facebook, and Amazon
  - Once the ML model is serialized in the ONNX format,
  - can be consumed by onnx-enabled runtime libraries (also called inference engines)and then make predictions

ML Model serialization formats(3)

Vendor-specific exchange formats

- Scikit-Learn
  - saves models as pickled python objects, with a .pkl file extension
- H2O
  - allows to convert the models to either POJO (Plain Old Java Object) or MOJO (Model Object, Optimized)
- SparkML models
  - can be saved in the MLeap file format and served in real-time using an MLeap model server
  - The MLeap runtime is a JAR that can run in any Java application
  - supports Spark, Scikit-learn, and Tensorflow for training pipelines and exporting them to an MLeap Bundle
- TensorFlow
  - saves models as .pb files; which is the protocol buffer files extension
- PyTorch
  - serves models by using their proprietary Torch Script as a .pt file
  - can be served from a C– application
- Keras
  - saves a model as a .h5 file, which is known as a data file saved in the Hierarchical Data Format (HDF)- contains multidimensional arrays of data
- Apple
  - has its proprietary file format with the extension .mlmodel to store models embedded in iOS applications
  - The Core ML framework has native support for Objective-C and Swift programming languages
  - Applications trained in other ML frameworks, such as TensorFlow, Scikit-Learn, and other frameworks need to use tools like such as coremltools and Tensorflow converter to translate their ML model files to the .mlmodel format for use on iOS

Different forms of ML workflows

- Operating an ML model might assume several architectural styles
- Primarily Four architectural patterns which are classified along two dimensions:
  - ML Model Training
  - ML Model Prediction
- For sake of simplicity disregard the third dimension - ML Model Type
  - which denotes the type of machine learning algorithm such as
    - Supervised
    - Unsupervised
    - Semi-supervised
    - and Reinforcement Learning

BITS Pilani WILP

Model Training Patterns

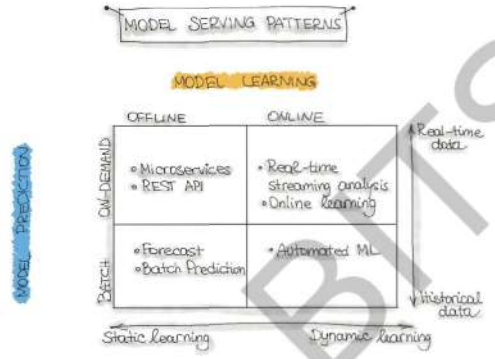
- Two ways to perform ML Model Training:
  - Offline learning
  - Online learning
- Offline learning (aka batch or static learning)
  - The model is trained on a set of already collected data
  - After deploying to the production environment, the ML model remains constant until it re-trained
  - Model will see a lot of real-live data and becomes stale - 'model decay' and should be carefully monitored
- Online learning (aka dynamic learning)
  - The model is regularly being re-trained as new data arrives, e.g. as data streams
  - Usually the case for ML systems that use time-series data, such as sensor, or stock trading data to accommodate the temporal effects in the ML model

Model Prediction Patterns

- Two modes to denote the mechanics of the ML model to makes predictions
  - Batch predictions
  - Real-time predictions
- Batch predictions
  - The deployed ML model makes a set of predictions based on historical input data
  - often sufficient for data that is not time-dependent, or when it is not critical to obtain real-time predictions as output
- Real-time predictions (aka on-demand predictions)
  - Predictions are generated in real-time using the input data that is available at the time of the request

ML architecture patterns

Forecast, Web-Service, Online Learning, and AutoML



Forecast

- Widely spread in academic research or data science education (e.g., Kaggle or DataCamp)
  - used to experiment with ML algorithms and data as it is the easiest way to create a machine learning system
  - not very useful in an industry setting for production systems (e.g. mobile applications)
- Usually involves steps
  - take an available dataset
  - train the ML model
  - run this model on another (mostly historical) data
  - makes predictions

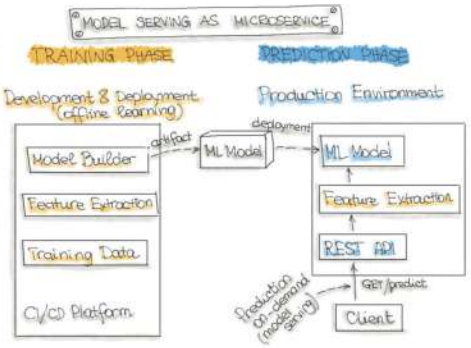
BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Web-Service (or Microservices)

Architecture for wrapping trained models as deployable services

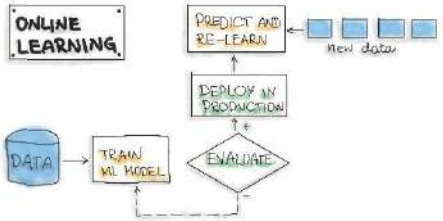
- The most commonly described deployment architecture for ML models
- The web service takes input data and outputs a prediction for the input data points
  - Model is trained offline on historical data, but it uses real-live data to make predictions
  - Model remains constant until it is re-trained and re-deployed into the production system.
- The difference from a forecast (batch predictions) is that
  - the ML model runs near real-time
  - handles a single record at a time instead of processing all the data at once



Online Learning (Real-time streaming analytics)

Most dynamic way to embed machine learning into a production system

- Confusing name because the core learning or ML model training is usually not performed on the live system
  - call it incremental learning- term online learning is already established within the ML community
- In this type of ML workflow
  - ML learning algorithm is continuously receiving a data stream, either as single data points or in small groups called mini-batches
  - System learns about new data on the fly as it arrives, so the ML model is incrementally being re-trained with new data
  - Continually re-trained model is instantly available as a web service
  - Technically works well with the lambda architecture in big data systems
  - Model would typically run as a service on a Kubernetes cluster or similar
- A big difficulty with the online learning system in production is that
  - if bad data is entering the system, the ML model, as well as the whole system performance, will increasingly decline

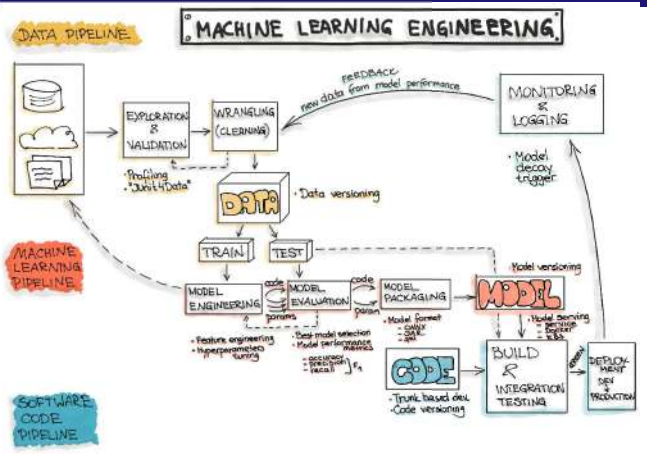


AutoML

Sophisticated version of online learning

- AutoML is getting a lot of attention
  - considered the next advance for enterprise ML
  - promises training ML models with minimal effort and without machine learning expertise
  - User needs to provide data, and the AutoML system automatically selects an ML algorithm and configures the selected algorithm
  - very experimental way to implement ML workflows
  - usually provided by big cloud providers, such as Google or MS Azure
  - Instead of updating the model, need to execute an entire ML model training pipeline in production that results in new models on the fly

Machine Learning Workflow



BITS Pilani WILP

innovate achieve lead

Code: Deployment Pipelines

- The final stage of delivering an ML project includes the following three steps:
  - Model Serving
  - Model Performance Monitoring
  - Model Performance Logging
- Model Serving
  - The process of deploying the ML model in a production environment
- Model Performance Monitoring
  - The process of observing the ML model performance based on live and previously unseen data, such as prediction or recommendation
  - interested in ML-specific signals, such as prediction deviation from previous model performance
  - signals might be used as triggers for model re-training
- Model Performance Logging
  - Every inference request results in a log-record.

Model Serving Patterns

- Three components should be considered when ML model is served in a production environment
  - The inference is the process of getting data to be ingested by a model to compute predictions
  - requires a model, an interpreter for the execution, and input data
- Deploying an ML system to a production environment includes two aspects,
  - First deploying the pipeline for automated retraining and ML model deployment
  - Second, providing the API for prediction on unseen data
- Model serving is a way to integrate the ML model in a software system
- Five patterns to put the ML model in production:
  - Model-as-Service
  - Model-as-Dependency
  - Precompute
  - Model-on-Demand
  - and Hybrid-Serving

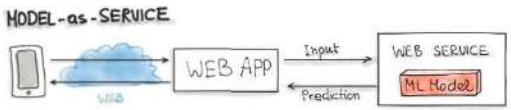
Model Serving Patterns(2)

Machine Learning Model Serving Taxonomy

Machine Learning Model Serving Taxonomy			
	ML Model		
Service & Versioning	Together with the consuming application	Independent from the consuming application	
Compile/ Runtime Availability	Build & runtime available	Available remotely through REST API/RPC	Available at the runtime scope
Serving Patterns	"Model-as-Dependency"	"Model-as-Service"	"Precompute" and "Model on Demand"
	Hybrid Model Serving (Federated Learning)		

Model-as-Service

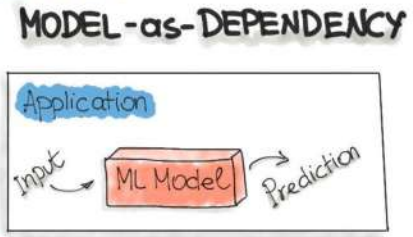
- A common pattern for wrapping an ML model as an independent service
  - can wrap the ML model and the interpreter within a dedicated web service
  - applications can request through a REST API or consume as a gRPC service
- Used for various ML workflows
  - Forecast
  - Web Service
  - Online Learning



BITS Pilani WILP

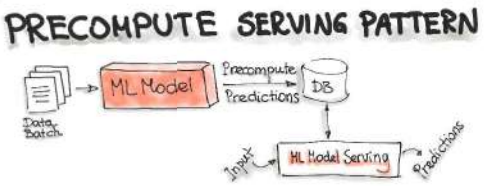
Model-as-Dependency

- Probably the most straightforward way to package an ML model
  - mostly used for implementing the Forecast pattern.
- A packaged ML model is considered as a dependency within the software application
  - For example, the application consumes the ML model like a conventional jar dependency by invoking the prediction method and passing the values
  - The return value of such method execution is some prediction that is performed by the previously trained ML model



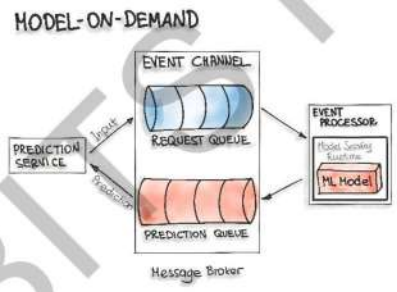
Precompute Serving Pattern

- Tightly related to the Forecast ML workflow
- Use an already trained ML model and precompute the predictions for the incoming batch of data
  - Resulting predictions are persisted in the database
  - For any input request, query the database to get the prediction result



Model-on-Demand

- Treats the ML model as a dependency that is available at runtime
  - contrary to the Model-as-Dependency pattern, has its own release cycle and is published independently
- Message-broker architecture is typically used for such on-deman model serving
  - contains two main types of architecture components:
    - a broker component
    - an event processor component
  - Broker component is the central part that contains the event channel that are utilised within the event flow
  - The event channels, which are enclosed in the broker component, ar message queues
  - Message broker allows one process to write prediction-requests in a input queue
  - Event processor contains the model serving runtime and the ML model
    - connects to the broker, reads these requests in batch from the queue an sends them to the model to make the predictions
- The model serving process runs the prediction generation on the input data
  - writes the resulted predictions to the output queue
  - queued prediction results are pushed to the prediction service that initiated the prediction request



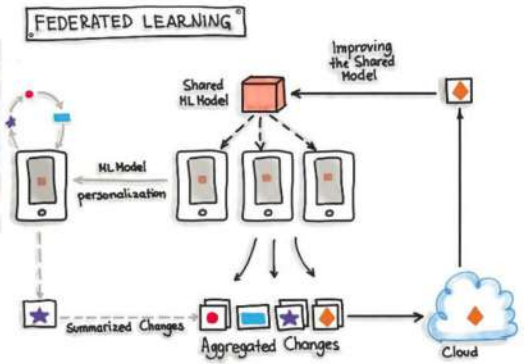
Hybrid-Serving (Federated Learning)

- Unique in the way it does, there is not only one model that predicts the outcome, but there are also lots of it
  - Exactly spoken there are as many models as users exist, in addition to the one that's held on a server
- Start with the unique model, the one on the server
  - Model on the server-side is trained only once with the real-world data
  - sets the initial model for each user
  - a relatively general trained model so it fits for the majority of users
- On the other side, there are the user-side models - really unique models
  - possible for the devices to train their own models due to hardware enhancements
  - devices will train their own highly specialized model for their own user
  - Once in a while, the devices send their already trained model data (not the personal data) to the server
- Then the server model will be adjusted
  - the actual trends of the whole user community will be covered by the model
  - this updated server model is set to be the new initial model that all devices are using

BITS Pilani WILP

Hybrid-Serving (Federated Learning) 2

- Not having any downsides for the users, while the server model gets updated, this happens only when the device is idle, connected to WiFi and charging
  - testing is done on the devices, therefore the newly adopted model from the server is sent to the devices and tested for functionality
- Big benefit
  - data used for training and testing, which is highly personal, never leaves the devices while still capturing all data that is available
  - possible to train highly accurate models while not having to store tons of (probably personal) data in the cloud
- Constraints
  - mobile devices are less powerful
  - the training data is distributed across millions of devices and these are not always available for training
  - Exactly for this TensorFlow Federated (TFF) has been created - lightweight form of TensorFlow

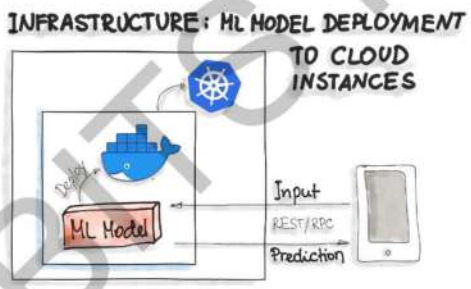


Deployment Strategies

- Common ways for wrapping trained models as deployable services, namely deploying ML models as
  - Docker Containers to Cloud Instances
  - Serverless Functions

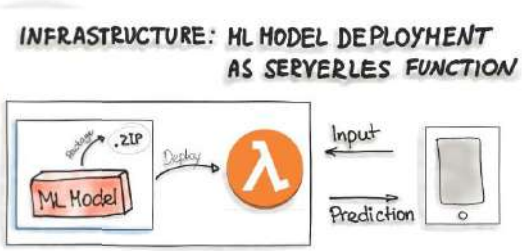
Deploying ML Models as Docker Containers

- No standard, open solution to ML model deployment!
- Containerization becomes the de-facto standard for delivery
  - as ML model inference being considered stateless, lightweight, and idempotent
  - means deploy a container that wraps an ML model inference code
- Docker is considered to be de-facto standard containerization technology
  - for on-premise, cloud, or hybrid deployments
- One ubiquitous way is to package the whole ML tech stack (dependencies) and the code for ML model prediction into a Docker container
  - Then Kubernetes or an alternative (e.g. AWS Fargate) does the orchestration
  - The ML model functionality, such as prediction, is then available through a REST API (e.g. implemented as Flask application)



Deploying ML Models as Serverless Functions

- Various cloud vendors already provide machine-learning platforms - can deploy model with their services
  - Amazon AWS Sagemaker, Google Cloud AI Platform, Azure Machine Learning Studio, and IBM Watson Machine Learning
  - Commercial cloud services also provide containerization of ML models such as AWS Lambda and Google App Engine servlet host
- In order to deploy an ML model as a serverless function,
  - the application code and dependencies are packaged into .zip files, with a single entry point function
  - could be managed by major cloud providers such as Azure Functions, AWS Lambda, or Google Cloud Functions
  - However, attention should be paid to possible constraints of the deployed artifacts such as the size of the artifacts



BITS Pilani WILP

Data science and ML in Real World

Gaining importance in businesses

- Becoming core capabilities for solving complex real-world problems,
  - transforming industries, and delivering value in all domains
- Currently, the ingredients for applying effective ML are :
  - Large datasets
  - Inexpensive on-demand compute resources
  - Specialized accelerators for ML on various cloud platforms
  - Rapid advances in different ML research fields (such as computer vision, natural language understanding, and recommendations AI systems)
- Many businesses are investing in their data science teams and ML capabilities
  - to develop predictive models that can deliver business value to their users

Real Challenge

Continuously deploy and operate

- Data scientists can implement and train an ML model with predictive performance on an offline holdout dataset
  - Real challenge isn't building an ML model, the challenge is building an integrated ML system and to continuously operate it in production
- MLOps is an ML engineering culture and practice that aims at unifying ML system development (Dev) and ML system operation (Ops)
- Practicing MLOps means that advocating for automation and monitoring at all steps of ML system construction, including
  - integration,
  - testing,
  - releasing,
  - deployment and infrastructure management

Elements of ML System

Vast and Complex

- Can be many pitfalls in operating ML-based systems in production
  - Some are summarized in Machine Learning: The high-interest credit card of technical debt
- Only a small fraction of a real-world ML system is composed of the ML code
  - Required surrounding elements are vast and complex
- Rest of the system is composed of configuration
  - automation,
  - data collection,
  - data verification
  - testing and debugging,
  - resource management,
  - model analysis,
  - process and metadata management,
  - serving infrastructure,
  - and monitoring.
- To develop and operate complex systems like these,
  - need to apply DevOps principles to ML systems (MLOps)

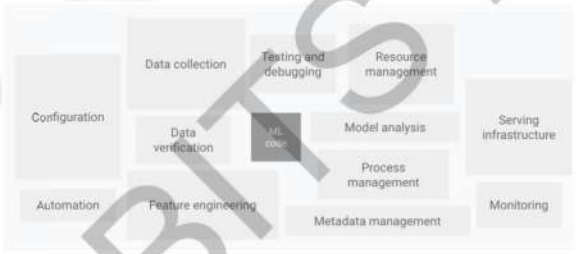


Figure 1. Elements for ML systems. Adapted from Hidden Technical Debt in Machine Learning Systems (https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf).

DevOps versus MLOps

DevOps - A popular practice in developing and operating large-scale software systems

- Provides benefits such as
  - shortening the development cycles,
  - increasing deployment velocity,
  - and ensures dependable releases
- To achieve these benefits, introduces two concepts in the software system development:
  - Continuous Integration (CI)
  - Continuous Delivery (CD)
- Continuous Integration (CI)
  - practice of merging all developers' working copies to a shared mainline several times a day
  - the practice of frequently integrating one's new or changed code with the existing code repository
    - should occur frequently enough that no intervening window remains between commit and build
    - such that no errors can arise without developers noticing them and correcting them immediately
- Continuous Delivery (CD)
  - a software engineering approach in which teams produce software in short cycles,
  - ensuring that the software can be reliably released at any time and, following a pipeline through a "production-like environment", without doing so manually
  - aims at building, testing, and releasing software with greater speed and frequency
  - helps reduce the cost, time, and risk of delivering changes by allowing for more incremental updates to applications in production
  - A straightforward and repeatable deployment process is important for continuous delivery

BITS Pilani WILP

DevOps versus MLOps(2)

ML systems differ from other software systems - Phases

- Team skills
  - In an ML project, the team usually includes data scientists or ML researchers,
    - who focus on exploratory data analysis, model development, and experimentation
  - might not be experienced software engineers who can build production-class services
- Development
  - ML is experimental in nature - need to try different features, algorithms, modeling techniques, and parameter configurations
  - Challenge is tracking what worked and what didn't, and maintaining reproducibility while maximizing code reusability
- Testing
  - More involved than testing other software systems
  - In addition to typical unit and integration tests, need data validation, trained model quality evaluation, and model validation
- Deployment
  - In ML systems, deployment isn't as simple as deploying an offline-trained ML model as a prediction service
  - Required to deploy a multi-step pipeline to automatically retrain and deploy model
    - pipeline adds complexity and requires to automate steps that are manually done before deployment by data scientists to train and validate new models
- Production
  - ML models can have reduced performance not only due to suboptimal coding, but also due to constantly evolving data profiles
  - Models can decay in more ways than conventional software systems - need to consider this degradation
  - Need to track summary statistics of data and monitor the online performance of model
    - to send notifications or roll back when values deviate from expectations

DevOps versus MLOps(3)

ML systems differ from other software systems - Porcess

- An ML system is a software system
  - similar practices apply to help guarantee reliably to build and operate ML systems at scale
- ML and other software systems are similar wrt
  - continuous integration of source control, unit testing, integration testing
  - continuous delivery of the software module or the package
- However, in ML, there are a few notable differences:
- CI is no longer only about testing and validating code and components
  - but also testing and validating data, data schemas, and models
- CD is no longer about a single software package or a service,
  - but a system (an ML training pipeline) that should automatically deploy another service (model prediction service)
- CT is a new property, unique to ML systems
  - concerned with automatically retraining and serving the models

Data science steps for ML

Steps can be completed manually or can be completed by an automatic pipeline

- Data extraction
  - select and integrate the relevant data from various data sources for the ML task.
- Data analysis
  - perform exploratory data analysis
- Data preparation
  - prepare date for the ML task - involves data cleaning, data transformations splitting and feature engineering
- Model training
  - implement different algorithms with the prepared data to train various ML models
- Model evaluation
  - evaluate on a holdout test set to assess the model quality
- Model validation
  - confirm whether model is adequate for deployment — predictive performance is better than a certain baseline
- Model serving
  - deploy to a target environment to serve predictions
- Model monitoring
  - monitor models predictive performance to potentially invoke a new iteration in the ML process

Maturity of the ML process

MLOps levels

- The level of automation of steps defines the maturity of the ML process
  - reflects the velocity of training new models given new data or training new models given new implementations
- Three levels of MLOps
  - starting from the most common level, which involves no automation, up to automating both ML and CI/CD pipelines
- MLOps level 0: Manual process
- MLOps level 1: ML pipeline automation
- MLOps level 2: CI/CD pipeline automation

BITS Pilani WILP

MLOps level 0: Manual process

Data scientists and ML researchers builds state-of-the-art models, but their process for building and deploying ML models is entirely manual

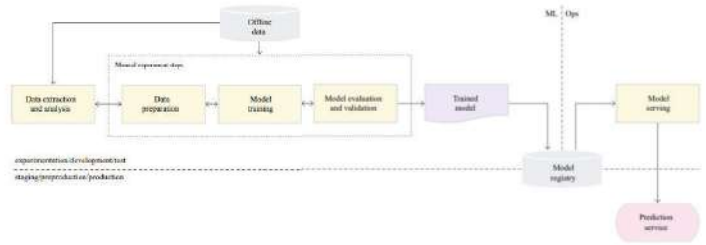


Figure 2. Manual ML steps to serve the model as a prediction service.

MLOps level 0: Manual process(2)

Characteristics

- Manual, script-driven, and interactive process
  - Every step is manual, including data analysis, data preparation, model training, and validation
  - Requires manual execution of each step, and manual transition from one step to another
  - usually driven by experimental code that is written and executed in notebooks by data scientist interactively, until a workable model is produced
- Disconnection between ML and operations
  - Separates data scientists who create the model and engineers who serve the model as a prediction service
  - Data scientists hand over a trained model as an artifact to the engineering team to deploy on their API infrastructure
    - can include putting the trained model in a storage location, checking the model object into a code repository, or uploading it to a models registry
  - Engineers who deploy the model need to make the required features available in production for low-latency serving
    - can lead to training-serving skew
- Infrequent release iterations
  - Assumes that data science team manages a few models that don't change frequently

MLOps level 0: Manual process(3)

Characteristics(2)

- No CI
  - Because few implementation changes are assumed, CI is ignored
  - Usually, testing the code is part of the notebooks or script execution
  - Scripts and notebooks that implement the experiment steps are source controlled
    - produce artifacts such as trained models, evaluation metrics, and visualizations
- No CD
  - Because there aren't frequent model version deployments, CD isn't considered
- Deployment refers to the prediction service
  - Concerned only with deploying the trained model as a prediction service
    - (for example, a microservice with a REST API)
- Lack of active performance monitoring
  - Doesn't track or log the model predictions and actions, which are required in order to detect model performance degradation and other model behavioral drifts

MLOps level 0: Manual process(4)

Challenges

- Common in many businesses that are beginning to apply ML to their use cases
  - manual, data-scientist driven process might be sufficient when models are rarely changed or trained
  - in practice, models often break when they are deployed in the real world
- To address these challenges and to maintain model's accuracy in production, requires to:
- Actively monitor the quality of model in production
  - to detect performance degradation and model staleness
  - acts as a cue to a new experimentation iteration and (manual) retraining of the model on new data
- Frequently retrain production models
  - need to retrain model with the most recent data - to capture the evolving and emerging patterns
- Continuously experiment with new implementations to produce the model
  - To harness the latest ideas and advances in technology
  - To try out new implementations such as feature engineering, model architecture, and hyper parameters
- To address the challenges of this manual process, MLOps practices for CI/CD and CT are helpful!

BITS Pilani WILP

MLOps level 1: ML pipeline automation

Aim to perform continuous training of the model by automating the ML pipeline

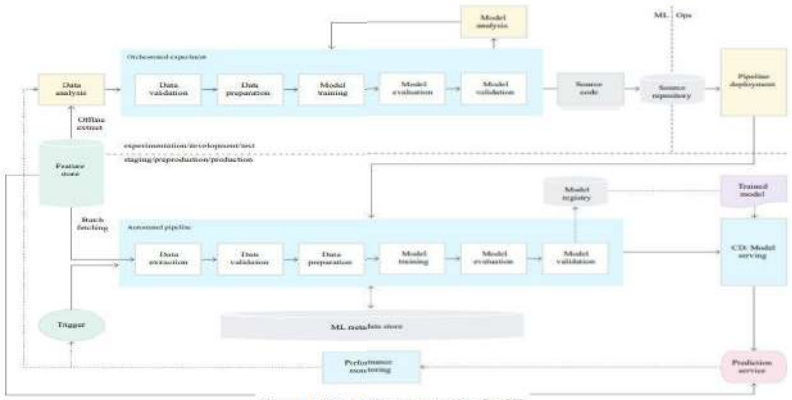


Figure 3. ML pipeline automation for CT.

MLOps level 1: ML pipeline automation(2)

Characteristics

- Rapid experiment
  - Steps of the ML experiment are orchestrated
  - Transition between steps is automated, which leads to rapid iteration of experiments and better readiness to move the whole pipeline to production
- CT of the model in production
  - Model is automatically trained in production using fresh data based on live pipeline triggers
- Experimental-operational symmetry
  - The pipeline implementation that is used in the development or experiment environment is used in the preproduction and production environment
  - key aspect of MLOps practice for unifying DevOps!
- Continuous delivery of models
  - ML pipeline in production continuously delivers prediction services to new models that are trained on new data
  - The model deployment step, which serves the trained and validated model as a prediction service for online predictions, is automated

MLOps level 1: ML pipeline automation(3)

Characteristics(2)

- Modularized code for components and pipelines
  - To construct ML pipelines, components need to be reusable, composable, and potentially shareable across ML pipelines
  - Source code for components must be modularized
  - In addition, components should ideally be containerized to do the following:
    - Decouple the execution environment from the custom code runtime
    - Make code reproducible between development and production environments
    - Isolate each component in the pipeline
- Pipeline deployment
  - In level 0, a trained model is deployed as a prediction service to production
  - In level 1, a whole training pipeline is deployed
    - which automatically and recurrently runs to serve the trained model as the prediction service

MLOps level 1: ML pipeline automation(4)

Data validation

- When ML pipeline is deployed to production, one or more of the triggers executes the pipeline
- Pipeline expects new, live data to produce a new model version that is trained on the new data
- Automated data validation and model validation steps are required in the production pipeline
- Data validation
  - Required before model training to decide whether to retrain the model or stop the execution of the pipeline
  - automatically make a decision if the following was identified by the pipeline
  - Data schema skews
    - presence of anomalies in the input data, which means that the downstream pipeline steps, including data processing and model training, receives data that doesn't comply with the expected schema
    - should stop the pipeline so the data science team can investigate
      - include receiving unexpected features, not receiving all the expected features, or receiving features with unexpected values
  - Data values skews
    - significant changes in the statistical properties of data, which means that data patterns are changing
    - need to trigger a retraining of the model to capture these changes

BITS Pilani WILP

MLOps level 1: ML pipeline automation(5)

Model validation

- Model validation
  - Occurs after successfully train the model given the new data to evaluate and validate the model before it's promoted to production
- Consists of
  - Producing evaluation metric values using the trained model on a test dataset to assess the model's predictive quality
  - Comparing the evaluation metric values produced by newly trained model to the current model,
  - Making sure that the performance of the model is consistent on various segments of the data
  - Making sure to test model for deployment, including infrastructure compatibility and consistency with the prediction service API
  - In addition to offline model validation, a newly deployed model undergoes online model validation
    - in a canary deployment or an A/B testing setup — before it serves prediction for the online traffic

MLOps level 1: ML pipeline automation(6)

Feature store

- An optional additional component - centralized repository to standardize the definition, storage, and access of features for training and serving
- Provide an API
  - for both high-throughput batch serving and low-latency real-time serving for the feature values
  - to support both training and serving workloads
- Helps data scientists to:
  - Discover and reuse available feature sets for their entities, instead of re-creating the same or similar ones
  - Avoid having similar features that have different definitions by maintaining features and their related metadata
  - Serve up-to-date feature values from the feature store
  - Avoid training-serving skew by using the feature store as the data source for experimentation, continuous training, and online serving
  - This approach makes sure that the features used for training are the same ones used during serving:
    - For experimentation, data scientists can get an offline extract from the feature store to run their experiments.
    - For continuous training, the automated ML training pipeline can fetch a batch of the up-to-date feature values of the dataset that are used for the training task.
    - For online prediction, the prediction service can fetch in a batch of the feature values related to the requested entity, such as customer demographic features, product features, and current session aggregation features

MLOps level 1: ML pipeline automation(7)

Metadata management

- Information about each execution of the ML pipeline is recorded in order to help
  - with data and artifacts lineage, reproducibility, and comparisons
  - to debug errors and anomalies
- Each time pipeline is executed, the ML metadata store records the following metadata:
  - The pipeline and component versions that were executed
  - The start and end date, time, and how long the pipeline took to complete each of the steps
  - The executor of the pipeline
  - The parameter arguments that were passed to the pipeline
  - The pointers to the artifacts produced by each step of the pipeline
    - location of prepared data, validation anomalies, computed statistics, and extracted vocabulary from the categorical features
  - A pointer to the previous trained model
    - if needs to roll back to a previous model version
    - if needs to produce evaluation metrics for a previous model version when the pipeline is given new test data during the model validation step
  - The model evaluation metrics produced during the model evaluation step
    - for both the training and the testing sets
    - helps to compare the performance of a newly trained model to the recorded performance of the previous model

MLOps level 1: ML pipeline automation(8)

ML pipeline triggers

- Can automate the ML production pipelines to retrain the models with new data, depending on use case:
- On demand
  - Ad-hoc manual execution of the pipeline.
- On a schedule
  - New, labelled data is systematically available for the ML system on a daily, weekly, or monthly basis
  - Retraining frequency also depends on how frequently the data patterns change, and how expensive it is to retrain models
- On availability of new training data
  - New data is available on an ad-hoc basis when new data is collected and made available in the source databases
- On model performance degradation
  - The model is retrained when there is noticeable performance degradation
- On significant changes in the data distributions (concept drift)
  - notice significant changes on the data distributions of the features that are used to perform the prediction
  - changes suggest that model has gone stale, and that needs to be retrained on fresh data

BITS Pilani WILP

MLOps level 1: ML pipeline automation(9)

Challenges

- Suitable when need deploy new models based on new data, rather than based on new ML ideas
  - New implementations of the pipeline aren't frequently deployed
  - Managing only a few pipelines
  - Manually test the pipeline and its components
- When need to try new ML ideas and rapidly deploy new implementations of the ML components
  - Need to manage many ML pipelines in production
  - Need a CI/CD setup to automate the build, test, and deployment of ML pipelines

MLOps level 2: CI/CD pipeline automation

Implementation of the ML pipeline using CI/CD

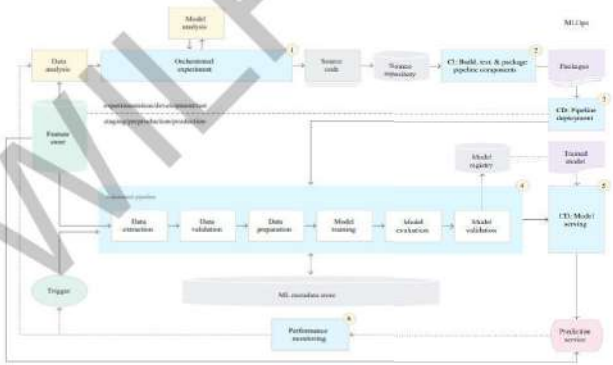


Figure 4. CI/CD and automated ML pipeline.

MLOps level 2: CI/CD pipeline automation(2)

Stages of the ML CI/CD automation pipeline

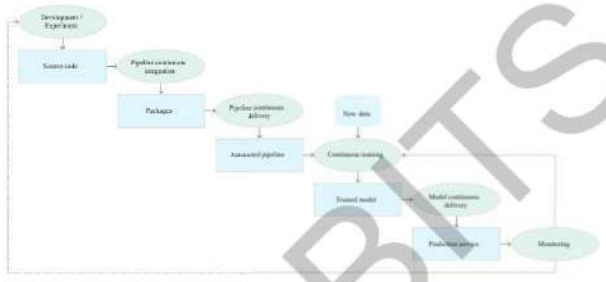


Figure 5. Stages of the CI/CD automated ML pipeline.

MLOps level 2: CI/CD pipeline automation(3)

Stages of the ML CI/CD automation pipeline(2)

- Development and experimentation
  - Iteratively try out new ML algorithms and new modeling where the experiment steps are orchestrated
  - Output - the source code of the ML pipeline steps that are then pushed to a source repository
- Pipeline continuous integration
  - Build source code and run various tests
  - Outputs - pipeline components (packages, executables, and artifacts) to be deployed in a later stage
- Pipeline continuous delivery
  - Deploy the artifacts produced by the CI stage to the target environment
  - Output - a deployed pipeline with the new implementation of the model
- Automated triggering
  - The pipeline is automatically executed in production based on a schedule or in response to a trigger
  - Output - a trained model that is pushed to the model registry
- Model continuous delivery
  - Serve the trained model as a prediction service for the predictions
  - Output - a deployed model prediction service
- Monitoring
  - Collect statistics on the model performance based on live data
  - Output - a trigger to execute the pipeline or to execute a new experiment cycle
  - The data analysis step is still a manual process for data scientists before the pipeline starts a new iteration of the experiment
  - The model analysis step is also a manual process

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

MLOps level 2: CI/CD pipeline automation(4)

Continuous integration

- When new code is committed or pushed to the source code repository
  - the pipeline and its components are built, tested, and packaged
- CI process can include the following tests:
  - Unit testing feature engineering logic
  - Unit testing the different methods implemented in model
    - a function that accepts a categorical data column and encode the function as a one-hot feature
  - Testing model training doesn't produce NaN values due to dividing by zero or manipulating small or large values
  - Testing that each component in the pipeline produces the expected artifacts
  - Testing integration between pipeline components

MLOps level 2: CI/CD pipeline automation(5)

Continuous delivery

- System continuously delivers new pipeline implementations to the target environment
  - that in turn delivers prediction services of the newly trained model.
- Rapid and reliable continuous delivery of pipelines and models depend on
  - Verifying the compatibility of the model with the target infrastructure before model is deployed
    - need to verify that the packages that are required by the model are installed in the serving environment
    - whether the memory, compute, and accelerator resources that are available
  - Testing the prediction service by calling the service API with the expected inputs
    - making sure that you get the response that you expect
    - captures problems that might occur when you update the model version and it expects a different input
  - Testing prediction service performance, which involves load testing the service
    - to capture metrics such as queries per seconds (QPS) and model latency
  - Validating the data either for retraining or batch prediction
  - Verifying that models meet the predictive performance targets before they are deployed
    - Automated deployment to a test environment - triggered by pushing code to the development branch
    - Semi-automated deployment to a pre-production environment - triggered by merging code to the main branch after reviewers approve the changes
    - Manual deployment to a production environment after several successful runs of the pipeline on the pre-production environment

Summary

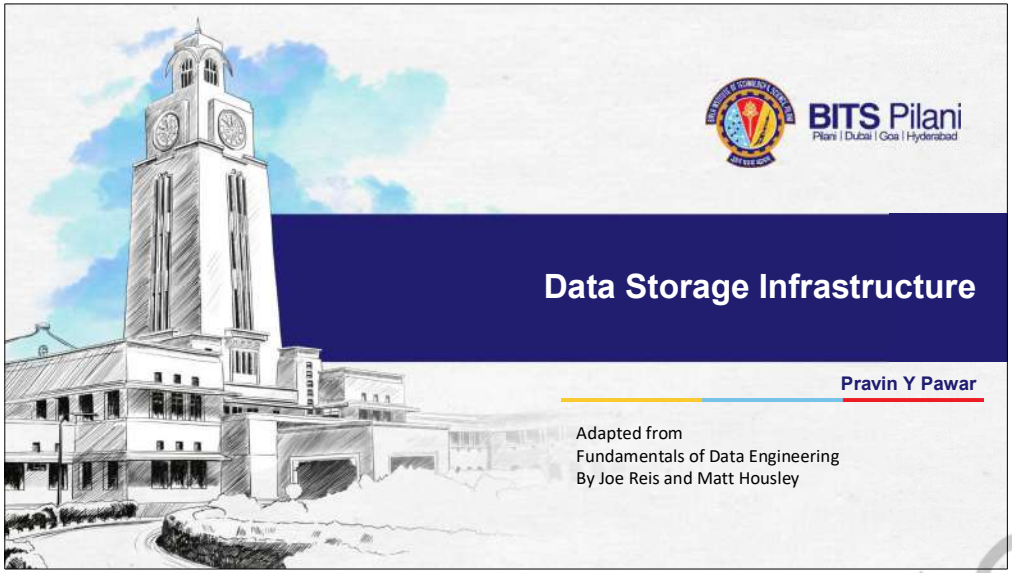
- Implementing ML in a production environment doesn't only mean deploying model as an API for prediction!
- Means deploying an ML pipeline that can automate the retraining and deployment of new models
  - Setting up a CI/CD system enables to automatically test and deploy new pipeline implementations
  - Allows coping with rapid changes in data and business environment
  - Don't have to immediately move all of processes from one level to another
  - Can gradually implement these practices to help improve the automation of your ML system development and production

Thank You!

In our next session:

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

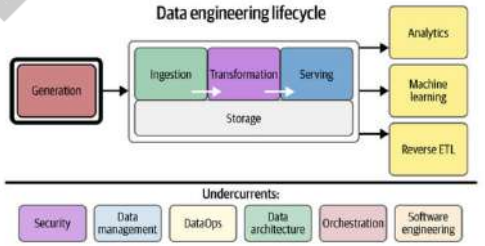
Data Storage Infrastructure

Pravin Y Pawar

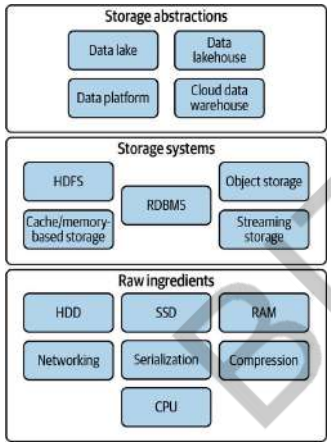
Adapted from
Fundamentals of Data Engineering
By Joe Reis and Matt Housley

Storage

- Cornerstone in the data engineering life cycle
  - Underlies major stages – ingestion, transformation and serving
  - Get stored many times when it moved though lifecycle



Storage Layering



Raw Ingredients of Data Storage

- Hardware
  - Magnetic Disk Drive
  - Solid-State Drive
  - Random Access Memory
  - Networking and CPU
- Associated features
  - Serialization
  - Compression
  - Caching

Table 6-1. A heuristic cache hierarchy displaying storage types with approximate pricing and performance characteristics

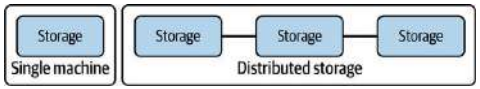
Storage type	Data fetch latency*	Bandwidth	Price
CPU cache	1 nanosecond	1 TB/s	N/A
RAM	0.1 microseconds	100 GB/s	$10/GB
SSD	0.1 milliseconds	4 GB/s	$0.20/GB
HDD	4 milliseconds	300 MB/s	$0.03/GB
Object storage	100 milliseconds	10 GB/s	$0.02/GB per month
Archival storage	12 hours	Same as object storage once data is available	$0.004/GB per month

BITS Pilani WILP

Data Storage Systems

Single Machine Versus Distributed Storage

- As data storage and access patterns become more complex and outgrow the usefulness of a single server
  - distributing data to more than one server becomes necessary
- Data can be stored on multiple servers, known as distributed storage
  - with purpose is to store data in a distributed fashion
- Distributed storage coordinates the activities of multiple servers
  - to store, retrieve, and process data faster and at a larger scale, all while providing redundancy in case a server becomes unavailable
  - provides built-in redundancy and scalability for large amounts of data
- Examples, object storage, Apache Spark, and cloud data warehouses rely on distributed storage architectures



Data Storage Systems(2)

File Storage

- A file is a data entity with specific read, write, and reference characteristics used by software and operating systems
- A file
  - has finite length - stream of bytes
  - supports append operations - append bytes to the files
  - supports random access - read from any location in the file or write updates to any location
- File storage systems organize files into a directory tree
  - /Users/Pravin/file.txt
- The filesystem stores each directory as metadata about the files and directories that it contains
  - consists of the name of each entity, relevant permission details, and a pointer to the actual entity
  - To find a file on disk, the operating system looks at the metadata at each hierarchy level and follows the pointer to the next subdirectory entity until finally reaching the file itself.

Data Storage Systems(3)

File Storage - Types

- Local disk storage
  - The most familiar type of file storage is an operating system–managed filesystem on a local disk partition of SSD or magnetic disc
  - New Technology File System (NTFS) and ext4 are popular filesystems on Windows and Linux respectively
  - OS handles the details of storing directory entities, files, and metadata
  - designed to write data to allow for easy recovery in the event of power loss during a write, though any unwritten data will still be lost.
  - support full read after write consistency; reading immediately after a write will return the written data
  - employs various locking strategies to manage concurrent writing attempts to a file
- Network-attached storage
  - provide a file storage system to clients over a network
  - prevalent solution for servers; they quite often ship with built-in dedicated NAS interface hardware
  - has performance penalties to accessing the filesystem over a network
  - significant advantages includes redundancy and reliability, fine-grained control of resources and file sharing across multiple machines
- Cloud filesystem services
  - provide a fully managed filesystem for use with multiple cloud VMs and applications, potentially including clients outside the cloud environment
  - should not be confused with standard storage attached to VMs—generally, block storage with a filesystem managed by the VM operating system
  - behave much like NAS solutions, but the details of networking, managing disk clusters, failures, and configuration are fully handled by the cloud vendor
  - Amazon Elastic File System (EFS) is an extremely popular example of a cloud filesystem service

Data Storage Systems(4)

Block Storage

- Fundamentally type of raw storage provided by SSDs and magnetic disks
- In the cloud, virtualized block storage is the standard for VMs
  - allow fine control of storage size, scalability, and data durability beyond that offered by raw disks.
- A block is the smallest addressable unit of data supported by a disk
  - was often 512 bytes of usable data on older disks, but it has now grown to 4,096 bytes for most current disks
  - also contain extra bits for error detection/correction and other metadata
- Blocks on magnetic disks are geometrically arranged on a physical platter
  - Two blocks on the same track can be read without moving the head, while reading two blocks on separate tracks requires a seek
  - Seek time can occur between blocks on an SSD, but this is infinitesimal compared to the seek time for magnetic disk tracks
- Block storage applications
  - Transactional database systems generally access disks at a block level to lay out data for optimal performance
  - Operating system boot disks on cloud VMs

BITS Pilani WILP

Data Storage Systems(5)

Block Storage - Types

- RAID
  - redundant array of independent disks
  - simultaneously controls multiple disks to
    - improve data durability
    - enhance performance
    - combine capacity from multiple drives
  - an array can appear to the operating system as a single block device
- Storage area network (SAN)
  - provide virtualized block storage devices over a network, typically from a storage pool
  - allow fine-grained storage scaling and enhance performance, availability, and durability
  - common with on-premises storage systems

Data Storage Systems(6)

Block Storage - Types

- Cloud virtualized block storage
  - similar to SAN but free engineers from dealing with SAN clusters and networking details
  - Example, Amazon Elastic Block Store (EBS) - default storage for Amazon EC2 virtual machines
    - store data separate from the instance host server but in the same zone to support high performance and low latency
    - allows EBS volumes to persist when an EC2 instance shuts down, when a host server fails, or even when the instance is deleted
    - suitable for applications such as databases, where data durability is a high priority
    - replicates all data to at least two separate host machines, protecting data if a disk fails
- Local instance volumes
  - Cloud providers also offer block storage volumes that are physically attached to the host server running a virtual machine
  - generally very low cost (included with the price of the VM in the case of Amazon's EC2 instance store) and provide low latency
    - when a VM shuts down or is deleted, the contents of the locally attached disk are lost, whether or not this event was caused by intentional user action
    - ensures that a new virtual machine cannot read disk contents belonging to a different customer

Data Storage Systems(7)

Object Storage

- Object Storage
  - contains objects of all shapes and sizes - a specialized file-like construct —TXT, CSV, JSON, images, videos, or audio
  - have grown in importance and popularity with the rise of big data and the cloud
  - Amazon S3, Azure Blob Storage, and Google Cloud Storage (GCS) are widely used
  - many cloud data warehouses (and a growing number of databases) utilize object storage as their storage layer
  - cloud data lakes generally sit on object stores
- Object storage was arguably one of the first "serverless" services
  - engineers don't need to consider the characteristics of underlying server clusters or disks
- An object store is a key-value store for immutable data objects.
  - don't support random writes or append operations; instead, they are written once as a stream of bytes
  - After this initial write, objects become immutable
  - To change data in an object or append data to it, must rewrite the full object

Data Storage Systems(8)

Cache and Memory-Based Storage Systems

- RAM offers excellent latency and transfer speeds but extremely vulnerable to data loss
- RAM-based storage systems are generally focused on caching applications, presenting data for quick access and high bandwidth
  - Data should generally be written to a more durable medium for retention purposes
  - These ultra-fast cache systems are useful when data engineers need to serve data with ultra-fast retrieval latency
- Memcached and lightweight object caching
  - a key-value store designed for caching database query results, API call responses, and more
  - uses simple data structures, supporting either string or integer types
  - deliver results with very low latency while also taking the load off backend systems
- Redis, memory caching with optional persistence
  - a key-value store, but it supports somewhat more complex data types (such as lists or sets)
  - with a typical configuration, Redis writes data roughly every two seconds
  - Redis is thus suitable for extremely high-performance applications but can tolerate a small amount of data loss

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Data Storage Systems(9)

The Hadoop Distributed File System

- HDFS is
  - based on Google File System (GFS) and was initially engineered to process data with the MapReduce programming model
  - similar to object storage but with a key difference: Hadoop combines compute and storage on the same nodes, where object stores typically have limited support for internal processing
- Management
  - breaks large files into blocks, chunks of data less than a few hundred megabytes in size
  - managed by the NameNode, which maintains directories, file metadata, and a detailed catalog describing the location of file blocks in the cluster
  - each block of data is replicated to three nodes - increases both the durability and availability of data
- Claims that Hadoop is dead! This is only partially true!
  - no longer a hot, bleeding-edge technology
  - Many Hadoop ecosystem tools such as Apache Pig are now on life support and primarily used to run legacy jobs
  - The pure MapReduce programming model has fallen by the wayside
- but HDFS remains widely used in various applications and organizations
  - HDFS is a key ingredient of many current big data engines, such as Amazon EMR
  - In fact, Apache Spark is still commonly run on HDFS clusters

Data Storage Systems(10)

Streaming Storage

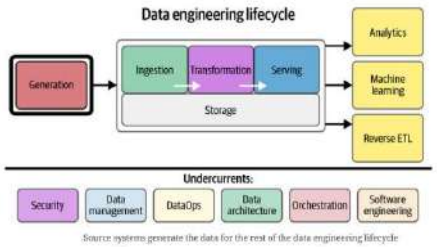
- Streaming data has different storage requirements than nonstreaming data - retention, replay!
- Retention
  - In the case of message queues, stored data is temporal and expected to disappear after a certain duration
  - Distributed, scalable streaming frameworks like Apache Kafka now allow extremely long-duration streaming data retention
  - Kafka competitors (including Amazon Kinesis, Apache Pulsar, and Google Cloud Pub/Sub) also support long data retention
- Replay
  - allows a streaming system to return a range of historical stored data
  - standard data-retrieval mechanism for streaming storage systems
  - can be used to run batch queries over a time range or to reprocess data in a streaming pipeline

Data Storage Abstractions

- Data engineering storage abstractions are data organization and query patterns that sit at the heart of the data engineering lifecycle
  - are built atop the data storage systems
- The main types of abstractions are those that support data science, analytics, and reporting use cases
  - Data warehouse
  - Data lake
  - Data lakehouse
  - Data platforms

Sources of Data: How Is Data Created?

- Essential to understand how data is created, as
  - Data is an unorganized, context-less collection of facts and figures
  - Can be created in many ways, both analog and digital
- Analog data
  - creation occurs in the real world, such as vocal speech, sign language, writing on paper, or playing an instrument
  - often transient
- Digital data
  - either created by converting analog data to digital form or is the native product of a digital system
  - A mobile texting app that converts analog speech into digital text
  - A credit card transaction on an ecommerce platform



BITS Pilani WILP

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Source Systems

Relational databases

- Most common application backends - developed at IBM in the 1970s and popularized by Oracle in the 1980s
  - growth of the internet saw the rise of the LAMP stack (Linux, Apache web server, MySQL, PHP) and an explosion of vendor and open source RDBMS options
- Data is stored in a table of relations (rows), and each relation contains multiple fields (columns)
- Each relation in the table has the same schema
- Rows are typically stored as a contiguous sequence of bytes on disk
- Tables are typically indexed by a primary key, a unique field for each row in the table
  - The indexing strategy for the primary key is closely connected with the layout of the table on disk
- Tables can also have various foreign keys —fields with values connected with the values of primary keys in other tables,
  - facilitating joins, and allowing for complex schemas that spread data across multiple tables
- Normalization is a strategy for ensuring that data in records is not duplicated in multiple places
  - avoiding the need to update states in multiple locations at once and preventing inconsistencies
- RDBMS systems are typically ACID compliant
- Relational database systems ideal for storing rapidly changing application states
  - Combining a normalized schema, ACID compliance, and support for high transaction rates
  - Challenge is to determine how to capture state information over time

Source Systems(2)

Non-relational databases: NoSQL

- NoSQL, not only SQL, refers to a whole class of databases that abandon the relational paradigm
  - widely adopted to describe a universe of “new school” databases, alternatives to relational databases
  - dropping relational constraints can improve performance, scalability, and schema flexibility
  - abandon various RDBMS characteristics, such as strong consistency, joins, or a fixed schema
- Numerous flavors of NoSQL database designed for almost any imaginable use case
  - key-value
  - document
  - wide-column
  - graph
  - search
  - time series
- all wildly popular and enjoy widespread adoption
- Should understand
  - these types of databases
  - including usage considerations
  - the structure of the data they store
  - and how to leverage each in the data engineering lifecycle

Source Systems(3)

Key-Value stores

- A non-relational database that retrieves records using a key that uniquely identifies each record
  - similar to hash map or dictionary data structures presented in many programming languages but potentially more scalable
  - encompass several NoSQL database types —document stores and wide column databases
- Different types of key-value databases offer a variety of performance characteristics to serve various application needs
- In-memory key-value databases are popular for caching session data for web and mobile applications, where ultra-fast lookup and high concurrency are required
  - Storage in these systems is typically temporary - if the database shuts down, the data disappears
  - Such caches can reduce pressure on the main application database and serve speedy responses
- Key-value stores can also serve applications requiring high-durability persistence
  - An ecommerce application may need to save and update massive amounts of event state changes for a user and their orders
  - A user logs into the ecommerce application, clicks around various screens, adds items to a shopping cart, and then checks out
  - Each event must be durably stored for retrieval
  - Key-value stores often persist data to disk and across multiple nodes to support such use cases

Source Systems(4)

Document stores

- A specialized key-value store
  - A document is a nested object - each document as a JSON object - are stored in collections and retrieved by key
  - A collection is roughly equivalent to a table in a relational database
- Does not support joins - data cannot be easily normalized, i.e., split across multiple tables
  - Ideally, all related data can be stored in the same document.
  - In many cases, the same data must be stored in multiple documents spread across numerous collections
- Generally embrace all the flexibility of JSON and don’t enforce schema or types - blessing and a curse
  - allows the schema to be highly flexible and expressive - can also evolve as an application grows
  - become absolute nightmares to manage and query - data may become inconsistent and bloated over time
- Support the creation of indexes and lookup tables to allow retrieval of documents by specific properties
  - invaluable in application development when you need to search for documents in various ways
- Generally not ACID compliant - many are eventually consistent
  - Allowing data distribution across a cluster is a boon for scaling and performance but can lead to catastrophes when engineers and developers don’t understand the implications
- To run analytics on document stores, engineers generally must run a full scan to extract all data from a collection
  - can have both performance and cost implications

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Source Systems(6)

Search and Time series databases

- Search
  - Non-relational database used to search data's complex and straightforward semantic and structural characteristics
  - Search databases are popular for fast search and retrieval and can be found in various applications
    - Elasticsearch, Apache Solr or Lucene, or Algolia
  - Two prominent use cases exist for a search database: text search and log analysis
    - Text search involves searching a body of text for keywords or phrases, matching on exact, fuzzy, or semantically similar matches
    - Log analysis is typically used for anomaly detection, real-time monitoring, security analytics, and operational analytics

- Time series
  - A series of values organized by time - stock prices might move as trades are executed throughout the day
    - Any events that are recorded over time — either regularly or sporadically — are time-series data
  - Optimized for retrieving and statistical processing of time-series data
    - Address the needs of growing, high-velocity data volumes from IoT, event and application logs, ad tech, and fintech
    - Often utilize memory buffering to support fast writes and reads
  - Measurement data is generated regularly, such as temperature or air-quality sensors
  - Event-based data is irregular and created every time an event occurs — for instance, when a motion sensor detects movement

  - The schema for a time series typically contains a timestamp and a small set of fields
    - makes time-series databases suitable for operational analytics but not great for BI use cases
    - Joins are not common, though some quasi time-series databases such as Apache Druid support joins

Source Systems(7)

APIs - REST

- APIs are now a standard and pervasive way of exchanging data in the cloud, for SaaS platforms, and between internal company systems

- REST
  - Currently the dominant API paradigm - REpresentational State Transfer
    - set of practices and philosophies for building HTTP web APIs was laid out by Roy Fielding in 2000 in a PhD dissertation
    - built around HTTP verbs, such as GET and PUT; in practice, modern REST uses only a handful of the verb mappings outlined in the original dissertation
  - Interactions are stateless - each REST call is independent
    - can change the system's state, but these changes are global, applying to the full system rather than a current session

  - Great variation in levels of API abstraction
    - APIs are merely a thin wrapper over internals that provides the minimum functionality required to protect the system from user requests
    - API is a masterpiece of engineering that prepares data for analytics applications and supports advanced reporting

  - Data providers frequently supply client libraries in various languages, especially in Python
    - remove much of the boilerplate labor of building API interaction code
    - handle critical details such as authentication and map fundamental methods into accessible classes

  - Various services and open source libraries have emerged to interact with APIs and manage data synchronization
    - Many SaaS and open source vendors provide off-the-shelf connectors for common APIs
    - Platforms also simplify the process of building custom connectors as required
    - Numerous data APIs without client libraries or out-of-the-box connector support

Source Systems(8)

Other API systems

- GraphQL
  - Created at Facebook as a query language for application data and an alternative to generic REST APIs
  - REST APIs generally restrict queries to a specific data model, GraphQL opens up the possibility of retrieving multiple data models in a single request
  - allows for more flexible and expressive queries than with REST
  - built around JSON and returns data in a shape resembling the JSON query

- Webhooks
  - A simple event-based data-transmission pattern - aka reverse APIs
  - Data source can be an application backend, a web page, or a mobile app
  - When specified events happen in the source system, this triggers a call to an HTTP endpoint hosted by the data consumer
  - Endpoint can do various things with the POST event data, potentially triggering a downstream process or storing the data for future use

- RPC and gRPC
  - A remote procedure call (RPC) is commonly used in distributed computing - allows to run a procedure on a remote system
  - gRPC is a remote procedure call library developed internally at Google in 2015 and later released as an open standard
    - Many Google services, such as Google Ads and GCP, offer gRPC APIs. gRPC is built around the Protocol Buffers open data serialization standard
  - gRPC emphasizes the efficient bidirectional exchange of data over HTTP/2.
    - Efficiency refers to aspects such as CPU utilization, power consumption, battery life, and bandwidth
    - imposes much more specific technical standards than REST
    - Allows the use of common client libraries and allowing engineers to develop a skill set that will apply to any gRPC interaction code.

Source Systems(10)

Third-Party Data Sources

- Every company is essentially now a technology company
  - consequence is want to make their data available to their customers and users, either as part of their service or as a separate subscription

- For example,
  - US Bureau of Labor Statistics publishes various statistics about the US labor market
  - NASA publishes various data from its research initiatives
  - Facebook shares data with businesses that advertise on its platform

- Why would companies want to make their data available?
  - Data is sticky, and a flywheel is created by allowing users to integrate and extend their application into a user's application
  - Greater user adoption and usage means more data, which means users can integrate more data into their applications and data systems
  - Side effect is there are now almost infinite sources of third-party data

- Direct third-party data access is commonly done via APIs, through data sharing on a cloud platform, or through data download
  - APIs often provide deep integration capabilities, allowing customers to pull and push data

- For example,
  - Many CRMs offer APIs that their users can integrate into their systems and applications
  - A common workflow to get data from a CRM, blend the CRM data through the customer scoring model, and then use reverse ETL to send that data back into CRM for salespeople to contact better-qualified leads

BITS Pilani WILP

Source Systems(11)

Message Queues

- Event-driven architectures are pervasive in software applications and are poised to grow their popularity even further
  - Message queues and event-streaming platforms—critical layers in event-driven architectures—are easier to set up and manage in a cloud environment
  - Rise of data apps—applications that directly integrate real-time analytics—are growing from strength to strength
- As source systems, message queues and event-streaming platforms are used in numerous ways
  - from routing messages between microservices ingesting millions of events per second of event data from web, mobile, and IoT applications
- Message queues
  - A mechanism to asynchronously send data (usually as small individual messages, in the kilobytes) between discrete systems using a publish and subscribe model
  - Data is published to a message queue and is delivered to one or more subscribers
  - Subscriber acknowledges receipt of the message, removing it from the queue
- Message queues
  - allow applications and systems to be decoupled from each other and are widely used in microservices architectures
  - buffers messages to handle transient load spikes and makes messages durable through a distributed architecture with replication
  - critical ingredient for decoupled microservices and event-driven architectures

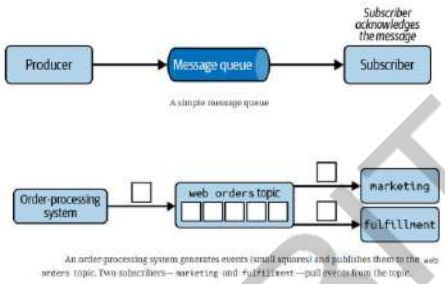
Source Systems(12)

Event-Streaming Platforms

- Is a continuation of a message queue in that messages are passed from producers to consumers
- Big difference between messages and streams is that
  - A message queue is primarily used to route messages with certain delivery guarantees
  - An event-streaming platform is used to ingest and process data in an ordered log of records
- In an event-streaming platform, data is retained for a while, and it is possible to replay messages from a past point in time.
- Topics
  - A producer streams events to a topic, a collection of related events
  - might contain fraud alerts, customer orders, or temperature readings from IoT devices
  - can have zero, one, or multiple producers and customers on most event-streaming platforms
- Stream partitions
  - are subdivisions of a stream into multiple streams
  - A good analogy is a multilane freeway - Having multiple lanes allows for parallelism and higher throughput
  - Messages are distributed across partitions by partition key
  - Messages with the same partition key will always end up in the same partition

Source Systems(13)

Message Queues and Event-Streaming Platforms

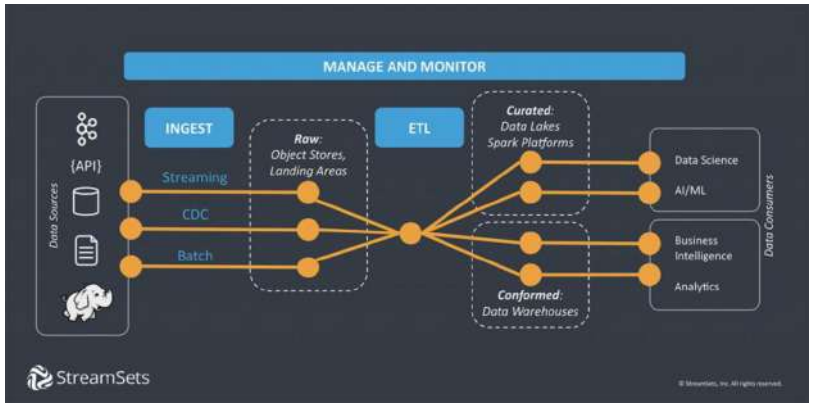


Data Ingestion

- Process of moving data from a source into a landing area or an object store where it can be used for ad hoc queries and analytics.
  - A simple data ingestion pipeline consumes data from a point of origin, cleans it up a bit, then writes it to a destination
- Importance
  - Helps teams go fast!
  - Scope of any given data pipeline is deliberately narrow, giving data teams flexibility and agility at scale
  - Once parameters are set, data analysts and data scientists can easily build a single data pipeline to move data to their system of choice
- Common examples of data ingestion include:
  - Move data from Salesforce.com to a data warehouse then analyze with Tableau
  - Capture data from a Twitter feed for real-time sentiment analysis
  - Acquire data for training machine learning models and experimentation
- Modern Data Integration Begins with Data Ingestion
  - Data engineers use data ingestion pipelines to better handle the scale and complexity of business demands for data
  - Lots of intent-driven data pipelines operating continuously across the organization without direct involvement of a development team
- Data ingestion has become a key component of self-service platforms for analysts and data scientists to access data for real-time analytics, machine learning and AI workloads.

BITS Pilani WILP

Working



Working(2)

- Data ingestion extracts data from the source where it was created or originally stored
  - Then loads data into a destination or staging area
- A simple data ingestion pipeline
  - might apply one or more light transformations
  - enriching or filtering data
  - before writing it to some set of destinations, a data store or a message queue
- More complex transformations
  - such as joins, aggregates, and sorts
  - for specific analytics, applications and reporting systems
  - can be done with additional pipelines

Components

- Data Sources
  - Data teams have moved way beyond the walled garden of the enterprise data center for insight
  - increasingly load data from across business units as well as 3rd party and unstructured data, start data loads, when and where they need it
  - Common data source types:
    - Apache Kafka
    - JDBC
    - Oracle CDC
    - HTTP Clients
    - HDFS
- Data Destinations
  - A data ingestion pipeline may simply send data to an application or messaging system, or store ingested data in a data lake or cloud object storage for use in relational and NoSQL databases or data warehouses
  - Common destination:
    - Apache Kafka
    - JDBC
    - Snowflake
    - Amazon S3
    - Databricks
- Cloud Data Migration
  - Enterprise business processes are moving to cloud-based platforms for storing, processing and applications
  - Data ingestion workloads have become essential to cloud migration
  - But moving data out of silos and into agile cloud data lakes or powerful cloud data warehouses, generates some uncomfortable questions
- The more data platforms can automate and operationalize the what-ifs of data ingestion, the better they can support the growing demand for continuous, reliable data.

Data Ingestion vs Data Integration

- Data ingestion originated as a small part of data integration, a more complex process required to make data consumable in new systems before loading it
- Data integration usually requires advance specification from source to schema to transformation to destination
- With data ingestion, a few light transformations may be made, such as masking personally identifiable information (PII)
  - but most of the work depends on the end use and happens after landing the data
- Think of it this way:
  - Data integration includes processes to prepare data to be consumable at its final destination
  - Data ingestion gets data to places where preparation happens in response to downstream needs
- Data ingestion works well for streaming data that can be used immediately with very few transformations or as a way to collect data for ad hoc analysis
  - By focusing on the ingestion part of the data lifecycle, companies have been able to speed up the availability of data for innovation and growth

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Data Ingestion Challenges

- With the rise of big data, cloud computing, and the demand for real-time analytics, the capacity for data increased dramatically
  - old ETL processes began to slow data teams down compared to the ELT model
- Complexity Takes Time
  - The to-do list for data engineering is long and getting longer
  - Building data pipelines from scratch every time a new source or business need comes up slows down the whole data team
- Change Takes Time
  - Every change or evolution of a target system generates 10-20 hours of work for a data engineer
  - 90% of time will be spent on maintenance and break-fix, changes that are considered data drift
- Maintenance and Rework Takes Time
  - Doing the same thing over and over with a lot of troubleshooting and debugging doesn't leave much time for innovation or ramping up on new technologies

Types of Data Ingestion Tools

- Hand Coding
  - One way to ingest data may be to hand code a data pipeline, assuming know how to code and are familiar with the languages needed
  - gives the greatest control
  - may spend a lot of time working and reworking code
- Single-purpose Tools
  - Basic data ingestion tools provide a drag-and-drop interface with lots of pre-built connectors and transformation
  - a quick way to get a lot done or to enable less skilled data consumers,
  - management of many drag-and-drop data pipelines will be cumbersome
  - can't share work with team or the analysts and data scientists knocking on door
- Data Integration Platforms
  - Traditional data integration platforms incorporate features for every step of the data value chain
  - means most likely need developers and architectures specific to each domain
  - making it difficult to move fast and adapt easily to change
- A DataOps Approach
  - Applying agile methodologies to data, a DataOps approach to data pipelines automates as much as possible and abstracts away the "how" of implementation
  - Data engineers can focus on the "what" of the data, and responding to business needs

Key Engineering Considerations for the Ingestion Phase

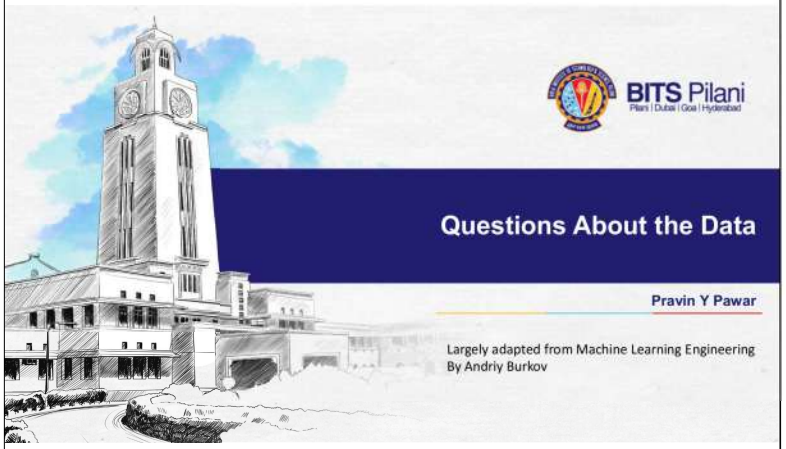
Primary considerations related to data ingestion

- What's the use case for the data is ingesting?
- Can this data be reused and avoid ingesting multiple versions of the same dataset?
- Where is the data going? What's the destination?
- How often should the data be updated from the source?
- What is the expected data volume?
- What format is the data in? Can downstream storage and transformation accept this format?
- Is the source data in good shape for immediate downstream use? Is the data of good quality?
- What post-processing is required to serve it? What are data-quality risks?
- Does the data require in-flight processing for downstream ingestion if the data is from a streaming source?

Thank You!

In our next session:

BITS Pilani WILP



Questions About the Data

Pravin Y Pawar

Largely adapted from Machine Learning Engineering
By Andriy Burkov

Need

- The data available to the analyst is
  - not always "right"
  - not always in a form that a machine learning algorithm can use
- Discuss
  - the properties of good quality data,
  - typical problems a dataset can have
- Questions about data
  - Is the Data Accessible?
  - Is the Data Sizeable?
  - Is the Data Useable?
  - Is the Data Understandable?
  - Is the Data Reliable?

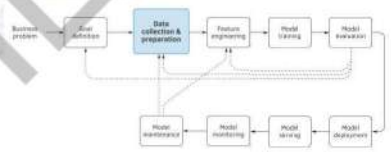


Figure 1: Machine learning project life cycle.

Is the Data Accessible?

- Does the data you need already exist?
  - If yes, is it accessible (physically, contractually, ethically, or from a cost perspective)?
  - If you are purchasing or re-using someone else's data sources, have you considered how that data might be used or shared?
  - Do you need to negotiate a new licensing agreement with the original supplier?
- If the data is accessible, is it protected by copyright or other legal norms?
  - If so, have you established who owns the copyright in your data?
  - Might there be joint copyright?
  - Is the data sensitive (e.g., concerning your organization's projects, clients, or partners, or it is classified by the government), and are there any potential privacy issues? If so, have you
  - discussed data sharing with the respondents from whom you collected the data?
  - Can you preserve personal information for a long-term so that it can be used in the future?
  - Do you need to share the data along with the model? If so, do you need to get written consent from owners or srespondents?
  - Do you need to anonymize data?
- Even if it's physically possible to get the data you need, don't work with it until all the above questions are resolved.

Is the Data Sizeable?

- The question for which would like to have a definitive answer is whether there's enough data
  - However, as already found out, it's usually not known how much data is needed to reach the goal,
  - especially if the minimum model quality requirement is stringent
- If have doubts about the immediate availability of sufficient data,
  - need find out how frequently new data gets generated
  - For some projects, can start with what's initially available and, while working on it, new data might gradually come in
- One practical way to find out if have collected sufficient data is to plot learning curves
  - More specifically, plot the training and validation scores of learning algorithm for varying numbers of training examples

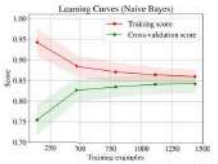


Figure 2: Learning curves for the Naive Bayes learning algorithm applied to the standard "digits" dataset of scikit-learn.

BITS Pilani WILP

Is the Data Useable?

- The data quality is one of the major factors affecting the model performance
- Imagine that want to train a model that predicts a person's gender, given their name
  - might acquire a dataset of people that contains gender information
  - however, if use this dataset blindly, might realize that no matter how hard one tries to improve the quality of model
  - its performance on new data is low.
- A tidy dataset can
  - have missing values - consider data imputation techniques to fill the missing values
  - have magic number - 9999 or -1
  - contains duplicates - check why those are added
  - have data which expired or be significantly not up to date
  - have data which is incomplete or unrepresentative of the phenomenon

Is the Data Understandable?

- Crucial to understand from where each attribute in the dataset came
  - equally important to understand what each attribute exactly represents
- One frequent problem observed in practice is when the variable that the analyst tries to predict is found among the features in the feature vector
  - Imagine that working on the problem of predicting the price of a house from its attributes
    - such as the number of bedrooms, surface, location, year of construction, and so on
  - Without spending too much time analyzing each column, remove only the transaction price from the attributes and use that value as the target to be learnt
    - predicts the transaction price with accuracy near 100%
  - Model is delivered to the client, they deploy it in production
    - the tests show that the model is wrong most of the time.
  - What did happen is called data leakage (also known as target leakage)
  - After a more careful examination of the dataset, realize that one of the columns in the spreadsheet contained the real estate agent's commission
  - Of course, the model easily learned to convert this attribute into the house price perfectly
    - However, this information is not available in the production environment before the house is sold

Is the Data Reliable?

- The reliability of a dataset varies depending on the procedure used to gather that dataset
- Can you trust the labels?
  - If the data was produced by the workers on Mechanical Turk , then the reliability of such data might be very low
  - In some cases, the labels assigned to feature vectors might be obtained as a majority vote
  - better to do additional validation of quality on a small random sample of the dataset
- The reliability of labels can also be affected by the delayed or indirect nature of the label.

Is the Data Reliable?(2)

Delayed labels

- The label is considered delayed when the feature vector to which the label was assigned represents something that happened significantly earlier than the time of label observation
- Take the churn prediction problem
  - a feature vector describing a customer, and want to predict whether the customer will leave at some point in the future
    - typically six months to one year from now
  - feature vector represents what we know about the customer now, but the label ("left" or "stayed") will be assigned in the future
  - This is an important property, because between now and the future, many events, not reflected in our feature vector might happen which would affect the customer's decision to stay or leave.
    - Therefore delayed labels make our data less reliable

BITS Pilani WILP

Is the Data Reliable?(3)

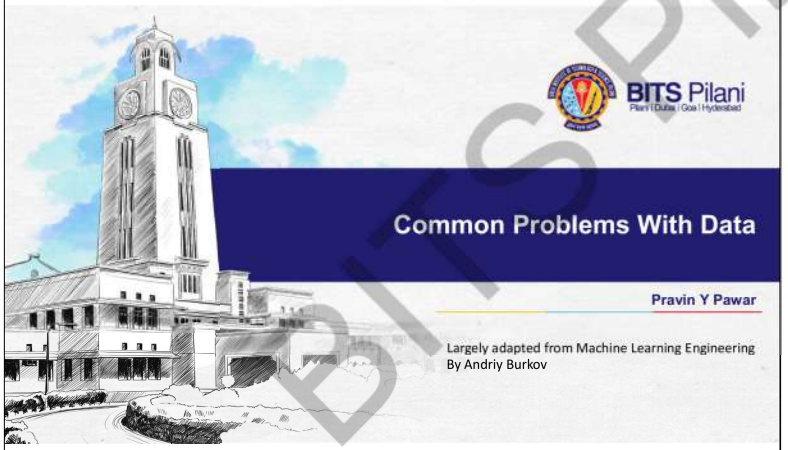
Direct or indirect labels

- Whether a label is direct or indirect also affects reliability, depending, of course, on what we are trying to predict.
- For example,
  - let's say goal is to predict whether the website visitor will be interested in a webpage
    - might acquire a certain dataset containing information about users, webpages, and labels "interested"/"not_interested" reflecting whether a specific user was interested in a particular webpage
  - A direct label would indeed indicate interest, while an indirect label could suggest some interest.
    - if the user pressed the "Like" button, have the direct indicator of interest
    - if the user only clicked on the link, this could be an indicator of some interest, but it's an indirect indicator
  - The user could have clicked by mistake, or because the link text was a clickbait, we cannot know for sure
    - If the label is indirect, this also makes such data less reliable
    - Of course, it's less reliable for predicting the interest, but can be perfectly reliable for predicting clicks

Is the Data Reliable?(4)

Feedback loops

- A feedback loop is a property in the system design when the data used to train the model is obtained using the model itself
- Imagine that working on a problem of predicting whether a specific user of a website will like the content
  - only have indirect labels – clicks
- If the model is already deployed on the website and the users click on links recommended by the model
  - means that the new data indirectly reflects not only the interest of users to the content, but also how intensively the model recommended that content
- If the model decided that a specific link is important enough to recommend to many users,
  - more users would likely click on that link, especially if the recommendation was made repeatedly during several days or weeks



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Common Problems With Data

Pravin Y Pawar

Largely adapted from Machine Learning Engineering
By Andriy Burkov

High Cost

Labelling

- Getting unlabeled data can be expensive
  - however, labeling data is the most expensive work, especially if the work is done manually
  - also when it must be gathered specifically for your problem
  - must be assigned manually, and paying someone to do that work is expensive
- Google has a clever technique outsourcing the labeling to random people with its free reCAPTCHA service
  - solves two problems:
  - reducing spam on the Web
  - providing cheap labeled data to Google
- The goal of a good labeling process design is to make the labeling as streamlined as pos

Original photo Labeled photo

Figure 3: The unlabeled and labeled aerial photo. Photo credit: Tom Fish.

BITS Pilani WILP

High Cost(2)

Tool-based labeling

- Well-designed labeling tools
  - will minimize mouse use (including menus activated by mouse clicks), maximize hotkeys,
  - reduce costs by increasing the speed of data labeling.
- Whenever possible, reduce decision-making to a yes/no answer
  - If the labeler clicks "Not Sure," can save this example to analyze later or simply not use such examples for training the model
- Another trick allowing for accelerated labeling is noisy pre-labeling consisting of prelabeling the example using the current best model
  - start by labeling a certain quantity of examples "from scratch" (that is, without using any support)
  - build the first model that works reasonably well, using this initial set of labeled examples
  - Next, use the current model and label each new example in place of the human labeler
  - Ask whether the automatically assigned label is correct
  - If the labeler clicks "Yes," save this example as usual
  - If they click "No," then ask to label this example manually

Bad Quality

- Data quality is one of the major factors affecting the performance of the model
- Data quality has two components:
  - raw data quality and labeling quality
- Some common problems with raw data are
  - noise,
  - bias,
  - low predictive power,
  - outdated examples,
  - outliers,
  - leakage

Noise

- Noise in data is a corruption of examples
  - Images can be blurry or incomplete
  - Text can lose formatting, which makes some words concatenated or split
  - Audio data can have noise in the background
  - Poll answers can be incomplete or have missing attributes, such as the responder's age or gender
- Noise is often a random process that corrupts each example independently of other examples in the collection
  - If tidy data has missing attributes, data imputation techniques can help in guessing values for those attributes
  - Blurred images can be deblurred using specific image deblurring algorithms
  - Noise in audio data can be algorithmically suppressed
- Noise is more a problem when the dataset is relatively small (thousands of examples or less),
  - because the presence of noise can lead to overfitting
  - the algorithm may learn to model the noise contained in the training data, which is undesirable
- In the big data context, on the other hand, noise is typically "averaged out" over multiple examples

Bias

- Bias in data is an inconsistency with the phenomenon that data represents
  - occur for a number of reasons (which are not mutually exclusive)
- Types of Bias
  - Selection bias
  - Omitted variable bias
  - Sponsorship or funding bias
  - Sampling bias (also known as distribution shift)
  - Prejudice or stereotype bias
  - Systematic value distortion
  - Experimenter bias
  - Labeling bias

BITS Pilani WILP

Low Predictive Power

- Is an issue that's often not considered until one has spent fruitless energy trying to train a good model
- Don't know whether
  - the model underperform because it is not expressive enough?
  - the data not contain enough information from which to learn?
- Suppose the goal is to predict whether a listener will like a new song on a music streaming service
  - data is the name of the artist, the song title, lyrics, and whether that song is in their playlist
  - model trained with this data will be far from perfect!
- Artists who are not in the listener's playlist are unlikely to receive a high score from the model
  - many users will only add some songs of a specific artist to their playlist
  - musical preferences are significantly influenced by the song arrangement, choice of instruments, sound effects, tone of voice, and subtle changes in tonality, rhythm, and beat
  - These are properties of songs that cannot be found in lyrics, title, or the artist's name; they have to be extracted from the sound file

Outdated Examples

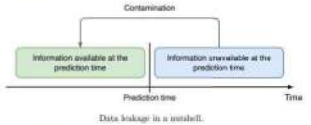
- Once you build the model and deploy it in production, the model usually performs well for some time!
  - a certain model quality monitoring procedure is deployed in the production environment
  - Once an erratic behavior is detected, new training data is added to adjust the model
  - the model is then retrained and redeployed
- Often, the cause of an error is explained by the finiteness of the training set.
  - In such cases, additional training examples will solidify the model
  - However, in many practical scenarios, the model starts to make errors because of concept drift.
- Concept drift is a fundamental change in the statistical relationship between features and label
- Imagine your model predicts whether a user will like certain content on a website
  - Over time, the preferences of some users may start to change, perhaps due to aging, or because a user discovers something new
  - examples added to the training data in the past no longer reflect some user's preferences and start hurting the model performance rather than contributing to it - aka concept drift

Outliers

- Outliers are examples that look dissimilar to the majority of examples from the dataset
  - up to the data analyst to define "dissimilar"
  - Typically, measured by some distance metric, such as Euclidean distance
- Shallow algorithms are particularly sensitive to outliers
  - linear or logistic regression, and some ensemble methods, such as AdaBoost
- Debatable
  - Whether to exclude outliers from the training data, or to use machine learning algorithms and models robust to outliers
  - Deleting examples from a dataset is not considered scientifically or methodologically sound, especially in small datasets
  - In the big data context, outliers don't typically have a significant influence on the model
- From a practical standpoint, if excluding some training examples results in better performance of the model on the holdout data, the exclusion may be justified
- Which examples to consider for exclusion can be decided based on a certain similarity measure

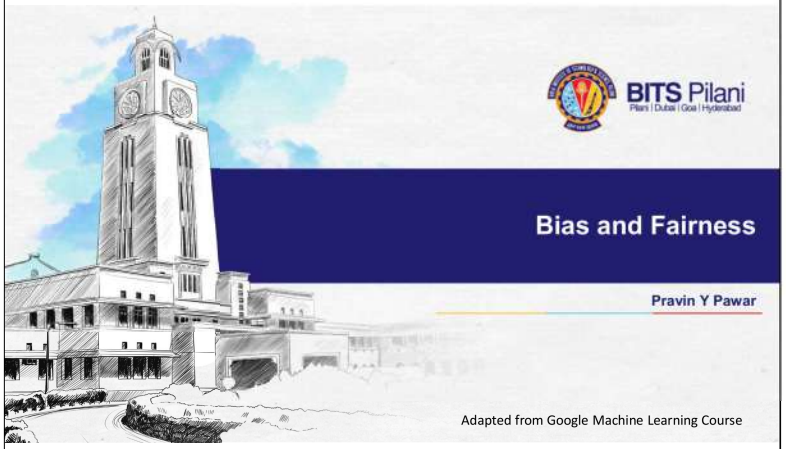
Data Leakage

- Data leakage, also called target leakage,
  - a problem affecting several stages of the machine learning life cycle, from data collection to model evaluation
- Data leakage in supervised learning is the unintentional introduction of information about the target that should not be made available
  - also called as "contamination"
  - Training on contaminated data leads to overly optimistic expectations about the model performance



Data leakage in a nutshell.

BITS Pilani WILP



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Bias and Fairness

Pravin Y Pawar

Adapted from Google Machine Learning Course

Fairness

- Evaluating a machine learning model responsibly requires doing more than just calculating loss metrics
  - critical to audit training data and evaluate predictions for bias
- Machine learning models are not inherently objective
  - Engineers train models by feeding them a data set of training examples
  - This human involvement in the provision and curation of this data can make a model's predictions susceptible to bias.
- Important to be aware of common human biases that can manifest in data
  - proactive steps can be taken to mitigate their effects

Types of Bias

Reporting Bias

- Occurs when the frequency of events, properties, and/or outcomes captured in a data set does not accurately reflect their real-world frequency
- Can arise because people tend to focus on documenting circumstances that are unusual or especially memorable
- EXAMPLE:
- A sentiment-analysis model is trained to predict whether book reviews are positive or negative based on a corpus of user submissions to a popular website. The majority of reviews in the training data set reflect extreme opinions (reviewers who either loved or hated a book), because people were less likely to submit a review of a book if they did not respond to it strongly. As a result, the model is less able to correctly predict sentiment of reviews that use more subtle language to describe a book.

Types of Bias

Automation Bias

- Tendency to favor results generated by automated systems over those generated by non-automated systems, irrespective of the error rates of each
- EXAMPLE:
- Software engineers working for a sprocket manufacturer were eager to deploy the new "groundbreaking" model they trained to identify tooth defects, until the factory supervisor pointed out that the model's precision and recall rates were both 15% lower than those of human inspectors.

BITS Pilani WILP

Types of Bias

Selection Bias

- Selection bias occurs if a data set's examples are chosen in a way that is not reflective of their real-world distribution
- Coverage bias: Data is not selected in a representative fashion.
  - EXAMPLE: A model is trained to predict future sales of a new product based on phone surveys conducted with a sample of consumers who bought the product. Consumers who instead opted to buy a competing product were not surveyed, and as a result, this group of people was not represented in the training data.
- Non-response bias (or participation bias): Data ends up being unrepresentative due to participation gaps in the data-collection process.
  - EXAMPLE: A model is trained to predict future sales of a new product based on phone surveys conducted with a sample of consumers who bought the product and with a sample of consumers who bought a competing product. Consumers who bought the competing product were 80% more likely to refuse to complete the survey, and their data was underrepresented in the sample.
- Sampling bias: Proper randomization is not used during data collection.
  - EXAMPLE: A model is trained to predict future sales of a new product based on phone surveys conducted with a sample of consumers who bought the product and with a sample of consumers who bought a competing product. Instead of randomly targeting consumers, the surveyer chose the first 200 consumers that responded to an email, who might have been more enthusiastic about the product than average purchasers.

Types of Bias

Group Attribution Bias

- Tendency to generalize what is true of individuals to an entire group to which they belong
- Two key manifestations are:
- In-group bias
  - A preference for members of a group to which you also belong, or for characteristics that you also share.
  - EXAMPLE: Two engineers training a résumé-screening model for software developers are predisposed to believe that applicants who attended the same computer-science academy as they both did are more qualified for the role.
- Out-group homogeneity bias
  - A tendency to stereotype individual members of a group to which you do not belong, or to see their characteristics as more uniform.
  - EXAMPLE: Two engineers training a résumé-screening model for software developers are predisposed to believe that all applicants who did not attend a computer-science academy do not have sufficient expertise for the role.

Types of Bias

Implicit Bias

- Occurs when assumptions are made based on one's own mental models and personal experiences that do not necessarily apply more generally
- EXAMPLE:
  - An engineer training a gesture-recognition model uses a head shake as a feature to indicate a person is communicating the word "no." However, in some regions of the world, a head shake actually signifies "yes."
- Confirmation bias
  - Model builders unconsciously process data in ways that affirm preexisting beliefs and hypotheses
- Experimenter's bias
  - In some cases, a model builder may actually keep training a model until it produces a result that aligns with their original hypothesis
- EXAMPLE:
  - An engineer is building a model that predicts aggressiveness in dogs based on a variety of features (height, weight, breed, environment). The engineer had an unpleasant encounter with a hyperactive toy poodle as a child, and ever since has associated the breed with aggression. When the trained model predicted most toy poodles to be relatively docile, the engineer retrained the model several more times until it produced a result showing smaller poodles to be more violent.

Fairness: Identifying Bias

Missing Feature Values

- If your data set has one or more features that have missing values for a large number of examples, that could be an indicator that certain key characteristics of your data set are under-represented.
- For example, the table below shows a summary of key stats for a subset of features in the California Housing dataset
- Suppose instead that three features (population, households, and median_income) only had a count of 3000—in other words, that there were 14,000 missing values for each feature.
- These 14,000 missing values would make it much more difficult to accurately correlate average income of households with median house prices
- Before training a model on this data, it would be prudent to investigate the cause of these missing values to ensure that there are no latent biases responsible for missing income and population data.

BITS Pilani WILP

Fairness: Identifying Bias

Unexpected Feature Values

- When exploring data, you should also look for examples that contain feature values that stand out as especially uncharacteristic or unusual
- These unexpected feature values could indicate problems that occurred during data collection or other inaccuracies that could introduce bias
- Can you pinpoint any unexpected feature values?

	longitude	latitude	total_rooms	population	households	median_income	median_house_value
1	-121.7	38.0	7105.0	3523.0	1088.0	5.0	0.2
2	-122.4	37.8	2479.0	1819.0	496.0	3.1	0.3
3	-122.0	37.0	2813.0	1337.0	477.0	3.7	0.3
4	-103.5	49.8	2212.0	803.0	144.0	5.3	0.2
5	-117.1	32.6	2963.0	1162.0	556.0	3.6	0.2
6	-118.0	33.7	3396.0	1542.0	472.0	7.4	0.4

Fairness: Identifying Bias

Data Skew

- Any sort of skew in your data, where certain groups or characteristics may be under- or over-represented relative to their real-world prevalence, can introduce bias into your model.

- Figure visualizes a subset of data drawn from the full housing data set that exclusively represents the northwest region of California
- If this unrepresentative sample were used to train a model to predict California housing prices statewide, the lack of housing data from southern portions of California would be problematic
- The geographical bias encoded in the model might adversely affect homebuyers in unrepresented communities

Fairness: Evaluating for Bias

Example

When evaluating a model, metrics calculated against an entire test or validation set don't always give an accurate picture of how fair the model is.

Consider a new model developed to predict the presence of tumors that is evaluated against a validation set of 1,000 patients' medical records. 500 records are from female patients, and 500 records are from male patients. The following confusion matrix summarizes the results for all 1,000 examples:

True Positives (TPs): 16	False Positives (FPs): 4
False Negatives (FNs): 6	True Negatives (TNs): 974

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{16}{16 + 4} = 0.800$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{16}{16 + 6} = 0.727$$

These results look promising: precision of 80% and recall of 72.7%.

Fairness: Evaluating for Bias(2)

Example Continued

But what happens if we calculate the result separately for each set of patients?
Let's break out the results into two separate confusion matrices: one for female patients and one for male patients.

Female Patient Results

True Positives (TPs): 10	False Positives (FPs): 1
False Negatives (FNs): 1	True Negatives (TNs): 488

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{10}{10 + 1} = 0.909$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{10}{10 + 1} = 0.909$$

Male Patient Results

True Positives (TPs): 6	False Positives (FPs): 3
False Negatives (FNs): 5	True Negatives (TNs): 486

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{6}{6 + 3} = 0.667$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{6}{6 + 5} = 0.545$$

When we calculate metrics separately for female and male patients, we see stark differences in model performance for each group.

BITS Pilani WILP

Fairness: Evaluating for Bias(3)

Example Continued

Female patients:

- Of the 11 female patients who actually have tumors, the model correctly predicts positive for 10 patients (recall rate: 90.9%). In other words, **the model misses a tumor diagnosis in 9.1% of female cases**.
- Similarly, when the model returns positive for tumor in female patients, it is correct in 10 out of 11 cases (precision rate: 90.9%); in other words, **the model incorrectly predicts tumor in 9.1% of female cases**.

Male patients:

- However, of the 11 male patients who actually have tumors, the model correctly predicts positive for only 6 patients (recall rate: 54.5%). That means **the model misses a tumor diagnosis in 45.5% of male cases**.
- And when the model returns positive for tumor in male patients, it is correct in only 6 out of 9 cases (precision rate: 66.7%); in other words, **the model incorrectly predicts tumor in 33.3% of male cases**.

We now have a much better understanding of the biases inherent in the model's predictions, as well as the risks to each subgroup if the model were to be released for medical use in the general population.

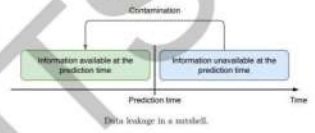
Data Leakage

Pravin Y Pawar

Largely adapted from Machine Learning Engineering
By Andriy Burkov

Data Leakage

- Data leakage, also called target leakage,
  - a problem affecting several stages of the machine learning life cycle, from data collection to model evaluation
- Data leakage in supervised learning is the unintentional introduction of information about the target that should not be made available
  - also called as "contamination"
  - Training on contaminated data leads to overly optimistic expectations about the model performance



Data leakage in a nutshell.

Causes of Data Leakage

- Three most frequent causes of data leakage that can happen during data collection and preparation:
  - target being a function of a feature,
  - feature hiding the target,
  - feature coming from the future

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Target being a function of a feature

Example

- Let goal be to predict a country's GDP based on various attributes:
  - area, population, geographic region, and so on
  - Gross Domestic Product (GDP) is defined as the monetary measure of all finished goods and services in a country within a specific period
  - If you don't do a careful analysis of each attribute and its relation to GDP, you might let a leakage happen:
    - two columns, Population and GDP per capita, multiplied, equal GDP
  - The model will perfectly predict GDP by looking at these two columns only
  - The fact that GDP be one of the features, though in a slightly modified form (devised by the population), constitutes contamination and, therefore, leads to data leakage

- Imagine training a model to predict the yearly salary, given the attributes of an employee
  - The training data is a table that contains both monthly and yearly salary, among many other attributes
  - If monthly salary is not removed from the list of features, that attribute alone will perfectly predict the yearly salary.
    - making believe that model is perfect
  - Once the model is put in production, it will likely stop receiving information about a person's monthly salary: otherwise, the modeling would not be needed

Country	Population	Region	...	GDP per capita	GDP
France	67M	Europe	...	38,900	2.6T
Germany	80M	Europe	...	44,578	3.7T
...	...	...	...	...	...
China	1366M	Asia	...	8,802	12.2T

An example of the target (GDP) being a simple function of two features: Population and GDP per capita.

Feature hiding the target

- Sometimes the target is not a function of one or more features, but rather is "hidden" in one of the features.

- Using customer data to predict their gender
  - Look at the column Group
  - If you closely investigate the data in the column Group, represents a demographic value to which each existing customer was related in the past.
  - If the data about a customer's gender and age is factual, then the column Group constitutes a form of data leakage
    - the value want to predict is "hidden" in the value of a feature

- On the other hand, if the Group values are predictions provided by another, possibly less accurate model
  - then it can be used to build a potentially stronger model - model stacking

Customer ID	Group	Yearly Spendings	Yearly Paycheck	...	Gender
1	M18-25	1360	11,987	...	M
2	F25-35	2365	8,543	...	F
...	...	...	...	...	...
18879	F65+	3651	6,775	...	F

An example of the target being hidden in one of the features.

Feature coming from the future

- Feature from the future is a kind of data leakage that is hard to catch
  - if don't have a clear understanding of the business goal

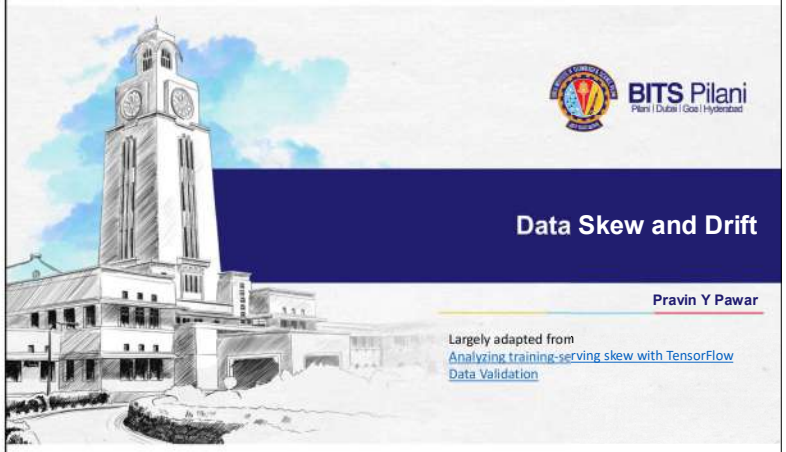
- Imagine a client asked to train a model that predicts whether a borrower will pay back the loan
  - based on attributes such as age, gender, education, salary, marital status, and so on
  - might decide to use all available attributes to predict the value in the column Will Pay Loan, including the data from the column Late Payment Reminders

- Model will look accurate at testing time and model doesn't work well in the production environment
  - in the production environment, the value of Late Payment Reminders is always zero
  - makes sense because the client uses model before the borrower gets the credit, so no reminders have yet been made!

- However, model most likely learned to make the "No" prediction when Late Payment Reminders is 1 or more and pays less attention to the other features
  - Understanding the business context in which the model will be used is, thus, crucial to avoid data leakage.

Borrower ID	Demographic Group	Education	...	Late Payment Reminders	Will Pay Loan
1	M35-50	High school	...	0	Y
2	F25-35	Master's	...	1	N
...	...	...	...	...	...
65725	M25-35	Master's	...	3	N

A feature unavailable at the prediction time: Late Payment Reminders.



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Data Skew and Drift

Pravin Y Pawar

Largely adapted from
Analyzing training-serving skew with TensorFlow Data Validation

BITS Pilani WILP

innovate achieve lead

Production ML is different!

- Machine learning (ML) in production is quite different from ML in a Kaggle competition
  - One of the biggest difference is in the data sets
- Kaggle data sets are always static
  - given a fixed data set for training and you will be evaluated on a fixed data during testing/evaluation.
- Production data tends to be constantly changing over different dimension
  - i.e. time-wise and system-wise
- Monitoring the predictive performance of an ML model in production has emerged as a crucial area of MLOps
- Two common causes of decay in a model's predictive performance over time are the following:
  - Data Drfit
  - Concept drift

Data drift

- Data drift
  - A skew grows between training data and serving data
- In data drift, the production data that a model receives for scoring has diverged from the dataset that was used to train, tune, and evaluate the model
- The discrepancies between training data and serving data can usually be classified as
  - Schema skew
  - Distribution skew

Schema skew

- Schema skew occurs when training data and serving data don't conform to the same schema
  - often caused by faults or changes in the upstream process that generates the serving data
- Schema deviations between training and serving data can include the following:
- Inconsistent features
  - a new feature is added to the serving data
- Inconsistent feature types
  - a numerical feature that was an integer number in training data is a real number in serving data
- Inconsistent feature domains
  - a value in a categorical feature disappears, or there's a change in the range of numerical features

Distribution skew

- Distribution skew occurs when the distribution of feature values for training data is significantly different from serving data
  - can be the result of choosing the wrong training dataset to represent real-world data
  - can also happen naturally as new trends and patterns emerge in the data due to the changes in the dynamics of the environment
- Examples include
  - changes in the prices of real estate
  - a change in the popularity of fashion items

BITS Pilani WILP

Concept drift

- Concept drift means that interpretation of the data has changed
  - As interpretation of the relationship between the input predictors and the target feature evolves
- Often, implies that
  - the mapping of input features to labels that are used during training is no longer valid,
  - or that a novel class or a range of label values has appeared
- Often the result of a change in the process you're attempting to model
  - can also be an evolution of your understanding of this process

Monitoring and detecting drifts and skews

- Prerequisites
  - The system needs to log all incoming features and predictions of the request and response
  - Log the ground truth as well of the associated request, which can be used as additional training data (this is for detecting label drift)
- Use statistical methods to actually detect it
- Supervised monitoring
  - Statistical Process Control
  - Sequential Analysis
  - Adaptive windowing
- Unsupervised monitoring
  - Clustering/novelty detection
  - Feature distribution monitoring
- Model dependent monitoring
  - MD3 – Margin Density Drift Detection

Source: ProductionML: Skews, Data Drift, and Concept Drift

Training-Serving skew

"Past performance is no guarantee of future results."

- Model decay or drift
  - ML models in production can experience reduced performance over time
    - not only due to being fed bad data and poor programming
    - but also due to datasets and profiles that are constantly evolving
- Natural occurrence in ML models and the speed of decay can vary greatly
  - In some models, it can take years
  - In others, it can happen over the course of a few days
- One of the biggest post-production problems that can lead to an expedited rate of decay is data-serving skew
  - a problem that can arise quite easily and be difficult to detect

What is training-serving skew in Machine Learning?

Training-Serving skew

- Training-serving skew is a difference between ML model outputs during the training and during serving (deployment)
  - essentially a discrepancy between an ML model's feature engineering code during training and during deployment
- Can be caused by:
  - A discrepancy between how data is handled in the training and serving pipelines
  - A change in the data between training and serving
  - A feedback loop between model and algorithm
- Very easy for training-serving skew to crop up
  - via a bug in model's code and cause serious repercussions further down the line, potentially stopping model from working entirely

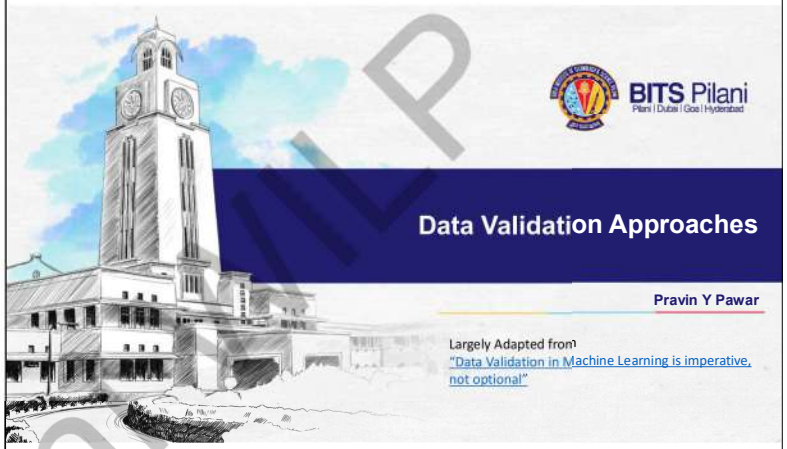
What is training-serving skew in Machine Learning?

BITS Pilani VLP

Avoiding training-serving skew

- Feature Engineering Code
  - Ideally, engineers should be re-using the same feature engineering code
  - to ensure that any given raw data input maps to the same feature vector during training and deployment (serving)
- Computational resources
  - One of the most common reasons for this skew is a mismatch in computational resources at training and deployment time

What is training-serving skew in Machine Learning?



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Data Validation Approaches

Pravin Y Pawar

Largely Adapted from[1]
"Data Validation in Machine Learning is imperative, not optional"

Data validation

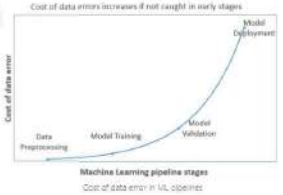
Definition

- Data Validation
  - means checking the accuracy and quality of source data before training a new model version
  - ensures that anomalies that are infrequent or manifested in incremental data are not silently ignored
  - focuses on checking that the statistics of the new data are as expected (e.g. feature distribution, number of categories, etc)
- Examples of such validations in the machine learning pipeline:
  - Are there any anomalies or data errors in the incremental data?
  - Are there any assumptions on data that are taken during model training and are getting violated during serving?
  - Are there significant differences between training and serving data?
  - Are there differences between successive data that are getting added into training data?
- Output from the data validation steps should be informative enough for a data engineer to take action
  - needs to have high precision as too many false alarms will easily be lost credibility

Data validation(2)

Requirement

- "garbage in garbage out"
  - Machine learning models are vulnerable to poor data quality
- In Production
  - model gets re-trained with a fresh set of incremental data added periodically (as frequent as daily)
  - updated model is pushed to the serving layer
  - deployed model makes predictions with new incoming data while serving and the same data is added with actual labels and used for retraining
  - ensures that the newly generated model adapts to the changes in data characteristics
- New incoming data in the serving layer can change due to various reasons
  - with time, the erroneous ingested data will become part of the training data, which will start degrading the model accuracy
- Catching the data errors at an early stage is very important
  - will reduce the cost of data error which is bound to increase as the error propagates further in the pipeline



BITS Pilani WILP

Challenges with the data validation

- Creating data validation rules for a dataset with few columns does sound simple
  - when the number of columns in datasets increases, it becomes a humongous task
- Tracking and comparing metrics from the historical datasets to find anomalies in historical trends for each column needs a good amount of recurring time from a data scientist
- Today applications are expected to run 24-by-7 and in such scenarios, data validation needs to be automated
  - data validation component should be smart enough to refresh validation rules

Working of data validation component

- Data validation component serves as a guard post of the ML application
  - does not let bad quality data in
  - keeps a check on each and every new data entry that is going to add to the training data
-
- Steps
  - Calculate the statistics from the training data against a set of rules
  - Calculate the statistics of the ingested data that has to be validated
  - Compare the statistics of the validation data with the statistics from the training data
  - Store the validation results and takes automated actions like removing the row, capping, or flooring the values
  - Sending the notification and alerts for approval

Unit-test approach by Amazon Research

Deequ

- In software engineering, engineers write unit tests to test their code
  - Similarly, unit tests should also be defined to test the incoming data
- AWS Deequ framework follows the below principles
- Declare constraints:
  - User defines how their data should look like
  - by declaring checks on their data by composing constraints on various columns
- Compute metrics
  - Based on the declared constraint, translate them to measurable metrics
  - metrics can be computed and compared between the data in hand and the incremental data
- Analyze and report
  - Based on the collected metrics over time, predict if the metric on the incremental data is an anomaly or not
  - As a rule, the user can have the system issue "a warning" if the new metric is more than three standard deviations away from the previous mean or throw "an error" if it is more than four standard deviations away.
  - Basis the analysis, report the constraints that fail including the value(s) that made the constraint fail

Unit-test approach by Amazon Research (2)

Deeque - Constraints

Table 1: Constraints available for composing user-defined data quality checks.

constraint	arguments	semantic
dimension completeness		
isComplete	column	check that there are no missing values in a column
hasCompleteness	column, udf	custom validation of the fraction of missing values in a column
dimension consistency		
isUnique	column	check that there are no duplicates in a column
hasUniqueness	column, udf	custom validation of the unique value ratio in a column
hasDistinctness	column, udf	custom validation of the unique row ratio in a column
isInRange	column, value range	validation of the fraction of values that are in a valid range
statistics (can be used to verify dimension consistency)		
hasSize	udf	custom validation of the number of records
hasTypeConsistency	column, udf	custom validation of the maximum fraction of values of the same data type
hasCountDistinct	column	custom validation of the number of distinct non-null values in a column
hasApproxCountDistinct	column, udf	custom validation of the approx. number of distinct non-null values
hasMin	column, udf	custom validation of a column's minimum value
time		
hasAnomalies	metric, detector	validation of anomalies in time series of metric values

Table 2: Computable metrics to base constraints on.

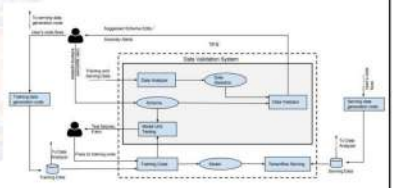
metric	semantic
dimension completeness	
Completeness	fraction of non-missing values in a column
dimension consistency	
Size	number of records
Compliance	ratio of columns matching predicate
Uniqueness	unique value ratio in a column
Distinctness	unique row ratio in a column
statistics (can be used to verify dimension consistency)	
Minimum	minimal value in a column
Maximum	maximal value in a column
Mean	mean value in a column
StandardDeviation	standard deviation of the value distribution in a column

BITS Pilani WILP

Data schema approach for data validation by Google Research

Tensorflow data validation

- Google Research has come up with a very similar technique
  - adopted "battle-tested" principles from the data management system and customized it for ML
  - first codifies the expectation from correct data and then using these expected statistics along with user-defined validation schema performs data validation
- Components
- Data Analyzer
  - computes a predefined set of data statistics required to define data
- Data Validator
  - checks for properties of data as specified through a schema
  - schema is a precursor for a data validator to perform
  - schema list out all the constraints on the features for basic checks and ML-related checks
- Model Unit Tester
  - checks for errors in training code using synthetic data generated through the schema



Differences between Deequ (Amazon) Tensorflow data validation (Google)

	Deequ (Amazon)	Tensorflow data validation (Google)
1	No visualization available	Provides visualization using Google Facets. It summarises statistics for each feature and compares the traini and validation data.
2	Recalculates training statistics by aggregating prior saved training statistics and new data statistics.	Calculates statistics on whole training data in every run unless specified. Thi may become computationally expensive.
3	Provides capability to do anomaly detection based on running average and standard deviation in addition to the threshold or absolute/relative difference from training data.	Provides capability to do anomaly detection based on the threshold or absolute/relative difference from training data.
4	Supports data only in SparkDataFrame.	Supports pandas dataframe, CSV, and best works with TFRecord.

Differences between Data Validation libraries

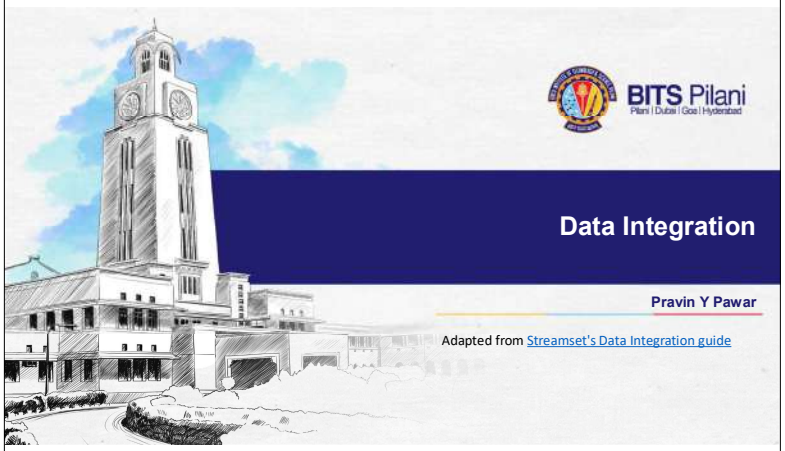
References
S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Viessmann and A. Grafberger, "Automating Large-Scale Data Quality Verification" in Proceedings of the VLDB endowment, volume 11, Issue 12: 1781-1794, 2018.

E. Breck, M. Zinkevich, N. Polyzotis, S. Whang and S. Roy, "Data validation for machine learning", in Proceedings of the 2nd SysML Conference, Palo Alto, CA, USA, 2019

Thank You!

In our next session:

BITS Pilani WILP



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Data Integration

Pravin Y Pawar

Adapted from Streamset's Data Integration guide

Data integration

Meaning

- Data integration combines various types and formats of data from any source across an organization into a data lake or data warehouse
  - to provide a unified fact base for analytics
- Unified datasets allows businesses to
  - make better decisions,
  - aligns departments to work better together,
  - and drives better customer experience
- Means consolidating data from multiple sources into a single dataset
  - to be used for consistent business intelligence or analytics
- Very simple explanation for a complex topic that has evolved over its 30 year history
  - data integration has transitioned from a backend, retrospective process into core real-time infrastructure

Data integration(2)

Working

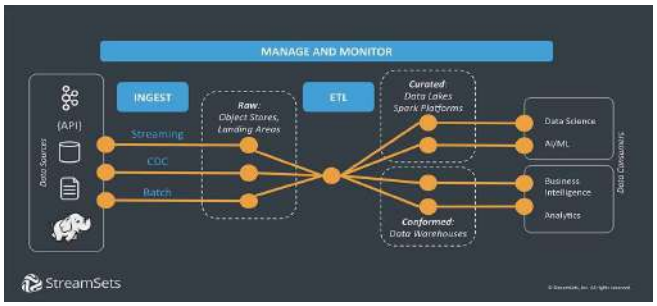
- To move data from one system to another requires a data pipeline that
  - understands the structure and meaning of the data
  - as well as defines the path it will take through the technical systems
- A relatively simple and common type of data integration is data ingestion,
  - where data from one system is integrated on a regular basis into another system
- More complex way
  - may also include cleansing, sorting, enrichment and other processes to make the data ready for use at its final destination
  - Sometimes this happens before the data is stored and the process is called ETL (extract, transform, load)
  - Other times it makes more sense to store the data first, then prepare it for use known as ELT (extract, load, transform)

Data Integration Evolution

- In the early 1990s, when companies began adopting data warehouses to collect data from multiple systems to fuel analysis,
  - there were no smartphones or ecommerce
  - Salesforce and Software as a Service as a category did not yet exists
  - Amazon had not sold a single book, much less on-demand computing
- Back then:
  - Data came from business applications and operational databases in a structured format that could be mapped to the structure required for analysis
  - Data arrived and was processed in batches, creating snapshots of the business in time and stored in data warehouses or data marts
  - Data was used for financial reporting, sales dashboards, supply chain analytics, and other essential functions of the enterprise
- Data integration was primarily the responsibility of ETL developers,
  - who used hand coding or specialized software to create ETL mappings and jobs
  - developed specialized skills related to the source and target systems they integrated
- Data integration was owned and governed by enterprise IT
  - with control of the hardware and software used to collect data, store it, and analyze it
  - focused on performance, security, and cost of the monolithic data management systems

BITS Pilani WILP

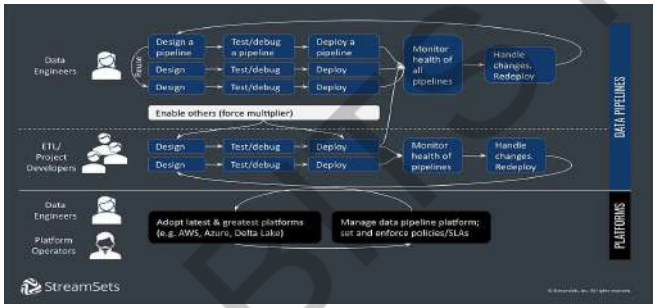
Modern Data Integration Space



Modern Data Integration Challenges

- The world according to data looks very different today!
- Enterprise data integration have been transformed by
  - explosion of data, data sources (IoT, APIs, cloud applications, on premise data, various databases, and more)
  - data structures combined with radical innovation in infrastructure services, compute power, analytic tools and machine learning
- Demand for (continuous) data
  - Real-time decision making and real-time services require continuous data that is transformed in flight
  - DevOps and agile software development practices have spread throughout the organization
    - increasing the demand for always on, self-service data
  - The move from on-premises to the cloud for applications as well as computing services requires cloud data integration,
    - data integration beyond the walled garden of the corporate data center
- Suddenly, the full lifecycle of data integration matters as much as the initial implementation
  - has to support continuous integration of data from different sources, and continuous data delivery as well as continuous innovation,
    - and that takes automation
  - focus is not just on the "how" of implementation, but on "what" is needed by the business

New Roles and New Responsibilities



Data Integration Tools

- Data Integration tools are software-based tools that ingest, consolidate, transform, and move data from source(s) to destination,
  - performing mappings, transformations, and data cleansing along the way
- Ultimately, they integrate the data into a 'single source of truth' destination,
  - such as a data lake or data warehouse
  - allows consistent, reliable data for use in analytics and business intelligence

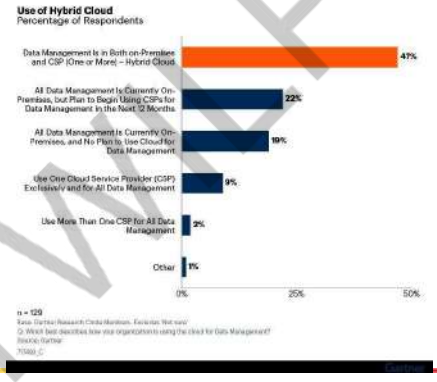
BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Data Integration Tool Considerations for the Age of Data Engineering

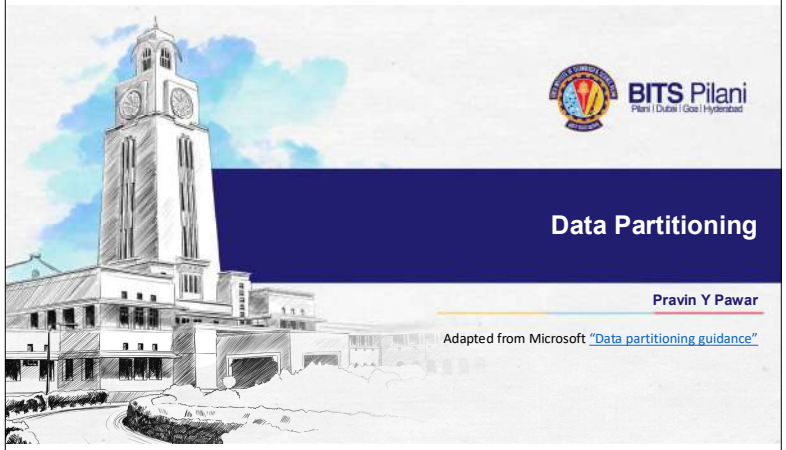
- When choosing a tool for data integration, there are a few important considerations to take into account:
- What type of data will be in your data pipeline?
  - Structured,
  - Unstructured
  - and Semi-structured Data
- How will that data be processed?
  - Batch,
  - Micro-batch,
  - and Stream Processing
- Where will the data come from and go to?
  - On-premises,
  - Cloud,
  - Multi-cloud,
  - and Hybrid Architectures

Moving to Hybrid Cloud



Thank You!

In our next session:



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Data Partitioning

Pravin Y Pawar

Adapted from Microsoft "Data partitioning guidance"

innovate achieve lead

BITS Pilani WILP

Data partitioning guidance

- In many large-scale solutions, data is divided into partitions
  - can be managed and accessed separately
- Partitioning can
  - improve scalability,
  - reduce contention,
  - and optimize performance
- Can also provide a mechanism for dividing data by usage pattern
  - can archive older data in cheaper data storage
- However, the partitioning strategy must be chosen carefully to maximize the benefits while minimizing adverse effects

Why partition data?

Reasons

- Improve scalability
  - When a single database system scaled up, it will eventually reach a physical hardware limit
  - If data is divided across multiple partitions, each hosted on a separate server, can scale out the system almost indefinitely
- Improve performance
  - Data access operations on each partition take place over a smaller volume of data
  - Correctly done, partitioning can make system more efficient
  - Operations that affect more than one partition can run in parallel
- Improve availability
  - Separating data across multiple servers avoids a single point of failure
  - If one instance fails, only the data in that partition is unavailable, operations on other partitions can continue
- Improve security
  - In some cases, sensitive and non-sensitive data can be separated into different partitions and apply different security controls to the sensitive data

Designing partitions

Three typical strategies for partitioning data

- Horizontal partitioning (aka sharding)
  - each partition is a separate data store, but all partitions have the same schema
  - each partition is known as a shard and holds a specific subset of the data
  - such as all the orders for a specific set of customers
- Vertical partitioning
  - each partition holds a subset of the fields for items in the data store
  - fields are divided according to their pattern of use
  - frequently accessed fields might be placed in one vertical partition and less frequently accessed fields in another
- Functional partitioning
  - data is aggregated according to how it is used by each bounded context in the system
  - an e-commerce system might store invoice data in one partition and product inventory data in another
- These strategies can be combined
  - might divide data into shards and then use vertical partitioning to further subdivide the data in each shard

Horizontal partitioning (sharding)

Example Product data

- Product inventory data is divided into shards based on the product key
- Each shard
  - holds the data for a contiguous range of shard keys (A-G and H-Z) organized alphabetically
- Sharding spreads the load over more computers, which reduces contention and improves performance

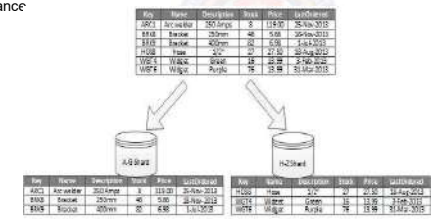


Figure – Horizontally partitioning (sharding) data based on a partition key

BITS Pilani WILP

Horizontal partitioning (sharding) - 2

Practices

- Sharding key
  - most important factor is the choice of a sharding key
  - can be difficult to change the key after the system is in operation
  - key must ensure that data is partitioned to spread the workload as evenly as possible across the shards
  - Choose a sharding key that minimizes any future requirements to split large shards, coalesce small shards into larger partitions,
  - Changing the schema can be very time consuming, and might require taking one or more shards offline while they are performed
- Size
  - The shards don't have to be the same size
  - more important to balance the number of requests
  - Some shards might be very large, but each item has a low number of access operations
  - Other shards might be smaller, but each item is accessed much more frequently
- Access
  - Avoid creating "hot" partitions that can affect performance and availability
  - using the first letter of a customer's name causes an unbalanced distribution, because some letters are more common
  - instead, use a hash of a customer identifier to distribute data more evenly across partitions
- Replication
  - If shards are replicated, it might be possible to keep some of the replicas online while others are split, merged, or reconfigured
  - the system might need to limit the operations that can be performed during the reconfiguration
  - the data in the replicas might be marked as read-only to prevent data inconsistences

Vertical partitioning

Example of vertical partitioning

- Different properties of an item are stored in different partitions
  - One partition holds data that is accessed more frequently, including product name, description, and price
  - Another partition holds inventory data: the stock count and last-ordered date

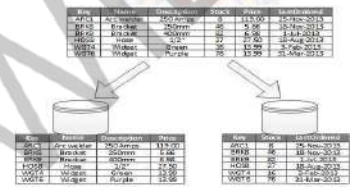


Figure - Vertically partitioning data by its pattern of use.

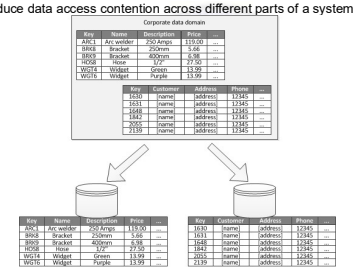
Vertical partitioning (2)

Advantages

- The most common use is to reduce the I/O and performance costs associated with fetching items that are frequently accessed
- Vertical partitioning operates at the entity level within a data store,
  - partially normalizing an entity to break it down from a wide item to a set of narrow items
  - ideally suited for column-oriented data stores such as HBase and Cassandra
- Relatively slow-moving data (product name, description, and price) can be separated from the more dynamic data (stock level and last ordered date)
  - Slow moving data is a good candidate for an application to cache in memory
- Sensitive data can be stored in a separate partition with additional security controls
- Vertical partitioning can reduce the amount of concurrent access that's needed

Functional partitioning

- Functional partitioning is a way to improve isolation and data access performance
  - suitable when possible to identify a bounded context for each distinct business area in an application
  - can be used to separate read-write data from read-only data
  - can help reduce data access contention across different parts of a system



Functionally partitioning data by bounded context or subdomain.

BITS Pilani WILP

Rebalancing partitions

- As a system matures, need to adjust the partitioning scheme!
- For example,
  - individual partitions might start getting a disproportionate volume of traffic and become hot, leading to excessive contention
  - might have underestimated the volume of data in some partitions, causing some partitions to approach capacity limits
- Some data stores, such as Azure Cosmos DB, can automatically rebalance partitions
- In other cases, rebalancing is an administrative task that consists of two stages:
  - Determine a new partitioning strategy.
    - Which partitions need to be split (or possibly combined)?
    - What is the new partition key?
  - Migrate data from the old partitioning scheme to the new set of partitions.
- Migrations
  - Depending on the data store, might be able to migrate data between partitions while they are in use - online migration
  - If that's not possible, might need to make partitions unavailable while the data is relocated - offline migration

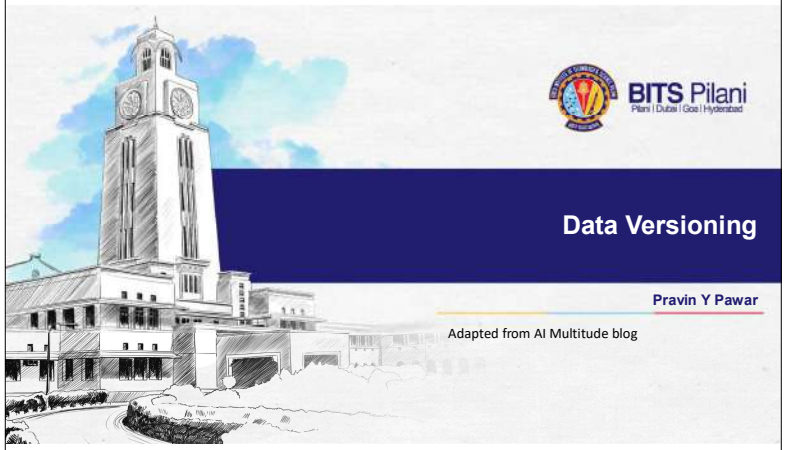
Rebalancing partitions

Migrations

- Offline migration
  - is typically simpler because it reduces the chances of contention occurring
  - Works as follows:
    - Mark the partition offline
    - Split-merge and move the data to the new partitions
    - Verify the data
    - Bring the new partitions online
    - Remove the old partition
    - Optionally, you can mark a partition as read-only in step 1, so that applications can still read the data while it is being moved
- Online migration
  - more complex to perform but less disruptive
  - similar to offline migration, except the original partition is not marked offline
  - Depending on the granularity of the migration process, the data access code in the client applications might have to handle reading and writing data that's held in two locations, the original partition and the new partition

Thank You!

In our next session:

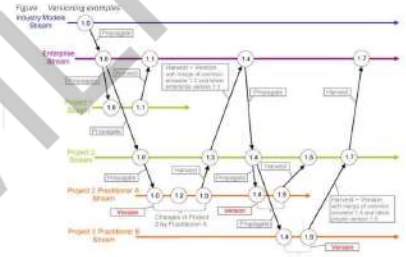


BITS Pilani WILP

Data Versioning

Data Versioning(2)

- Data versioning is the storage of different versions of data that were created or changed at specific points in times
- Many different reasons for making changes to the data
  - Data scientists might test the ML models to increase efficiency
    - therefore make certain changes to the dataset
  - Datasets can also change over time due to the flow of information
    - storing older versions of data can help organizations replicate the previous environment



Source: IBM

Benefits of data versioning

Old versions of data is saved and kept at the company's disposal

- Preserving the working version while testing
  - AI/ML models work with the goal of maximizing business efficiency - normal for development teams to test new ways to increase efficiency
  - Introducing a new dataset into systems could be one of those exercises
  - However, in search of the uncertain better, no one wants to risk the previous working version
  - most of the engineers' attempts end up as inefficient trials - need to save the previous dataset
  - If an attempt fails, they simply reload the old working data set into the pipeline, preventing potential loss of business
- Measuring the business performance
  - Without the intervention of engineers, datasets can change over time - Sales data is changed by each transaction
  - Storing sales data from different years over a period of time can be insightful for businesses to understand consumer preferences
  - Consequently, versioning data can lead to a more profitable business by analyzing changes in customer trends
- Compliance and auditing benefits
  - Data versioning can help with both internal and external audits and compliance processes by ensuring data is stored from specific times
  - Some data protection regulations, such as GDPR, force companies to store certain data sources
  - Data versioning can save companies' time in meeting such requirements
  - can also be easier for companies to detect fraud if they have versioned their data

Main formats for data versioning

- There is no standard model for data versioning, but there are some common formats that are widely used:
- The three-part semantic version number convention
  - is the most common format for indicating different versions
  - For example, 3.2.4 indicates a specific data version
    - The left-hand side number (3) indicates a significant change between data versions
    - The middle number (2) indicates new features in a compatible manner
    - and the right-hand side number (4) indicates minor bug fixes compared to older versions
- Naming data versions depending on their status
  - For example, a dataset could be incomplete-complete, filtered-unfiltered, cleaned-uncleaned, etc.
  - Specifying this information could be helpful for practitioners, especially when they work together on a dataset via a cloud system
- Naming data versions depending upon process
  - Data version can be named subject to the latest process it is exposed to
  - For instance, normalized or adjusted according to something etc.

BITS Pilani WILP

Options for versioning the data

Use file versioning

- File versioning
  - Manually saving versions to computer is one of the options for data versioning
- Appropriate for:
  - Small firms
    - Small firms with only a few data engineers or scientists working in the same location
  - Protecting sensitive information
    - If data contains particularly sensitive information, it should be viewed and interpreted only by a few executives and data engineers
  - Individual work
    - If the task is not suitable for teamwork, where different people cannot work together to achieve a final goal

Options for versioning the data(2)

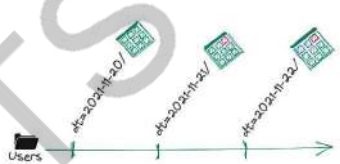
Using a data versioning tool

- Data versioning tool
  - Specialized tools offer an alternative to file versioning
  - can either develop own software or outsource it - providers offering such services, such as DVC, Delta Lake, and Pachyderm
- Data versioning tools are more suitable for companies that need:
  - Real-time editing
    - If more than one person is working on a dataset, using a specialized tool is more efficient
    - because file versioning does not allow real-time editing with a group of people
  - Collaboration from different locations
    - When people need to collaborate in different locations, using a software is more efficient than file versioning
  - Accountability
    - Data versioning software makes it possible to determine in which steps errors occur and who makes the error.
    - Consequently, the accountability of the team is enhanced

Data Versioning Implementation

Versioning Approach #1: Full Duplication

- One option is to save a full copy of it under a new location each time want a version of it
  - works best for smaller datasets with something like a daily versioning frequency
  - does create versioned data, it does so in the least space-efficient way
  - code or queries that interact with this versioned data will be error-prone with the correct date value having to be manually hardcoded in different places
  - Although not the most elegant solution, it is an easy way to get started versioning data

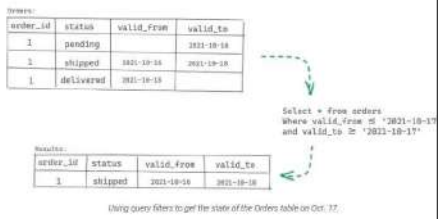


Versioning via saving a full copy of an example users dataset daily.

Data Versioning Implementation (2)

Versioning Approach #2: "Valid_from/to" Metadata

- A more space-efficient and incremental approach to versioning works by adding and maintaining two metadata fields in a tabular dataset
  - often named valid_from and valid_to
  - When updating a record in this dataset, make sure to never overwrite an existing record
  - Instead, append new records and update the valid_to field to the current timestamp for any record that would have been overwritten
- This approach works quite well for "time traveling" throughout a single collection of tabular data
- However, it provides only one method of interacting with the versions—which is to add filters to queries on the metadata fields



Using query filters to get the state of the Orders table on Oct. 17.

BITS Pilani WILP

Data Versioning Implementation (3)

Versioning Approach #3: First-class Data Version Control

- First two approaches can be summarized as "Let me add a bit of versioning to the data I already have"
  - Instead, should think of versioning as a first-class citizen of data environment
  - An inherent property of any data we introduce into the system
- To make this possible - need to solve a few challenges.
  - Minimize the storage footprint of data versioning
    - means not creating copies of data objects that remain unchanged between versions
  - Expose operations that let us interact directly with the versions
    - Things like "create a version", "delete a version", "compare two versions" are examples
  - Work the same over any scale of data, data format, and both structured and unstructured data
- Now, these are not simple problems
  - And probably won't hack way to a solution in an afternoon or even a weekend
  - One of the more popular approaches to solving these extends the git version control model to data
    - projects like lakeFS, DVC, and git LFS

Challenges to data versioning

- Limited storage space
  - Each versioning of data means that more storage space is required
  - For companies that produce or use large amounts of data, it would therefore be costly to version the data too often
  - important for companies to find an optimal balance between the benefits of versioning and the costs incurred by storage
- Security issues
  - Ensuring data security is essential for businesses to protect their reputation
  - However, as more and more versions of data are stored, the risk of data loss or leakage increases
  - For cloud users, in particular, this risk is even greater as they simply outsource their IT functions, giving them less control over their data
  - important for organizations to assess and understand this risk in order to determine an optimal data versioning strategy
- Choosing the right provider
  - If decide to use a data versioning tool, want to choose the most suitable option that meets business requirements
  - Different cloud providers offer different features and charge different prices
  - advisable to evaluate the different options in order to ensure cloud cost optimization
  - Should compare the tools according to the following criteria:
    - Open source or not
    - Storage capacity
    - Has a user friendly interface or not
    - Support of most common clouds (like AWS) and storage types or not
    - Cost

Thank You!

In our next session:

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Analytics

Pravin Y Pawar

Analytics

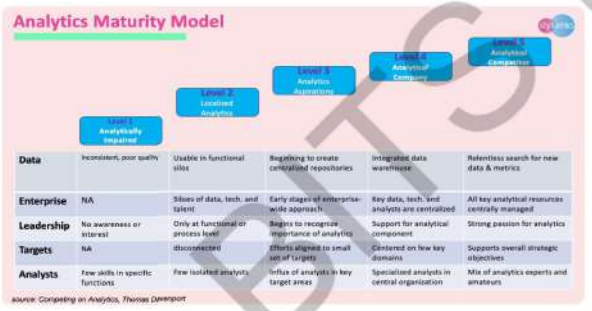
Defined

- Extensive use of data, statistical and quantitative analysis, explanatory and predictive models, and fact based management to drive decisions and actions

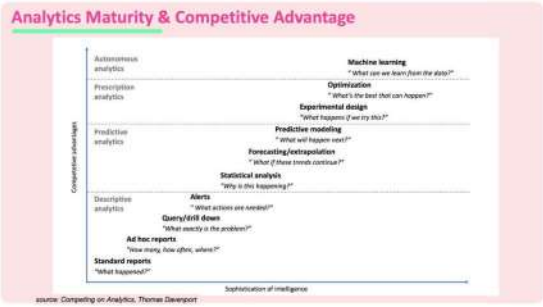
- Purpose
  - ✓ To unearth hidden patterns
  - ✓ To decipher unknown correlations
  - ✓ Understand the rationale behind trends
  - ✓ Mine useful business information

- Helps in
  - ✓ Effective marketing
  - ✓ Better customer service and satisfaction
  - ✓ Improved operational efficiency
  - ✓ Competitive advantage over rivals

Analytics Maturity Model



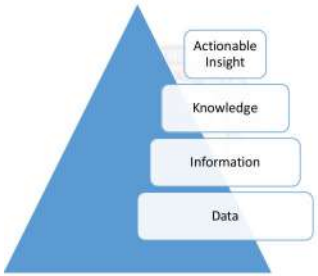
Analytics Maturity and Competitive Advantage



BITS Pilani WILP

Process of Analysis

Transformation of Data



Types of Analytics (1)

Descriptive Analytics

- aka Business Intelligence (BI) or Performance reporting
- Provides access to historical and current data
- Provides ability to alert, explore and report using internal and external data
- Reports on events , occurrences of the past
- Usually data from legacy systems used for analysis
- Based on relational databases
- Sometimes also referred as Analytics 1.0
- Era : mid 1950s to 2009
- Questions asked
  - ✓ What happened?
  - ✓ Why did it happen?

Types of Analytics (2)

Predictive Analytics

- Uses quantitative techniques like segmentation, forecasting etc, but also makes use of descriptive analytics for data exploration
- Era : from 2005 to 2012
- Uses technologies like models and rule based systems
- Uses past data to predict the future
- Based on huge data gathered over period of time
- Externally sourced data also used
- aka Analytics 2.0
- Key questions
  - ✓ What will happen?
  - ✓ Why it will happen?

Types of Analytics (3)

- Prescriptive
  - ✓ Uses a variety of quantitative techniques like optimization and technologies like models, machine learning and recommendations engines
  - ✓ Suggests optimal behaviors and actions
  - ✓ Aka Analytics 3.0 = Descriptive + Predictive + Prescriptive
  - ✓ Uses data from past to make prophecies of future and at the same time make recommendations to leverage the situation to one's advantage
  - ✓ Data is blend from Big data and legacy systems , ERP, CRM etc.
- Autonomous
  - ✓ Uses AI or cognitive technologies to create and improve models and learn from data
  - ✓ All without human hypothesis and with less involvement by human analysis

BITS Pilani WILP

Another thought

- Basic Analytics
  - ✓ Slicing and dicing of data to help with basic insights
  - ✓ Reporting on historical data, basic visualizations etc.
- Operationalized Analytics
  - ✓ Analytics integrated in business processes
- Advanced Analytics
  - ✓ Forecasting the future by predictive modelling

6.1 Introduction to OLAP

Characteristics of Strategic Information

Integrated	Must have a single, enterprise-wide view
Data Integrity	Information must be accurate and must conform to business rules
Accessible	Easily accessible with intuitive access paths, and responsive for analysis
Credible	Every business factor must have one and only one value
Timely	Information must be available within the stipulated time frame

BITS Pilani WILP

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

What is OLAP?

OLAP is a category of software technology that enables analysts, managers, and executives to gain insight into data through fast, consistent, interactive access to a wide variety of possible views of information that has been transformed from raw data to reflect the real dimensionality of the enterprise as understood by the user.

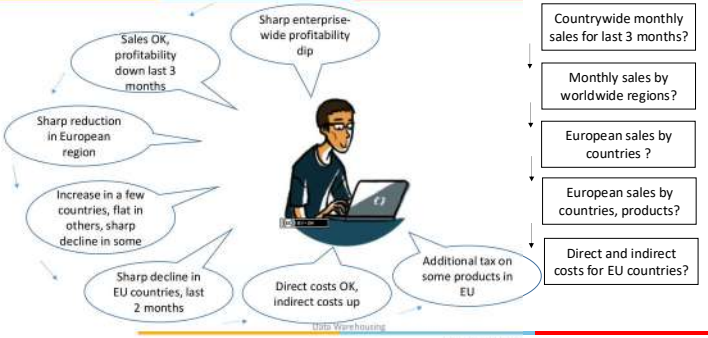
Codd's Rules for OLAP

1. **Multidimensional Conceptual View**. Provide a multidimensional data model that is intuitively analytical and easy to use. Business users' view of an enterprise is multidimensional in nature. Therefore, a multidimensional data model conforms to how the users perceive business problems.
2. **Transparency** Make the technology, underlying data repository, computing architecture, and the diverse nature of source data totally transparent to users. Such transparency, supporting a true open system approach, helps to enhance the efficiency and productivity of the users through front-end tools that are familiar to them.
3. **Accessibility** Provide access only to the data that is actually needed to perform the specific analysis, presenting a single, coherent, and consistent view to the users. The OLAP system must map its own logical schema to the heterogeneous physical data stores and perform any necessary transformations.
4. **Consistent Reporting Performance** Ensure that the users do not experience any significant degradation in reporting performance as the number of dimensions or the size of the database increases. Users must perceive consistent run time, response time, or machine utilization every time a given query is run.
5. **Client/Server Architecture** Conform the system to the principles of client/server architecture for optimum performance, flexibility, adaptability, and interoperability. Make the server component sufficiently intelligent to enable various clients to be attached with a minimum of effort and integration programming.
6. **Generic Dimensionality** Ensure that every data dimension is equivalent in both structure and operational capabilities. Have one logical structure for all dimensions. The basic data structure or the access techniques must not be biased toward any single data dimension.

Codd's Rules for OLAP

7. **Dynamic Sparse Matrix Handling** Adapt the physical schema to the specific analytical model being created and loaded that optimizes sparse matrix handling. When encountering a sparse matrix, the system must be able to dynamically deduce the distribution of the data and adjust the storage and access to achieve and maintain consistent level of performance.
8. **Multiuser Support** Provide support for end users to work concurrently with either the same analytical model or to create different models from the same data. In short, provide concurrent data access, data integrity, and access security.
9. **Unrestricted Cross-dimensional Operations** Provide ability for the system to recognize dimensional hierarchies and automatically perform roll-up and drill-down operations within a dimension or across dimensions. Have the interface language allow calculations and data manipulations across any number of data dimensions, without restricting any relations between data cells, regardless of the number of common data attributes each cell contains.
10. **Intuitive Data Manipulation** Enable consolidation path reorientation (pivoting), drill-down and roll-up, and other manipulations to be accomplished intuitively and directly via point-and-click and drag-and-drop actions on the cells of the analytical model. Avoid the use of a menu or multiple trips to a user interface.
11. **Flexible Reporting** Provide capabilities to the business user to arrange columns, rows, and cells in a manner that facilitates easy manipulation, analysis, and synthesis of information. Every dimension, including any subsets, must be able to be displayed with equal ease.
12. **Unlimited Dimensions and Aggregation Levels** Accommodate at least fifteen, preferably twenty, data dimensions within a common analytical model. Each of these generic dimensions must allow a practically unlimited number of user-defined aggregation levels within any given consolidation path.

An Analysis Session



BITS Pilani WILP

innovate achieve lead

Typical Calculations

- Roll-ups to provide summaries and aggregations along the hierarchies of the dimensions.
- Drill-downs from the top level to the lowest along the hierarchies of the dimensions,
- in combinations among the dimensions.
- Simple calculations, such as computation of margins (sales minus costs).
- Share calculations to compute the percentage of parts to the whole.
- Algebraic equations involving key performance indicators.
- Moving averages and growth percentages.
- Trend analysis using statistical methods.

OLAP operations

LINE	TOTAL SALES
Clothing	$12,836,450
Electronics	$16,068,300
Video	$21,262,190
Kitchen	$17,704,400
Appliances	$19,600,800
Total	$87,472,140

Drill down

LINE	1998	1999	2000	TOTAL
Clothing	$3,457,000	$3,590,050	$5,789,400	$12,836,450
Electronics	$5,894,800	$4,078,900	$6,094,600	$16,068,300
Video	$7,198,700	$6,057,890	$8,005,600	$21,262,190
Kitchen	$4,875,400	$5,894,500	$6,934,500	$17,704,400
Appliances	$5,947,300	$6,104,500	$7,549,000	$19,600,800
Total	$27,373,200	$25,725,840	$34,373,100	$87,472,140

Rotate / Pivot

YEAR	Clothing	Electronics	Video	Kitchen	Appliances	TOTAL
1998	$3,457,000	$5,894,800	$7,198,700	$4,875,400	$5,947,300	$27,373,200
1999	$3,590,050	$4,078,900	$6,057,890	$5,894,500	$6,104,500	$25,725,840
2000	$5,789,400	$6,094,600	$8,005,600	$6,934,500	$7,549,000	$34,373,100
Total	$12,836,450	$16,068,300	$21,262,190	$17,704,400	$19,600,800	$87,472,140

Limitations of Other Tools

- Users need ability to analyse the data along multiple dimensions and their hierarchies rapidly
- Spreadsheets can be cumbersome to use, particularly for large volumes of data.
- Multidimensional data entered in spreadsheet has lot of redundancy
- It will require enormous effort to do create multidimensional view

Limitations of Other Tools

- SQL was originally meant to be end-user query language
- Except for very simple operations, the syntax is not easy to conceptualize for end-users
- The vocabulary is not suitable for analysis, comparisons are a challenge
- SQL is not good with complex calculations and time-series data.

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Limitations of Other Tools

- A real-world analysis session requires many queries following one after the other.
- Each query may translate into a number of statements invoking full table scans, multiple joins, aggregations, groupings, and sorting.
- The overhead on the systems would be enormous and seriously impact the response times

Features of OLAP

Multidimensional analysis	Consistent performance	Fast response times for interactive queries
Drill-down and roll-up	Navigation in and out of details	Slice-and-dice or rotation
Multiple view modes	Easy scalability	Time intelligence (yearto-date, fiscal period)

Basic Features

Advanced Features

Powerful calculations	Cross-dimensional calculations	Pre-calculation or pre-consolidation
Drill-through across dimensions or details	Sophisticated presentation & displays	Collaborative decision making
Derived data values through formulas	Application of alert technology	Report generation with agent technology

CUBE Operator in SQL

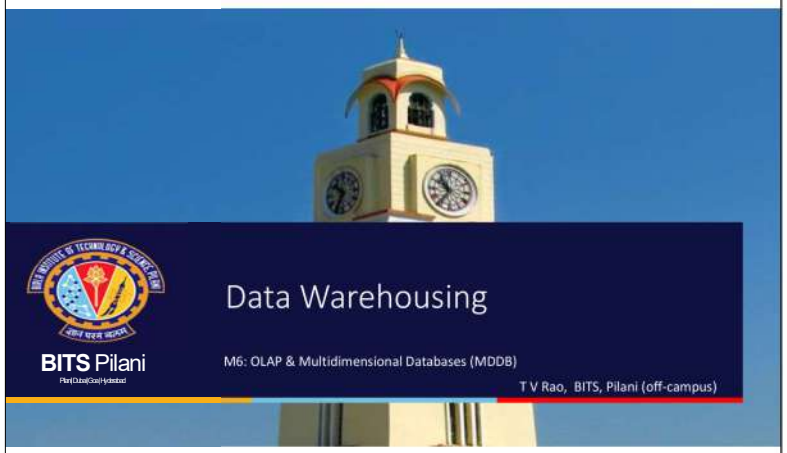
- A cube aggregates the facts in each level of each dimension in a given OLAP schema
  - Data cubes are not "cubes" in the strictly mathematical sense because they do not have equal sides.
  - Most likely, there are more than 3 dimensions
- Major SQL vendors provide cube operator in their products
- Typical sequence for Cube computation:
  - Identify physical sources of data
  - Specify logical views built upon physical source
  - Build cube for specified measures and dimensions

Prescribed Text Books

	Author(s), Title, Edition, Publishing House
T1	Ponniah P, "Data Warehousing Fundamentals", Wiley Student Edition, 2012
T2	Kimball R, "The Data Warehouse Toolkit", 3e, John Wiley, 2013

BITS Pilani WILP

innovate achieve lead



Why Multidimensional Database

- In 1960s, when a research scholar at MIT was doing analytical work on product sales, he realized that
  - he spent most of his time wrestling with reformatting the data for his analysis,
  - not on the statistical algorithms or the true data analysis
- Once he had modeled the data in a multidimensional form, he was able to report the data in many different formats
- By abstracting the data model from the data itself, the user could work with the data in an ad hoc fashion, asking questions that had not been formulated when developing the specifications

Multidimensional Database

- A multidimensional database, or MDB, stores dimensional information in a format called a *cube*.
- The basic concept of a cube is to precompute the various combinations of dimension values and fact values so they can be studied interactively
- Data in an MDB is accessed through an interface, which is often proprietary, although MDX (MultiDimensionalEXpressions) has gained wide acceptance as a standard
- MDBs support many statistical functions

BITS Pilani WILP

A Motivating Example

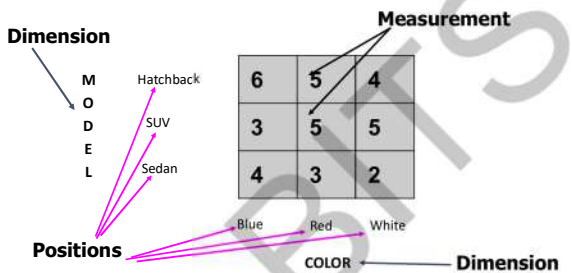
An automobile manufacturer wants to increase sale volumes by examining sales data collected throughout the organization. The evaluation would require viewing historical sales volume figures from multiple dimensions such as

- Sales volume by model
- Sales volume by color
- Sales volume by dealer
- Sales volume over time

Sales Data in Relational Form

MODEL	COLOR	SALES VOLUME
Hatchback	BLUE	6
Hatchback	RED	5
Hatchback	WHITE	4
SUV	BLUE	3
SUV	RED	5
SUV	WHITE	5
SEDAN	BLUE	4
SEDAN	RED	3
SEDAN	WHITE	2

Multidimensional Structure



Comments on Multidimensional Structure

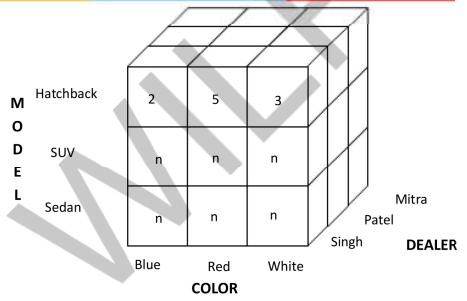
- The relational structure tells us nothing about the possible contents of those fields.
- The structure of the array, on the other hand, tells us not only that there are two dimensions, COLOR and MODEL, but it also presents all possible values of each dimension as positions along the dimension.
- Because of this structured presentation, all possible combinations of perspectives containing a specific attribute (the color BLUE, for example) line up along the dimension position for that attribute.
- This makes data browsing and manipulation highly intuitive to the end-user. As a result, this "intelligent" array structure lends itself very well to data analysis.

BITS Pilani WILP

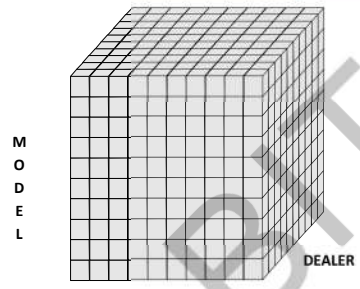
Let us add Dealer to the table

MODEL	COLOR	DEALERSHIP	VOLUME
Hatchback	BLUE	Mitra	6
Hatchback	BLUE	Patel	6
Hatchback	BLUE	Singh	2
Hatchback	RED	Mitra	3
Hatchback	RED	Patel	5
Hatchback	RED	Singh	5
Hatchback	WHITE	Mitra	2
Hatchback	WHITE	Patel	4
Hatchback	WHITE	Singh	3
SUV	BLUE	Mitra	2
SUV	BLUE	Patel	3
SUV	BLUE	Singh	2
SUV	RED	Mitra	7
SUV	RED	Patel	5
SUV	RED	Singh	2
SUV	WHITE	Mitra	4
SUV	WHITE	Patel	5
SUV	WHITE	Singh	1
SEDAN	BLUE	Mitra	6
SEDAN	BLUE	Patel	4
SEDAN	BLUE	Singh	2
SEDAN	RED	Mitra	1
SEDAN	RED	Patel	3
SEDAN	RED	Singh	4
SEDAN	WHITE	Mitra	2
SEDAN	WHITE	Patel	2
SEDAN	WHITE	Singh	3

Multidimensional Structure



Multidimensional Structure



If each dimension has 10 positions the relational table requires 10*10*10 i.e. 1000 records

Performance Advantages with MDB

Volume figure when car type = SEDAN, color=BLUE, & dealer=PATEL?

- RDBMS – all 1000 records might need to be searched to find the right record
- MDB has more 'knowledge' about where the data lies
  - Maximum of 30 position searches
- Average case
  - MDB 15 vs. 500 for RDBMS

BITS Pilani WILP

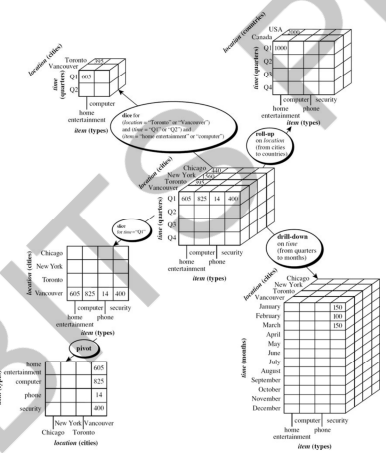
Performance Advantages with MDB

- To generalize the performance advantage
  - In case of RDBMS, size of search space gets multiplied, each time a new dimension is added; accordingly access time is affected
  - In case of MDB, the search space increases by the size of new dimension, each time a new dimension is added.
- At what cost?
  - MDB is a separate proprietary implementation from SQL
  - Since all business data is in RDBMS, the MDB has to be precomputed. Larger the data, more the dimensions, higher the precomputation effort.
  - Higher the interval of precomputation, higher the latency in MDB

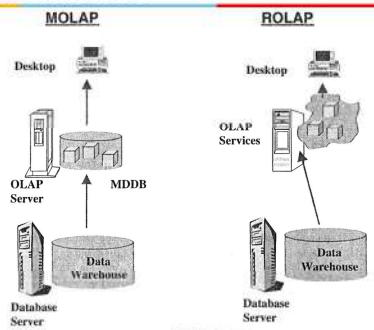
OLAP Operations

- Roll up (drill-up): summarize data
  - *by climbing up hierarchy or by dimension reduction*
- Drill down (roll down): reverse of roll-up
  - *from higher level summary to lower level summary or detailed data, or introducing new dimensions*
- Slice and dice: *project and select*
- Pivot (rotate):
  - *reorient the cube, visualization, 3D to series of 2D planes*
- Other operations
  - *drill across: involving (across) more than one fact table*
  - *drill through: through the bottom level of the cube to its back-end relational tables (using SQL)*

OLAP Operations



Distinct OLAP models



BITS Pilani WILP

innovate achieve lead

Data Storage

ROLAP	MOLAP
Data stored as relational tables in the warehouse. Detailed and light summary data available. Very large data volumes. All data access from the warehouse storage.	Data stored as relational tables in the warehouse. Various summary data kept in proprietary databases (MDBs) Moderate data volumes. Summary data access from MDB, detailed data access from warehouse.

Underlying Technologies

ROLAP	MOLAP
Use of complex SQL to fetch data from warehouse. ROLAP engine in analytical server creates data cubes on the fly. Multidimensional views by presentation layer.	Creation of pre-fabricated data cubes by MOLAP engine. Proprietary technology to store multidimensional views in arrays, not tables. High speed matrix data retrieval. Sparse matrix technology to manage data sparsity in summaries.

Functions and Features

ROLAP	MOLAP
Known environment and availability of many tools. Limitations on complex analysis functions. Drill-through to lowest level easier. Drill-across not always easy.	Faster access. Large library of functions for complex calculations. Easy analysis irrespective of the number of dimensions. Extensive drill-down and slice-and-dice capabilities.

HOLAP (Hybrid OLAP)

The intermediate architecture type, *HOLAP*, aims at mixing the advantages of both basic solutions. It takes advantage of the standardization level and the ability to manage large amounts of data from ROLAP implementations, and the query speed typical of MOLAP systems. HOLAP has the largest amount of data stored in an RDBMS, and a multidimensional system stores only the information users most frequently need to access. If that information is not enough to solve queries, the system will transparently access the part of the data managed by the relational system. Important market actors have adopted HOLAP solutions to improve their platform performance.

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Prescribed Text Books

	Author(s), Title, Edition, Publishing House
T1	Ponniah P, "Data Warehousing Fundamentals", Wiley Student Edition, 2012
T2	Kimball R, "The Data Warehouse Toolkit", 3e, John Wiley, 2013

References

	Author(s), Title, Edition, Publishing House
	Star Schema: The Complete Reference by Christopher Adamson McGraw-Hill/Osborne
	Kenan Technologies, An introduction to Multidimensional Database Technology
	Data Mining: Concepts and Techniques, Third Edition by Jiawei Han, Micheline Kamber and Jian Pei Morgan Kaufmann Publishers

Thank You

BITS Pilani WILP

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Features

Pravin Y Pawar

Adapted from Reliable Machine Learning by Cathy Chen

Features

Definitions

- The data is just the data. Features are what make it useful for ML!
  - A feature is any aspect of the data that we determine is useful in accomplishing the goals of the model
    - "we" includes the humans building the model or many times can now include automatic feature-engineering systems
- A feature is a specific, measurable aspect of the data, or a function of it.
- Features are used to build the models
  - Structures that connects models back to data used to assemble the model
- Model is set of rules that takes data and uses it to generate predictions about world
  - true for model architecture and configured model
- Trained model is formula for combining a collection of features definitions used to extract feature values from real data
- More than just pieces of raw data, but rather involve some kind of preprocessing!

Features

Examples

- Concrete examples of features
  - From a web log, information about the customer (say, browser type).
  - Individual words or combinations of words from text a human enters into an application.
  - The set of all the pixels in an image, or a structured subset thereof.
  - Current weather at the customer's location when they loaded the page.
  - Any combination or transformation of features can itself be a feature.
- Typically, features contain smaller portions of structured data extracted from the underlying training data
  - As modeling techniques continue to develop, it seems likely that more features will start to resemble raw data rather than these extracted elements
- For example, a model training on text currently might train on words or combinations of words
  - but expect to see more training on paragraphs or even whole documents in the future
  - has significant implications for modeling

Feature Definition and Feature Values

- Feature definition
  - code (or an algorithm or other written description) that describes the information getting extracted from the underlying data
  - not any specific instance of that data being extracted
- For example, feature definition is
  - "Source country of the current customer obtained by taking the customer's source IP address and looking it up in the geolocation data service" is a feature definition
  - Any particular country—for example, the "Dominican Republic" or "Russia" — would not be a feature definition
- Feature value
  - specific output of a feature definition applied to incoming data
  - "Dominican Republic" is a feature value that might be obtained by determining that the current customer's source IP address

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Feature Selection and Engineering

- Feature selection is process by which features are identified in data useful for models
- Feature engineering is process by which features are extracted and transformed into usable state
- Figure out what matters and what doesn't!
- Building and managing features has changed over years and will continue to evolve
  - Moving from entirely manual process to a mostly automated one to completely automated one
- Refer to processes of selecting and transforming features jointly as feature engineering

Human-driven Feature Engineering

- Process normally starts with human intuition based on an understanding or the domain of the problem
  - or at least detailed consultation with experts
- Understanding the underlying problem area is key, and more specificity is better
  - ML engineer spends time with a dataset and a problem
  - uses a set of statistical tools to evaluate the likelihood that a particular feature, or several features in combination with one another, will be useful in the task
- Next, ML engineers typically brainstorm to generate a list of possible features
  - Of these features, one of them is may be much more likely to be useful than the others
  - up to humans to generate and evaluate those ideas by using the ML platform and model metrics

Algorithmic / Automated Feature Engineering

- For algorithmic feature engineering, the process is considerably more automatic and data bound
  - AutoML is capable of not only selecting from identified features but also being able to programmatically apply common transforms (like log scaling or thresholding) to existing features
  - Embedding is one of other ways which enables to learn something (an embedding) about the data without explicitly specifying it
- Still, algorithms are generally able to identify only features that exist in the data
  - whereas humans can imagine new data that could be collected that might be relevant
- Even more importantly, humans understand the process of data collection, including any likely systemic bias
  - might have a material impact on the value of the data

Lifecycle of a Feature

- Feature definitions are created to fill a need, evaluated against that need, and eventually discarded as either the model is discarded or better features are found to accomplish the same goal
- Simplified version of the lifecycle of a feature (both the definition and the representative values):
  - Data collection/creation
  - Data cleaning/normalization/preprocessing
  - Candidate feature definition creation
  - Feature value extraction
  - Storage of feature values in a feature store
  - Feature definition evaluation
  - Model training and serving using feature values
  - (Usually) Update of feature definitions
  - (Sometimes) Deletion of feature values
  - (Eventually) Discontinuation of a feature definition

BITS Pilani WILP

Lifecycle of a Feature(2)

- Data collection/creation
  - No data, no features!
  - need to collect or create data in order to create features
- Data cleaning/normalization/preprocessing involves coarser preprocessing steps:
  - eliminating obviously malformed examples,
  - scaling input samples to a common set of values,
  - possibly even deleting specific data that we should not train on for policy reasons
  - might seem outside the feature-engineering process, but no features can exist until the data exists and is in a usable form
- Candidate feature definition creation
  - Using either subject matter expertise plus human imagination or automated tools
  - develop a hypothesis for which elements or combinations of data are likely to accomplish the models goals

Lifecycle of a Feature(3)

- Feature value extraction
  - Need to write code that reads the input data and extracts the features from the data
  - In some simple situations, might need to do this inline as part of the training process
  - But if we expect to train on the same data more than a few times,
    - it's probably sensible to extract the feature from the raw data and store it for later efficient and consistent reading
- If application involves online serving
  - need a version of this code to extract the same features from the values available at serving
    - in order to use them to perform inference in model
  - Under ideal circumstances, the same code can extract features for training and for serving
    - but may have additional constraints in serving that are not present in training
- Storage of feature values in a feature store
  - place to put the features
  - is just a place to write extracted feature values so that they can be quickly and consistently read during training a model

Lifecycle of a Feature(4)

- Feature definition evaluation
  - Once features are extracted, most likely will build a model using them or add new features to an existing model in order to evaluate how well they work
  - looking for evidence that the features provide the value that we were hoping they would
- Evaluation of feature definitions comes in two distinct phases that are connected
  - First, need to determine whether the feature is useful at all - coarse-grained evaluation
    - simply trying to decide whether to continue working on integrating the feature into model
  - Next phase occurs if decide to keep that feature
    - need a process to continuously evaluate the quality and value (compared to cost) of the future to determine that it is still working the way we expect and providing the value we expect even several years from now
- Model training and serving using feature values
  - Perhaps this is obvious, but the entire point of having features is to use them to train models
  - then use the resulting models for a particular purpose

Lifecycle of a Feature(5)

- (Usually) Update of feature definitions
  - Frequently have to update the definition of a feature, either to fix a bug or simply to improve it in some way
  - If keep track of versions for feature definitions, this will be much easier
    - can update the version and then, optionally, reprocess older data to create a new set of feature values for the new version
- (Sometimes) Deletion of feature values
  - Sometimes need to be able to delete feature values from feature store
    - can be for policy/governance reasons;
    - can also be for quality/efficiency reasons- corrupted, sampled in a biased way or just too old to be useful
- (Eventually) Discontinuation of a feature definition
  - Either found a better way to provide the value that this feature definition provides
  - or find that the world has changed enough that this feature no longer provides any value
  - Eventually, will decide to retire the feature definition (and values) entirely
  - Need to
    - remove any serving code that refers to the feature,
    - remove the feature values in feature store,
    - cancel the code that extracts from the data
    - proceed to delete the feature code

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Engineering Good Features

- Generally adding more features leads to better model performance
  - List of features used for model in production only grows over time
  - However, more feature doesn't always mean better model performance
- Two factors might need to consider when evaluating whether a feature is good for model
  - Feature Importance to the model
  - Feature Generalization to unseen data

Too many features

Can be bad both during training and serving model because

- More features means more opportunities for data leakage
- Too many features can cause overfitting
- Too many features can increase memory required to serve a model,
  - which in turn, might require to use a more expensive machine/instance to serve model
- Too many features can increase inference latency when doing online prediction
  - Especially if needed to extract these features from raw data for predictions online
- Useless features become technical debts
  - Whenever data pipeline changes, all the affected features need to be adjusted accordingly
- Feature removal
  - Might help models learn faster if not useful features are removed, prioritizing good features
  - Can store removed features to add them back later
  - Can store general feature definitions to reuse and share across teams in organization
  - Feature stores can be used for the same!

Feature Importance

Many methods to measure importance

- Use built-in feature importance functions
  - For classical ML algorithms like boosted gradient trees implemented by XGBoost
- For model agnostic methods, look at
  - SHAP (SHapely Additive explanations)
  - InterpretML
- Exact algorithms for feature importance measurement is complex
  - A features importance to a model is measured by how much that model's performance deteriorates if that feature is removed from model
  - SHAP not only measures a importance but also measures each features contribution to a models specific prediction
- Often a small number of features accounts for large portion of models feature importance
- Facebook team while measuring importance of click-through rate prediction model, found out that
  - top 10 features are responsible for about half of models total feature importance
  - Last 300 features contributes less that 1% of feature importance

Feature Generalization

- Goal of ML model is to make correct predictions on unseen data,
  - Features used for model should generalize to unseen data
  - Not all features generalize equally!
  - To predict whether a comment is spam, identifier of each comment is not generalizable but username of user who posts comment might be more meaningful
- Measuring feature generalization is lot less scientific than measuring feature importance
  - Requires both intuition and subject matter expertise on top of statistical knowledge
- Two aspects might want to consider with regards to generalization
  - Feature coverage
  - Distribution of feature values

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Feature coverage

- Coverage is percentage of samples that has values for feature in data
  - Fewer values that are missing, higher the coverage
  - Feature that appears in a very small percentage of data, not very generalizable
  - To predict whether a person will purchase house in next 12 months, number of children values is present only for 1% of data, feature might not be useful
- Some features can still be useful even if they are missing in most of data
  - Common when missing values are present not at random
- Coverage differs wildly between different slices of data or in even in same slice of date over time
  - If coverage differs a lot between train and test – indication of different distribution of data
  - Need to investigate whether the way used to split data makes sense and whether this feature is cause of data leakage

Distribution of feature values

- Feature might hurt models performance
  - If set of values appears in seen data (train split) has no overlap with set of values appears in unseen data (test split / real time data)
- For example,
- To build a model to estimate time it will take for given taxi ride
  - Might retrain model every week and use data from last six days to predict ETA for today
  - DAY_OF_WEEK might be useful feature as traffic on weekdays in worse than weekends
    - Feature coverage is 100% - as its present for all data records
    - In train split, values are from MONDAY to SATURDAY
    - In test split, value is SUNDAY
    - If used as is it wont generalize the test split and hurt models performance
  - HOUR_OF_DAY can be a good feature
    - Time in day affects traffic and range of values in train split overlaps with test split 100%

Specificity

- When considering features generalization, trade-off between generalization and specificity comes in
- For example,
- Might realize that traffic during an hour only changes depending on whether hour is rush hour
  - So generate IS_RUSH_HOUR feature to have value as 1 if hour is between 7 to 9 am or between 4 to 6 pm
  - IS_RUSH_HOUR is more generalizable but less specific than HOUR_OF_DAY
  - Using IS_RUSH_HOUR without HOUR_OF_DAY might cause models to lose important information about hour

Feature Systems

- Need to decompose the work into several subsystems to
  - successfully manage the flow of data through systems
  - turn data into features that are usable by training system and manageable by our modelers
- Let's walk through the feature systems starting with raw data and ending up with features stored in a format ready to be read by a training system
  - Data Ingestion system
  - Feature store
  - Feature quality evaluation system

BITS Pilani WILP

Data ingestion system

- Have to write software that reads raw data, applies feature extraction code to that data, and stores the resulting feature values in the feature store
- In the case of one-time extraction, even for a very large amount of data
  - may be a relatively ad hoc process with code designed to be run once
- But in many cases, the process of extracting data is a separate production system all its own
- When have many users or repeated use of the data ingestion system
  - it should be structured as an on-demand, repeatable, monitored data processing pipeline
    - the biggest variable is user code
- Need a system whereby feature authors write code that identifies features and extracts them to store in the feature store
  - an either let feature authors run their code themselves, which imposes a substantial operational burden on them
  - can provide them a development engineering environment
    - helping them writing reliable feature-extraction code so that it can be run on that code reliably in data ingestion system

Data ingestion system(2)

- Need to build a few systems to facilitate feature authors writing reliable and correct features
- Versioning
  - should note that features should be versioned
  - likely want to substantially change a feature over time, perhaps because of changes in data that it merges
  - with or other factors related to the data a feature version helps keep the transition clear and avoid unintended consequences of the change
- Test Systems
  - need a test system that checks feature-extraction code for basic correctness
  - need a staging environment for running proposed feature extraction on a certain number of examples
    - provide those to the feature author along with basic analysis tools to ensure that the feature is extracting what it is expected to extract
  - may want to allow the feature to be run or may want additional human review for reliability concerns
    - dependence on external data, for example
  - The more work we do here, the more productive feature authors will be.

Feature Store

- A storage system designed to store extracted feature (and label) values
  - so that they can be quickly and consistently read during training a model and even during inference
  - most useful, however, in larger, centrally managed services, especially when the features (definitions and values both) are shared among multiple models
- Recognizing the importance of putting feature and label data in a single, well-managed place has significantly improved the production readiness of ML in the industry.
- Most ML training and serving systems have some kind of a feature store even if it is not called that!
- Feature stores do not solve every problem
  - The most important characteristic of a feature store is its API
  - Different commercial and open source feature stores have different APIs

Feature Store(2)

A few basic features

- Store feature definitions
  - Usually these are stored as code that extracts the feature in a raw data format and outputs the feature data in the desired format
- Store feature values themselves
  - Ultimately, need to write features in a format that is easy to write, easy to read, and easy to use
  - will be largely determined by proposed use cases and most commonly is divided into ordered and unordered data
- Serve feature data
  - Provide access to feature data quickly and efficiently at a performance level suitable to the task
  - absolutely do not want expensive CPUs or accelerators stalled, waiting on the I/O of reading from feature store
- Coordinate metadata writes with the metadata system
  - To get the most out of feature store, should keep information about the data stored in it in the metadata system
  - helps model builders
    - Feature metadata is most useful for model developers
    - Pipeline metadata is most useful for ML reliability or production engineers

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Feature Store(3)

Data Access Patterns requirements

- Consider asking the following questions as we think about what we need in a feature store:
  - Are we reading data in a particular order (log lines that are timestamped) or in no order that matters (a large collection of images in a cloud storage system)?
  - Will we read the features frequently or only when training a new model?
    - In particular, what is the ratio of bytes/records read to bytes/records written?
  - Is the feature data ingested once, never appended to, and seldom updated? Or is the data frequently appended to while older data is continuously deleted?
  - Can feature values be updated, or is the store append-only?
  - Are there special privacy or security requirements for the data we are storing?
    - In most cases, the extracted features of a dataset with privacy and use restrictions will also have privacy and use restrictions.
- After thinking about these questions, should be able to determine needs for a feature storage system
  - If lucky, will be able to use one of the existing commercial or open source feature stores on the market
  - If not, have to implement this functionality ourselves, whether we do it in an uncoordinated fashion or as part of a more coherent system

Feature Store(4)

Types

- Feature store fails into one of two buckets
  - Based on requirements of API and have a clearer understanding of data access needs
- Columns
  - The data is structured and decomposable into columns, not all of which will be used in all models
  - Typically, the data is also ordered in some way, often by time
  - column-oriented storage is the most flexible and efficient
- Blobs
  - binary large objects
  - the data is fundamentally unordered, mostly unstructured, and is best stored in a manner that's more efficient at storing a bunch of bytes
- Many feature stores will need to be replicated, partially or completely, in order to store data near the training and serving stacks
  - Since ML computation requirements are generally considerable for training, often prefer to replicate data to the place or places where can get the best value for training dollar
  - Having the feature store implemented as a service facilitates the management of this replication

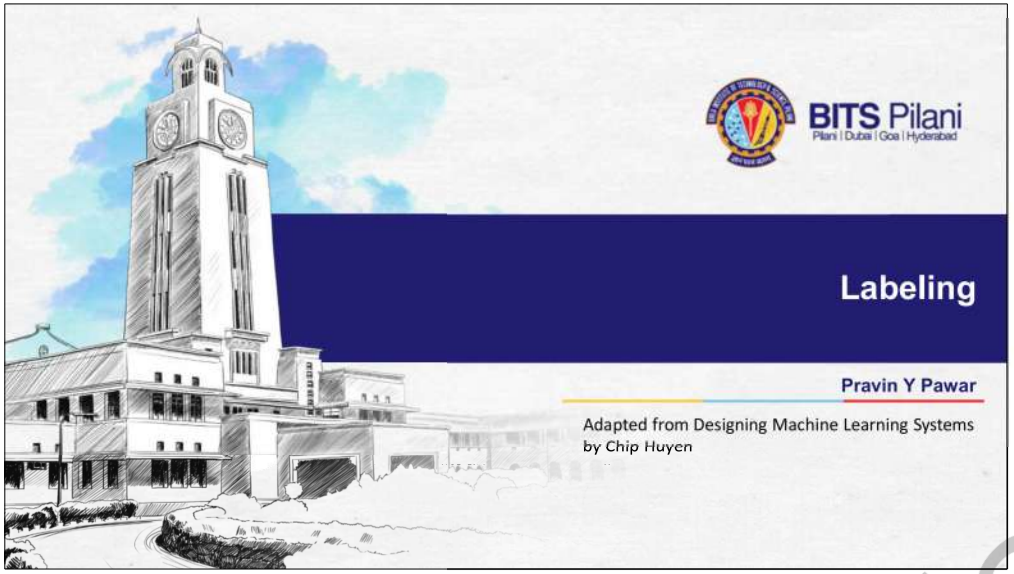
Feature quality evaluation system

- As develop new features, need to evaluate what those features add to the quality of the overall model
  - in combination with existing features!
- General idea is to combine
  - the approaches of using slightly different models
  - A/B testing
  - and a model quality evaluation system
- in order to effectively evaluate the benefit of each new feature under development
  - can do this quickly and at relatively low cost.
- One common approach is to take an existing model and retrain it by using a single additional feature
  - then direct a fraction of our user requests to new model and evaluate its performance on a task
  - doing so can inform the choice of whether to keep the new feature or to eliminate it and try other ideas
- Even a feature that adds value may not be worth the cost to collect, process, store, and maintain it
  - For all but the most trivially obvious features, it is a good habit to calculate a rough return on investment (ROI) for every new feature added to the system
  - will help avoid useful features that are still more expensive than the value that they add

Thank You!

In our next session:

BITS Pilani WILP



Labeling

- Despite the promise of unsupervised ML, most ML models in production today are supervised
  - means that they need labels to learn
- Performance of an ML model still depends heavily on the quality and quantity of the labeled data it's trained on
- There are tasks where data has natural labels or it's possible to collect natural labels on the fly
  - For predicting the click-through rate on an ad, labels are whether users click on an ad or not
  - Similarly, for recommendation systems, labels are whether users click on a recommended item or not
- However, for most tasks, natural labels are not available or not accessible
  - will need to obtain labels by other means
- The challenges of obtaining labels for data includes
  - label multiplicity problem
  - what to do when lack hand labeled data

Hand Labels

- Acquiring hand labels for data is difficult for many, many reasons!
- First, hand-labeling data can be expensive, especially if subject matter expertise is required
  - To classify whether a comment is spam, might be able to find 200 annotators on a crowdsourcing platform and train them in 15 minutes to label data
  - However, if want to label chest X-rays, need to find board-certified radiologists, whose time is limited and expensive
- Second, hand labeling poses a threat to data privacy
  - Hand labeling means that someone has to look at data, which isn't always possible if data has strict privacy requirements
  - For example, can't just ship patient's medical records or company's confidential financial information to a third-party service for labeling
  - In many cases, data might not even be allowed to leave organization, and might have to hire or contract annotators to label data on-premise
- Third, hand labeling is slow
  - For example, accurate transcription of speech utterance at phonetic level can take 400 times longer than the utterance duration
  - Slow labeling leads to slow iteration speed and makes model less adaptive to changing environments and requirements
  - If the task changes or data changes, have to wait for data to be relabeled before updating model

Label Multiplicity

- Often, to obtain enough labeled data, companies have to
  - rely on multiple annotators who have different levels of expertise
  - use data from multiple sources
- Leads to the problem of label ambiguity or label multiplicity:
  - what to do when there are multiple possible labels for a data instance
- Consider this simple task of entity recognition given to three annotators asking them to annotate all entities they can find
  - "Darth Sidious, known simply as the Emperor, was a Dark Lord of the Sith who reigned over the galaxy as Galactic Emperor of the First Galactic Empire."
  - Receive back three different solutions
  - Three annotators have identified different Entitie
  - Which one should your model train on?

Annotator	# entities	Annotation
1	3	[**Darth Sidious**], known simply as the Emperor, was a [**Dark Lord of the Sith**] who reigned over the galaxy as [**Galactic Emperor of the First Galactic Empire**]
2	6	[**Darth Sidious**], known simply as the [**Emperor**], was a [**Dark Lord**] of the [**Sith**] who reigned over the galaxy as [**Galactic Emperor**] of the [**First Galactic Empire**].
3	4	[**Darth Sidious**], known simply as the [**Emperor**], was a [**Dark Lord of the Sith**] who reigned over the galaxy as [**Galactic Emperor of the First Galactic Empire**].

BITS Pilani WILP

Label Multiplicity (2)

Rely on multiple annotators who have different levels of expertise

- Disagreements among annotators are extremely common!
  - The higher level of domain expertise required, the higher the potential for annotating disagreement
  - If one human-expert thinks the label should be A while another believes it should be B, how do we resolve this conflict to obtain one single ground truth?
  - If human experts can't agree on a label, what does human-level performance even mean?
- To minimize the disagreement among annotators, it's important to
- First, have a clear problem definition
  - For example, in the entity recognition task above, some disagreements could have been eliminated if clarified that in case of multiple possible entities, pick the entity that comprises the longest substring
    - means Galactic Emperor of the First Galactic Empire instead of Galactic Emperor and First Galactic Empire
- Second, need to incorporate that definition into training
  - to make sure that all annotators understand the rules

Label Multiplicity (3)

Use data from multiple sources

- Indiscriminately using data from multiple sources, generated with different annotators
  - without examining their quality can cause model to fail mysteriously
- Consider a case when trained a moderately good model with 100K data samples
  - ML engineers are confident that more data will improve the model performance
    - so spend a lot of money to hire annotators to label another million data samples
  - However, the model performance actually decreases after being trained on the new data
    - reason is that the new million samples were crowdsourced to annotators who labeled data with much less accuracy than the original data
    - can be especially difficult to remedy this if already mixed data and can't differentiate new data from old data
- Data Lineage
  - Good practice to keep track of the origin of each of data samples as well as its labels - data lineage
  - helps both in flagging potential biases in data as well as debug models

Handling the Lack of Hand Labels

- Because of the challenges in acquiring sufficient high-quality labels, many techniques have been developed to address the problems that result
- cover four of them: weak supervision, semi-supervision, transfer learning, and active learning

Method	How	Ground truths required?
Weak supervision	Leverages (often noisy) heuristics to generate labels	No, but a small number of labels are recommended to guide the development of heuristics
Semi-supervision	Leverages structural assumptions to generate labels	Yes. A small number of initial labels as seeds to generate more labels
Transfer learning	Leverages models pretrained on another task for your new task	No for zero-shot learning Yes for fine-tuning, though the number of ground truths required is often much smaller than what would be needed if you train the model from scratch.
Active learning	Labels data samples that are most useful to your model	Yes

Summaries for four techniques for handling the lack of hand labeled data.

Weak supervision

Approach

- If hand labeling is so problematic, what if we don't use hand labels altogether?
  - One approach that has gained popularity is weak supervision
  - One of the most popular open-source tools for weak supervision is Snorkel, developed at the Stanford
- The insight behind weak supervision is that people rely on heuristics, which can be developed with subject matter expertise, to label data.
  - For example, a doctor might use the following heuristics to decide whether a patient's case should be prioritized as emergent.
  - If the nurse's note mentions a serious condition like pneumonia, the patient's case should be given priority consideration
- Libraries like Snorkel are built around the concept of a labeling function (LF): a function that encodes heuristics
- The above heuristics can be expressed by the following function

```
def labeling_function(note):
    if "pneumonia" in note:
        return "EMERGENT"
```

- After LFs are written, can apply them to the samples want to be labeled

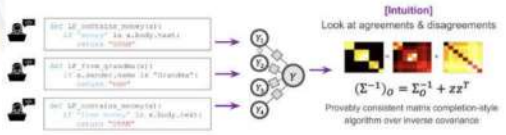
BITS Pilani WILP

Weak supervision(2)

Heuristics and Labeling Functions

- LFs can encode many different types of heuristics
  - Keyword heuristic, such as the example above.
  - Regular expressions, such as if the note matches or fails to match a certain regular expression
  - Database lookup, such as if the note contains the disease listed in the dangerous disease list
  - The outputs of other models, such as if an existing system classifies this as EMERGENT

Because LFs encode heuristics, and heuristics are noisy, LFs are noisy!
- Multiple labeling functions might apply to the same data examples, and they might give conflicting labels
- One function might think a note is EMERGENT but another function might think it's not
- One heuristic might be much more accurate than another heuristic
- important to combine, denoise, and reweight all LFs to get a set of most likely-to-be-correct labels



A high level overview of how labeling functions are combined.
Image by Ratner et al.

Weak supervision(3)

Solution

- In theory, don't need any hand labels for weak supervision
  - However, to get a sense of how accurate LFs are, a small amount of hand labels is recommended
  - hand labels can help discover patterns in data to write better LFs
- Weak supervision can be especially useful when data has strict privacy requirements
  - only need to see a small, cleared subset of data to write LFs
  - which can be applied to the rest of data without anyone looking at it
- With LFs, subject matter expertise can be versioned, reused, and shared
  - Expertise owned by one team can be encoded and used by another team
  - If data changes or requirements change, can just reapply LFs to data samples
- Programmatic labeling
  - Approach of using labeling functions to generate labels for data

Weak supervision(4)

Hand labelling vs Programmatic labeling

Hand labeling	Programmatic labeling
Expensive: Especially when subject matter expertise required	**Cost saving**: Expertise can be versioned, shared, and reused across an organization
Non-private: Need to ship data to human annotators	**Privacy**: Create LFs using a cleared data subsample then apply LFs to other data without looking at individual samples.
Slow: Time required scales linearly with # labels needed	**Fast**: Easily scale from 1K to 1M samples
Non-adaptive: Every change requires re-labeling the data	**Adaptive**: When changes happen, just reapply LFs!

The advantages of programmatic labeling over hand labeling.

Weak supervision(5)

Drawbacks

- If heuristics work so well to label data, why do we need machine learning models?
  - One reason is that labeling functions might not cover all data samples
  - need to train ML models to generalize to samples that aren't covered by any labeling function
- Weak supervision is a simple but powerful paradigm. However, it's not perfect.
  - In some cases, the labels obtained by weak supervision might be too noisy to be useful
  - But it's often a good method to get started when want to explore the effectiveness of ML without wanting to invest too much in hand labeling upfront

BITS Pilani WILP

Semi-supervision

- Semi-supervision leverages structural assumptions to generate new labels based on a small set of initial labels
  - Unlike weak supervision, semi-supervision requires an initial set of labels
- Self-training - A classic semi-supervision method
  - Start by training a model on existing set of labeled data, and use this model to make predictions for unlabeled samples
  - Assuming that predictions with high raw probability scores are correct, add the labels predicted with high probability to training set
  - then train a new model on this expanded training set
  - goes on until are happy with model performance
- Another semi-supervision method assumes that data samples that share similar characteristics share the same labels
  - The similarity might be obvious, such as in the task of classifying the topic of Twitter hashtags
  - In most cases, the similarity can only be discovered by more complex methods
    - use a clustering method or a K-nearest neighbor method to discover samples that belong to the same cluster

Semi-supervision(2)

- Perturbation-based method has gained popularity in recent years
  - based on the assumption that small perturbations to a sample shouldn't change its label
  - apply small perturbations to training samples to obtain new training samples
  - perturbations might be applied directly to the samples (e.g. adding white noise to images) or to their representations (e.g. adding small values to embeddings of words)
  - perturbed samples have the same labels as the unperturbed samples
- Semi-supervision is the most useful when the number of training labels is limited
  - One thing to consider when doing semi-supervision is how much of this limited amount should be used for evaluation
  - If evaluate multiple model candidates on the same test set and choose the one that performs best on the test set,
    - might have chosen a model that overfits the most on the test set
  - On the other hand, if choose models based on a validation set,
    - the value gained by having a validation set might be less than the value gained by adding the validation set to the limited training set

Transfer learning

Approach

- Refers to the family of methods where a model developed for a task is reused as the starting point for a model on a second task
- First, the base model is trained for a base task such as language modeling
  - The base task is usually a task that has cheap and abundant training data
  - Language modeling is a great candidate because it doesn't require labeled data
  - Can collect any body of text — books, Wikipedia articles, chat histories — and the task is: given a sequence of tokens, predict the next token
- Then use this pretrained base model on the task that you're interested in, - called a downstream task
  - such as sentiment analysis, intent detection, question answering, etc
  - In zero-shot learning scenarios, might be able to use the base model on a downstream task directly
  - In many cases, might need to fine-tune the base model
  - Fine-tuning means making small changes to the base model, which can be continuing training the entire base model or a subset of the base model on data from a given downstream task

Transfer learning(2)

Discussion

- Especially appealing for tasks that don't have a lot of labeled data
  - Even for tasks that have a lot of labeled data, using a pretrained model as the starting point can often boost the performance significantly compared to training from scratch
- Transfer learning has gained a lot of interest in recent years
  - enabled many applications that were previously impossible due to the lack of training samples
  - A non-trivial portion of ML models in production today are the results of transfer learning,
    - including object detection models that leverage models pretrained on ImageNet and text classification models that leverage pretrained language models such as BERT or GPT-3.
  - lowers the entry barriers into ML, as it helps reduce the upfront cost needed for labeling data to build ML applications
- A trend that has emerged in the last five years is that usually,
  - the larger the pretrained base model, the better its performance on downstream tasks
- Large models are expensive to train
  - Based on the configuration of GPT-3, it's estimated that the cost of training this model is in the tens of millions USD
  - Many have hypothesized that in the future, only a handful of companies can afford to train large pretrained models
  - The rest of the industry will use these pretrained models directly or fine-tune them for their specific needs

BITS Pilani WILP

Active learning

Approach

- Method for improving the efficiency of data labels
  - The hope here is that ML models can achieve greater accuracy with fewer training labels if they can choose which data samples to learn from
- Aka query learning
  - because a model (active learner) sends back queries in the form of unlabeled samples to be labeled by annotators (usually humans)
- Instead of randomly labeling data samples, label the samples that are most helpful to models according to some heuristics
  - The most straightforward heuristic is uncertainty measurement — label the examples that model is the least certain about hoping that they will help model learn the decision boundary better
    - For example, in the case of classification problems where model outputs raw probabilities for different classes, it might choose the data examples with the lowest probabilities for the predicted class
  - Another common heuristic is based on disagreement among multiple candidate models - query-by-committee
    - need a committee of several candidate models, which are usually the same model trained with different sets of hyperparameters
    - each model can make one vote for which examples to label next, which it might vote based on how uncertain it is about the prediction
    - then label the examples that the committee disagrees on the most

Human-Generated Labels

- Lets Turn attention to the large classes of problems requiring human annotations to provide the model training data
  - For example, building a system to analyze and interpret human voice
    - needs human annotation to ensure that the transcription is accurate and to understand what the speaker meant
  - Image analyses and understanding
    - often needs example annotated images for image classifications or detection problems
- Getting these human annotations at the scale needed to train an ML model is challenging
  - from both implementation and cost perspectives
  - Effort must be dedicated to designing efficient systems to produce these annotations
- Humans need to be trained on the task
  - Because of the large cost associated with acquiring human-generated labels,
  - training systems should be designed to get as much mileage from them as possible
- While some kinds of tremendously complex data can best be labeled by humans,
  - other types of data definitely cannot be labeled by humans at all
    - abstract data with high dimensionality that makes it difficult for a human to determine the correct answer quickly
  - Sometimes humans can be provided with augmentation software
  - to assist in these tasks, but other times they are just the wrong option for performing the labeling

Annotation Workforces

- The first question that often comes up with human annotation problems is who will do the labeling
  - question of scale and equity!
- For simpler models, for which a small amount of data is sufficient to train the model,
  - typically the engineer building the model will do their own labeling, often with hacky, homebuilt tools
- For more complex models, dedicated annotation teams are used to provide human annotations at a scale not otherwise possible
  - dedicated teams can be collocated with the model builder or remotely provided by third-party annotation providers
  - teams range in size from a single person to hundreds of people, all generating annotated data for a single model
- The Amazon Mechanical Turk service was the original platform used for this
  - but since then a proliferation of crowdsourcing platforms and services have developed
  - Some of these services use paid volunteers, and others use teams of employees to label the data
- Cost, quality, and consistency trade-offs arise in the choice of labeling
  - Crowdsourced labeling often requires additional effort to verify quality and consistency, but paid labeling staff can be expensive
  - The costs for these annotation teams can easily exceed the computational costs of training the models

Measuring Human Annotation Quality

- As the quality of any model is only as good as the data used to train the model,
  - quality must be designed into the system from the start
  - becomes increasingly true as the size of the annotation team grows
- Quality can be achieved in multiple ways depending on the task, but the most frequent techniques used include the following:
  - Multiple labeling (also called consensus labeling)
    - The same data is given to multiple labelers to check for agreement among them.
  - Golden set test questions
    - Trusted labelers (or the model builder) produce a set of test questions that are randomly included in the unlabeled data to evaluate the quality of the produced labels
  - A separate QA step
    - A fraction of the labeler data is reviewed by a more trusted QA team

BITS Pilani WILP

Measuring Human Annotation Quality (2)

- Once measured, quality metrics can be improved!
- Quality issues are best addressed by managing the annotation team with humility
- and by understanding that they will produce higher-quality results when they have the following:
  - More training and documentation
  - Recognition for quality and not just throughput
  - A variety of tasks over the workday
  - Easy-to-use tools
  - Tasks with a balanced set of answers
  - An opportunity to provide feedback on the tools and instructions
- However, even the best, most conscientious labeler will miss things occasionally
  - so processes should be designed to detect or accept these occasional errors

An Annotation Platform

- A labeling platform organizes the flow of data to be annotated and the results of the annotations
  - while providing quality and throughput metrics of the overall process
- Are primarily work-queuing systems to divide the annotation work among the annotators
  - The actual labeling tool that allows the labelers to view the data and provide their annotations
- With a team or organization that is working on multiple models simultaneously,
  - the same annotation team may be shared among multiple annotation projects
  - each annotator might have different sets of skills (e.g., language skills)
  - queuing systems can be relatively complex and require careful design to avoid problems such as scalability issues or queue starvation
- Pipelines enabling the output of one annotation task to serve as the input to another can be useful for complex workflows
  - Quality measurement should be designed into the system from the start
  - so project owners can understand labeling throughput and quality of all their annotation tasks

An Annotation Platform(2)

- Although historically many companies have implemented their own labeling platforms with their own set of these features,
  - many options exist for prebuilt labeling platforms
- The major cloud providers and many smaller startups offer labeling platform services
  - that can be used with arbitrary annotation workforces
- many annotation workforce providers have their own platform options that can be used
- Rapidly changing area with new features being added to existing platforms all the time
  - Publicly available tools are moving beyond simple queuing systems
  - are starting to provide dedicated tools for common tasks, including advanced features like AI-assisted labeling
- In many cases the most sensible place to store completed labels is in the feature store

Active Learning and AI-Assisted Labeling

- Active learning techniques
  - focus the annotation effort on the cases in which the model and the human annotators disagree or the model is most uncertain
  - thereby improve overall label quality
- A semi-supervised system
  - allows the modeler to bootstrap the system with weak heuristic functions that imperfectly predict the labels of some data
  - then use humans to train a model that takes these imperfect heuristics to high-quality training data
  - can be particularly valuable for problems with complex, frequently changing category definitions requiring models to be retrained quickly and frequently
- Efficient annotation techniques for particularly complex labeling tasks is an ongoing field of research
  - a quick review of available tools from cloud and annotation providers is well worth time
  - are often adding new capabilities for AI-assisted annotations

BITS Pilani WILP

Documentation and labeler training systems

- Most commonly overlooked parts of an annotation platform!
- While labeling instructions often start simply
  - inevitably get more complex as data is labeled and various corner cases are discovered
- Labeling definitions and directions should be updated as new corner cases are discovered
  - annotation and modeling teams should be notified about the changes
  - If the changes are significant, previously labeled data might have to be re-annotated to correct data labeled with old instructions
- Annotation teams often have significant turnover
  - investing in training for using annotation tools and for understanding labeling instructions will almost always give big wins in label quality and throughput

Data Augmentation

- A family of techniques that are used to increase the amount of training data
  - Traditionally, used for tasks that have limited training data, such as in medical imaging projects
  - However, in the last few years, shown to be useful even when have a lot of data
    - because augmented data can make models more robust to noise and even adversarial attacks
- Data augmentation has
  - become a standard step in many computer vision tasks
  - finding its way into natural language processing (NLP) tasks
- Three main types of data augmentation:
  - simple label-preserving transformations,
  - perturbation, which is a term for "adding noises",
  - data synthesis

Simple Label-Preserving Transformations

Computer vision

- In computer vision, the simplest data augmentation technique is to randomly modify an image while preserving its label
  - can modify the image by
    - cropping,
    - flipping,
    - rotating,
    - inverting (horizontally or vertically),
    - erasing part of the image, and more
  - makes sense because a rotated image of a dog is still a dog
- Common ML frameworks like PyTorch and Keras both have support for image augmentation
- According to Krizhevsky et al., in their legendary AlexNet paper,
  - "the transformed images are generated in Python code on the CPU while the GPU is training on the previous batch of images. So these data augmentation schemes are, in effect, computationally free."

Simple Label-Preserving Transformations

NLP

- In NLP, can randomly replace a word with a similar word,
  - assuming that this replacement wouldn't change the meaning or the sentiment of the sentence
- Similar words can be found
  - either with a dictionary of synonymous words,
  - or by finding words whose embedding's are close to each other in a word embedding space

Original sentences	I'm so happy to see you.
Generated sentences	I'm so **glad** to see you. I'm so happy to see **y'all**. I'm **very** happy to see you.

Three sentences generated from an original sentence by replacing a word with another word with similar meaning.

- This type of data augmentation is a quick way to double, even triple training data

BITS Pilani WILP

Perturbation

- Perturbation is also a label-preserving operation,
  - but sometimes, used to trick models into making wrong predictions
- Neural networks, in general, are sensitive to noise.
  - In the case of computer vision, this means that by adding a small amount of noise to an image can cause a neural network to misclassify it.
- Adversarial attacks
  - Using deceptive data to trick a neural network into making wrong predictions
  - Adding noise to samples to create adversarial samples is a common technique
  - Success of adversarial attacks is especially exaggerated as the resolution of images increases

Perturbation

- Aadversarial augmentation
  - Adding noisy samples to training data can help models recognize the weak spots in their learned decision boundary and improve their performance
  - Noisy samples can be created by either adding random noise or by a search strategy
  - Moosavi-Dezfooli et al. proposed an algorithm, called DeepFool, that finds the minimum possible noise injection needed to cause a misclassification with high confidence
- Adversarial augmentation is less common in NLP but perturbation has been used to make models more robust
  - One of the most notable examples is BERT

Data Synthesis

- Since collecting data is expensive and slow with many potential privacy concerns
  - it'd be a dream if could sidestep it altogether and train models with synthesized data
- Still far from being able to synthesize all training data,
  - it's possible to synthesize some training data to boost a model's performance
- In NLP,
  - templates can be a cheap way to bootstrap model.

A template might look like:
"Find me a [CUISINE] restaurant within [NUMBER] miles of [LOCATION]." With lists of all possible cuisines, reasonable numbers (you would probably never want to search for restaurants beyond 1000 miles), and locations (home, office, landmarks, exact addresses) for each city, you can generate thousands of training queries from a template.

Template	Find me a [CUISINE] restaurant within [NUMBER] miles of [LOCATION].
Generated queries	• Find me a **Vietnamese** restaurant within **2** miles of **my office**. • Find me a **Thai** restaurant within **5** miles of **my home.** • Find me a **Mexican** restaurant within **3** miles of **Google headquarters**.

Three sentences generated from a template

Data Synthesis

Computer vision

- In computer vision, a straightforward way to synthesize new data
  - is to combine existing examples with discrete labels to generate continuous labels
- Mixup
  - Consider a task of classifying images with two possible labels: DOG (encoded as 0) and CAT (encoded as 1)
  - From example x1 of label DOG and example x2 of label CAT, can generate x' such as:

$$x' = \gamma x_1 + (1 - \gamma) x_2$$

  - The label of x' is a combination of the labels of x1and x2: $\gamma * 0 + (1 - \gamma) * 1$
- Research showed mixup
  - improves models' generalization
  - reduces their memorization of corrupt labels
  - increases their robustness to adversarial examples
  - and stabilizes the training of generative adversarial networks
- Using neural networks to synthesize training data is an exciting approach
  - actively being researched but not yet popular in production

BITS Pilani WILP

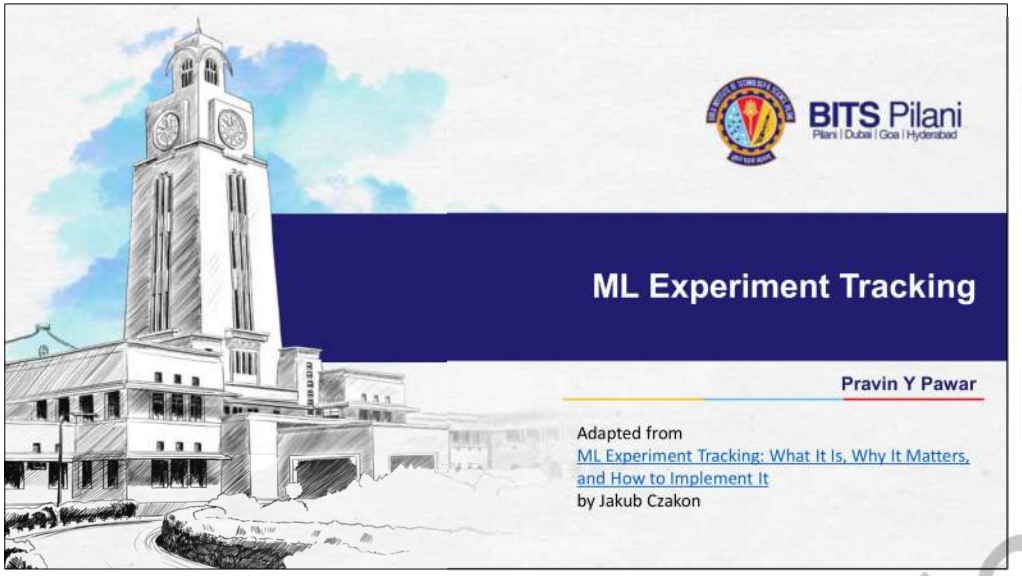
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED
LEARNING PROGRAMMES

Thank You!

In our next session:

BITS Pilani WILP

innovate achieve lead



ML Experiment Tracking

Pravin Y Pawar

Adapted from
ML Experiment Tracking: What It Is, Why It Matters, and How to Implement It
by Jakub Czakon

a story that have heard too many times

"… So far we have been doing everything manually and sort of ad hoc.

Some people are using it, some people are using that, it's all over the place.

We don't have anything standardized.

But we run many projects, the team is growing, and we are scaling pretty fast.

So we run into a lot of problems. How was the model trained? On what data? What parameters we used for different versions? How can we reproduce them?

We just feel the need to control our experiments…"

– unfortunate Data Scientist.

A lot of experiments

- And the truth is, when develop ML models, will run a lot of experiments
- Those experiments may:
  - use different models and model hyper parameters
  - use different training or evaluation data,
  - run different code (including this small change that you wanted to test quickly)
  - run the same code in a different environment (not knowing which PyTorch or Tensorflow version was installed)
  - And as a result, they can produce completely different evaluation metrics.
- Keeping track of all that information can very quickly become really hard
- Especially
  - if want to organize and compare those experiments
  - feel confident that you know which setup produced the best result
- This is where ML experiment tracking comes in!

ML experiment tracking

What?

- Experiment tracking is the process of saving all experiment related information that you care about for every experiment you run
- This "metadata you care about" will strongly depend on your project, but it may include:
  - Scripts used for running the experiment
  - Environment configuration files
  - Versions of the data used for training and evaluation
  - Parameter configurations
  - Evaluation metrics
  - Model weights
  - Performance visualizations (confusion matrix, ROC curve)
  - Example predictions on the validation set (common in computer vision)
- Of course, you want to have this information available after the experiment has finished,
  - but ideally, you'd like to see some of it as your experiment is running as well.

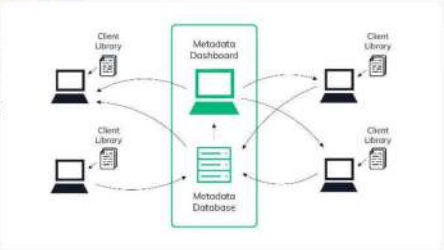
BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

ML experiment tracking

Components

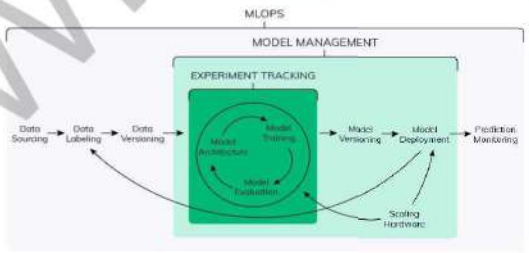
- To do experiment tracking properly, need some sort of a system that deals with all this metadata
- Typically, such a system will have 3 components:
  - Experiment database: A place where experiment metadata is stored and can be logged and queried
  - Experiment dashboard: A visual interface to your experiment database - A place where can see experiment metadata
  - Client library: Which gives methods for logging and querying data from the experiment database
- Of course, you can implement each component in
- many different ways, but the general picture will be very s



Experiment tracking vs ML model management vs MLOps

isn't experiment tracking like MLOps or something?

- Experiment tracking (also referred to as experiment management) is a part of MLOps:
  - a larger ecosystem of tools and methodologies that deals with the operationalization of machine learning
- MLOps deals with every part of ML project lifecycle
  - from developing models by scheduling distributed training jobs, managing model serving,
  - monitoring the quality of models in production, and re-training those models when needed



Experiment tracking vs ML model management vs MLOps(2)

how is experiment tracking different from ML model management?

- Experiment tracking focuses on the iterative model development phase
  - when you try many things to get your model performance to the level you need
- ML model management starts when models go to production:
  - streamlines moving models from experimentation to production
  - helps with model versioning
  - organizes model artifacts in an ML model registry
  - helps with testing various model versions in the production environment
  - enables rolling back to an old model version if the new one seems to be going crazy
  - But not every model gets deployed.
- Experiment tracking is useful even if your models don't make it to production (yet).
  - And in many projects, especially those that are research-focused, they may never actually get there
  - But having all the metadata about every experiment you run ensures that you will be ready when this magical moment happens.

Why does experiment tracking matter?

4 ways in which experiment tracking can actually improve ML workflow

BITS Pilani WILP

All ML experiments are organized in a single place

- There are many ways to run ML experiments or model training jobs:
  - Private laptop
  - PC at work
  - A dedicated instance in the Cloud
  - University cluster
  - Kaggle kernel or Google Colab
  - And many more.
- Sometimes just want to test something quickly and run an experiment in a notebook
- Sometimes want to spin up a distributed hyper parameter tuning job
- Either way, during the course of a project (especially when there are more people working on it),
  - can end up having your experiment results scattered across many machines

ML experiments are organized in a single place(2)

- With the experiment tracking system, all of experiment results are logged to one experiment repository by design
  - keeping all of experiment metadata in a single place, regardless of where its run, makes experimentation process so much easier to manage
- Specifically, a centralized experiment repository makes it easy to:
  - Search and filter experiments to find the information you need quickly
  - Compare their metrics and parameters with no additional work
  - Drill down and see what exactly it was that you tried (code, data versions, architectures)
  - Reproduce or re-run experiments when you need to
  - Access experiment metadata even if you don't have access to the server where you ran them
- Additionally, can sleep peacefully knowing that all the ideas tried are safely stored, and can always go back to them later

Compare experiments, analyze results, debug model training with little extra work

- Whether debugging training runs, looking for improvement ideas, or auditing current best models, comparing experiments is important
- But when don't have any experiment tracking system in place:
  - the way you log things can change,
  - you may forget to log something important
  - you may simply lose some information accidentally
- In those situations, something as simple as comparing and analyzing experiments can get difficult or even impossible
- With an experiment tracking system, experiments are stored in a single place,
  - follow the same protocol for logging them, so those comparisons can go really deep

Benefits of tracking

- Proper experiment tracking makes it easy to:
  - Compare parameters and metrics
  - Overlay learning curves
  - Group and compare experiments based on data versions or parameter values
  - Compare Confusion Matrices, ROC curves, or other performance charts
  - Compare best/worst predictions on test or validation sets
  - View code diffs (and/or notebook diffs)
  - Look at hardware consumption during training runs for various models
  - Look at prediction explanations like Feature Importance, SHAP or Lime
  - Compare rich-format artifacts like video or audio
  - … Compare anything else you logged
- Modern experiment tracking tools will give many of those comparison features (almost) for free
  - Some tools even go as far as to automatically find diffs between experiments or show which parameters have the biggest impact on model performance
- When have all the pieces in one place, might be able to find new insights and ideas
  - just by looking at all the metadata logged
  - especially true when you are not working alone!

BITS Pilani WILP

What should be tracked in any ML experiment?

- Things that should be keep track of regardless of the project are:
  - Code:
    - preprocessing, training and evaluation scripts, notebooks used for designing features, other utilities.
    - All the code that is needed to run (and re-run) the experiment
  - Environment:
    - The easiest way to keep track of the environment is to save the environment configuration files like `Dockerfile` (Docker), `requirements.txt` (pip) or `conda.yml` (conda)
    - Can also save the Docker image on Docker Hub, but I find saving configuration files easier
  - Data:
    - saving data versions (as a hash or locations to data files) makes it easy to see what your model was trained on
    - can also use modern data versioning tools like DVC (and save the .dvc files to your experiment tracking tool)
  - Parameters:
    - saving your run configuration is absolutely crucial
  - Metrics:
    - logging evaluation metrics on train, validation, and test sets for every run is pretty obvious
- Keeping track of those things will let you reproduce experiments, do basic debugging, and understand what happened at a high-level.
  - That said, can always log more things to gain even more insights.

How to set up experiment tracking?

- There are (at least) a few options. The most popular being:
  - Spreadsheets + naming conventions
  - Versioning configuration files with Github
  - Using modern experiment tracking tools

Spreadsheets + naming conventions

(but please don't use it)

- A common approach
  - Simply create a big spreadsheet to put all of the information (metrics, parameters, etc) and a directory structure
    - where things are named in a certain way
    - names usually end up being really long like 'model_v1_lr01_ batchsize64_ no_preprocessing_ result_accuracy082.h5'
  - Whenever an experiment is run, look at the results and copy them to the spreadsheet
- In some situations, it can be just enough to solve experiment tracking problems
  - may not be the best solution but it is quick and simple
  - …things can fall apart really quickly!

Spreadsheets + naming conventions(2)

(but please don't use it)

- A few major reasons why tracking experiments in spreadsheets doesn't work for many people:
  - have to remember to track them
    - If something doesn't happen automatically, things get messy, especially with more people involved
  - have to be sure that you or your team will not overwrite things in the spreadsheet by accident
    - Spreadsheets are not easy to version, so if this happens, you are in trouble
  - have to remember to use the naming conventions
    - If someone on your team messes this up, you may not know where the experiment artifacts (model weights, performance charts) for the experiments you ran are.
  - have to back up your artifact directories (remember that things break)
  - When your spreadsheet grows, it becomes less and less usable
    - Searching for things and comparing hundreds of experiments in a spreadsheet (especially if you have multiple people that want to use it at the same time) is not a great experience.

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Versioning configuration files with Github

Can version metadata files in GitHub

- Another option is to version all of experiment metadata in Github
  - to commit metrics, parameters, charts, and whatever want to keep track of to Github when running experiment
  - can be done with post-commit hooks where you create or update some files (configs, charts, etc) automatically after experiment finishes
- … Github wasn't built for … machine learning
- It can work in some setups but:
  - .git and Github wasn't built for comparing machine learning objects
  - Comparing more than two experiments is not going to work. Compare in .git systems was designed for comparing two branches, master and develop, for example. If you want to compare multiple experiments, take a look at metrics and overlay learning curves you are out of luck.
  - Organizing many experiments is difficult (if not impossible). You can have branches with ideas or a branch per experiment but the more experiments you run the less usable it becomes
  - You will not be able to monitor your experiments live, the information will be saved after your experiment is finished.

Using modern experiment tracking tools

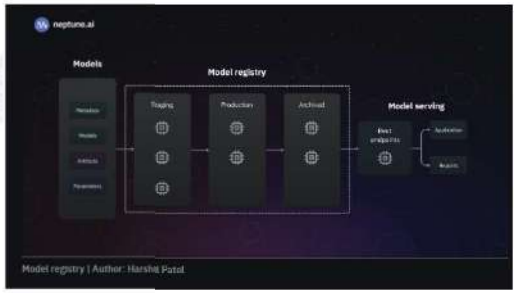
- While can try and adjust general tools to work for machine learning experiments,
  - could just use one of the solutions built specifically for tracking, organizing, and comparing experiments
- Experiment is logged to a central experiment database and displayed in the experiment dashboard,
  - where it can be searched, compared, and drill down to whatever information needed
- Today there are at least a few good tools for experiment tracking!
- Designed to treat machine learning experiments as first-class citizens, and they will always:
  - be easier to use for a machine learning person than general tools
  - have better integrations with the ML ecosystem
  - have more experiment-focused features than the general solutions

Questions from cross-functional team

- Where can we find the best version of this model so we can audit, test, deploy, or reuse it?
- How was this model trained?
- How can we track the docs for each model to make sure they are compliant and people can know the necessary details about it including the metadata?
- How can we review models before they are put to use or even after they have been deployed?
- How can we integrate with tools and services that make shipping new projects easier?
- They want to understand
  - what is running in production,
  - how to improve it
  - or roll back to previous versions
- It makes perfect sense!

What is a model registry?

- An ML model registry serves as a centralized repository, enabling effective model management and documentation
- Allows for
  - clear naming conventions,
  - comprehensive metadata,
  - and improved collaboration between data scientists and operations teams,
  - ensuring smooth deployment and utilization of trained models
- A data scientist can push trained models to the model registry
  - Once in the registry, models are ready to be tested, validated, and deployed to production



BITS Pilani WILP

innovate achieve lead

Model registry vs model repository

- Model Repository is a storage location for machine learning models
- Model Registry is a more comprehensive system that tracks and manages the full lifecycle of machine learning models.
- However, both are often used interchangeably, and the specific definitions may vary depending on the context or the tools and platforms being used.

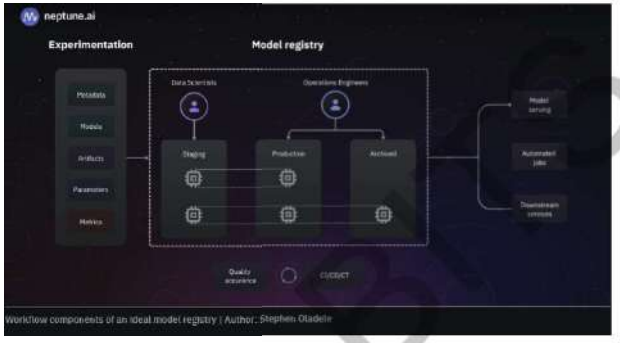
Feature	Model Repository	Model Registry
Storage	Stores machine learning models in a file format	Stores machine learning models and associated metadata
Version control	No	Yes
Lineage tracking	No	Yes
Access control	No	Yes
Other features	None	Can track the performance of models over time, ensure reliability and reproducibility of models, and protect sensitive models from unauthorized access

Model registry vs Experiment tracking

- Model registry has to integrate with the Experiment management system (which tracks the experiments) to register models from various experiment runs to make them easier to find and work with.

Parameters	Model registry	Experiment tracking
Purpose	To store trained, production-ready, and retired models in a central repository.	To track experiment runs of different parameter configurations and combinations.
Priority	To make sure models are discoverable and can be accessed by any user or system.	To make sure experiments are easier to manage and collaborate on.
Integration	Integrates with the experiment tracking system to register models from successful experiments including the model and experiment metadata.	Integrates with the training pipeline to perform experiment runs and track experiment details including the dataset version and metadata.
MLOps	A crucial piece of MLOps and production models.	Most useful in the model development phase, has an indirect impact on operationalizing the model.

Model registry key components

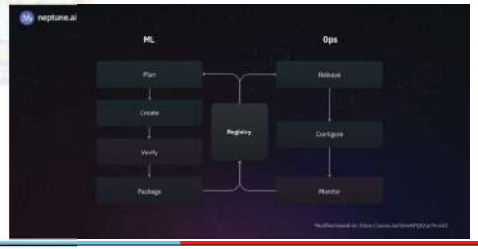


Workflow components of an ideal model registry | Author: Stephen Oladele

Model registry key features and functionalities

- Acts as a centralized storage for effective collaboration
  - Model registry provides a central storage unit that holds models (including model artifacts) for easy retrieval by an application (or service)
  - Without the model registry, the model artifacts would be stored in files that are difficult to track and saved to whatever source code repository is established
  - The centralized storage also enables data teams to have a single view of the status of all models, making collaboration easier

Bridges the gap between experiment and production activities
- Model registry acts as a glue between ML experimentation and Operations,
- enabling model development, software development, and operational teams to collaborate.



BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Model registry key features and functionalities(2)

Providing a central UI

- Model registry provides teams with visibility over their models
- With a central interface, teams can:
  - Search for models,
  - View the status of models (if they are being staged, deployed, or retired),
  - Approve or disapprove models across different stages,
  - And view the necessary documentation.
- This makes model discovery easier for everyone on the team.
- If a model needs to be deployed, the operations teams can easily:
  - Search for it,
  - Look up the validation results and other metrics,
  - Package the model (if needed),
  - And move it from the staging environment to the production environment.
- This improves the way cross-functional teams collaborate on ML projects.

Model registry key features and functionalities(3)

Enables model versioning by tracking different versions of a model

- Model registry enables model versioning by tracking the different versions of a model as they are developed and improved
- This allows to compare different versions of the model, track its performance, and select the best version for deployment
- Here's how it typically works:
  - Model Registration: When a new model is developed or trained, it is registered in the model registry.
  - Version Control: The model registry maintains a history of all registered models and their versions.
  - Model Comparison: The model registry allows users to compare performance metrics, model architectures, hyperparameters, and other relevant information in different versions of a model.
  - Model Tracking: As new versions of the model are developed or trained, they are registered in the model registry as well, incrementing the version number.
  - Retention and Archiving: The model registry typically retains older versions of the models, ensuring a complete history and traceability.
  - By enabling model versioning, the model registry ensures that different iterations of a model can be stored, tracked, compared, and accessed conveniently.

Model registry key features and functionalities(4)

Integrates with experiment management systems or training pipelines

- Model registry integrates with systems that output the trained model
- The trained models could be the raw artifacts (model weights, configuration, and metadata) or models that have been serialized into a file (e.g., an ONNX file) for compatibility with the production environment or containerized (using Docker) to be exported to the production environment.
- Model registries:
  - Register the model,
  - Assign a version to it,
  - Note the version of the dataset the model was trained on,
  - Add annotations and tags,
  - Retrieve the parameters, validation results (including metrics and visualizations), and other relevant model metadata on the model from the experiment management system.
- To make collaboration easier, the registry also includes details such as:
  - The model owner or developer,
  - Experiment run id the model was trained under,
  - Versioned model source code,
  - Environment runtime dependencies used to train the model (and the versions),
  - Comments and model change history,
  - And the model documentation.

Model registry key features and functionalities(5)

Integrates with the staging environment for trained models

- Model Registry provides the functionality for integrating with the staging environment for running all types of checks and balances on the model
  - can include integration testing (with other applications) and other QA tests before the model can be promoted to the production environment
- In the staging environment, the model reviewers should also be able to perform fairness checks on the model to ensure it:
  - Outputs explainable results,
  - Complies with regulatory requirements,
  - And provides useful business benefits.

BITS Pilani WILP

Model registry key features and functionalities(6)

Integrate with model delivery (CI/CD) tools and services for automation

- Automation is a critical part of building any scalable software
  - In machine learning, building automated pipelines will allow to spend more time building new products rather than maintaining old models
- A Model registry integrates with pipeline automation tools and provides custom APIs that can allow to plug into custom workflow tools
  - For example, using webhooks to trigger downstream actions based on predefined events in the registry.
- Should also be able to configure model promotion schemes through different environments like development (training), staging (testing), and production (serving)
- Performance is a crucial requirement for building automated pipelines
  - Model registries should be highly available for automated jobs that are event or schedule-based to enable continuous training and delivery of the model

Model registry key features and functionalities(7)

- Provides an interface for downstream systems to consume models
  - Model registries provide interfaces that enable downstream services to consume the model through API integration
  - The integration can also track offline and online evaluation metrics for the models
  - makes it easy to build an automated setup with CI/CD/CT with ML pipelines
  - downstream service could be either a model user, an automated job, or a REST serving that can consume the most stable—or any—version of the model
- Integrate with model deployment tools
  - Eventually, models have to be deployed, and the more efficient the deployment process, the better.
  - Model Registries:
    - Integrate with downstream services and REST serving services that can consume the model and serve it in the production environment.
    - Collect real-time (or aggregated) metrics on the production model to log performance details of the model. This will be helpful for comparison between models (deployed and staged), as well as auditing the production model for review

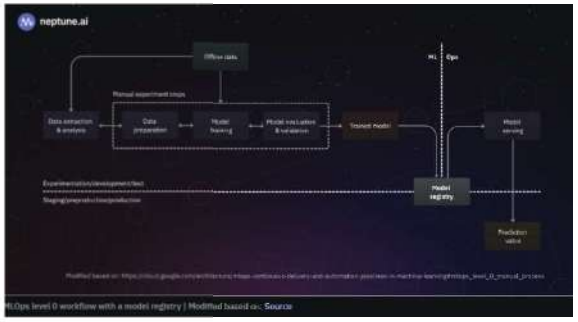
Where does a model registry fit in the MLOps stack?

- If want to run machine learning projects efficiently and at scale,
  - most likely need to add a model registry to MLOps stack.
- Depending on what level of implementation you are in your MLOps stack,
  - needs and requirements for a model registry would differ.
- Where does it fit?
  - the model registry sits between machine learning development and deployment.

Where does a model registry fit in the MLOps stack?(2)

Model registry in MLOps level 0

- The output from the experimentation step is fed into the model registry.
  - involves a manual process where the data scientist prepares the model artifact and metadata and could also package them (serialization, containerization) before registering them
- The operations team can push the packaged model to the staging environment
  - for testing before deploying it to a prediction service engine that can integrate with other applications.



MLOps level 0 workflow with a model registry | Modified based on: Source

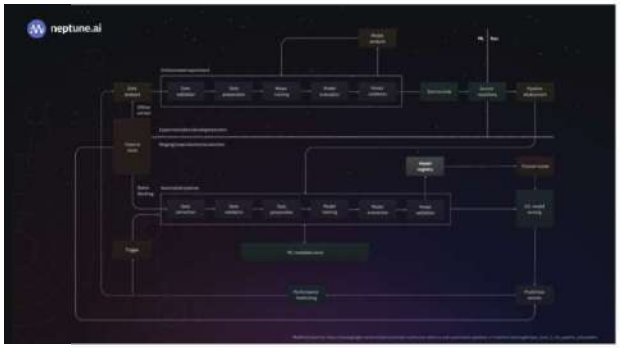
BITS Pilani WILP

Where does a model registry fit in the MLOps stack? (3)

Model registry in MLOps level 1

- As opposed to level 0 (where the workflow is a manual process), the goal of the workflow in level 1 is to perform continuous training of the model by automating the ML pipeline. This is one process a model registry enables well because of its ability to integrate with the pipeline.

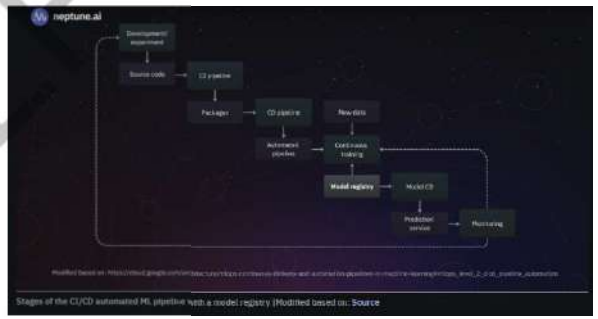
- At this level, the entire pipeline is deployed, and when models are trained on the provided dataset, the output (trained model and metadata) is fed into the model registry where it can be staged, and if it passes the necessary tests and checks, it can be fed to the continuous delivery pipeline for release.



Where does a model registry fit in the MLOps stack?(4)

Model registry in MLOps level 2

- The role of the model registry in level 2 of the MLOps workflow is also the same as that of level 1—the automated pipeline delivers the trained model to the model registry where it is staged, may be passed through QA checks, and sent to the continuous delivery pipeline:
- The model registry serves as a crucial component in any automated pipeline because event triggers can be integrated with it to promote models with good metrics upon re-training on fresh data or archive models.



Stages of the CI/CD automated ML pipeline with a model registry [Modified based on: Source]

Setting up ML model registry – build, maintain or buy?

- Setting up a model registry for MLOps workflow will require you to decide on either
  - Building one,
  - Maintaining one,
  - or Buying one

Setting up ML model registry – build, maintain or buy?(2)

Building a model registry solution

- Like any software solution, if you understand the key functionalities and requirements, you can build a system yourself! This is the case with a model registry.

- May want to set up the following:
  - Object storage for models and artifacts
  - Database for logging model details
  - API integration for both receiving models, promoting models across various environments, and collecting model information from the different environments
  - User interface (UI) for ML teams to interact with a visual workflow

- While building the solution yourself might seem ideal, you should consider the following factors:
  - Incentive: What's the incentive to build out your solution? Is it for customization or for owning a proprietary license to the solution?
  - Human resources: Do you have the talents and skills to build out your solution?
  - Time: How long would it take you to build out a solution and is it worth the wait?
  - Operations: When the solution is eventually built out, who would maintain its operations?
  - Cost: What would it cost you to build a solution, including the maintenance of the solution?

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Setting up ML model registry – build, maintain or buy?(3)

Maintaining a self-hosted model registry

- Another option to consider is to maintain an existing solution yourself
  - solution has been built out already, but might have to manage some features, such as object storage and database
  - most of these existing solutions are open-source solutions

- The following are the factors to consider:
  - Type of solution: Are you going to opt for an open-source solution with no license cost or a closed-source solution with license cost?
  - Operations: Who is going to manage the solution? Does the solution support consistent maintenance and software updates?
  - Cost: What is the cost of operating the solution in terms of the infrastructure to host it and the running cost?
  - Features: What features have already been implemented, and what features do you have to build and manage yourself? Is it worth adopting compared to building out your solution?
  - Support: What type of support is available in case things break during operations? Is there a community or dedicated customer support channel? For open-source solutions, while you might have a community, you will likely lack the necessary developer support required to fix things.
  - Accessibility: How easy is it to get started with the solution? Is the documentation comprehensive enough? Can everyone from the model reviewers to the model developers and software engineers intuitively use the solution?

Setting up ML model registry – build, maintain or buy?(4)

Purchase the license to a fully-managed solution

- The final option to consider is subscribing to a fully managed solution where the operations and management of the registry are handled by the solution vendor
  - do not have to worry about building or maintaining a solution
  - just have to ensure systems and services can integrate with the registry

- Here are the factors to consider:
- Industry type: What type of industry is the model built for? What sensitive information have the models learned? Are there data privacy compliance measures? Is the model only allowed to stay on-premise?
- Features: Are the key features and functionalities of any model registry available in this solution? What extra features are available, and how relevant are they to your workflow?
- Cost: What's the cost of purchasing a license, and do the features justify the cost?
- Security: How secure is the platform hosting the solution? Is it resistant to third-party attacks?
- Performance: Is the registry highly performant? For situations where models are too large, can the registry provide models for services to consume at low latency?
- Availability: What's the uptime of the solution, and does it meet your required service level agreement (SLA)?
- Support: What level of support is available in case things go south?
- Accessibility: How easy is it to get started with the solution? Are the documentation and learning support decent enough? What's the learning curve in terms of usage?

ML model registry solutions out there

The most popular ones available out there that offer a host of Model Registry features.

Model Registry Features	Azure ML Studio	Neptune.ai	MLFlow	Verta.ai	AWS Sagemaker
Model Versioning	✓	✓	✓	✓	✗
CI/CD/CT compatibility	✓	✗	✓	✓	✗
Model packaging	✗	✗	✓	✓	✓
Model searching	✓	✓	✓	✓	✓
Access control, model review, and promoting models	✗	✓	✓	✓	✓
Model lineage and evaluation history	✓	✗	✗	✓	✓

Dataset Metadata

Metadata about features and labels

- Dataset provenance
  - Where did the data come from?
  - Depending on the source of data, might have a lookup table of logs from various systems, a key for an external data provider with data about when we downloaded the data, or even a reference to the code that generated the data

- Dataset location
  - For some datasets, will store raw, unprocessed data
    - should store a reference to where dataset is kept, as well as perhaps information about where we got it from
  - Some data is created for on an ongoign basis, such as logs from systems - should store the log or datastore reference where
    - that data is stored, or where we are permitted to read from it.

- Dataset responsible person or team
  - Should track which person or team is responsible for the dataset
  - In general, this is the team that chose to download or create the dataset, or the team that owns the system that produces the data

- Dataset creation date or version date
  - Often useful to know the first date a particular dataset was used

- Dataset use restrictions
  - Many datasets have restrictions on their use, either because of licensing or governance constraints
  - should document that in the metadata system for easy analysis and compliance later

BITS Pilani WILP

Feature Metadata

Track of metadata about feature definitions

- Feature version definition
  - a reference to code or another durable description to what data the feature reads and how it processes the data to create the feature
  - should be updated for every updated version of the feature definition
  - versioning these definitions (and restricting the versions in use) will make the resulting codebase more predictable and maintainable
- Feature definition responsible person or team
  - There are two good use cases for storing this information:
  - figuring out what a feature is for
  - and finding someone who can help resolve an incident when the feature might be at fault
  - In both cases, it is useful to store authorship or maintainer information about the feature
- Feature definition creation date or current version date
  - useful to get a change history of when a feature was most recently updated and when it was originally created
- Feature use restrictions
  - important but trickier to store
  - Features may be restricted from use in some contexts.
  - For example, it may be illegal to use a particular feature in some jurisdictions.
    - Age and gender may be a reasonable predictor of automobile insurance risk models, but insurance is highly regulated
    - may not be permitted to take those fields into account

Label Metadata

Intended to help with the maintenance and development of the labeling system itself, but might also be used by the training system as it uses the labels

- Label definition version
  - Switching to metadata specific to labels and analogously to features,
  - must store the version of any label definitions to understand which labeling instructions the labels were made with
- Label set version
  - Changes to the labels may occur because of incorrect labels getting corrected or new labels being added
  - If the dataset is being used for comparison with an older model, using an older version of the labels may be desirable
- Label source
  - Although not typically needed for training, it is sometimes necessary to know the source of each label in a dataset
  - may be the source that particular label was licensed from,
  - the human who produced the label(along with any QA that was applied to the label),
  - or the algorithm that produced the label if an automated labeling approach was used
- Label confidence
  - Depending on how the labels are produced, might have different estimates of the confidence of correctness for different labels
  - Users of these labels might choose different thresholds to decide which labels to use in training their models

Pipeline Metadata

- Won't spend as much time on: metadata about the pipeline processes themselves
  - data about the intermediate artifacts we have,
  - which pipeline runs they came from,
  - and which binaries produced the
- Type of metadata is produced automatically by some ML training systems
  - for example, ML Metadata (MLMD) is automatically included in TensorFlow Extended (TFX),
  - which uses it to store artifacts about training runs
- Such systems are either integrated into training systems or are somewhat difficult to implement later!

Metadata Systems Overview

Feature systems and labeling systems both benefit from efficient tracking of metadata

- A metadata system is designed to keep track of what we're doing
  - In the case of features and labels, it should minimally keep track of the feature definitions and the versions used in each model's definitions and trained models
- Most organizations start building their data sciences and ML infrastructure without a solid metadata system,
  - only to regret it later!
- The next most common approach is to build several metadata systems, each targeted at solving a particular problem
  - make one for tracking feature definitions and mappings to feature stores
  - need a system for mapping model definitions to trained models, along with data about the engineers or teams responsible for those models
  - need a model serving system is also going to need to keep track of trained model versions, when they were put into production
  - need a model quality or fairness evaluation systems will need to be read from all of these systems in order to identify and track the likely contributing causes of changes in model quality or violations of our proposed fairness metrics

BITS Pilani WILP

Choices for a metadata system

One system

- Build one system to track metadata from all of these sources
  - makes it simple to make correlations across multiple subsystems, simplifying analysis and reporting
- Such a large system is difficult to get right from a data schema perspective
  - will be constantly adding columns when we discover data we would like to keep track of (and backfilling those columns on existing data)
  - be difficult to stabilize such a system and make it reliable
- From a systems design perspective, should ensure that our metadata systems are never in the live path of either model training or model serving
  - But it's difficult to imagine how feature engineering or labeling can take place without the metadata system being functional
  - can still cause production problems for our humans working on those tasks
- If the needs are simple and well understood, prefer a single system.

Choices for a metadata system (2)

Multiple systems (that work together)

- Can build separate metadata systems for each task we identify
  - Perhaps would have one for features and labels, one for training, one for serving and one for quality monitoring
- Decoupling the system provides the standard advantages and costs that decoupling always does
  - allows to develop each system separately without concern for the others, making them nimbler and simpler to modify and extend
  - an outage of one metadata system has limited production impact on others
  - The cost, though, is the added difficulty of analysis and reporting across those systems
  - Will have processes that need to join data across the features, labeling, training, and serving systems
  - The area is rapidly developing and teams expect to continue extending what they track and how they work, multiple systems will simplify the development over time.

Most Ignored!

- More generally, metadata systems are often overlooked or deprioritized
  - They should not be
  - They are one of the most effective and direct contributors to productive use of the data in an ML system
  - Metadata unlocks value and should be prioritized

Need for metadata

But the problems people have with storing and managing ML model metadata are different.

- For some, it is messy experimentation that is the issue
- Others have already deployed the first models to production, but they don't know how those models were created or which data was used
- Some people already have many models in production, but
  - orchestrating model A/B testing,
  - switching challengers and champions,
  - or triggering, testing, and monitoring re-training pipelines is not great.
- ML metadata store can help with all of those things and then some others as well.

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

What is ML metadata anyway?

- When you do machine learning, there is always a model involved. It is just what machine learning is.
- Ml Model could be a classic, supervised model like a lightGBM classifier, a reinforcement learning agent, a bayesian optimization algorithm, or anything else really.
  - But it will take some data, run it through some numbers and output a decision.
  - … and it takes a lot of work to deliver it into production.
- One has to:
  - train,
  - tune,
  - debug,
  - evaluate,
  - explain,
  - and compare it to baseline.
- If model makes it past the research phase, it also has to :
  - package,
  - deploy,
  - monitor,
  - and re-train it.

What is ML metadata anyway? (2)

- A lot of steps!
  - Don't go through those steps linearly, and you don't go through them once.
  - Takes a lot of iterations of fixing data, fixing model, fixing preprocessing, and all that good stuff.
  - Each of those steps produces meta-information about the model, or as many people call it, ML model metadata.
- Those could be:
  - training parameters,
  - evaluation metrics,
  - prediction examples,
  - dataset versions,
  - testing pipeline outputs,
  - references to model weights files,
  - and other model-related things.
- This information helps you know your models.
  - Where the particular version of the model is and quickly rollback to the previous version
  - How the model was built and who created it
  - Which data/parameters/code it used at training
  - How does the new experiment or model version compare to the baseline or previous production models
  - How did it perform at various evaluation stages
  - And when your models are important to someone, when they touch users or help clients make decisions, you better know a lot about them.

Metadata about experiments and model training runs

- During experimentation, you usually care about debugging, visualizing, monitoring your model training to get to the best model.
- To do that, it is a good practice to log anything that happens during the ML run, including:
  - data version: reference to the dataset, md5 hash, dataset sample to know which data was used to train the model
  - environment configuration: requirements.txt, conda.yml, Dockerfile, Makefile to know how to recreate the environment where the model was trained
  - code version: git SHA of a commit or an actual snapshot of code to know what code was used to build a model
  - hyperparameters: configuration of the feature preprocessing steps of the pipeline, model training, and inference to reproduce the process if needed
  - training metrics and losses: both single values and learning curves to see whether it makes sense to continue training
  - record of hardware metrics: CPU, GPU, TPU, Memory to see how much your model consumes during training/inference
  - evaluation and test metrics: f2, acc, roc on test and validation set to know how your model performs
  - performance visualizations: ROC curve, Confusion matrix, PR curve to understand the errors deeply
  - model predictions: to see the actual predictions and understand model performance beyond metrics
  - …and about a million other things that are specific to your domain

Metadata about artifcats

- Apart from experiments and model training runs, there is one more concept used in ML projects: artifact
  - input or output of those runs can be used in many runs across the project
  - Artifacts can change during the project, and typically have many versions of the same artifact at some point in your ML lifecycle
  - Artifacts could be datasets, models, predictions, and other file-like objects.
- For artifacts you may want to log:
  - Reference: Paths to the dataset or model (s3 bucket, filesystem)
  - Version: Dataset or model md5 hash to let you quickly see the diff
  - Preview: Dataset/prediction preview (head of the table, snapshot of the image folder) to see what this dataset is about
  - Description: additional info about the artifact that will help you understand what it is. For example, you may want to log column names for a tabular dataset artifact
  - Authors: who created modified this artifact and when
  - And many other things that may be specific to your project as the size of the dataset, type of the ML model and others

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Metadata about trained models

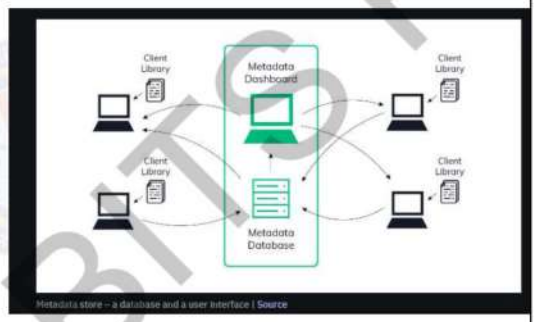
- Trained models are such an important type of artifact in ML projects
  - Once your model is trained and ready for production, needs change from debugging and visualization to knowing how to deploy a model package, version it, and monitor the performance on prod.
- So the ML metadata may want to log are:
  - Model package: Model binary or location to your model asset
  - Model version: code, dataset, hyperparameters, environment versions
  - Evaluation records: History record of all the evaluations on test/validation that happened over time
  - Experiment versions: Links to recorded model training (and re-training) runs and other experiments associated with this model version
  - Model creator/maintainer: who build this model, and who should you ask if/when things go wrong
  - Downstream datasets/artifacts: references of datasets, models, and other assets used downstream to build a model. This can be essential in some orgs for compliance.
  - Drift-related metrics: Data drift, concept drift, performance drift, for all the "live" production
  - Hardware monitoring: CPU/GPU/TPU/Memory that is consumed in production.
  - And anything else that will let you sleep at night while your model is sending predictions to the world

Metadata about pipeline

- You may be at a point where your ML models are trained in a pipeline that is triggered automatically:
  - When the performance drops below certain thresholds
  - When new labeled data arrives in the database
  - When a feature branch is merged to develop
  - Or simply every week
- Likely have some CI/CD workflow connected to a pipeline and orchestration tool like Airflow, Kubeflow, or Kedro
  - every trigger starts the execution of a computation DAG (directed acyclic graph) where every node produces metadata
- Need for the metadata is a bit different than for experiments or models
  - This metadata is needed to compute the pipeline (DAG) efficiently:
  - Input and output steps: information about what goes into a node and what goes out from a node and whether all the input steps are completed
  - Cached outputs: references to intermediate results from a pipeline so that you can resume calculations from a certain point in the graph

What is an ML metadata store?

- ML metadata store is a "store" for ML model-related metadata
  - place where one can get anything one need when it comes to any and every ML model you build and deploy.
- More specifically, ML metadata store lets you:
  - log,
  - store,
  - display,
  - monitor,
  - compare,
  - organize,
  - filter,
  - and query all model-related metadata.
- In short, it gives a single place to manage all the ML metadata about experiments, artifacts, models, and pipelines we have listed in the previous section.
- You can think of it as a database and a user interface built specifically to manage ML model metadata
- It typically comes with an API or an SDK (client library) to simplify logging and querying ML metadata.



Metadata store -- a database and a user interface | Source

Repository vs Registry vs Store

- In the context of ML metadata,
  - all of those things are basically databases created to store metadata that have slight functional differences
- A metadata repository
  - place where store metadata objects along with all the relationships between those objects
  - can use GitHub as a repository where save all the evaluation metrics as a file created via post-commit hook or log parameters and losses during training to an experiment tracking tool which in this context is actually a repository
- A metadata registry
  - place where you "checkpoint" the important metadata
  - register something you care about and want to easily find and access it later
  - A registry is always for something specific. There are no general registries
  - For example, you may have a model registry that lists all the production models with references to the ML metadata repository where the actual model-related metadata is.
- A metadata store
  - place where you go "shopping" for metadata
  - For ML models, it is a central place where you can find all the model, experiment, artifact, and pipeline-related metadata
  - The more you need to come to the "shop", search metadata "products", compare them and "buy" them, the more it is a store rather than a repository.

BITS Pilani WILP

innovate achieve lead

Two flavors of ML metadata store

Pipeline-first vs model-first ML metadata store

- As your ML organization matures, you get to a point when training models happen in some automated, orchestrated pipelines
  - At that moment, running experiments, training, and re-training productions models is always associated with executing a pipeline.
- ML metadata store can:
  - Treat pipelines as first-class objects and associate resulting models and experiments with them
  - Treat models and experiments as first-class objects and treat pipelines as a mode of execution
  - Depending on what you put in a center, your ML metadata store will do slightly different things
- If you focus on the process, the model building pipeline, you get something like MLMD, which powers pipeline and orchestration tools like Kubeflow and TensorFlow Extended (TFX).
- If you focus on the output, the model, you get something like MLflow or Neptune.

How do you set up an ML metadata management system?

Build vs maintain vs buy

- It's an old dilemma!
  - People who work in software face it many times in their careers.
- To make a decision, you should:
  - understand what problem you are actually solving
  - see if there are tools that can solve this problem before building it
  - assess how much time it will take to build, maintain, and customize it
  - don't underestimate the amount of effort that goes into it

How do you set up an ML metadata management system?(2)

Build a system yourself

Pros	Cons
No licence cost	You have to implement it
You can create it exactly for your use case	You have to set up and maintain the infrastructure
You can improve/change it however you like	You have to document it for other people
	You have to fix bugs
	You have to implement improvements and integrations with other tools
	When software doesn't work it is your fault

How do you set up an ML metadata management system?(3)

Maintain open source

Pros	Cons
No licence cost	You have to set up and maintain the infrastructure
You don't have to implement it	You have to adjust it to your needs
Documentation and community support is already there	You have to count on community support
Other people are creating improvements that you may care about	You can push for new features but may have to implement them yourself
	When software doesn't work it is your fault
	Project may get abandoned by the creators/maintainers/community

BITS Pilani WILP

How do you set up an ML metadata management system?(4)

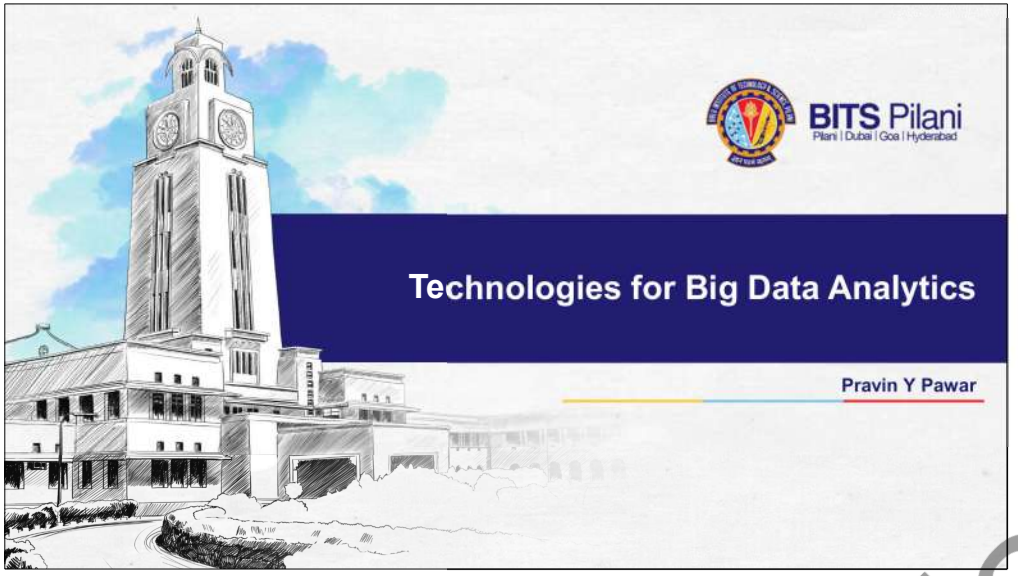
Buy a solution

Pros	Cons
You don't have to implement it	Licence fee
Documentation and support is provided by the vendor	You may not be able to fix bugs/create improvements yourself and the vendor may be slow to fix them
Infrastructure is already there (for hosted options)	Company may go out of the market
When you need features you can request them	
When software doesn't work it is their fault (and they have to fix it)	

Thank You!

In our next session:

BITS Pilani WILP



Technologies for Big Data Analytics

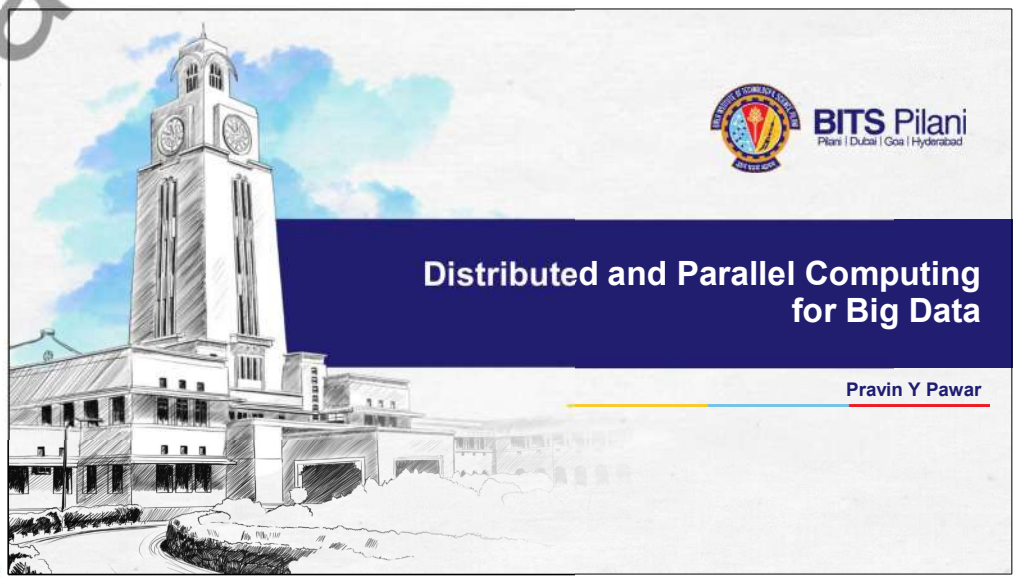
Problem of managing data

- How to manage voluminous, varied and scattered data?
  - ✓ Traditional data storage and processing systems not able to cope up with this type of data
  - ✓ Prompted the requirement of systems for storage and processing of big data
- Most effective and popular ones are
  - ✓ Distributed and parallel processing
  - ✓ Hadoop
  - ✓ Big Data Cloud
  - ✓ In-memory computing

Popular Technologies

Three options

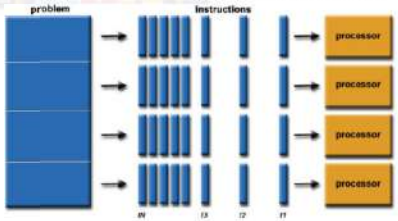
- Big Data Clouds
  - ✓ Helps to save cost and better management of resources
  - ✓ Everything based on a services model
- Hadoop
  - ✓ Most popular open source platform for big data processing
  - ✓ Used by data-driven organizations to derive value which can be used in decision making
- In-memory Computing (IMC)
  - ✓ By usage of main memory (RAM) helps to manage data processing tasks better and faster
- Choice depends heavily on the requirements!



BITS Pilani WILP

Parallel Computing

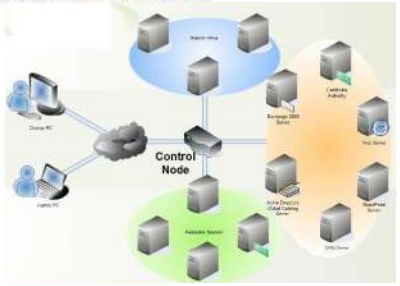
- In Parallel Computing,
  - ✓ Additional computational resources are added to the same system to improve the processing capability of system
  - ✓ Helps in dividing complex computations into subtasks, which can be handled individually by processing units that are running in parallel
  - ✓ Multiple computing systems are running parallel
  - ✓ Concept is processing capability will increase with the increase in the level of parallelism



Distributed Computing

Need

- In distributed computing,
  - ✓ Multiple computing resources are connected in network and computing tasks are distributed across these resources
  - ✓ Results into increase in speed and efficiency of system
  - ✓ Faster and more efficient than traditional methods of computing
  - ✓ More suitable to process huge amounts of data in limited time



Techniques for High Data Processing

Method	Description	Usage
Cluster or Grid Computing	• Based on a connection of multiple servers in a network • Servers share the workload among them • Systems can be homogeneous or heterogeneous	• Used in Hadoop • Cluster can be created by hardware components that were acquired long back
Massively Parallel Processing (MPP)	• Single machine working as grid • Capable of handling activities of storage, memory and computing	• Used in Data warehouses • EMC Greenplum • ParAccel
High-Performance Computing (HPC)	• Known to offer high performance and scalability by using IMC • Suitable for processing floating-point data at high speeds	• Used to develop specialty and custom applications for research where results is more valuable than cost

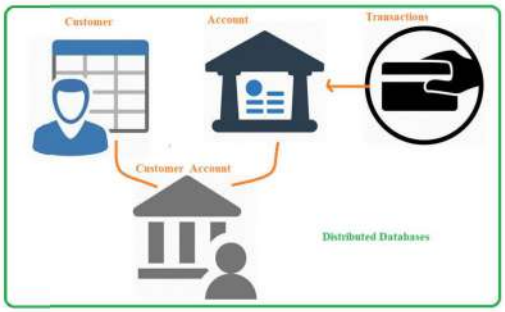
Comparing Parallel and Distributed Systems

Distributed System	Parallel System
• Independent, autonomous system connected in a network accomplishing specific tasks	• Computer system with several processing units attached to it
• Coordination is possible between connected computers with own memory and CPU	• A common shared memory can be directly accessed by every processing unit in a network
• Loose coupling of computers connected in network, providing access to data and remotely located resources	• Tight coupling of processing resources that are used for solving single, complex problem

BITS Pilani WILP

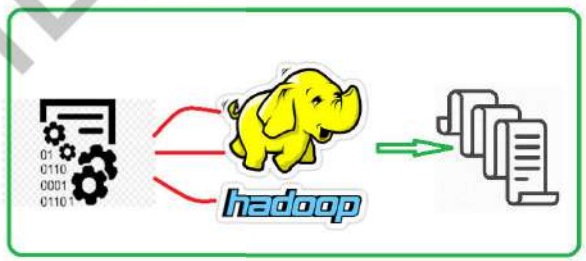
Data Models – Distributed Databases

- Deal with tables and relations
- Must have a schema for data
- Implements data partitioning



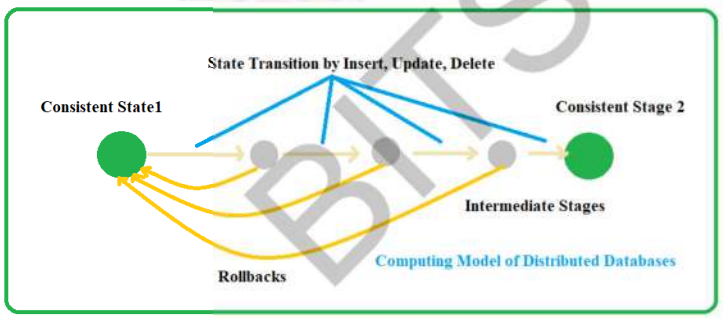
Data Models – Hadoop

- Deals with flat files in any format
- Operates on no schema for data
- Divides files automatically in blocks



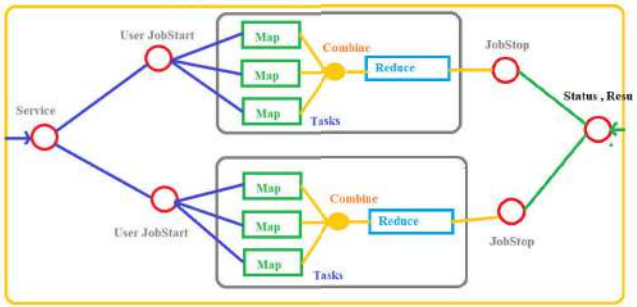
Computing model – Distributed Databases

- Generate notions of transaction
- Implement ACID transaction properties
- Allow distributed transactions

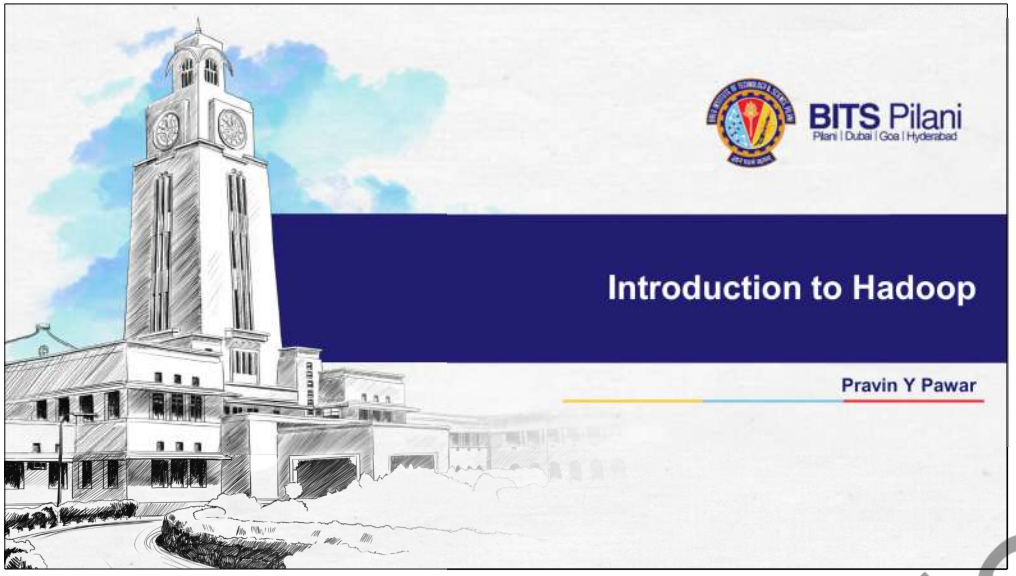


Computing model - Hadoop

- Generates notions of a job divided into tasks
- Implements MapReduce computing model
- Considers every task either as a Map or a Reduce



BITS Pilani WILP



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Introduction to Hadoop

Pravin Y Pawar

Big Data Challenges

Volume, Variety and Velocity

- "How to store terabytes of mounting data?"
  - ✓ VOLUME
- "How to handle structured, semi-structured and unstructured data?"
  - ✓ VARIETY
- "How to manage the data that is getting generated at very fast speed?"
  - ✓ VELOCITY

Image Source : amcham

Why Hadoop?

Key consideration

- Hadoop can handle
  - ✓ Massive amount of data
  - ✓ Different kinds of data
  - ✓ In fast manner
- Advantages
  - Low cost – open source
  - Computing power – many nodes can be used for computation
  - Scalability – simple to add nodes in system
  - Storage Flexibility – can store unstructured data easily
  - Inherent data protection – protects against hardware failures

Distributed Computing Challenges

Problems and Solutions

- Storage of huge amount of data
  - ✓ More systems , more failures
  - ✓ How to retrieve the data stored on the failed node?
  - ✓ Hadoop solves this by Replication Factor (RF)
  - ✓ Number of data copies of a given data item / data block stored across the network
- Processing the huge amount of data
  - ✓ Data is spread across systems, how to process it in quick manner?
  - ✓ Challenge is to integrate data from different machines before processing
  - ✓ Hadoop solves this by MapReduce Programming
  - ✓ Programming model to process huge amount of data at same time in quick manner

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

What is Hadoop?

Key Aspects

- Two Tasks
  - ✓ Massive Data Storage
    - ❑ Huge of amount of data across several nodes
    - ❑ Uses low cost commodity storage
  - ✓ Faster Data Processing
    - ❑ Has everything needed for data processing application development
    - ❑ Computation done parallel on several nodes at same time

Hadoop Ecosystem

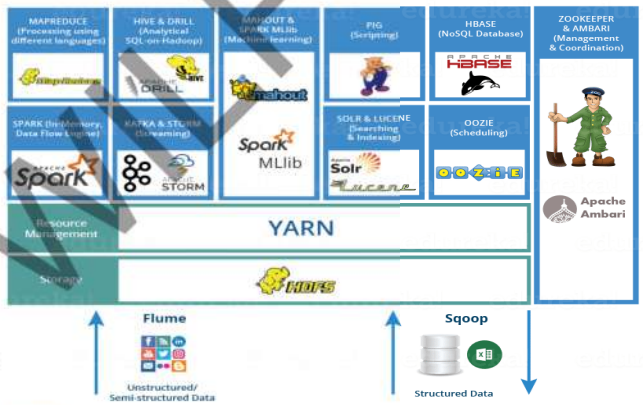


Image Source : Edureka

Hadoop High Level Architecture

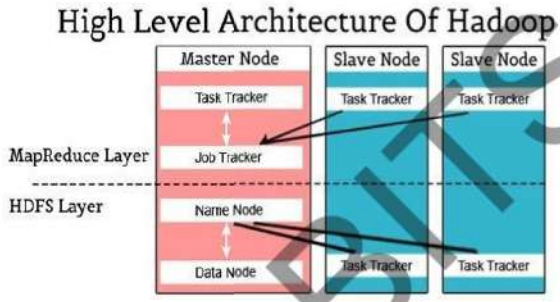
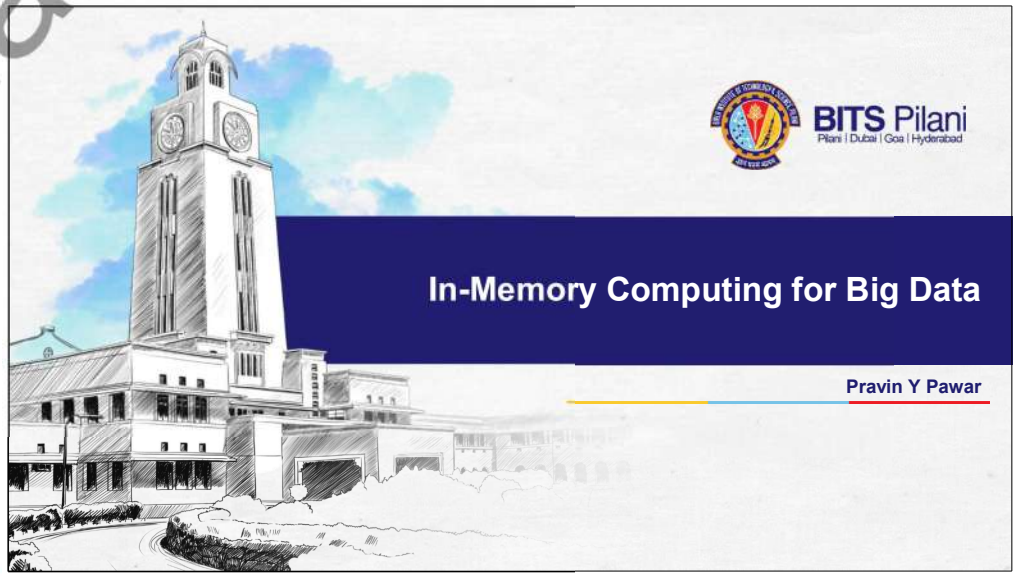


Image Source : intellipaat



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

In-Memory Computing for Big Data

Pravin Y Pawar

BITS Pilani WILP

innovate achieve lead

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Hadoop for Big Data

Issues with MapReduce on Hadoop

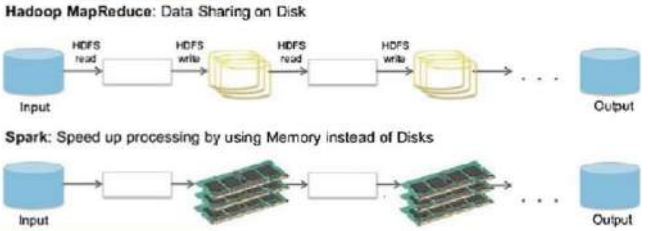
- Apache Hadoop
  - ✓ Revolutionized big data processing, enabling users to store and process huge amounts of data at very low costs.
  - ✓ An ideal platform to implement complex batch applications as diverse as
    - sifting through system logs
    - running ETL
    - computing web indexes
    - recommendation systems etc.
  - ✓ Its reliance on persistent storage to provide fault tolerance and its one-pass computation model make MapReduce a poor fit for
    - low-latency applications
    - iterative computations, such as machine learning and graph algorithms.

Adapted from : https://databricks.com/blog/2013/11/21/putting-spark-to-use.html

In-Memory Computing

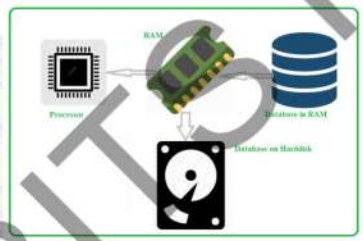
- In-memory computing
  - ✓ means using a type of middleware software that allows one to store data in RAM, across a cluster of computers, and process it in parallel
- For example,
  - ✓ Operational datasets typically stored in a centralized database which you can now store in "connected" RAM across multiple computers.
  - ✓ RAM is roughly 5,000 times faster than traditional spinning disk.
  - ✓ Add to the mix native support for parallel processing, and things get very fast. Really, really, fast!

- RAM storage and parallel distributed processing are two fundamental pillars of in-memory computing.



In-Memory Computing for Big Data

- RAM (or primary storage area)
  - ✓ is used for data processing
  - ✓ Helps to increase the computing speed
  - ✓ Reduction in prices made is affordable
- Data handling
  - ✓ Relational databases are used for structured data
  - ✓ Unstructured data is handled by NoSql databases
- Volume is addressed by IMC
- Diversity of big data managed by NoSQL databases



Fast and Easy Big Data Processing with Spark

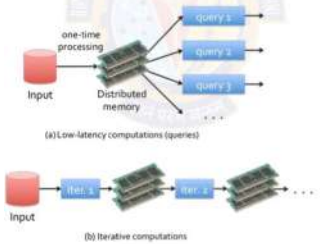
- At its core, Spark provides a general programming model that enables developers to write application by composing arbitrary operators, such as
  - ✓ mappers
  - ✓ reducers
  - ✓ joins
  - ✓ group-bys
  - ✓ and filters
- This composition makes it easy to express a wide array of computations, including iterative machine learning, streaming, complex queries, and batch.
- Spark keeps track of the data that each of the operators produces, and enables applications to reliably store this data in memory.
  - ✓ This is the key to Spark's performance, as it allows applications to avoid costly disk accesses.

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Apache Spark Applications

- Spark enables
- Low-latency computations
  - ✓ by caching the working dataset in memory and then performing computations at memory speeds
- Efficient iterative algorithm
  - ✓ by having subsequent iterations share data through memory, or repeatedly accessing the same dataset



Spark Overview

- Cluster computing platform
  - ✓ Designed to be fast and general purpose
- Speed
  - ✓ Is important in processing large datasets as it creates difference when data is being explored
  - ✓ Extends MapReduce model to efficiently support more types of computations
  - ✓ Runs computations in memory
- Generality
  - ✓ Covers a wide variety of workloads which earlier required different distributed systems
  - ✓ Including
    - ❖ Batch applications
    - ❖ Iterative algorithms
    - ❖ Interactive queries
    - ❖ Streaming
  - ✓ Easy and inexpensive to combine different processing types in data pipelines

APACHE Spark™

Spark Overview (2)

- Spark handles highly accessible, simple APIs in
  - ✓ Java
  - ✓ Python
  - ✓ Scala
  - ✓ SQL

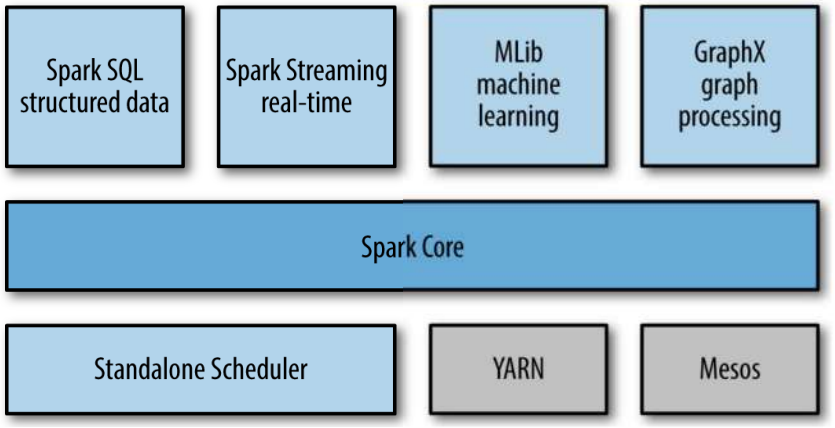
- Integrates easily with other big data tools like
  - ✓ Hadoop (HDFS) and other ecosystem tools
  - ✓ Kafka
  - ✓ AWS

kafka

aws

BITS Pilani WILP

Unified Stack



Ref : https://www.oreilly.com/library/view/learning-spark/9781449359034/ch01.html

Spark Core

- Contains the basic functionality of Spark, including components for
  - ✓ task scheduling
  - ✓ memory management
  - ✓ fault recovery
  - ✓ interacting with storage systems etc.
- Home to the API that defines resilient distributed datasets (RDDs), which are Spark's main programming abstraction
- Provides many APIs for building and manipulating these collections

Spark SQL

- Spark's package for working with structured data
- Allows querying data via SQL as well as the Apache Hive variant of SQL
  - ✓ called the Hive Query Language (HQL)
- Allows developers to intermix SQL queries with the programmatic data manipulations supported by RDDs in
  - ✓ Python
  - ✓ Java
  - ✓ Scala

  all within a single application
- Tight integration with the rich computing environment provided by Spark makes Spark SQL unlike any other open source data warehouse tool

Spark Streaming

- Spark component that enables processing of live streams of data
- Examples of data streams include web servers log files
- Provides an API for manipulating data streams that closely matches the Spark Core's RDD API
- Designed to provide the same degree of fault tolerance, throughput, and scalability as Spark Core

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

MLlib

- Built in library containing common machine learning (ML) functionality
- Provides multiple types of machine learning algorithms, including
  - ✓ Classification
  - ✓ Regression
  - ✓ Clustering
  - ✓ Collaborative filtering
  - ✓ Supporting functionality such as data import and model evaluation
- Provides some lower-level ML primitives, including a generic gradient descent optimization algorithm
- All of these methods are designed to scale out across a cluster

Spark MLlib

GraphX

- Library for
  - ✓ manipulating graphs (e.g. a social network's relations graph)
  - ✓ performing graph-parallel computations
- Extends the Spark RDD API, allowing us to create a directed graph with arbitrary properties attached to each vertex and edge
- Provides
  - ✓ various operators for manipulating graphs (e.g., subgraph and mapVertices)
  - ✓ library of common graph algorithms (e.g., PageRank and triangle counting)

Apache Spark GraphX

Cluster Managers

- Spark is designed to efficiently scale up from one to many thousands of compute nodes
- Can run over a variety of cluster managers, including
  - ✓ Hadoop YARN
  - ✓ Apache Mesos
  - ✓ Simple cluster manager included in Spark itself called the Standalone Scheduler
  - ✓ Kubernetis
- If you are just installing Spark on an empty set of machines
  - ✓ the Standalone Scheduler provides an easy way to get started
- If you already have a Hadoop YARN or Mesos cluster,
  - ✓ Spark's support for these cluster managers allows your applications to also run on them

Spark Standalone Cluster | hadoop YARN | MESOS | kubernetes

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Agenda

- Messaging Systems
- Kafka Architecture
- Kafka Fundamentals
- Kafka Workflow
- Kafka Command Line Interface
- Kafka Java APIs

Messaging System

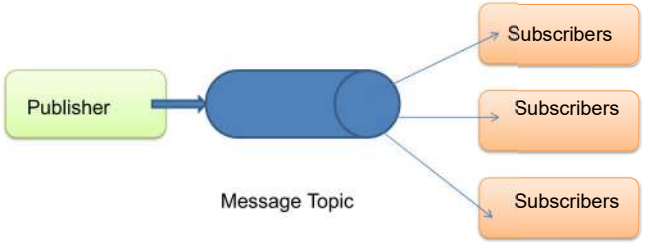
- System to transfer data from one application to another
- Application focuses on its core features whereas messaging system takes care of data transfer / sharing
- Two types
  - Point to point
  - Pub-sub

Point to Point messaging system

Messages are persisted in queue
One or more consumers can be connected to queue
But only one can consume the message
Message removed from queue,once any consumer reads it

Pub-sub Messaging System

Messages are persisted in topic
Many consumers can consume messages at same time
Message producers are Publishers
Message consumers are Subscribers



BITS Pilani WILP

innovate achieve lead

Apache Kafka

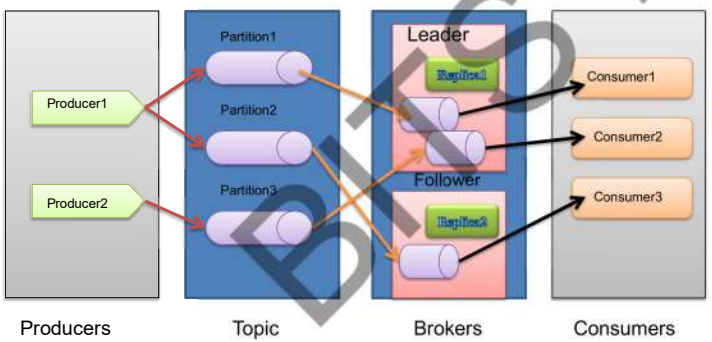
- Distributed pub-sub and queuing messaging system
- Designed for high throughput distributed systems
  - Better throughput
  - In-built partitioning mechanism
  - Replicas
  - Fault Tolerant
- Good fit for large scale data processing
- Suitable for both offline and online message transfer
- Messages are persisted on disk and replicated across cluster to prevent message loss
- Integrate with Storm and Spark easily

Kafka Benefits

- Reliability
  - distributed, partitioned, replicated and fault tolerant
- Scalability
  - scales easily without down time by addition of machines in cluster
- Durability
  - messages persists on disk as fast as possible
- Performance
  - has high throughput for both publishing and subscribing messages

Kafka Fundamentals

Kafka Architecture



Topics

- A stream of messages belonging to a particular category
  - Data is stored in topic
- Topics are split into partitions
  - For each topic, Kafka keeps a minimum of one partition
  - Each such partition contains messages in an immutable ordered sequence
- A partition is implemented as a set of segment files of equal sizes

BITS Pilani WILP

Partition

- Topics may have many partitions, so it can handle an arbitrary amount of data.
  - Each partitioned message has a unique sequence id called as "offset".
- Replicas are nothing but "backups" of a partition.
- Replicas are never read or write data. They are used to prevent data loss.

Brokers

- Simple system responsible for maintaining the published data
  - Each broker may have zero or more partitions per topic
  - Used to maintain load balance
  - are stateless, so they use ZooKeeper for maintaining their cluster state
- If there are N partitions in a topic and N number of brokers
  - ✓ each broker will have one partition
- If there are N partitions in a topic and more than N brokers
  - ✓ the first N broker will have one partition and the next M broker will not have any partition for that particular topic
- If there are N partitions in a topic and less than N brokers
  - ✓ each broker will have one or more partition sharing among them

Producers

- The publisher of messages to one or more Kafka topics
- Sends data to Kafka brokers
- Every time a producer publishes a message to a broker, the broker simply appends the message to the last segment file
  - Actually, the message will be appended to a partition
- Producer can also send messages to a partition of their choice
- Kafka producer doesn't wait for acknowledgements from the broker and sends messages as fast as the broker can handle.

Consumers

- Consumers read data from brokers.
- Consumers subscribes to one or more topics and consume published messages by pulling data from the brokers.
- Since Kafka brokers are stateless, which means that the consumer has to maintain how many messages have been consumed by using partition offset.
  - If the consumer acknowledges a particular message offset, it implies that the consumer has consumed all prior messages.
  - The consumers can rewind or skip to any point in a partition simply by supplying an offset value.
  - Consumer offset value is notified by ZooKeeper.

BITS Pilani WILP

Kafka Cluster

- Kafka's having more than one broker are called as Kafka cluster.
- A Kafka cluster can be expanded without downtime.
- These clusters are used to manage the persistence and replication of message data.

Leader and Follower node

- Leader
  - ✓ "Leader" is the node responsible for all reads and writes for the given partition.
  - ✓ Every partition has one server acting as a leader.
- Follower
  - ✓ Node which follows leader instructions are called as follower.
  - ✓ If the leader fails, one of the follower will automatically become the new leader.
  - ✓ A follower acts as normal consumer, pulls messages and updates its own data store.

ZooKeeper

- ZooKeeper is used for managing and coordinating Kafka broker.
- ZooKeeper service is mainly used to notify producer and consumer about the presence of any new broker in the Kafka system or failure of the broker in the Kafka system.
  - ✓ As per the notification received by the Zookeeper regarding presence or failure of the broker then producer and consumer takes decision and starts coordinating their task with some other broker.

Kafka Workflow

- Kafka is simply a collection of topics split into one or more partitions.
  - A Kafka partition is a linearly ordered sequence of messages, where each message is identified by their index .
  - All the data in a Kafka cluster is the disjointed union of partitions.
  - Incoming messages are written at the end of a partition and messages are sequentially read by consumers.
- Kafka provides both pub-sub and queue based messaging system in a fast, reliable, persisted, fault-tolerance and zero downtime manner.

BITS Pilani WILP

Workflow of Pub-Sub Messaging

- Producers send message to a topic at regular intervals.
- Kafka broker stores all messages in the partitions configured for that particular topic. It ensures the messages are equally shared between partitions.
- Consumer subscribes to a specific topic.
- Once the consumer subscribes to a topic, Kafka will provide the current offset of the topic to the consumer and also saves the offset in the Zookeeper ensemble.
- Consumer will request the Kafka in a regular interval for new messages.
- Once Kafka receives the messages from producers, it forwards these messages to the consumers.
- Consumer will receive the message and process it.
- Once the messages are processed, consumer will send an acknowledgement to the Kafka broker.
- Once Kafka receives an acknowledgement, it changes the offset to the new value and updates it in the Zookeeper. Since offsets are maintained in the Zookeeper, the consumer can read next message correctly even during server outrages.
- This above flow will repeat until the consumer stops the request.
- Consumer has the option to rewind/skip to the desired offset of a topic at any time and read all the subsequent messages.

Workflow of Queue Messaging / Consumer Group

- In a queue messaging system
  - Instead of a single consumer, a group of consumers having the same "Group ID" will subscribe to a topic.
  - Consumers subscribing to a topic with same "Group ID" are considered as a single group and the messages are shared among them.

Workflow of Queue Messaging / Consumer Group(2)

- Producers send message to a topic in a regular interval.
- Kafka stores all messages in the partitions configured for that particular topic.
- A single consumer subscribes to a specific topic.
- Kafka interacts with the consumer in the same way as Pub-Sub Messaging until new consumer subscribes the same topic.
- Once the new consumer arrives, Kafka switches its operation to share mode and shares the data between the two consumers.
- This sharing will go on until the number of consumers reach the number of partition configured for that particular topic.
- Once the number of consumer exceeds the number of partitions, the new consumer will not receive any further message until any one of the existing consumer unsubscribe.

Kafka Server Properties file

Config / server.properties

```
# The id of the broker. This must be set to a unique integer
  for each broker.
broker.id=1
# The port the socket server listens on
port=9092
# A comma separated list of directories under which to
  store log files
log.dirs=/tmp/kafka-logs
```


BITS Pilani WILP

Kafka CLI

- Server Management
- Zookeeper Server
  - ✓ bin/zookeeper-server-start.sh config/zookeeper.properties
- Kafka Server
  - ✓ bin/kafka-server-start.sh config/server.properties

Kafka CLI(2)

- Kafka Topic Lifecycle Management
  - bin/kafka-topics.sh
- Create
  - ✓ --create --zookeeper zkConnectString --replication-factor count --partitions count --topic topic-name
- List
  - ✓ --list --zookeeper zkConnectString
- Update
  - ✓ --alter -- zookeeper zkConnectString --topic topic_name --partitions count
- Delete
  - ✓ --delete --zookeeper zkConnectString --topic topic_name

Kafka CLI(3)

- Kafka Producer
  - bin/kafka-console-producer.sh --broker-list localhost:9092 --topic topic-name
- Kafka Consumer
  - bin/kafka-console-consumer.sh --zookeeper localhost:2181 —topic topic-name --from-beginning

Kafka Java API

- Kafka Producer API
  - ✓ Applications intending to write into Kafka topics
- Kafka Consumer API
  - ✓ Applications intending to read messages from Kafka topics
- Kafka Streaming API
  - ✓ Applications intending to do computations on messages

BITS Pilani WILP

Kafka Java API (2)

- Kafka Producer API
- Producer class
  - The central part of the Producer API
  - provides an option to connect Kafka broker in its constructor
  - provides close method to close the producer pool connections to all Kafka brokers.
- KafkaProducer class
  - Send - to send messages asynchronously to a topic
  - Flush - to ensure all previously sent messages have been actually completed
  - Metrics - helps in getting the partition metadata for a given topic
- ProducerRecord
  - key/value pair that is sent to Kafka cluster

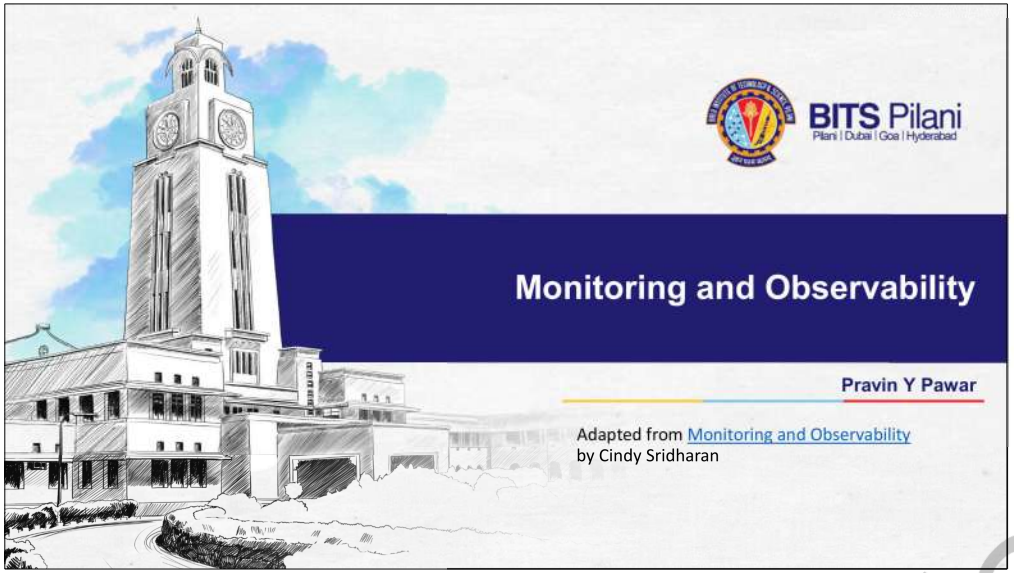
Kafka Java API (3)

- Kafka Consumer API
- KafkaConsumer class
  - used to consume messages from the Kafka Cluster
  - Subscribe - Subscribe to the given list of topics
  - Unsubscribe - Unsubscribe the topics from the given list of partitions
  - Poll - Fetch data for the topics or partitions specified using one of the subscribe/assign APIs
- ConsumerRecords class
  - acts as a container for ConsumerRecord
  - is used to keep the list of ConsumerRecord per partition for a particular topic
- ConsumerRecord class
  - used to create a consumer record with specific topic name, partition count and <key, value> pairs.

Reference

- Real-Time Analytics , Byron Ellis
  - https://kafka.apache.org/quickstart
  - https://kafka.apache.org/intro.html
  - http://tutorialspoint.com

BITS Pilani WILP



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Monitoring and Observability

Pravin Y Pawar

Adapted from Monitoring and Observability
by Cindy Sridharan

Only half joking – Monitoring to Observability

- Why call it monitoring? That's not sexy enough anymore.
- Observability, because rebranding Ops as DevOps wasn't bad enough,
  - now they're devopsifying monitoring too!
- Is that supposed to be like the second coming of DevOps? Or was it the Second Way?
  - It all felt so cultish anyway.
- What is the difference between "monitoring" and "observability", if any?
  - Or is the latter just the latest buzzword on the block,
  - to be flogged and shoved down throats until it has been milked for all its worth!

Monitoring

- "Monitoring" traditionally was a preserve of Operations engineers
  - term often invokes not very pleasant memories in minds of many who've been doing it for long enough
- In the eyes of many, "monitoring" harks back to many dysfunctional aspects of the old school way of operating software,
  - term this day causes some people to think of simple up/down checks
- True that a decade ago, up/down checks might've been all a "monitoring" tool would have been capable of
  - in the recent years "monitoring" tools have evolved greatly
  - to the point where many, many, many people no longer think of monitoring as just external pings
- While they might still call it "monitoring",
  - the methods and tools they use are more powerful and streamlined

Whitebox monitoring

- In general, tend to measure at three levels: network, machine, and application
  - Application metrics are usually the hardest, yet most important, of the three
    - Very specific to business, and they change as applications change
- Time series, logs and traces are all more in vogue than ever these days
  - are forms of "whitebox monitoring", which refers to a category of monitoring based on the information derived from the internals of systems
- Whitebox monitoring isn't really a revolutionary idea anymore
  - at least not since Etsy published the seminal blog post introducing statsd
- For all its flaws, statsd was a game changer,
  - most open source time series based systems as well as commercial monitoring solutions have supported statsd style metrics for years now
  - while not perfect, statsd style metrics collection was a huge improvement over the way one did "monitoring" previously

BITS Pilani WILP

Monitoring is for symptom based Alerting

- Monitoring system should address two questions: what's broken, and why?
  - the "what's broken" indicates the symptom
  - the "why" indicates a (possibly intermediate) cause
  - "What" versus "why" is one of the most important distinctions in writing good monitoring with maximum signal and minimum noise
- Blackbox monitoring
  - monitoring a system from the outside by treating it as a blackbox
  - is something is very good at answering the what is broken and alerting about a problem that's already occurring (and ideally end user impacting)
- Whitebox monitoring,
  - is fantastic for the signals that can be anticipated in advance and be on the lookout for
  - is for the known, hard failure modes of a system, the sort that lend themselves well toward exhibiting any deviation in a dashboardable manner, as it were.

"Monitorable" systems

One of the design goals while building systems should be to make it as monitorable as possible

- A good example of something that needs "monitoring"
  - a storage server running out of disk space
  - a proxy server running out of file descriptors
- Building "monitorable" systems requires being able to understand the failure domain of the critical components of the system proactively!
- The more mature a system, the better understood are its failure modes
  - Being able to entirely architect away a failure mode is the best thing one can do while building systems
  - The next best thing one can do is to be able to "monitor" impending failures and alert accordingly
- For monitoring to be effective, it becomes salient to be able to identify
  - a small set of hard failure modes of a system
  - or a core set of metrics that can indicate the health of the system accurately

Monitoring data needs to actionable

- In choosing what to monitor, keep the following guidelines to be considered:
  - The rules that catch real incidents most often should be as simple, predictable, and reliable as possible
  - Data collection, aggregation, and alerting configuration that is rarely exercised (e.g., less than once a quarter for some SRE teams) should be up for removal.
  - Signals that are collected, but not exposed in any prebaked dashboard nor used by any alert, are candidates for removal.
- When not used to directly drive alerts, monitoring data should be optimized for providing a bird's eye view of the overall health of a system
  - In the event of a failure, monitoring data should immediately be able to provide visibility into impact of the failure as well as the effect of any fix deployed
- The crucial thing to understand here is that monitoring doesn't guarantee that failure can be completely avoided
  - Monitoring provides a good approximation of the health of a system, but monitoring doesn't prevent failure entirely!
- Monitoring can furnish one with a panoramic view of systems' performance and behavior in the wild
  - however, does not make systems completely impregnable to failure, and that shouldn't be its goal either

Monitoring vs Observability

- The goals of "monitoring" and "observability" are different!
  - "Observability" isn't a substitute for "monitoring" nor does it obviate the need for "monitoring"
  - they are complementary
- "Observability" might be a fancy new term on the horizon, but it really isn't a novel idea
  - Events, tracing, exception tracking are all a derivative of logs
  - if one has been using any of these tools, one already has some form of "observability"
- In essence "observability" captures what "monitoring" doesn't (and ideally, shouldn't)!

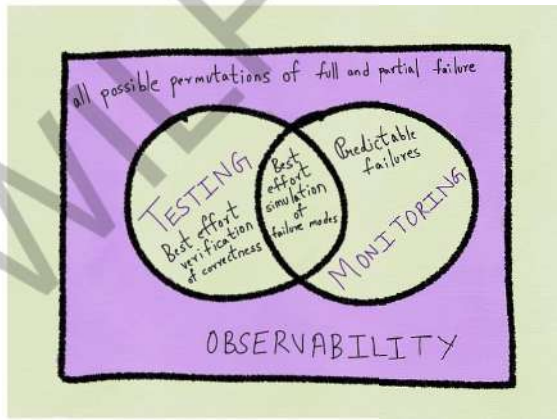
BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Monitoring vs Observability(2)

- "Monitoring" is best suited to report the overall health of systems
  - Aiming to "monitor everything" can prove to be an anti-pattern
  - is best limited to key business and systems metrics derived from time-series based instrumentation, known failure modes as well as blackbox tests
- "Observability", on the other hand, aims to provide highly granular insights into the behavior of systems
  - along with rich context
  - perfect for debugging purposes
- Since it's still not possible to predict every single failure mode a system could potentially run into or predict every possible way in which a system could misbehave,
  - it becomes important that we build systems that can be debugged armed with evidence and not conjecture

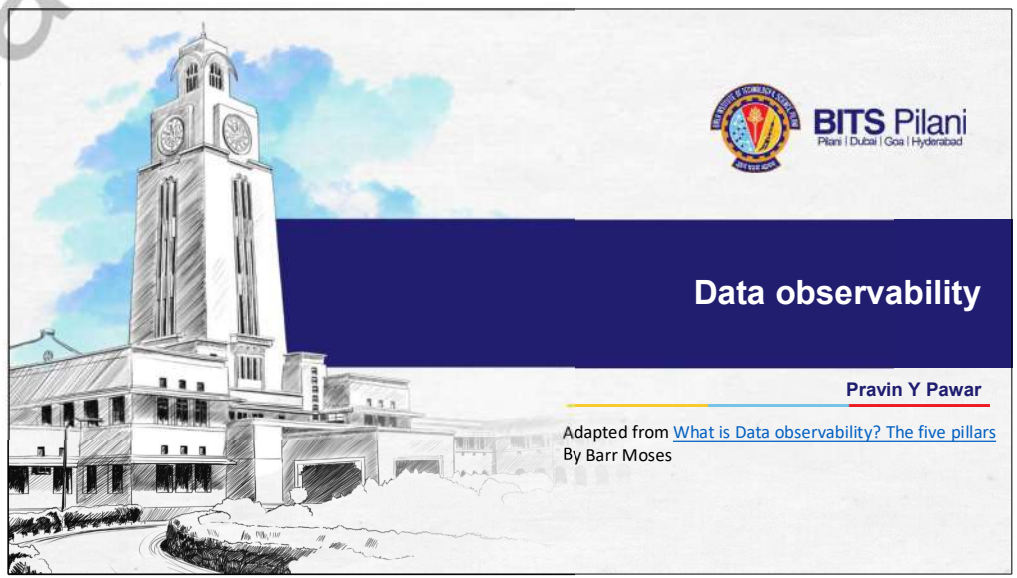
Monitoring vs Observability(3)



source

Observability at Twitter

- Twitter has since published a two part blog post on its observability stack
  - The posts are more about the architecture of the different components than the term itself
- These are the four pillars of the Observability Engineering team's charter:
  - Monitoring
  - Alerting/visualization
  - Distributed systems tracing infrastructure
  - Log aggregation/analytics
- "Observability", according to this definition, is a superset of "monitoring",
  - providing certain benefits and insights that "monitoring" tools provides



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Data observability

Pravin Y Pawar

Adapted from What is Data observability? The five pillars
By Barr Moses

BITS Pilani WILP

innovate achieve lead

Observability

- Observability is no longer just for software engineering!
  - With the rise of data downtime and the increasing complexity of the data stack,
  - observability has emerged as a critical concern for data teams, too.
- Developer Operations (DevOps) teams have become an integral component of most engineering organizations
  - remove silos between software developers and IT, facilitating the seamless and reliable release of software to production
- As organizations grow and the underlying tech stacks powering them become more complicated
  - it's important for DevOps teams to maintain a constant pulse on the health of their systems
- Observability, a more recent addition to the engineering lexicon, speaks to this need, and refers to the monitoring, tracking, and triaging of incidents to prevent downtime.

Observability engineering

- As a result of this industry-wide shift to distributed systems,
  - observability engineering has emerged as a fast-growing engineering discipline
- At its core, observability engineering is broken into three major pillars:
  - Metrics refer to a numeric representation of data measured over time
  - Logs, a record of an event that took place at a given timestamp, also provide valuable context regarding when a specific event occurred
  - Traces represent causally related events in a distributed environment
- Taken together, these three pillars give DevOps teams valuable insights to predict future behavior
  - and in turn, trust their system to meet SLAs

The rise of data downtime

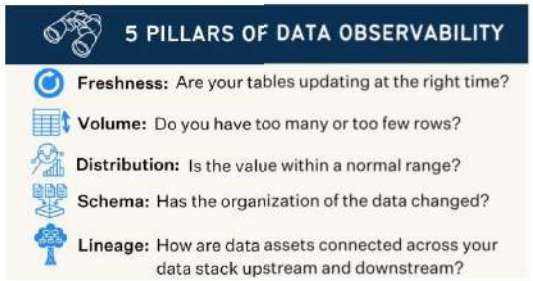
- Common phenomenon
  - Most of the time a report is delivered, only to be notified minutes later about issues with data
  - didn't matter how strong data pipelines were or how many times reviewed SQL: data just wasn't reliable!
- Data downtime tops the list of the problems!
  - periods of time when data is partial, erroneous, missing, or otherwise inaccurate
  - only multiplies as data systems become increasingly complex, supporting an endless ecosystem of sources and consumers
- For data engineers and developers, data downtime means wasted time and resources
- For data consumers, it erodes confidence in decision making

Data observability

- Instead of putting together a holistic approach to address data downtime,
  - teams often tackle data quality and lineage problems on an ad hoc basis
  - much in the same way DevOps applies observability to software
- Data observability refers to an organization's comprehensive understanding of the health and performance of the data within their systems.
- Data observability tools employ
  - automated monitoring,
  - root cause analysis,
  - data lineage,
  - and data health insights
  - to proactively detect, resolve, and prevent data anomalies
- Results in
  - healthier data pipelines,
  - increased team productivity,
  - enhanced data management practices,
  - and ultimately, higher customer satisfaction

BITS Pilani WILP

Five pillars of data observability



- Together, these components provide valuable insight into the quality and reliability of data.

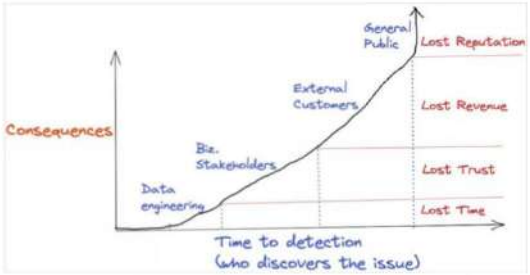
Five pillars of data observability(2)

- Freshness
  - seeks to understand how up-to-date your data tables are, as well as the cadence at which your tables are updated
  - is particularly important when it comes to decision making
  - after all, stale data is basically synonymous with wasted time and money
- Quality
  - Data pipelines might be in working order but the data flowing through them could be garbage
  - looks at the data itself and aspects such as percent NULLS, percent uniques and if data is within an accepted range
  - gives insight into whether or not tables can be trusted based on what can be expected from data
- Volume
  - refers to the completeness of data tables and offers insights on the health of data sources
  - If 200 million rows suddenly turns into 5 million, should know why

Five pillars of data observability(3)

- Schema
  - Changes in the organization of data, in other words, schema, often indicates broken data
  - Monitoring who makes changes to these tables and when is foundational to understanding the health of data ecosystem
- Lineage
  - When data breaks, the first question is always "where?"
  - Data lineage provides the answer by telling
    - which upstream sources and downstream ingestors were impacted,
    - which teams are generating the data
    - and who is accessing it
- Good lineage also collects information about the data (also referred to as metadata)
  - speaks to governance, business, and technical guidelines associated with specific data tables
  - serving as a single source of truth for all consumers

Why is data observability important?



Data observability is important because the consequences of data downtime can be severe. Image courtesy of Shane Murray.

- For data engineers and developers, data observability is important because data downtime means wasted time and resources
- For data consumers, it erodes confidence in your decision making

BITS Pilani WILP

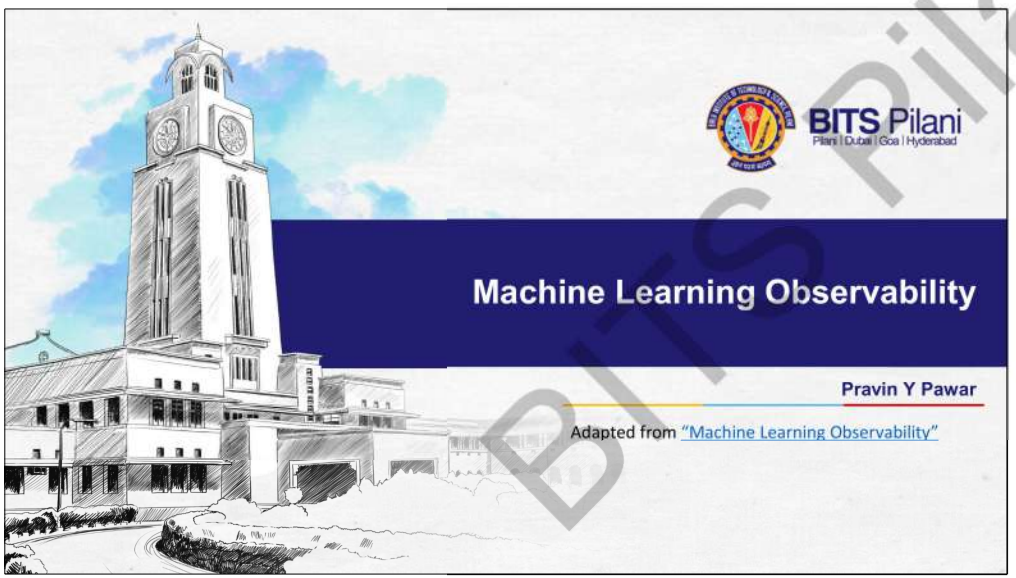
The key features of data observability tools

"what are data observability tools?"

- Connects to existing stack quickly and seamlessly and does not require modifying your data pipelines, writing new code, or using a particular programming language
  - allows quick time to value and maximum testing coverage without having to make substantial investments
- Monitors data at-rest and does not require extracting the data from where it is currently stored
  - allows the data observability solution to be performant, scalable and cost-efficient
- Requires minimal configuration and practically no threshold-setting
  - should use machine learning models to automatically learn environment and data
  - uses anomaly detection techniques to let know when things break
  - minimizes false positives by taking into account not just individual metrics, but a holistic view of data and the potential impact
- Requires no prior mapping of what needs to be monitored and in what way
  - helps identifying key resources, key dependencies and key invariants to get broad data observability with little effort
- Provides rich context that enables rapid triage and troubleshooting, and effective communication with stakeholders impacted by data reliability issues
  - shouldn't stop at "field X in table Y has values lower than Z today."
- Prevents issues from happening in the first place by exposing rich information about data assets
  - so that changes and modifications can be made responsibly and proactively

The future of data observability

- Data observability is a rapidly maturing but still evolving space
  - For example, G2 Crowd created a data observability category in late 2022,
  - but there is not a data observability Gartner Magic Quadrant
  - But Gartner did place data observability on their 2022 Data Management Hype Cycle!
- However, multiple companies and technologies are identifying with the term data observability
  - has been tremendous investor activity, and, most importantly, customer interest and values are at all-time highs
- Data observability is at the heart of the modern data stack,
  - whether it's enabling more self-service analytics and data team collaboration, adoption,
  - or working alongside dbt unit tests and Airflow circuit breakers to prevent bad data from entering the data warehouse (or data lake) in the first place
- A core component of important and emerging data quality best practices such as data mesh, data SLAs, and data contracts

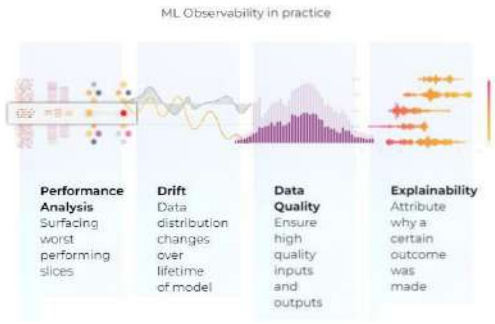


What is ML Observability?

- ML Observability is a tool used to monitor, troubleshoot, and explain machine learning models
  - as they move from research to production environments
- An effective observability tool should
  - not only automatically surface issues,
  - but drill down to the root cause of your ML problems
  - and act as a guardrail for models in production

BITS Pilani WILP

4 Pillars of ML Observability



Performance Analysis

- ML observability enables fast actionable performance information on models deployed in production
- While performance analysis techniques vary on a case-by-case basis depending on model type and its use case in the real world,
  - common metrics include: Accuracy, Recall, F-1, RMSE, and Precision
- Performance analysis in an ML observability system ensures that performance has not degraded drastically from when it was trained or when it was initially promoted to production.

Drift

- ML observability encompasses drift to monitor for a change in distribution over time,
  - measured for model inputs, outputs, and actuals of a model
- Measure drift to identify if models have grown stale, have data quality issues, or if there are adversarial inputs in model
- Detecting drift in models will help protect models from performance degradation and allow to better understand how to begin resolution

Data Quality

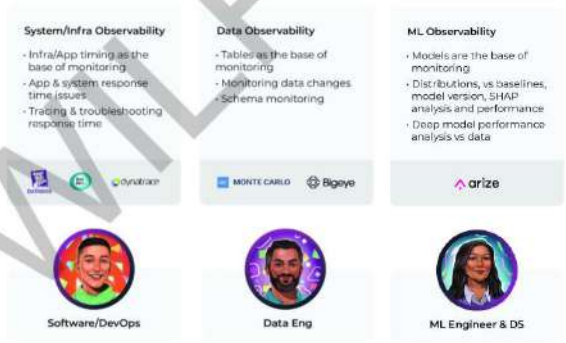
- Data quality checks in an ML observability system identify hard failures
  - within data pipelines between training and production
  - that can negatively impact a model's end performance
- Data quality includes
  - monitoring for cardinality shifts,
  - missing data,
  - data type mismatch,
  - out-of-range,

BITS Pilani WILP

Explainability

- Explainability in ML observability uncovers feature importance
  - across training, validation, and production environments
  - which provides the ability to introspect and understand why a model made a particular prediction
- Explainability is commonly achieved by
  - calculating metrics such as SHAP and LIME
  - to build confidence and continuously improve machine-learned models

ML observability in context



arize

How Can I Achieve ML Observability?

arize

ML Observability with Arize

- ML Observability is the practice of obtaining a deep understanding into model's data and performance across its lifecycle
  - Observability doesn't just stop at surfacing a red or green light,
  - but enables ML practitioners to root cause/explain why a model is behaving a certain way in order to improve it

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED
LEARNING PROGRAMMES

Thank You!

In our next session:

BITS Pilani WILP

Privacy

- Privacy is a notion that has proven notoriously difficult to define in scholarship
- The rise of big data has proven a dramatic example of Privacy.
- Experts used to think that de-identified data was appropriate and safe to release
  - Such data would have removed what was thought to be identifying information, such as name or address, that could easily be matched to a person
  - However, information about other factors associated with, but seemingly not directly related to, a specific person's identity was still released, such as birthday, race, or zip code.
- With the advent of big data, many datasets were compiled, often about overlapping groups of people,
  - became possible to use different datasets together to identify people from de-identified information
  - For example, for most Americans, it turned out to be possible to identify them in a de-identified dataset merely from knowing their birthday, zip code, and gender.

Privacy goals in ML

- An ML process involves many assets which may need to be protected such as
  - The identity of the data contributors
  - the raw dataset collected by them
  - the feature extracted from datasets
  - the model itself
  - its input (or part of it) whenever the model is queried

Privacy goals in ML(2)

- Identity Privacy
  - Protecting the data contributor's identity is necessary in many data collection processes,
    - especially when the data collected may reveal political beliefs, sexual preferences, health conditions etc.
  - Achieving identity privacy can be seen as ensuring anonymity of the data contributors
    - both from the model creators and the model owners, who are highly involved in the data collection stage of an ML process,
    - but also from adversaries which in many cases are acting as model consumers
- Raw Dataset Privacy
  - Often, the raw dataset Z contains sensitive information, such as the X-ray images from the cancer diagnosis example
    - Privacy violations could also introduce further new attack vectors
  - For instance, consider a scenario where an ML model can calculate energy efficient plans
    - previously it has been trained on a dataset which contains the energy consumption and location of houses
    - Compromising this dataset can expose the time intervals in which the residents of a specific house are absent
    - The latter can then be used as input in defining strategies for robberies
  - Often, compromising the privacy of the raw dataset could also result in compromising identity privacy,
    - since data in Z can be used to identify a specific data contributor
  - A common malpractice which eases the task of compromising Z is storing the dataset as plaintext
    - instead of using state of the art encryption

BITS Pilani WILP

Privacy goals in ML(3)

- Feature Datasets Privacy
  - The privacy of the feature datasets is as important as the privacy of the raw dataset
  - Both the training set $X1$ and the validation set $X2$ are extracted from Z
    - in case of them being compromised, information about Z could also be recovered
  - Unfortunately, the privacy of the feature datasets is threatened in many ML models, which by default store the set $X1$ inside the model
- Model Privacy
  - Privacy of the model $\mathcal{M}\theta$ refers to the secrecy of the produced model $\mathcal{M}\theta$ and its parameter θ
    - keeping the model secret is a common goal desired by the model owners
  - However, exposing part of the model's functionality to the model consumers is inevitable
    - since observing the responses to queries may reveal information about the internal structure of the model
- Input Privacy
  - Often the input (or part of it) to the model may be sensitive and needs to be protected
    - only the output of the model is revealed to the model consumers
  - Consider the scenario of an ML model that takes as input medical records to predict if a certain disease can be inherited
    - likely that the medical record contributors do not desire to disclose their medical records to each other
  - In many cases, achieving input privacy requires not only keeping the input x secret,
    - but also limiting the information about x disclosed through the response $y = \mathcal{M}\theta$

Two key ideas

- In terms of
  - how they relate to a notion of privacy
  - how they relate to taking an existing dataset with personal information and turning it into a dataset that can be used or released without compromising individual identity
- Broad Ideas
  - k-anonymity
  - Differential privacy

Threat Models and Attacks

- Adversary models and Type of attacks that target the assets of ML processes
- Two different adversary models
  - White box adversaries
  - Black box adversaries
- White box adversaries
  - who know the structure of the model $\mathcal{M}\theta$, its parameter θ,
  - while they may also know part of the raw dataset Z
  - finally, they can interact with the model (i.e., the adversary is a model consumer in this case)
- Black box adversaries
  - who have no knowledge about the model and its parameters
  - but they can query the model and observe its response

Threat Models and Attacks(2)

- Membership Inference Attacks
  - Can be performed by a black box adversary
  - Adversary wants to determine if a given data point z was part of the raw dataset Z, or if a particular data point
  - x was part of the training set $X1$
  - Usually exploit that ML models often behave differently on the data used for their training versus unseen data
  - Can compromise the raw dataset privacy or the feature datasets privacy
  - For instance,
    - this attack could be used by an adversary to learn whether a specific individual's record was used to train an ML model which determines the presence of a certain disease
- De-anonymisation Attacks
  - Aims to identify an individual who has contributed his data into a dataset
  - Many model owners publish the raw dataset Z or the feature datasets $X1$ and $X2$,
    - which may allow an adversary to compromise the identity privacy of the data contributors
    - even though the dataset has been first anonymized
  - Requires white box access to the dataset

BITS Pilani WILP

Threat Models and Attacks(3)

- Reconstruction Attacks
  - Goal of this attack is to construct the raw dataset Z
    - by reverse engineering the feature training dataset $X1$ or the validation dataset $X2$
  - Can compromise the raw dataset privacy
  - Requires white box access to a model that hard-codes the feature datasets inside it
- Model Extraction Attacks
  - Goal of this attack is to construct an approximation $\mathcal{M}\theta$ of the model $\mathcal{M}\theta$
  - The more accurate the approximation is, the more successful the attack is
  - Adversary has black box capabilities and trains his model $\mathcal{M}\theta$ by collecting pairs of queries and their responses received by the model $\mathcal{M}\theta$
  - Violates the model privacy
- Model Inversion Attacks
  - Aim at inferring properties about the training dataset $X1$, even when it is not explicitly stored in the model $\mathcal{M}\theta$
  - Can even be performed by a limited black box attacker who can interact with the model by querying it and collecting its responses
  - Adversary may first perform a model extraction attack and then utilise the model $\mathcal{M}\theta$ to accomplish a model inversion attack

k-anonymity

- The idea behind k-anonymity is that in a given dataset,
  - for any given combination of categories of interest,
  - there should be at least k individuals (externally specified) who fall into any given bucket
- For example, could apply k-anonymity to a dataset listing individuals in a town
  - by requiring that data be bucketed such that for any zip code / birth information / gender category there were at least 10 individuals
- Many ways to accomplish this
  - report birthdays at the month or year level
  - report only the first four digits of a zip code rather than all six
- The appropriate size for k will depend on the particular domain,
  - which could have different implications regarding sensitivity of the data,
  - possibility to build useful datasets with large k, and possibility of reidentification given other known potentially Linkable datasets

Differential privacy

- Mathematical method adds noise to data such that probabilistic guarantees can be made regarding the possibility (or more importantly, lack of possibility) to make inferences about a specific individual when given access to that noisified data
- Idea behind differential privacy is that it would be a privacy problem for one individual's inclusion, or non-inclusion, in a dataset to influence operations, such as the calculation of averages or other statistics, so that the dataset's information could be inferred based on aggregate reporting
- For example,
  - if the mean age of a class is reported and the size of the group, with and without an individual, possible to know that individual's age
  - If differential privacy is applied, it would, in probability, not be possible to infer that individual's age at a pre-specified level of precision and given a pre-specified query budget

Methods to Preserve Privacy

- k-anonymity and differential privacy refer to very specific notions of privacy and computational measures
  - Indeed, privacy is itself a highly technical and specialized field, perhaps best left to experts in terms of implementation
- A variety of accessible privacy-enhancing measures are transparent, straightforward to implement, and meaningful
  - Should include them in workflow and will likely need to customize them
- Likely to already be familiar concepts to systems administrators and those who deal with compliance issues
  - sometimes are painfully unfamiliar to data scientists and ML engineers
- Two aspects
  - Technical measures
  - Institutional measures

BITS Pilani WILP

Technical measures

- Access controls
  - A key way to preserve privacy and reduce threats is to implement robust access controls
  - Any data about people in a database should be treated as a "need to know" resource,
    - with ML engineers requesting access for specific purposes
    - rather than being able to freely access or browse data
  - Likewise, access should be revisited periodically
    - to make sure staff members do not retain access to data for which they no longer have a valid active reason for access
- Access logging
  - Keep track of who is accessing specific forms of data and when
  - Makes it possible to understand
    - data use patterns,
    - see when someone might be inappropriately accessing data,
    - preserve evidence in case allegations of inappropriate use are made later
  - An analysis of such logs may also indicate ways
    - in which data storage schema could be refactored to reduce the extent of data that different use patterns can access
  - For example, if an ML modeling calls for access to a sensitive table of data merely to access one column,
    - consider splitting off that column of data rather than granting access to a full table of additional but unnecessary pieces of information

Technical measures(2)

- Data minimization
  - The collection of data should be minimized, and likewise the use of data should be minimized
  - Data should not be collected merely because it "could be useful" in the future
  - Data should be logged only when there is an immediate use case for this data
- Data separation
  - The data needed for legitimate business uses should be separated from sensitive data(such as names and addresses) that is unlikely to be relevant to creating an ML model
  - For example, to predict users' clicks, there doesn't seem to be a justification for knowing a user's name or address
    - no reason for that information to be stored with information that might be useful for that particular prediction task, such as past browsing history or demographic information.
  - Studying data access logs can help to identify ways in which data storage can be refactored to minimize exposure of data to ML applications

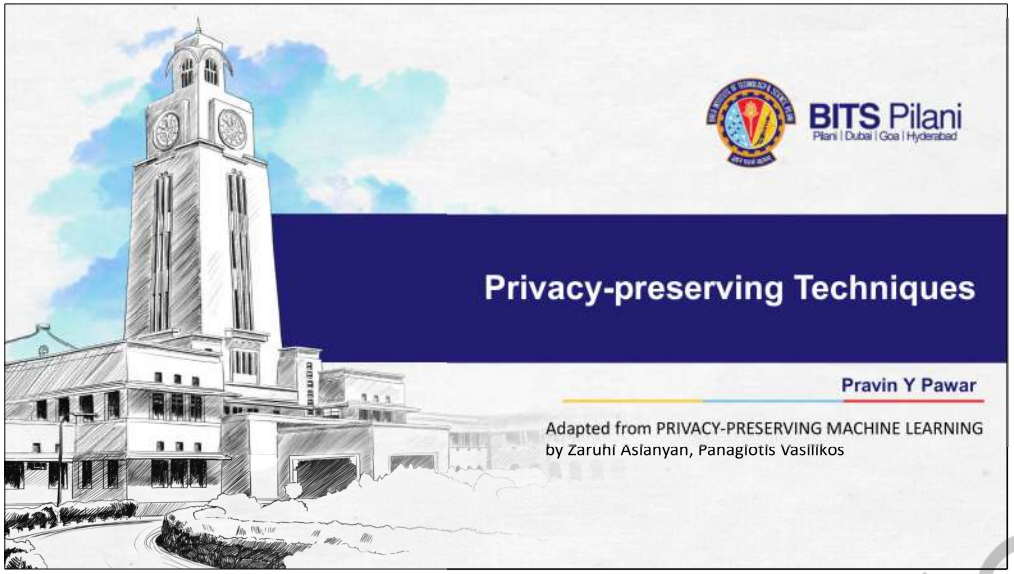
Institutional measures

- Ethics training
  - Everyone needs ethics training when they enter for designing ML products
  - ML engineers should be given — but usually are not — a thorough review
    - not only of general ethics training (such as the possibility of bias in data)
    - but also domain-specific training when building algorithms for a specific use case
- Data access guidelines
  - Makes sense to have explicit rules that are readily available
    - regarding what constitutes appropriate access to data and use of that data and what use cases are expressly prohibited
  - A lack of clear and explicit ethics rules can lead to an institutional culture without accountability
  - Should have clear data access and appropriate data use guidelines in place in a location that is readily apparent and accessible

Institutional measures(2)

- Privacy by design
  - Set of design principles that can apply to digital product, including ML pipelines and ML-driven products
  - The notion is that privacy is something that shouldn't be tacked on to the end of a process that already exists
  - Rather, privacy should be intrinsic to design considerations from the start
    - should be a question and concern addressed at all working stages
  - Privacy by design can provide a flexible but holistic way to ensure privacy in all elements of ML development

BITS Pilani WILP



Privacy-preserving techniques

- Privacy-preserving techniques that can be used to strengthen ML algorithms to protect individuals' data
  - Anonymization
  - Differential Privacy
  - Homomorphic Encryption
  - Multi-party Computation
  - Federated Learning

Anonymization

- Data anonymization aims at protecting the privacy of an individual in a given dataset
- Before releasing a dataset,
  - First step is to remove all direct identifiers, such as name, address and phone numbers in most of the cases this is not enough
  - Only removing the direct identifiers will still allow an attacker to re-identify an individual in the dataset by linking the data with other additional information that he/she has
- In order to avoid such cases, additional anonymization steps should be enforced
  - Several methods, such as k-anonymity and l-diversity, have been studied that make it harder for an adversary to learn anything about an individual from an anonymized dataset
- For a great many cases l-diversity together with k-anonymity provides a robust privacy model
  - k-anonymity is an anonymization method that ensures the information about an individual in the published dataset cannot be distinguished from at least k-1 other individuals in the same dataset
  - On the other hand, a dataset is called l-diverse if every equivalence class has at least $l \geq 2$ different values for the sensitive attributes
- Though challenging, anonymization needs to be applied aiming at providing data privacy while preserving utility.

Differential Privacy

- Differential privacy is a privacy method that helps reveal useful information about a dataset without revealing any private information about an individual in the dataset
- The method guarantees that even if an attacker knows all records in a dataset,
  - s/he will not be able to link a specific record to an individual on the basis of the output of a differentially-private method
- Outcome of the analysis is not(significantly) dependent on an individual's record being in the dataset
  - Hence, the privacy risk is essentially the same with or without the individual's participation in the dataset
- Differential privacy is achieved by adding random noise to the result,
  - which can be done through various differentially-private mechanisms,
  - such as the Laplace mechanism, the exponential mechanism and the randomized response mechanism

BITS Pilani WILP

BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
WORK INTEGRATED LEARNING PROGRAMMES

Homomorphic Encryption

- Cryptographic method that allows computing on encrypted data,
  - decrypted output is identical to the output of the computation on the original unencrypted input
- The method is typically used as follows:
  1. the owner of the data encrypts them by means of a homomorphic function and shares the result with a third party in charge of performing a given computation
  2. the third party performs the computation on the encrypted data and returns the result, which is encrypted because the input data are encrypted
  3. the owner of the data decrypts the result, thereby obtaining the result of the computation on the original plain-text data
- During this process the third party does not have access to the unencrypted input or output.
- Has a number of limitations
  - various encryption schemes that allow performing different computations on encrypted data,
  - they are generally limited to working with addition or multiplication over integers
- For example, an additively encryption scheme allows to take two encrypted messages, $m1$ and $m2$, and produce the encrypted message $m1 + m2$.
  - Moreover, for security purposes, some noise is typically added to the input data during the encryption process
  - noise accumulates with the operations performed on encrypted data
  - noise growth is particularly severe under multiplication
- In order to get the expected result when decrypting, the noise must be kept below a certain threshold
  - This threshold affects the number of computations that can be performed on encrypted data

Multi-party Computation (MPC)

- Technology that allows multiple parties to compute a function on their private inputs without revealing them
  - parties are independent and do not trust each other
  - main idea is to allow to perform a computation on the private data while keeping the data secret
  - guarantees that all participants learn nothing more than what they can learn from the output and their own input
- Both MPC and homomorphic encryption are powerful privacy techniques,
  - however they usually have high communication and computation costs

Federated Learning

- Federated learning allows ML processes to be decentralised limiting the information exposed from the contributor's datasets
  - controlling the risk of compromising the dataset's and identity privacy
- General idea of federated learning
  - A central ML model $\mathcal{M}\theta$ owned by some central authority (e.g., a company) can be further trained on new private datasets from data contributors
  - by having each contributor performing the training locally with its own dataset
  - then updating the central model $\mathcal{M}\theta$ (i.e., update the model's parameter θ)
- Works as follows
  (1) the central model $\mathcal{M}\theta$ is distributed among a number of n participants (i.e., the data contributors)
  (2) each participant updates locally the model $\mathcal{M}\theta$ by training it on its own local dataset $\mathcal{D}l$, producing a new local parameter θl
  (3) each participant sends its update θl to the central authority
  (4) the central authority aggregates the local parameters of each participant, producing a new parameter θ' which is used to update the central model
  (5) This process can be further continued until the central model has been trained enough
- Can be seen as a flexible approach for achieving privacy of raw datasets, feature datasets and identity privacy,
  - since it lifts the requirement of collecting vast amounts of information from the data contributors and then train on them in a centralised manner
- Communication in federated learning networks could create performance overhead, which is not present in centralised ML approaches,
  - put in conjunction with the availability of devices in the network, scalability also becomes a concern

Thank You!

In our next session:

BITS Pilani WILP